StrainDesign

Release 1.18

Philipp Schneider

Mar 13, 2026

CONTENTS:

1 A COBRApy-based package for computational design of metabolic networks **3**

2 Installation: **5**
- 2.1 Developer Installation: . . . 5
- 2.2 Apple Silicon (ARM64) note: . . . 5

3 Examples: **7**

4 How to cite: **9**
- 4.1 Solvers . . . 9
  - 4.1.1 3rd party solver installation . . . 9
    - 4.1.1.1 CPLEX . . . 9
    - 4.1.1.2 Gurobi . . . 10
    - 4.1.1.3 SCIP . . . 10
  - 4.1.2 Solver selection . . . 11
- 4.2 Network Analysis . . . 12
  - 4.2.1 Flux optimization (FBA/pFBA) . . . 12
    - 4.2.1.1 Parsimonious FBA (pFBA) . . . 13
  - 4.2.2 Flux variability analysis (FVA) . . . 14
  - 4.2.3 Gene knockout simulation . . . 15
  - 4.2.4 Yield optimization . . . 16
    - 4.2.4.1 Mathematical background . . . 17
- 4.3 Plotting the flux space . . . 17
  - 4.3.1 Production Envelopes . . . 17
  - 4.3.2 Yield Spaces . . . 20
  - 4.3.3 Mixed Plots . . . 21
  - 4.3.4 3D plots . . . 24
    - 4.3.4.1 Interactive and animated 3D plots . . . 26
- 4.4 Computational strain design: Growth-coupled production (GCP) . . . 27
  - 4.4.1 pGCP: potentially growth-coupled production . . . 28
  - 4.4.2 wGCP: weakly growth-coupled production . . . 29
  - 4.4.3 dGCP: directionally growth-coupled production . . . 30
  - 4.4.4 SUCP: substrate-uptake-coupled production . . . 31
- 4.5 Minimal Cut Sets (MCS) . . . 32
  - 4.5.1 Prerequisites . . . 32
    - 4.5.1.1 1) Add and verify production pathway . . . 32
  - 4.5.2 Example 1: Strain designs with a minimum product (1,4-butanediol) yield (SUCP strain design) 34
  - 4.5.3 Example 2: Enforce product (1,4-BDO) synthesis at all growth states (dGCP strain design) . 45
  - 4.5.4 Example 3: Suppress flux states that are optimal with respect to a pre-defined objective function (wGCP strain design) . . . 50

4.5.5 Example 4: Protect flux states that are optimal with respect to a pre-defined objective function (pGCP strain design) . . . 53
4.5.6 Example 5: All single gene knockouts that prohibit growth (synthetic lethals). . . . 56
4.5.7 Example 6: Genome-scale strain designs with a minimum product (1,4-butanediol) yield (SUCP strain design) . . . 58
4.5.8 Example 7: Suppress flux states in a toy network . . . 67
4.5.9 Example 8: Suppress and protect flux states in a toy network . . . 69
4.5.10 Theoretical background . . . 70
4.6 Multi-level strain optimization approaches . . . 71
4.6.1 OptKnock . . . 73
4.6.1.1 Example 9: OptKnock strain design . . . 73
4.6.1.2 Example 10: OptKnock strain design with a tilted objective function . . . 76
4.6.1.3 Example 11: Genome-scale OptKnock strain design . . . 79
4.6.2 RobustKnock . . . 84
4.6.2.1 Example 12: RobustKnock strain design . . . 85
4.6.3 OptCouple . . . 88
4.6.3.1 Example 13: OptCouple strain design . . . 89
4.6.4 Combining nested optimization strain design with MCS . . . 92
4.6.4.1 Example 14: Combining OptKnock with a tilted objective function and the MCS approach . . . 92
4.7 Standalone network compression . . . 95
4.7.1 Standalone GPR-integraton . . . 97
4.7.1.1 Gene perturbation studies . . . 98
4.8 CNApy interface . . . 99
4.9 StrainDesign API . . . 99
4.9.1 straindesign . . . 99
4.9.1.1 Submodules . . . 100
4.9.1.2 Package Contents . . . 158
4.10 Legacy Methods . . . 159
4.10.1 Java-based EFMTool compression (`backend='efmtool_rref'`) . . . 159
4.10.2 JAVA_HOME path . . . 159
4.11 StrainDesign Developer's Guide . . . 159
4.11.1 Table of Contents . . . 159
4.11.2 1. Architecture Overview . . . 161
4.11.3 2. Repository Structure . . . 161
4.11.4 3. Public API & Entry Points . . . 162
4.11.4.1 Primary function . . . 162
4.11.4.2 SDModule specification . . . 163
4.11.4.3 Result access . . . 163
4.11.4.4 Lower-level access . . . 163
4.11.5 4. Preprocessing Pipeline . . . 164
4.11.5.1 4.1 Model Cleaning . . . 164
4.11.5.2 4.2 Compression Pass 1 (before GPR extension) . . . 164
4.11.5.3 4.3 FVA . . . 164
4.11.5.4 4.4 GPR Integration . . . 166
4.11.5.5 4.5 Compression Pass 2 (after GPR extension) . . . 167
4.11.5.6 4.6 Network Compression — Algorithm Details . . . 167
4.11.5.7 4.7 Regulatory Interventions . . . 168
4.11.6 5. MILP Construction . . . 168
4.11.6.1 5.1 Overview & z-map Matrices . . . 168
4.11.6.2 5.2 Primal LP from COBRApy Model . . . 169
4.11.6.3 5.3 SUPPRESS Module — Farkas' Lemma Dual . . . 169
4.11.6.4 5.4 PROTECT Module . . . 170
4.11.6.5 5.5 OPTKNOCK — LP Duality for Bilevel Optimization . . . 170

4.11.6.6 5.6 ROBUSTKNOCK — Three-Level Duality . . . 170
4.11.6.7 5.7 OPTCOUPLE — Growth Coupling Potential . . . 171
4.11.6.8 5.8 Linking Binary Variables: `link_z` . . . 171
4.11.6.9 5.9 Big-M Bounding . . . 172
4.11.6.10 5.10 Indicator Constraints vs. Big-M . . . 172
4.11.7 6. Solver Backends . . . 173
4.11.7.1 6.1 MILP_LP Unified Interface . . . 173
4.11.7.2 6.2 CPLEX . . . 173
4.11.7.3 6.3 Gurobi . . . 173
4.11.7.4 6.4 SCIP . . . 174
4.11.7.5 6.5 GLPK . . . 174
4.11.8 7. Computation Approaches — SDMILP . . . 174
4.11.8.1 7.1 ANY . . . 174
4.11.8.2 7.2 BEST . . . 174
4.11.8.3 7.3 POPULATE . . . 175
4.11.8.4 7.4 Exclusion Constraints . . . 175
4.11.8.5 7.5 Solution Verification . . . 175
4.11.9 8. Post-processing & Result Container . . . 176
4.11.9.1 8.1 Solution Decompression . . . 176
4.11.9.2 8.2 GPR Translation . . . 176
4.11.9.3 8.3 SDSolutions . . . 176
4.11.10 9. Constraint & Expression Parsing . . . 177
4.11.11 10. Supporting Utilities . . . 177
4.11.11.1 `lptools.py` . . . 177
4.11.11.2 `networktools.py` — LP Suppression . . . 178
4.11.11.3 `pool.py` — SDPool . . . 180
4.11.11.4 `names.py` . . . 180
4.11.11.5 `__init__.py` . . . 180
4.11.11.6 `efmtool_cmp_interface.py` — JPype/JVM Initialization . . . 180
4.11.12 11. Known Issues, Urgent Actions & Future Work . . . 181
4.11.12.1 11.1 Urgent Action Items . . . 181
4.11.12.2 11.2 Low-Hanging Fruits . . . 182
4.11.12.3 11.3 Future Development Directions . . . 182
4.11.13 12. MILP Performance Notes . . . 187
4.11.14 13. Testing . . . 188
4.11.15 Quick Reference: Data Flow Through Key Functions . . . 188

5 References: **191**

6 Indices and tables **193**

Python Module Index **195**

Index **197**

CHAPTER

ONE

A COBRAPY-BASED PACKAGE FOR COMPUTATIONAL DESIGN OF METABOLIC NETWORKS

The comprehensive StrainDesign package for MILP-based strain design computation with the COBRApy toolbox supports MCS, MCS with nested optimization, OptKnock , RobustKnock and OptCouple , GPR-rule integration, gene and reaction knockouts and additions as well as regulatory interventions. The automatic lossless network and GPR compression allows strain design computations from genome-scale metabolic networks. Supported solvers are GLPK (available from COBRApy), CPLEX, Gurobi and SCIP .

The default compression uses a pure Python sparse RREF implementation. A legacy Java-based compression via EFMTool is optionally available (see *Legacy Methods*). Note that the Java backend (via JPype) is known to conflict with CPLEX's native library when both are loaded in the same Python session. If you use CPLEX, we recommend the default Python compression backend.

CHAPTER

TWO

INSTALLATION:

The StrainDesign package is available on pip and Anaconda. To install the latest release, run:

```
pip install straindesign
```

or

```
conda install -c cnapy straindesign
```

2.1 Developer Installation:

Download the repository and run

```
pip install -e .
```

in the main folder. Through the installation with -e, updates from a 'git pull' are at once available in your Python environment without the need for a reinstallation.

2.2 Apple Silicon (ARM64) note:

On macOS with Apple Silicon (M1/M2/M3), the default GLPK solver (shipped with COBRApy) may produce inaccurate results for larger metabolic models due to numerical precision differences on the ARM64 architecture. If you encounter unexpected infeasibility or incorrect flux values, we recommend installing an alternative solver:

- **Gurobi** (free academic license): `pip install gurobipy`
- **SCIP** (open source): `pip install pyscipopt`

Then specify the solver explicitly when calling StrainDesign functions, e.g. `sd_setup = {..., 'solver': 'gurobi'}` or `sd_setup = {..., 'solver': 'scip'}`. For standalone analysis functions: `fba(model, solver='gurobi')` or `fva(model, solver='scip')`.

CHAPTER

THREE

EXAMPLES:

Computation examples are provided in the different chapters of this documentation. The original Jupyer notebook files are located in the StrainDesign package at docs/source/examples.

CHAPTER

FOUR

HOW TO CITE:

Schneider P., Bekiaris P. S., von Kamp A., Klamt S. - StrainDesign: a comprehensive Python package for computational design of metabolic networks. Bioinformatics, btac632 (2022)

4.1 Solvers

4.1.1 3rd party solver installation

Through COBRApy, StrainDesign is already shipped with the free **GLPK** linear programming solver. Alternatively, the more powerful commercial solvers IBM CPLEX and Gurobi can be used by both COBRApy and StrainDesign, and the free solver SCIP can be used by StrainDesign. Using one of the GLPK alternatives is preferred, in particular, when using strain design algorithms like MCS, OptKnock etc. since their support of indicator contstraints renders computations significantly more stable.

In the following, you will find installation instructions for the individual solvers.

Warning

The **free community versions** of CPLEX and Gurobi can be used. However, with larger problems (100+ reactions) their problem size limitations may result in uncaught errors.

4.1.1.1 CPLEX

Together with Gurobi, CPLEX is the perfect choice for computing strain designs. Its stability and support of advanced features like indicator constraints and populating solution pools make it indispensible for genome-scale computations.

Note

You will need an academic or commercial licence to be able to use CPLEX.

Warning

CPLEX is not available for Python 3.13 and above. The newest CPLEX release (22.1.2) supports Python up to 3.12. If you are using Python 3.13+, please use Gurobi or SCIP instead.

Download and install the CPLEX suite to a location with non-root access since python will need to build some things to set up the CPLEX-API, later. Make sure that your CPLEX and Python versions are compatible. Currently, CPLEX is not officially available for Python 3.13 and above (the latest release 22.1.2 supports up to Python 3.12). Once the installation is completed, you may use the installation to set up the CPLEX-API with your Python/conda environment.

This can be done either with pip

```
pip install yourCPLEXhome/python/VERSION/PLATFORM
```

or with conda. For an installation with conda, make sure to activate the same Python/conda environment where `cobra` and `straindesign` are installed. Then call

```
python yourCPLEXhome/python/VERSION/PLATFORM/setup.py install
```

Now CPLEX should be available for your computations. If you face difficulties with building CPLEX, consider downgrading the setuptools package to setuptools==58.2.0.

The official instructions can be found here: https://www.ibm.com/docs/en/icos/22.1.0?topic=cplex-setting-up-python-api

4.1.1.2 Gurobi

Similar to CPLEX, Gurobi offers a fast MILP solvers with the advanced features of indicator constraints and solution pooling. The installation steps are similar to the ones of CPLEX.

> **Note**
>
> You will need an academic or commercial license and install the Gurobi solver software.

Ensure that the versions of Gurobi and Python versions are compatible, install Gurobi on your system and activate your license following the steps from the Gurobi manual. In the next step you will link your Gurobi installation to your Python/conda environment.

Using the command line, navigate to your CPLEX installation path and into the Python folder. The path should look similar to

```
C:/gurobi950/windows64
```

Make sure to activate the same Python/conda environment where `cobra` and `straindesign` are installed. Then call

```
python setup.py install
```

If your `gurobipy` package does not work right away, additionally install the gurobi package from conda or PyPi via

```
conda install -c gurobi gurobi
```

or

```
python -m pip install gurobipy
```

Now Gurobi is available for your computations.

The official instructions can be found here: https://support.gurobi.com/hc/en-us/articles/360044290292-How-do-I-install-Gurobi-for-Python-

4.1.1.3 SCIP

Less powerful than CPLEX and Gurobi, the open source solver SCIP still offers the solution of MILPs with indicator constraints, which gives it an edge above GLPK in terms of stability. If you want to use SCIP, you may install it via conda or pip:

```
conda install -c conda-forge pyscipopt
```

or

```
python -m pip install pyscipopt
```

> **Warning**
>
> If you encounter program crashes with SCIP (a dependency of pyscipopt), make sure you use a SCIP version of 8.0.0 or older since newer versions are unreliable in solving MILPs and can produce errors (as of December 2023), this issue might get fixed in the future. You can, manually install pyscipopt 4.2.0 and scip version 8.0.0 through `conda install -c conda-forge pyscipopt=4.2.0 scip=8.0.0`. Keep in mind that SCIP 4.2.0 is currently (December 2023) not available for python>3.10.

Official website: https://github.com/scipopt/PySCIPOpt

4.1.2 Solver selection

For any type of LP or MILP-based analysis or design method, four different sovers are supported: **GLPK** (which is built into COBRApy/optlang), **IBM CPLEX**, **Gurobi** and **SCIP**. You can query the available solvers by accessing the set `straindesign.avail_solvers`. For the subsequent steps we also import the cobra module and load the *E. coli* core model.

[1]:
```
import cobra
import straindesign as sd
model = cobra.io.load_model('e_coli_core')

sd.avail_solvers
```

```
Set parameter Username
Academic license - for non-commercial use only - expires 2023-07-20
```

[1]:
```
{'cplex', 'glpk', 'gurobi', 'scip'}
```

You may enforce the use of a specific solver by specifying the "solver"-keyword. E.g., to enforce the use of GLPK, use:

[2]:
```
solution = sd.fba(model, solver='glpk')

print(f"Maximum growth: {solution.objective_value}.")
```

```
Maximum growth: 0.873921506969.
```

By default, the automatic solver selection uses COBRApy's selection. Therefore, StrainDesign will try to use the model's selected solver:

[3]:
```
print(f"When the model\'s solver is \'{model.solver.configuration}', StrainDesign␣
↪selects {sd.select_solver(None,model)}.")
```

```
When the model's solver is '<optlang.gurobi_interface.Configuration object at␣
↪0x0000014FE00DA400>', StrainDesign selects gurobi.
```

Otherwise COBRApy's global configuration is used.

[4]:
```
print(f"COBRApy\'s solver is \'{cobra.Configuration().solver.__name__}', StrainDesign␣
↪selects {sd.select_solver()}.")
```

```
COBRApy's solver is 'optlang.gurobi_interface', StrainDesign selects gurobi.
```

[5]:
```
model.solver = 'cplex'
sd.select_solver('glpk',model)
```

[5]:
```
'glpk'
```

4.2 Network Analysis

StrainDesign provides canonical functions for maximizing and minimizing metabolic fluxes in network. The output format is identical to the ones of COBRApy's functions.

4.2.1 Flux optimization (FBA/pFBA)

Flux Balance Analysis (FBA) is a linear program that optimizes a flux rate under the given steady-state network constraints. For the case of a growth-rate maximization, the problem is written as:

$$\begin{aligned} \text{maximize} \quad & v_{growth} \\ \text{subject to} \quad & \\ & \mathbf{S}\,\mathbf{v} = \mathbf{0} \\ & \mathbf{lb} \leq \mathbf{v} \leq \mathbf{ub} \end{aligned}$$

Where $\mathbf{S}$ is the stoichiometric matrix of the metabolic model, $\mathbf{v}$ is the vector of metabolic flux rates and $\mathbf{lb}$ and $\mathbf{ub}$ are the physiological lower and upper bounds of each flux rate which also define whether a reaction can run in the reverse direction ($\mathbf{lb} < 0$) or not ($\mathbf{lb} \geq 0$). $\mathbf{S}\,\mathbf{v} = \mathbf{0}$ represents all steady-state constraints, and $\mathbf{lb} \leq \mathbf{v} \leq \mathbf{ub}$ the allowed flux ranges.

> **Note**
>
> All of the following computation examples will require the COBRApy and the StrainDesign package.

Here, we load both packages and the e_coli_core model from BiGG:

[1]:
```
import cobra
import straindesign as sd
model = cobra.io.load_model('e_coli_core')
```

```
Set parameter Username
Academic license - for non-commercial use only - expires 2023-07-20
```

An FBA is launched by a single function call. By default the model's objective function is optimized. The function returns a solution object, in which the objective value and the fluxes are stored in `solution.objective_value` and `solution.fluxes`.

[2]:
```
solution = sd.fba(model)

print(f"Maximum growth: {solution.objective_value}.")
```

```
Maximum growth: 0.8739215069684305.
```

You may also use a custom objective (in form of a linear expression) and change the optimization sense. Here we minimize the Glucose uptake rate through the PTS:

[3]:
```
solution = sd.fba(model,obj='GLCpts',obj_sense='minimize')

print(f"Minimum flux through GLCpts: {solution.objective_value}.")
```

```
Minimum flux through GLCpts: 0.4794285714285715.
```

We can also consider custom constraints (in this case limited oxygen uptake and an increased fixed ATP maintenance demand):

[4]:
```
solution = sd.fba(model,constraints=['-EX_o2_e <= 5', 'ATPM = 20'])

print(f"Maximum growth at limited oxygen uptake and high ATP maintenance: {solution.
↪objective_value}.")
```

```
Maximum growth at limited oxygen uptake and high ATP maintenance: 0.26305573292588313.
```

4.2.1.1 Parsimonious FBA (pFBA)

Parsimonious flux balance analysis optimizes a flux rate under the given steady-state network constraints, but also minimizes the sum of absolute fluxes to achieve this optimum. One can write:

$$\begin{aligned} &\text{minimize} && \Sigma|v_i| \\ &\text{subject to} \end{aligned}$$

$$\begin{aligned} &\text{maximize} && v_{growth} \\ &\text{subject to} \\ &&& \mathbf{S}\,\mathbf{v} = \mathbf{0} \\ &&& \mathbf{lb} \leq \mathbf{v} \leq \mathbf{ub} \end{aligned}$$

pFBA is the simpleset (although very rough) approach of emulating a cell's enzyme cost minimization (after an assumed growth maximization).

> **Note**
>
> pFBA solutions are more often "unique" than pure FBA solutions, since the outer minimization leaves fewer degrees of freedom in the solution space.

StrainDesign computes pFBA solutions when you pass the 'pFBA'-argument with a value of 1.

[5]:
```
fba_sol = sd.fba(model)
pfba1_sol = sd.fba(model,pfba=1)
print(f"The sum of fluxes of the regular FBA: {sum([abs(v) for v in fba_sol.fluxes.
↪values()])} "+\
      f"is usually higher than of the parsimoniuos FBA: {sum([abs(v) for v in pfba1_sol.
↪fluxes.values()])}")
```

```
The sum of fluxes of the regular FBA: 2508.293334194643 is usually higher than of the␣
↪parsimoniuos FBA: 518.4220855176071
```

Likewise, it is possible to minimize the number of active reactions to attain an optimal flux distribution. We therefore use pFBA mode 2. Most of the times modes 1 and 2 yield the same results.

[6]:
```
pfba1_sol = sd.fba(model,pfba=1)
pfba2_sol = sd.fba(model,pfba=2)
print(f"The number of active reactions in pFBA1: {sum([v!=0 for v in pfba1_sol.fluxes.
↪values()])}, "+\
      f"and pFBA2: {sum([v!=0 for v in pfba2_sol.fluxes.values()])}, is often identical.
↪")
```

```
The number of active reactions in pFBA1: 48, and pFBA2: 48, is often identical.
```

4.2.2 Flux variability analysis (FVA)

The fva function determines the possible maximal and minimal flux ranges under the given model constraints:

[7]:
```
solution = sd.fva(model)
print(solution)
```

```
         minimum  maximum
PFK          0.0   176.61
PFL          0.0    40.00
PGI        -50.0    10.00
PGK        -20.0    -0.00
PGL          0.0    60.00
...          ...      ...
NADH16       0.0   120.00
NADTRHD      0.0   378.22
NH4t         0.0    10.00
O2t          0.0    60.00
PDH          0.0    40.00

[95 rows x 2 columns]
```

The parameters 'solver' and 'constraints' can also be used in the FVA function call. As an example, we determine the flux ranges for the case that the flux sum of PDH and PFL is smaller than 8.

[8]:
```
solution = sd.fva(model, constraints='PDH + PFL <= 8')
print(solution)
```

```
         minimum     maximum
PFK          0.0  147.610000
PFL          0.0    8.000000
PGI        -50.0   10.000000
PGK        -20.0   -0.000000
PGL          0.0   60.000000
...          ...         ...
NADH16       0.0  120.000000
NADTRHD      0.0  375.220000
NH4t         0.0    8.300263
O2t          0.0   60.000000
PDH          0.0    8.000000

[95 rows x 2 columns]
```

4.2.3 Gene knockout simulation

StrainDesign allows you to simulate gene knockouts directly in FBA and FVA calls by passing gene IDs or gene names as constraints. When a constraint references a gene identifier (rather than a reaction ID), StrainDesign automatically evaluates the gene-protein-reaction (GPR) rules via AST parsing to determine which reactions lose function, and sets those reactions to zero flux.

Gene knockout constraint format

Use `gene_id = 0` or `gene_name = 0` to knock out a gene. The SD solution grammar values also work: `gene = -1` (KO) and `gene = 1` (ignored, gene is active). This can be mixed freely with regular reaction constraints. Gene identifiers are matched case-sensitively against both gene IDs and gene names in the model.

SD solution grammar (SDSolutions)

Strain design solutions use the following convention for intervention values:

- **-1** (negative): knockout (KO) — reaction/gene flux fixed to zero
- **+1** (positive): knock-in (KI) — reaction added/activated to its model bounds
- **0**: non-added KI (absence of knock-in, only shown with `show_no_ki=True`)

Gene knockout constraints are aligned with this grammar: `gene = 0` and `gene = -1` both trigger a knockout, while `gene = 1` is treated as "gene is active" and ignored. This means SD solution dict values can be passed directly as constraints.

A single non-essential gene knockout (b0727, sucB — part of the alpha-ketoglutarate dehydrogenase complex) has only a minor impact on growth:

```
[ ]: sol_wt = sd.fba(model)
     sol_ko = sd.fba(model, constraints='b0727 = 0')
     print(f"Wild-type growth:  {sol_wt.objective_value:.4f}")
     print(f"b0727 KO growth:   {sol_ko.objective_value:.4f}")
```

Gene names work as well. The knockout of an essential gene (b0720, gltA — citrate synthase) is lethal:

```
[ ]: sol_essential = sd.fba(model, constraints='gltA = 0')
     print(f"gltA KO growth:    {sol_essential.objective_value:.6f}")
```

Gene knockouts can be combined with each other and with reaction constraints. Here we demonstrate a synthetic lethal pair — b1761 (gdhA, glutamate dehydrogenase) and b3213 (gltD, glutamate synthase). Each alone is viable, but together they block both glutamate synthesis routes:

```
[ ]: sol_gdhA = sd.fba(model, constraints='b1761 = 0')
     sol_gltD = sd.fba(model, constraints='b3213 = 0')
     sol_both = sd.fba(model, constraints=['b1761 = 0', 'b3213 = 0'])
     print(f"b1761 (gdhA) KO:   {sol_gdhA.objective_value:.4f}")
     print(f"b3213 (gltD) KO:   {sol_gltD.objective_value:.4f}")
     print(f"Both KO (lethal):  {sol_both.objective_value:.6f}")
```

Gene knockouts also work with FVA. Here, the FVA after knocking out b0727 confirms that the AKGDH reaction is fixed at zero:

[ ]:
```
fva_ko = sd.fva(model, constraints='b0727 = 0')
print(fva_ko.loc[['AKGDH', 'BIOMASS_Ecoli_core_w_GAM']])
```

Gene and reaction constraints can be freely mixed:

[ ]:
```
sol_mixed = sd.fba(model, constraints=['b0727 = 0', 'EX_glc__D_e >= -5'])
print(f"b0727 KO + glucose limit: {sol_mixed.objective_value:.4f}")
```

For programmatic use, the function `gene_kos_to_constraints` translates a list of gene knockouts to reaction-level constraints explicitly:

[ ]:
```
from straindesign.networktools import gene_kos_to_constraints

constraints = gene_kos_to_constraints(model, ['b0727', 'b0116'])
print("Reactions knocked out:", [list(c[0].keys())[0] for c in constraints])
```

4.2.4 Yield optimization

Yield optmization aims to maximize a given flux expression (e.g., a product's synthesis rate) divided by another such expression (e.g., a substrate's uptake rate).

> **Warning**
>
> Yield optimization and Flux Balance Analysis are two different methods that produce distinct optimal values and also distinct optimal flux vectors.

Consider the following example of maximizing growth and biomass yield under limited oxygen uptake.

[9]:
```
numerator = 'BIOMASS_Ecoli_core_w_GAM'
denominator = '-EX_glc__D_e'
constraint = '-EX_o2_e <= 3'

sol_fba = sd.fba(model,obj='BIOMASS_Ecoli_core_w_GAM',obj_sense='maximize',
↪constraints=constraint)
fba_yield = sol_fba.fluxes[numerator] / -sol_fba.fluxes['EX_glc__D_e']

sol_yopt = sd.yopt(model,obj_num=numerator,obj_den=denominator,obj_sense='maximize',
↪constraints=constraint)
yopt_yield = sol_yopt.objective_value

print(f"Maximum yield (FBA): {fba_yield}.")
print(f"Maximum yield (yOpt): {yopt_yield}.")
```

```
Maximum yield (FBA): 0.031965425062067315.
Maximum yield (yOpt): 0.03629426243040193.
```

The best biomass yield is achieved when only respiration is used. The best growth rates use respiration and *additionally* overflow metabolism with poorer biomass yield.

The plotting chapter shows how relationships between yields and rates can be visualized.

4.2.4.1 Mathematical background

Constraint-based models ($\mathbf{S}\,\mathbf{v} = \mathbf{0}$, $\mathbf{lb} \leq \mathbf{v} \leq \mathbf{ub}$) can be rewritten in a single matrix-inequality term ($\mathbf{A}\,\mathbf{x} \leq \mathbf{b}$).

With this notation, the yield optimization is a linear fractional program (LFP):

$$\begin{array}{ll} \text{maximize} & \frac{\mathbf{c}^\intercal \mathbf{x}}{\mathbf{d}^\intercal \mathbf{x}} \\ \text{subject to} & \mathbf{A}\,\mathbf{x} \leq \mathbf{b}. \end{array}$$

Under the condition that the denominator term is strictly positive ($\mathbf{A}\,\mathbf{x} \leq \mathbf{b} \;\Rightarrow\; \mathbf{d}^\intercal \mathbf{x} > 0$), the LFP may be rewritten as an LP problem, using the Charnes-Cooper transformation. The formerly fixed boundaries $\mathbf{b}$ of the problem are then scaled by the auxiliary variable e while the variable $y = \frac{\mathbf{c}^\intercal \mathbf{x}}{\mathbf{d}^\intercal \mathbf{x}}$ expresses the original objective function:

$$\begin{array}{ll} \text{maximize} & y \\ \text{subject to} & \begin{bmatrix} \mathbf{A} & -\mathbf{b} & \mathbf{0} \\ \mathbf{d}^\intercal & 0 & 0 \\ \mathbf{c}^\intercal & 0 & -1 \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{x}} \\ e \\ y \end{bmatrix} \begin{matrix} \leq \\ = \\ = \end{matrix} \begin{bmatrix} \mathbf{0} \\ 1 \\ 0 \end{bmatrix} \\ & e \geq 0. \end{array}$$

Solutions of $\mathbf{x}$ to the LFP (first problem) can be retrieved from a solution of the LP through $\mathbf{x} = \frac{\tilde{\mathbf{x}}}{e}$.

4.3 Plotting the flux space

There are different ways to visualize the space of feasible steady-state flux vectors. The two most popular plot types are production envelopes and yield spaces.

> **Note**
>
> Production envelopes project the flux space of feasible steady-state flux vectors onto two dimensions whereas each dimension is a flux rate. Yield spaces map feasible yield combinations onto two dimensions whereas each dimension is a flux ratio.

StrainDesign provides functions for both of these plot types, but additionally support plotting of arbitrary other projections or mappings of rate and yield terms on two or three dimensions. Here, we use the e_coli_core example for demonstration purposes again.

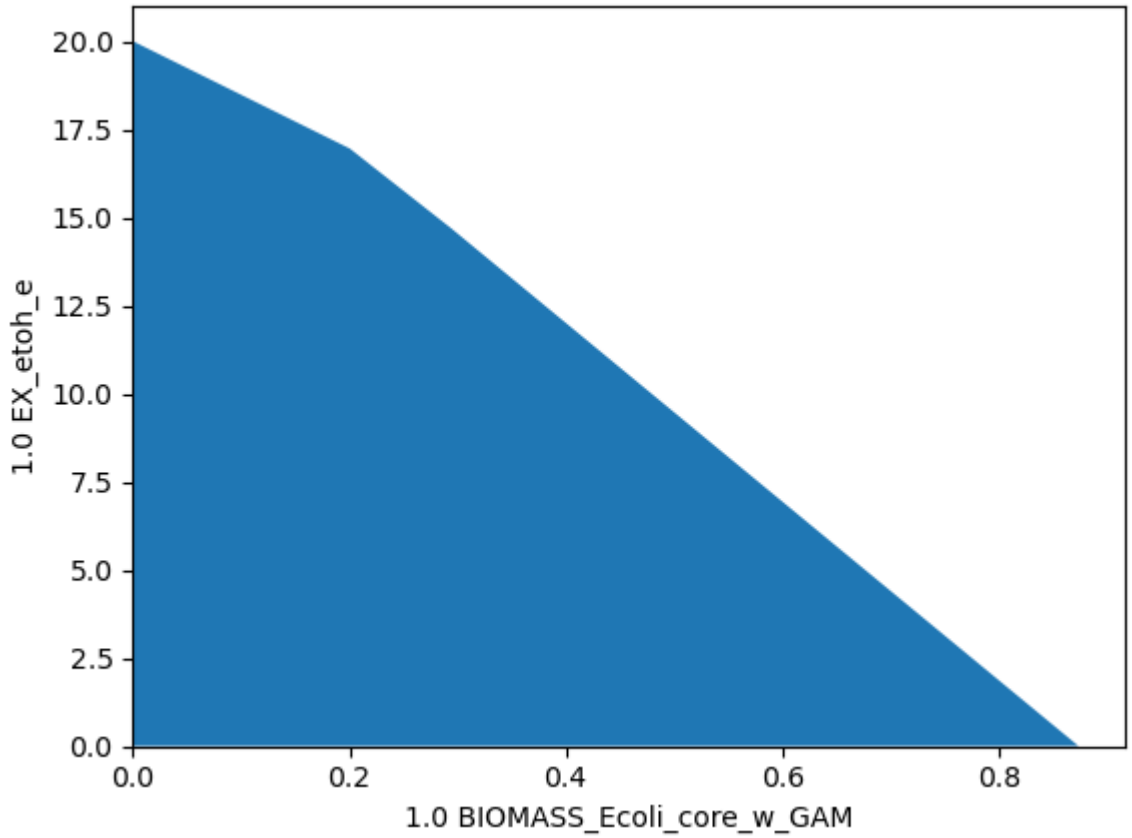
[1]:
```
import cobra
import straindesign as sd
model = cobra.io.load_model('e_coli_core')
```

```
Set parameter Username
Academic license - for non-commercial use only - expires 2026-12-09
```

4.3.1 Production Envelopes

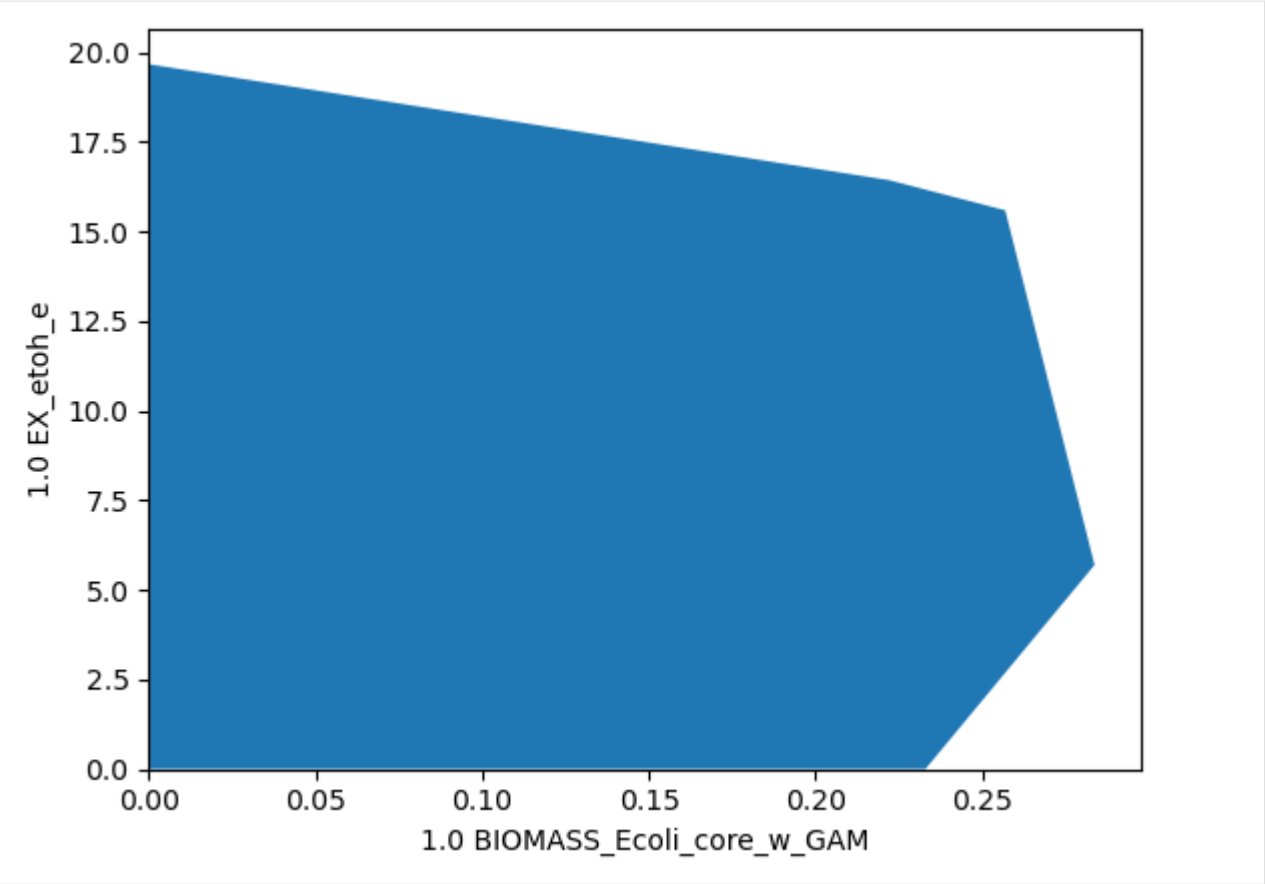
Production envelopes project the solution space of steady-state flux vectors onto the dimensions of growth rate and product synthesis rate. Such a plot can be generated by:

[2]:
```
sd.plot_flux_space(model,('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'));
```



Again, arbitrary constraints can be applied to the flux space to plot subspaces. Here, we plot the flux space within a small range of oxygen uptakes (1-2 $mmol_{0_2} g_{CDW}^{-1} h^{-1}$). This constraint reduces the maximum growth rate to a thrid, while maximum growth entails ethanol production.

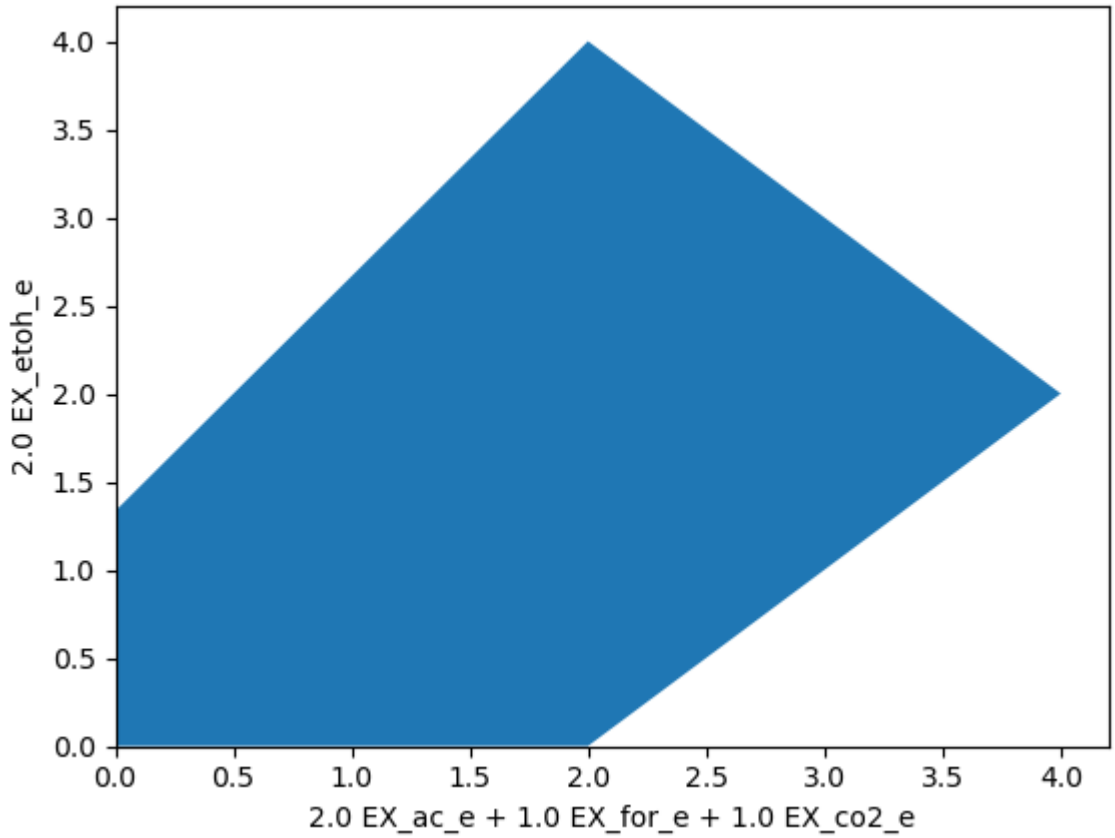
[3]:
```
sd.plot_flux_space(model,('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                   constraints=['-EX_o2_e >= 1', '-EX_o2_e <= 2']);
```



It is also possible to use arbitrary linear expressions for the axis. Here, for instance, we plot the carbon recovery in the oxidized products formate, acetate and CO2 versus the reduced product ethanol. The coefficients for each product are matched with their number of carbon atoms. Glucose uptake is set to 1 (so the input equals 6 carbon atoms). The plot now shows that, stoichiometrically, at most of 4 out of 6 atoms can be directed to either side, ethanol or oxidized products. Yet, to balance redox equivalents, it is necessary, to then direct the remaining 2 carbon atoms towars the other side.

[4]:
```
constraints=[
             'EX_h_e >= 0',
             'EX_co2_e >= 0',
             'EX_o2_e = 0'
             ]
model1 = model.copy()
model1.reactions.ATPM.lower_bound = 0
model1.reactions.EX_glc__D_e.lower_bound = -1
model1.reactions.EX_glc__D_e.upper_bound = -1
datapoints, triang, plot1 = sd.plot_flux_space(model1,('2 EX_ac_e + 1 EX_for_e + 1 EX_
↪co2_e','2 EX_etoh_e'),
                                               constraints = constraints);
```

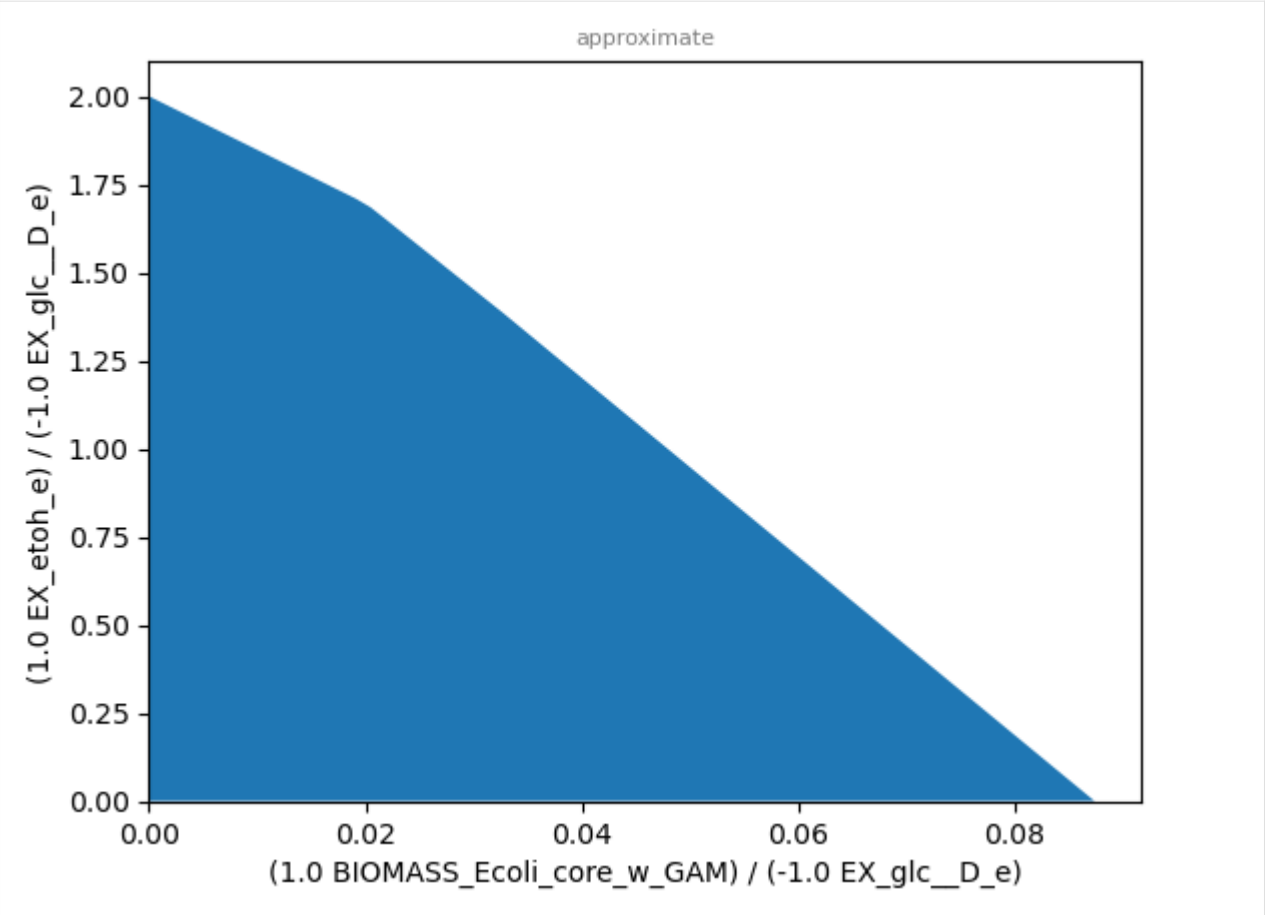
```
Read LP format model from file C:\Users\phili\AppData\Local\Temp\tmpfe1uvv3g.lp
Reading time = 0.00 seconds
: 72 rows, 190 columns, 720 nonzeros
```



4.3.2 Yield Spaces

The yield space plot can be used to depict the relationship between two yield terms in a metabolic networks. Here, we plot the biomass yield vs the etahnol production yield.

[5]:
```
sd.plot_flux_space(model,(('BIOMASS_Ecoli_core_w_GAM','-EX_glc__D_e'),('EX_etoh_e','-EX_
↪glc__D_e')));
```



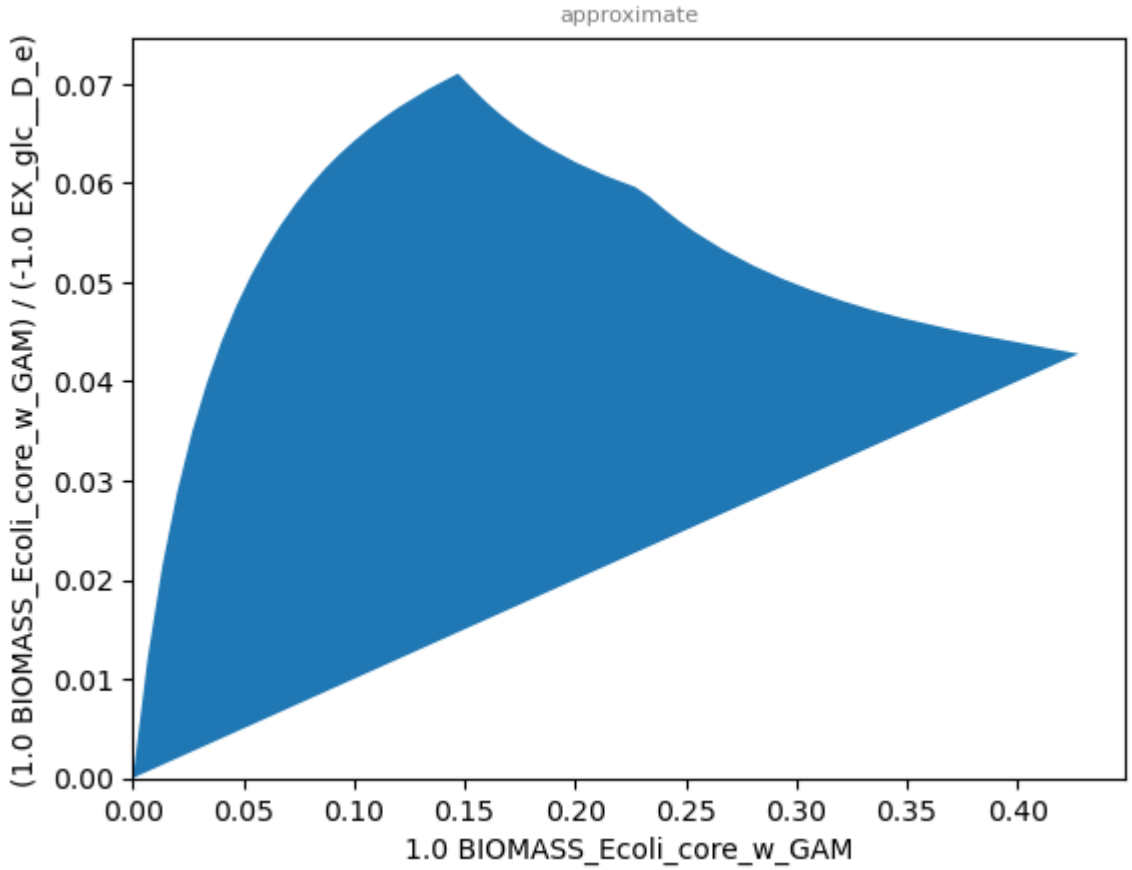
Yield space plots often look similar to their corresponding phase planes, as it is the case here.

> **Warning**
>
> It should be noted that yield terms are generally non-linear and, adversely to pure flux-space projections, yield spaces may therfore be non-polyhedral or even non-convex.

4.3.3 Mixed Plots

The following plot uses a rate and a yield for the different axes. It shows the relationship between possible growth rates and biomass yields under limited oxygen uptake.

[6]:
```
sd.plot_flux_space(model,('BIOMASS_Ecoli_core_w_GAM',('BIOMASS_Ecoli_core_w_GAM','-EX_
↪glc__D_e')),
                   constraints='-EX_o2_e <= 6');
```



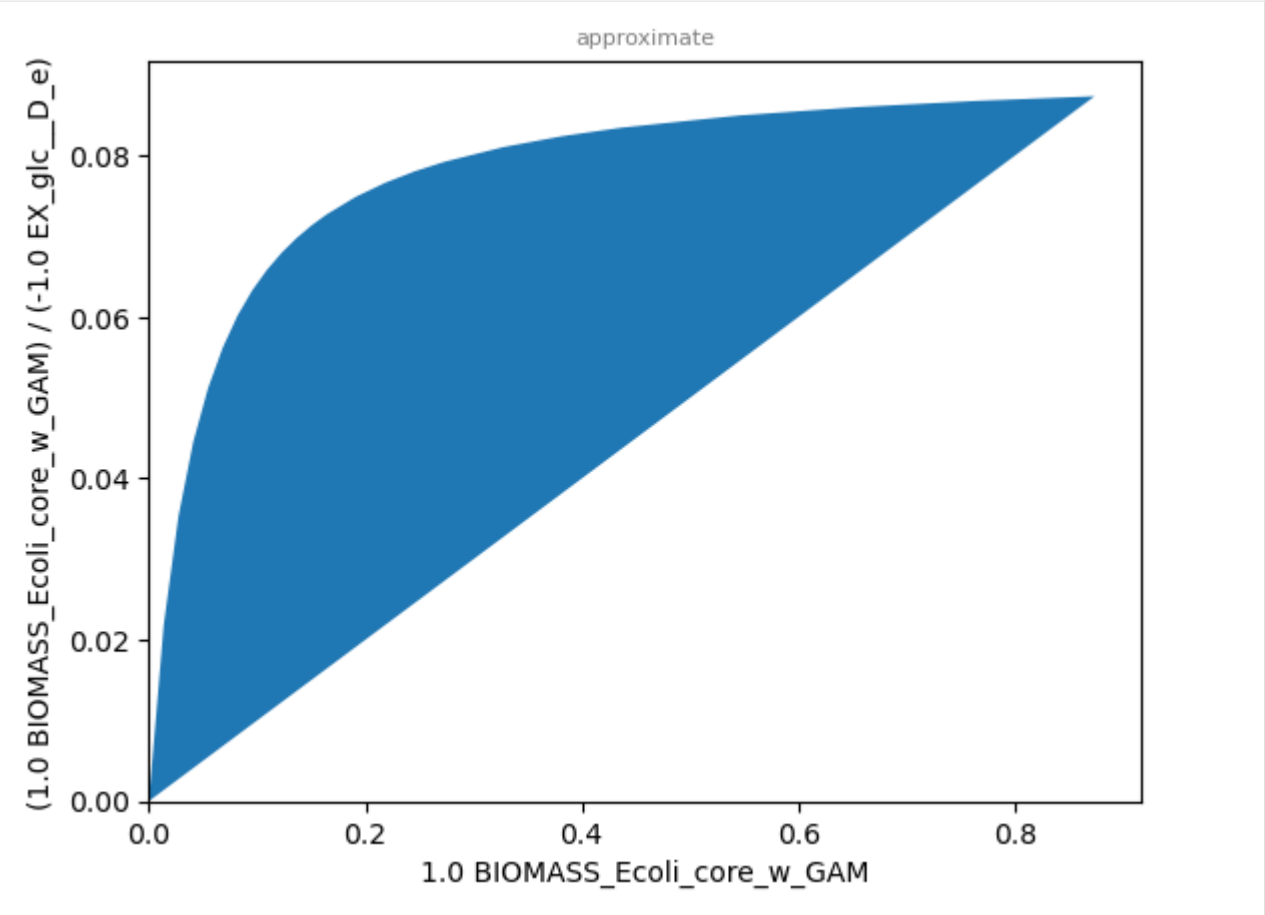
The graph shows that under limited oxygen uptake, maximum growth rate and maximum biomass yield do not coincide. As discussed in the chapter of yield optimization, overflow metabolism may be used to boost growth at the expense of biomass yield.

Note

If growth yield and growth rate could be used interchangably, we would observe a straight diagonal line here.

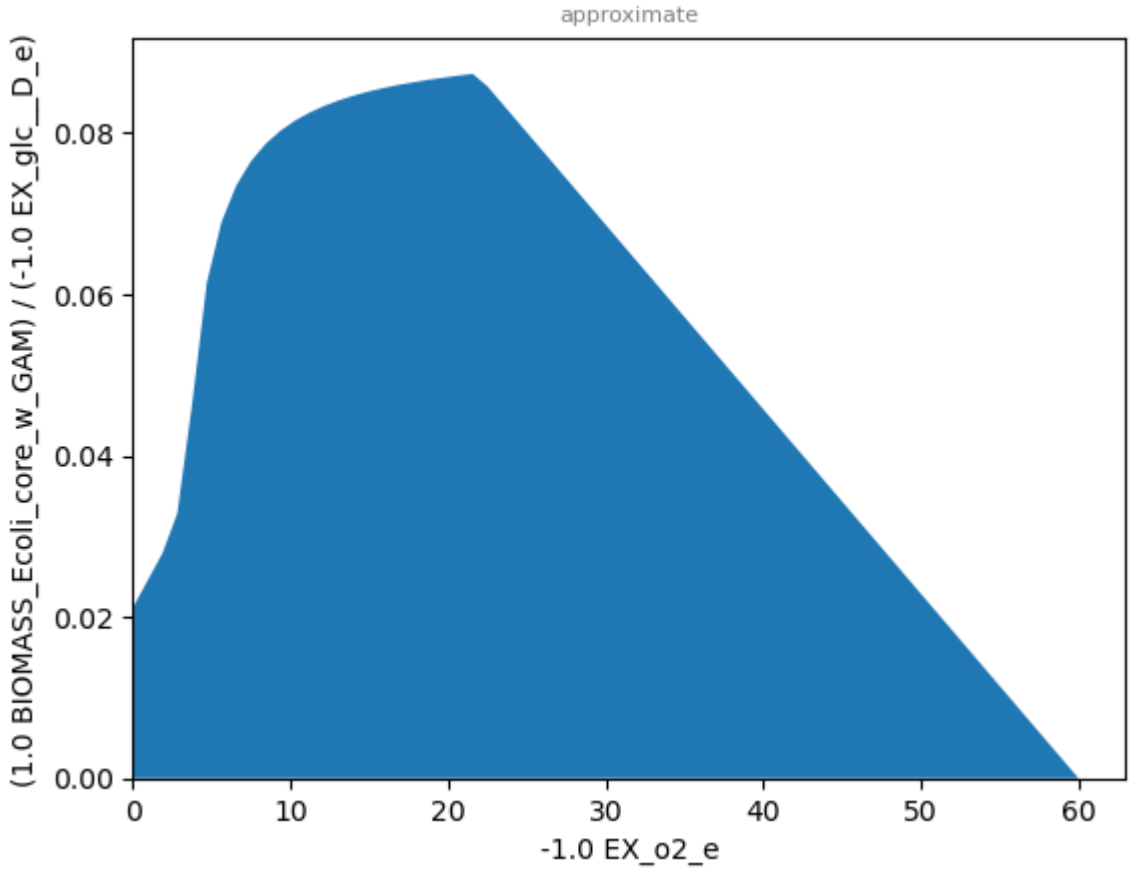
In the case of unlimited oxygen uptake, we observe that best yield and growth rate indeed coinside, since respiration can be fully exploited to attain the hightest possible biomass yield:

[7]:
```
sd.plot_flux_space(model,('BIOMASS_Ecoli_core_w_GAM',('BIOMASS_Ecoli_core_w_GAM','-EX_
↪glc__D_e')));
```



We may also use a mixed rate-yield (oxygen uptake vs biomass yield) plot to graphically determine the oxygen uptake rate needed to attain the maximum yield. There is a sigmoidal increase of the biomass yield with increased oxygen availability:

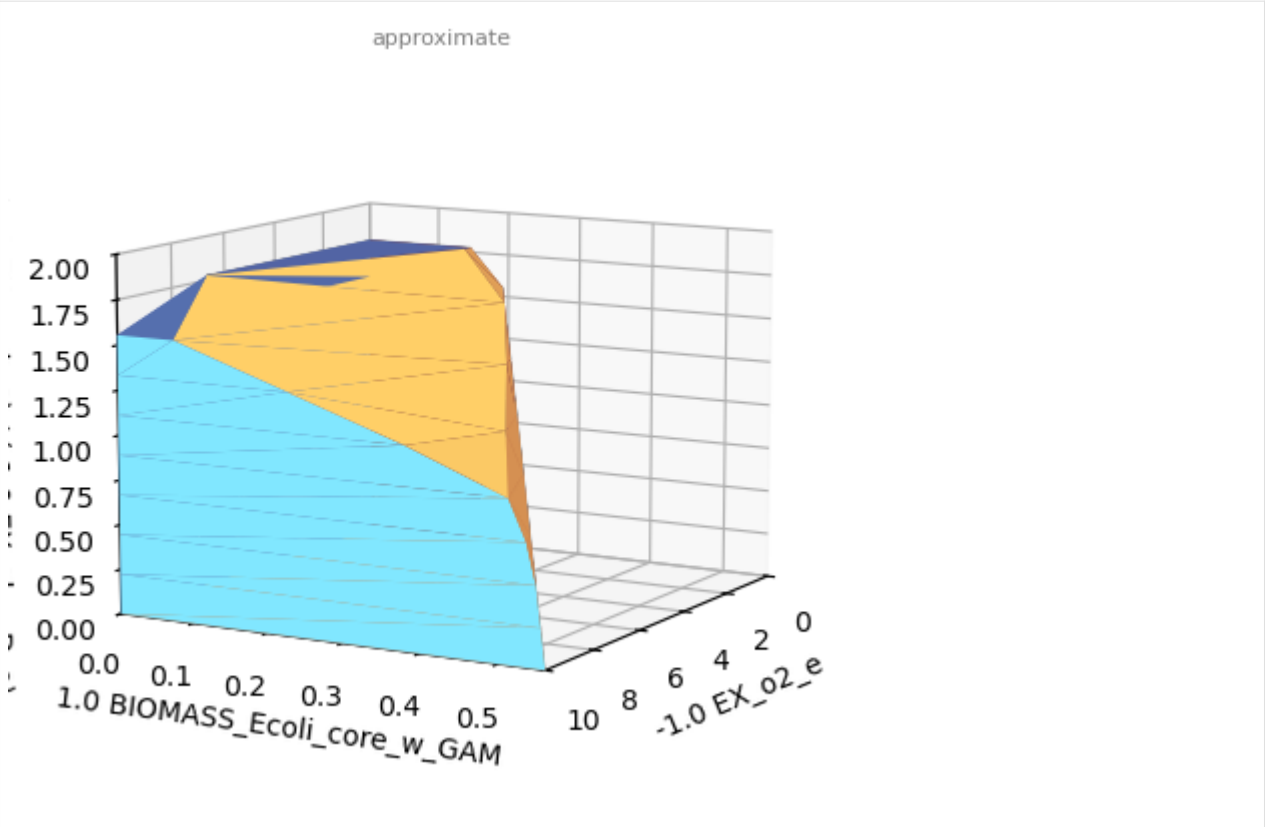
[8]:
```
sd.plot_flux_space(model,('-EX_o2_e',('BIOMASS_Ecoli_core_w_GAM','-EX_glc__D_e')));
```



4.3.4 3D plots

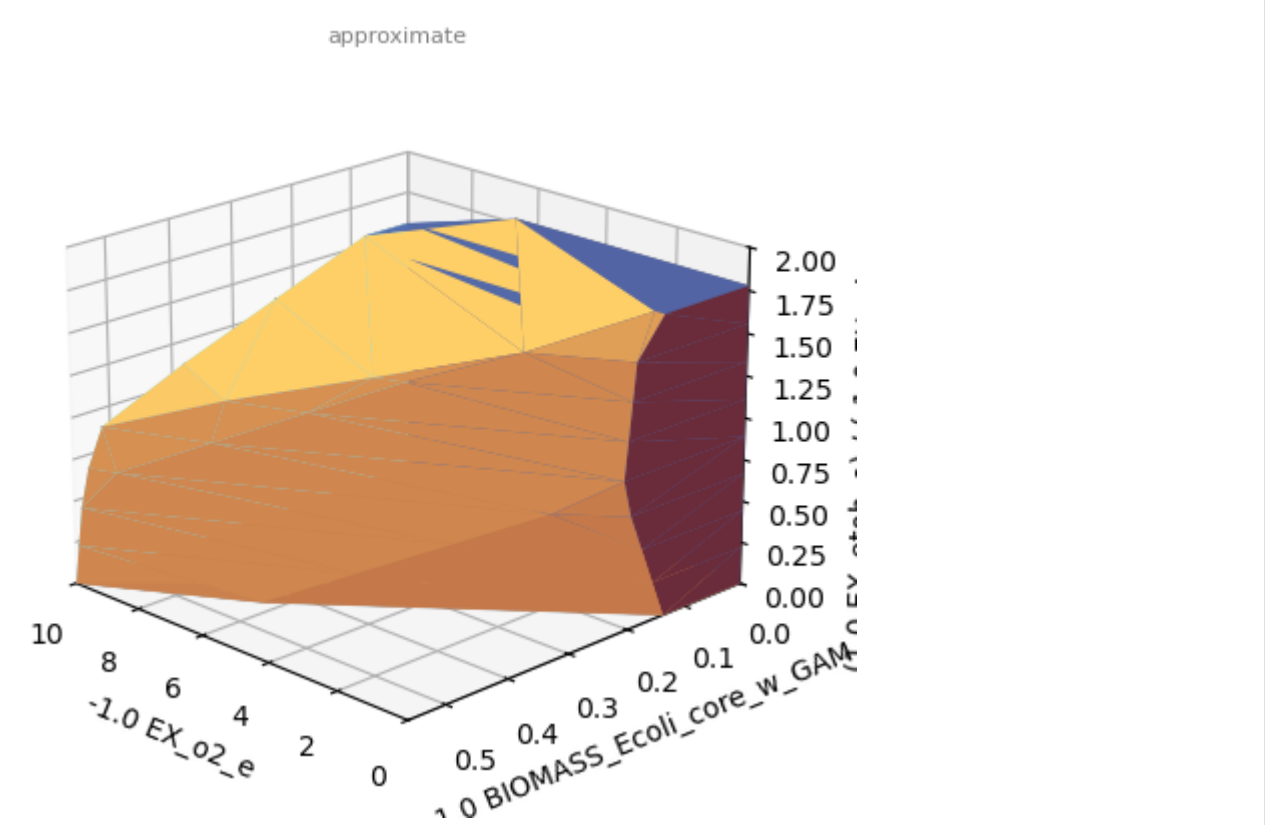
A 3-dimensional plot may help tp reveal more complex relationships between fluxes and yields. In the following plot, we analyze the relationship between oxygen uptake, growth rate and ethanol yield.

[9]:
```
sd.plot_flux_space(model,('-EX_o2_e','BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e','-EX_glc__D_
↪e')),
                   constraints='-EX_o2_e <= 10',points=10);
```



Since a 3D-plot does not show us the angle we are looking for, we may halt plotting and use matplotlib commands to rotate the plot before the actual plotting.

[10]:
```
%matplotlib inline
import matplotlib.pyplot as plt
_,_,plot1 = sd.plot_flux_space(model,('-EX_o2_e','BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e',
↪'-EX_glc__D_e')),
                    constraints='-EX_o2_e <= 10',points=10,show=False);
plot1._axes.view_init(20, 135)
plt.show()
```



The plot now shows that at low or no oxygen uptake, a minimum ethanol yield is guaranteed at maximum growth rates, while at higher growh rates this is not the case. The critical point seems to be reached at an oxygen uptake rate of about $7\,mmol_{0_2} g_{CDW}^{-1} h^{-1}$.

4.3.4.1 Interactive and animated 3D plots

You may set a custom rendering option, to generate an interactive that can be rotated or a plot in a separate window. This can be done either through the IPython magic command, e.g., `%matplotlib tk`:

[11]:
```
%matplotlib tk
sd.plot_flux_space(model,('-EX_o2_e','BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e','-EX_glc__D_
↪e')),
                   constraints='-EX_o2_e <= 10',points=10);
```

or by passing the parameter `plt_backend`:

[12]:
```
sd.plot_flux_space(model,('-EX_o2_e','BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e','-EX_glc__D_
↪e')),
                   constraints='-EX_o2_e <= 10',points=10,
                   plt_backend='TkAgg');
```

You may also animate 3D figures and save these animations to GIFs or movie files. Please note, that the ffmpeg codec needs to be installed and available. For that matter, refer to the matplotlib reference.

[ ]:
```
import matplotlib.animation as animation
from IPython import display
```

(continues on next page)

(continued from previous page)

```
r1 = ('-EX_o2_e')
r2 = ('BIOMASS_Ecoli_core_w_GAM')
y3 = ('EX_etoh_e', '-EX_glc__D_e')
constraints = ['NADTRHD = 0',
               'NADH16 = 0',
               'LDH_D = 0',
               'PPC = 0']
_,_,plot2 = sd.plot_flux_space(model, (r1, r2, y3),
                           constraints=constraints,
                           points=10,
                           show=False)
# specify animation
def animate(angle):
    plot2._axes.view_init(20, angle)
    return plot2

# generate animation
ani = animation.FuncAnimation(
    plot2.figure, animate, save_count=360)

# Save the animation in gif (requires ffmpeg)
try:
    writer = animation.FFMpegWriter(
        fps=25, bitrate=1000)
    ani.save("movie.gif", writer=writer)
except Exception:
    pass  # ffmpeg not available, use pre-generated gif

# Display the gif
display.display(display.Image(filename="movie.gif"))
```

Plot generation

For rate-only axes, boundary tracing finds exact vertices of the flux space projection via LP optimizations. In 2D, this traces the convex polygon boundary directly. In 3D with all rate axes, face-normal refinement recovers the exact polytope.

For yield axes, the boundary is approximated: in 2D via adaptive midpoint refinement of upper/lower boundaries, in 3D by slicing along the yield axis and tracing rate-rate polygons per slice.

The `points` parameter controls resolution of approximate (yield) plots. The `cmap` parameter controls the colormap used for 3D face coloring.

4.4 Computational strain design: Growth-coupled production (GCP)

A main principle for the design of bioproduction hosts is the coupling of product synthesis to microbial growth, such that the product of interest becomes a byproduct of growth. Different notions of growth-coupling principle prevail in literature. With gradually increasing coupling strength, we can distinguish four different classes:

- potentially growth-coupled production (pGCP): product synthesis is possible at maximum growth *(e.g., minimum demand in OptKnock computations)*

- weakly growth-coupled production (wGCP): product synthesis is enforced at maximum growth *(e.g., demanded by RobustKnock or OptCouple)*
- directionally growth-coupled production (dGCP): product synthesis is enforced at all growth rates greater than zero *(e.g., often demanded by MCS, often generated by OptCouple)*
- substrate-uptake-coupled production (SUCP): product is synthesized whenever substrate is taken up, i.e., a minimum product yield is guaranteed *(e.g., often demanded by MCS)*

Before moving on the computation of strain designs, we will provide strain design examples for the different coupling types and show their main property. To keep it as simple as possible, we will use ethanol as the desired product, use the small "textbook model" e_coli_core, and introduce only reaction knockouts (and no knock-ins).

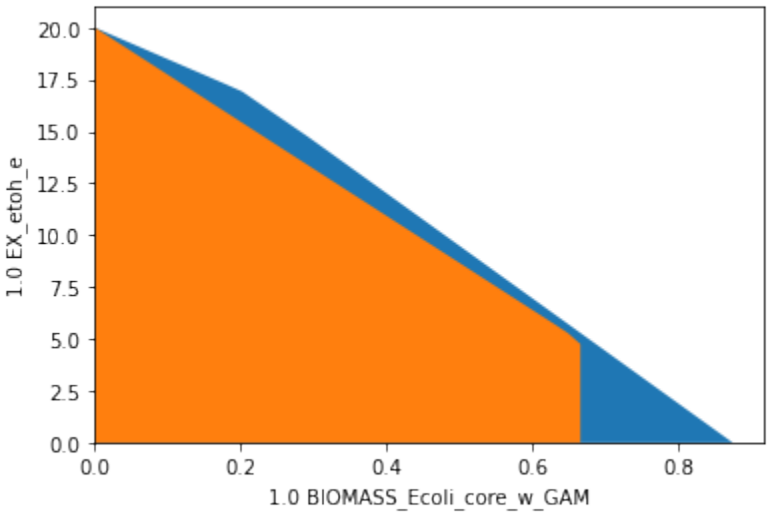
[1]:
```
import cobra
import straindesign as sd
model = cobra.io.load_model('e_coli_core')
```

```
Set parameter Username
Academic license - for non-commercial use only - expires 2023-07-20
```

4.4.1 pGCP: potentially growth-coupled production

When we knock out the reactions **PPS**, **THD2**, **TKT2** and **MDH** in e_coli_core, we obtain a potentially coupled strain design. Below, we plot the flux space of the wild type model (blue/background) on the dimensions of growth rate and the ethanol yield and compare it to the pGCP strain design (orange/foreground). In the wild type model, growth and ethanol production compete, whereas, in the strain designed for pGCP, product synthesis is also possible (yet *not required*) at growth-maximal flux states.

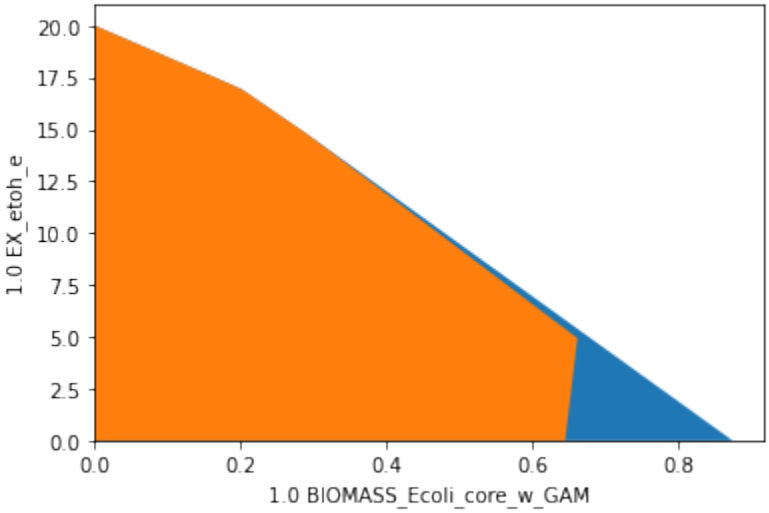
[2]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               show=False);
# pGCP design plot
constraints = ['PPS=0', 'THD2=0', 'TKT2=0', 'MDH=0']
_,          _,      plot2 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               constraints=constraints,
                                               show=False);
# adjust axes limits and show plot
plot2.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot2.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



4.4.2 wGCP: weakly growth-coupled production

A weakly growth-coupled production strain design can be generated by knocking out the reactions **ACALDt**, **PTAr**, **PPS**, **PYRt2** and **MDH** in e_coli_core. Below, we plot the flux space of the wild type model (blue/background) on the dimensions of growth rate and the ethanol yield and compare it to the wGCP strain design (orange/foreground). In the wild type model, growth and ethanol production compete, whereas, in the strain designed for wGCP, growth-maximal flux states can only be reached by simultanously producing ethanol.

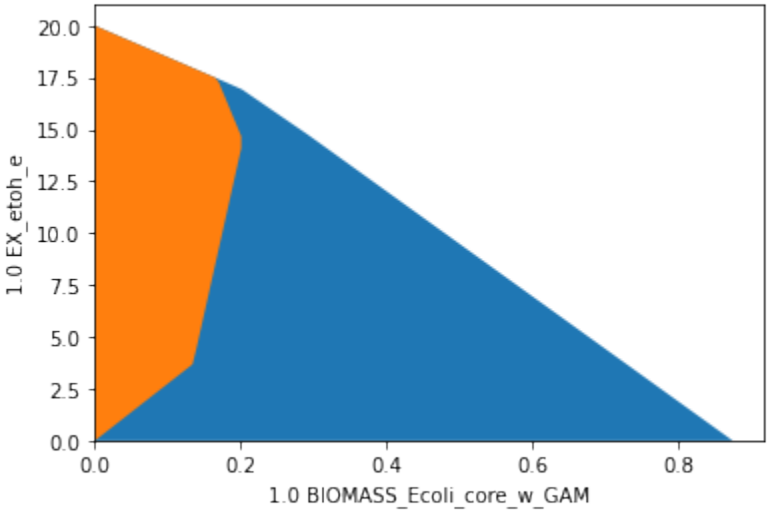
[3]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               show=False);
# pGCP design plot
constraints = ['ACALDt=0', 'PTAr=0', 'PPS=0', 'PYRt2=0', 'MDH=0']
_,          _,      plot2 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               constraints=constraints,
                                               show=False);
# adjust axes limits and show plot
plot2.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot2.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



4.4.3 dGCP: directionally growth-coupled production

By knocking out the reactions **PGL**, **ATPS4r** and **NADH16** in e_coli_core, we obtain a directionally coupled strain design. Below, we plot the flux space of the the dGCP strain design (orange/foreground) on the dimensions of growth rate and the ethanol production. In the strain designed for dGCP, product synthesis is occurs at all growth-associated flux states. In this example significantly higher specific production rates are reached than in the pGCP and wGCP scenario, however at the detriment of high growth rates.

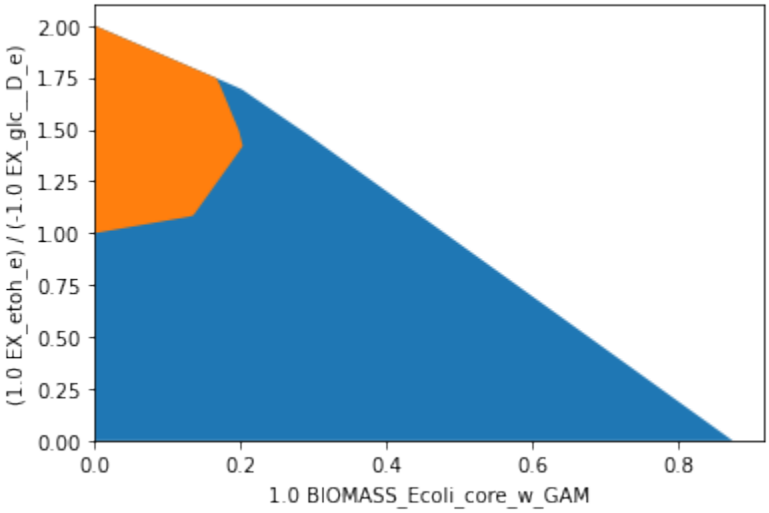
[4]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               show=False);
# pGCP design plot
constraints = ['PGL=0', 'ATPS4r=0', 'NADH16=0']
_,          _,      plot2 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'),
                                               constraints=constraints,
                                               show=False);
# adjust axes limits and show plot
plot2.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot2.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



4.4.4 SUCP: substrate-uptake-coupled production

In strain designs with substrate-uptake-coupled production, all feasible flux states of substrate uptake also entail production. To obtain a such a strain design for the production of ethanol, the reactions knockouts **LDH_D**, **ATPS4r** and **NADH16** can be introduced to e_coli_core. Other than in the previous examples, we here plot the flux space of the SUCP strain design (orange/foreground) on the dimensions of growth rate and ethanol *yield*. In the SUCP strain design, a minimum product yield is ensured in all remaining feasible flux states. Again, the improved production capacities often come at the expense of viability.

[5]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e',
↪'-EX_glc__D_e')),
                                               show=False);
# pGCP design plot
constraints = ['LDH_D=0', 'ATPS4r=0', 'NADH16=0']
_,          _,      plot2 = sd.plot_flux_space(model,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_etoh_e',
↪'-EX_glc__D_e')),
                                               constraints=constraints,
                                               show=False);
# adjust axes limits and show plot
plot2.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot2.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



In the following chapter, we discuss how different algorithms can be used to generate growth-coupled strain designs.

4.5 Minimal Cut Sets (MCS)

The minimal cut set approach aims to find a minimum set of network intervetions that enforce some pre-definde behavior in a metabolic network. It may be used to suppress growth, suppress or enforce the synthesis of a certain product or to couple or decouple metabolic fluxes. For a better conceptional understanding of the MCS approach, please refer to examples 7 and 8. Examples 1-5 treat strain design applications in a small network, while example 6 presents a genome-scale strain design computation.

4.5.1 Prerequisites

For the shown examples, we again load the COBRApy and the StrainDesign packages as well as the small E. coli textbook-network "e_coli_core" and the example network "SmallExample". For CNApy users a special project (e_coli_core_14bdo) is available that contains the modified metabolic network together with strain design files for the various examples which can be loaded from the strain design dialog.

The following steps show how the function `compute_strain_designs` is employed to find MCS strain designs:

1) The production pathways, i.e., their metabolites and reactions are added to the model.
2) Analysis tools are used to identify adequate strain design goals.
3) Set up the according strain design problem by specifying two strain design *modules*
4) The strain design function is called, passing the model and the strain design module(s) as function arguments.
5) The results are analyzed.

Step (1) is required for all examples shown hereafter. Steps (2)-(4) are example-dependent. We do step (5) for all examples to verify the computed strain designs.

4.5.1.1 1) Add and verify production pathway

[1]:
```
import straindesign as sd
import cobra
cobra.Configuration().solver = 'cplex'
```

(continues on next page)

(continued from previous page)

```
ecc = cobra.io.load_model('e_coli_core')
model = cobra.io.read_sbml_model('../../../tests/model_small_example.xml')

# Create copy of model to which pathway will be added
ecc_14bdo = ecc.copy()

# Add metabolites to model
ecc_14bdo.add_metabolites([ cobra.Metabolite('sucsal_c'),# Succinic semialdehyde
                            cobra.Metabolite('4hb_c'),   # 4-Hydroxybutanoate
                            cobra.Metabolite('4hbcoa_c'),# 4-Hydroxybutyryl-CoA
                            cobra.Metabolite('4hbal_c'), # 4-Hydroxybutanal
                            cobra.Metabolite('14bdo_c'), # Butane-1,4-diol (cytopl.)
                            cobra.Metabolite('14bdo_p'), # Butane-1,4-diol (peripl.)
                            cobra.Metabolite('14bdo_e')  # Butane-1,4-diol (extrac.)
                            ])

# Create reactions
SSCOARx    = cobra.Reaction('SSCOARx')
AKGDC      = cobra.Reaction('AKGDC')
HBD        = cobra.Reaction('4HBD')
HBCT       = cobra.Reaction('4HBCT')
HBDH       = cobra.Reaction('4HBDH')
HBDx       = cobra.Reaction('4HBDx')
BDOtpp     = cobra.Reaction('14BDOtpp')
BDOtex     = cobra.Reaction('14BDOtex')
EX_14bdo_e = cobra.Reaction('EX_14bdo_e')

# Add reactions to model
ecc_14bdo.add_reactions([SSCOARx,
                         AKGDC,
                         HBD,
                         HBCT,
                         HBDH,
                         HBDx,
                         BDOtpp,
                         BDOtex,
                         EX_14bdo_e])

# Define reaction equations
SSCOARx.reaction    = '1 h_c + 1 nadph_c + 1 succoa_c -> 1 coa_c + 1 nadp_c + 1 sucsal_c'
AKGDC.reaction      = '1 akg_c + 1 h_c -> 1 co2_c + 1 sucsal_c'
HBD.reaction        = '1 h_c + 1 nadh_c + 1 sucsal_c  -> 1 4hb_c + 1 nad_c'
HBCT.reaction       = '1 4hb_c + 1 accoa_c            -> 1 4hbcoa_c + 1 ac_c'
HBDH.reaction       = '1 4hbcoa_c + 1 h_c + 1 nadh_c  -> 1 4hbal_c + 1 coa_c + 1 nad_c'
HBDx.reaction       = '1 4hbal_c + 1 h_c + 1 nadh_c   -> 1 14bdo_c + 1 nad_c'
BDOtpp.reaction     = '1 14bdo_c                      -> 1 14bdo_p'
BDOtex.reaction     = '1 14bdo_p                      -> 1 14bdo_e'
EX_14bdo_e.reaction = '1 14bdo_e                      ->'

# Verify that pathway is operational
sol = sd.fba(ecc_14bdo,obj='EX_14bdo_e',obj_sense='max')
print(f"Maximum possible 1,4-BDO synthesis rate: {sol.objective_value}.")
```

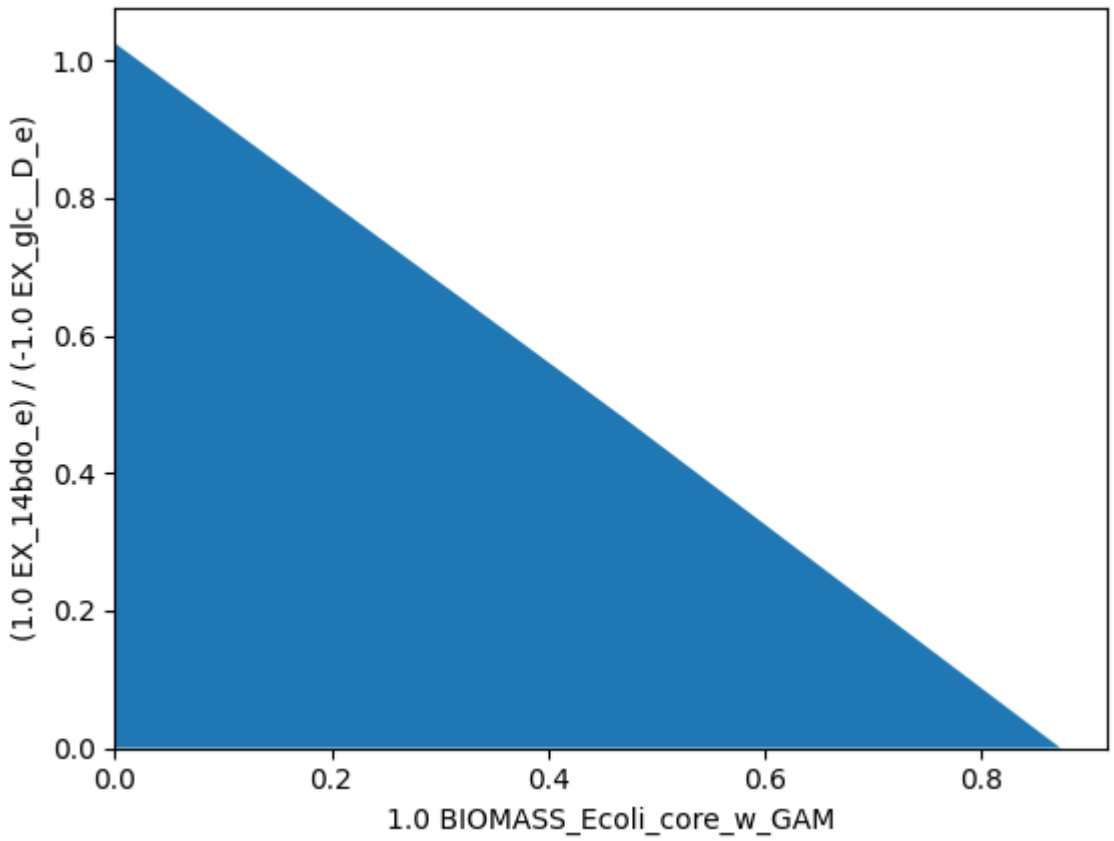
```
Maximum possible 1,4-BDO synthesis rate: 10.252923076923619.
```

4.5.2 Example 1: Strain designs with a minimum product (1,4-butanediol) yield (SUCP strain design)

We may use the MCS approach to generate strain designs with a guaranteed minimum yield of 1,4-butanediol (1,4-BDO) on glucose, i.e. *substrate-uptake-coupled production (SUCP)*.

We may plot the relationship between growth and product yield to get a feel for the production potential.

[2]:
```
sd.plot_flux_space(ecc_14bdo, ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e','-EX_glc__D_e
↪')));
```



Bacterial growth seems to compete with 1,4-BDO production. The maximum theoretical 1,4-BDO yield, is slightly above 1. We may now try to set realistic strain design goals. Assuming that knockout sets can be found the force a product yield above 0.3, while a growth rate of 0.2 is still attainable, we can specify according flux subspaces for protection or deletion through inequalities.

Since we decided to enforce a yield above 0.3, we aim to suppress all flux states with a yied inferior to this. Hence, we can describe the subspace of undesired fluxes with the inequality:

$$\frac{v_{1,4-BDO}}{v_{Glc,up}} \leq 0.3$$

which can be linearized unter the assumption that $v_{Glc_up} > 0$ to:

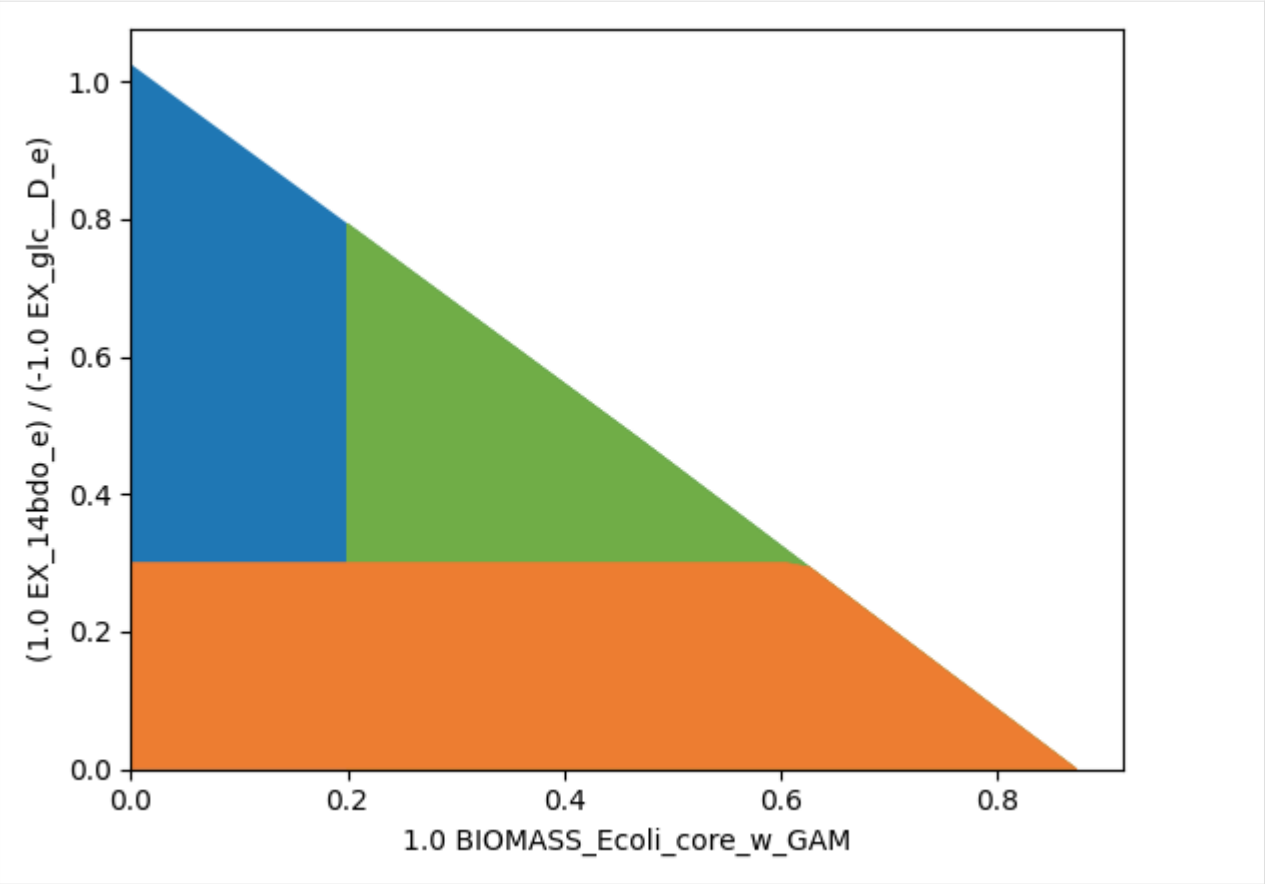
$$v_{1,4-BDO} - 0.3\, v_{Glc,up} \leq 0$$

The flux states, which we aim to protect (at least partically) can be described by:

$$v_{growth} \geq 0.2$$

We can use the plotting function to visualize the flux subspaces that we would like to suppress (orange) or protect (green).

[3]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.2
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints='EX_14bdo_e + 0.3 EX_glc__D_e
↪<= 0',
                                               show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



The StrainDesign package uses so-called "strain design modules" to specify strain design goals. In the case of MCS, the goal is to suppress and protect flux spaces denoted by (sets of) linear inequalities. In the following we specify these modules.

[4]:
```
module_suppress = sd.SDModule(ecc_14bdo,sd.SUPPRESS,constraints='EX_14bdo_e + 0.3 EX_glc_
↪_D_e <= 0')
module_protect  = sd.SDModule(ecc_14bdo,sd.PROTECT, constraints='BIOMASS_Ecoli_core_w_
↪GAM>=0.2')
```

It must be noted that suppressed or protected flux spaces are not only denoted by single inequalities, but are always subspaces of the original model. For instance, the set of flux vectors in the "suppressed" flux space is constrained by the specified inequality

$$v^{supp}_{1,4-BDO} - 0.3\, v^{supp}_{Glc,up} \leq 0$$

but also by the model constraints

$$\mathbf{S} \cdot \mathbf{v^{supp}} = \mathbf{0}$$

$$\mathbf{lb} \leq \mathbf{v^{supp}} \leq \mathbf{ub}$$

We must pay close attention when specifying *protect* and *suppress' modules, as, in either case, we avoid to include the zero flux vector :math: `mathbf{v = 0}` in the according subspace. If the zero vector were contained in the*suppressed* flux space, the subspace could not be suppressed, since the zero vector can always be attained, even if all reactions were to be blocked. If the zero vector is part of the *protected* flux space, the module will be ineffective, since reactions knockouts can never disrupt this vector.

In the case of the e_coli_core, it is unnecessary to exclude the zero vector explicitly, since it is already excluded by default due to the minimum ATP maintenance demand. If this was not the case, one would need a auxiliary constraint, e.g., $ v_{Glc,up} :nbsphinx-math:`ge `0.1$.

We can now proceed with the strain design computation. Since we normally don't know if solutions to our strain design problems exist, we will start the computation with the most relaxed settings possible. This means, we compute only one single solution, within a given MILP timelimit of 5 minutes (for genome-scale setups, this should be increased to an hour), while omitting the minimality demand in the solutions and allow up to 30 knockouts. For this initial approach, we also activate logging to follow the progress of the computation.

[5]:
```
import logging
logging.basicConfig(level=logging.INFO)
# Compute strain designs
sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_suppress, module_protect],
                                 time_limit = 300,
                                 max_solutions = 1,
                                 max_cost = 10,
                                 solution_approach = sd.ANY)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.reaction_sd)} solutions in the uncompressed netork.")
print(f"Example knockout set: {[s for s in sols.reaction_sd[0]]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 62859
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Compressing Network (104 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 61 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (61 reactions).
INFO:root:  Network compression completed. (1 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 61 reactions, 36 metabolites
INFO:root:  50 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding (also non-optimal) strain designs ...
INFO:root:Searching in full search space.
INFO:root:Minimizing number of interventions in subspace with 10 possible targets.
INFO:root:Strain design with cost 9.0: {'ACALD': -1, 'ATPS4r': -1, 'CO2t*EX_co2_e': -1,
↪'D_LACt2*EX_lac__D_e*LDH_D': -1, 'SUCCt3': -1, 'EX_h2o_e*H2Ot': -1, 'FORt2': -1,
↪'NADTRHD': -1, 'PDH': -1}
INFO:root:Finished solving strain design MILP.
```

(continues on next page)

(continued from previous page)

```
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:12 solutions found.
```

```
One compressed solution with cost 9.0 found and expanded to 12 solutions in the␣
↪uncompressed netork.
Example knockout set: ['ACALD', 'ATPS4r', 'SUCCt3', 'FORt2', 'NADTRHD', 'PDH', 'CO2t',
↪'D_LACt2', 'EX_h2o_e']
```

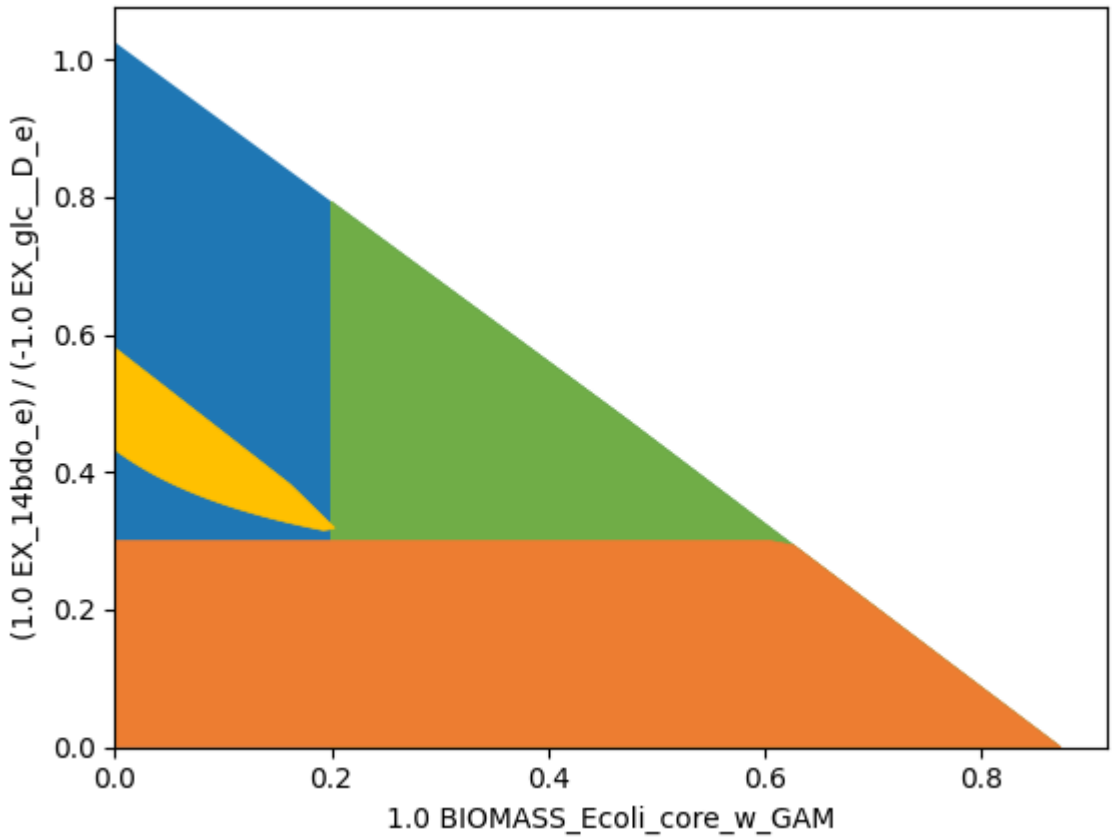
We may plot the computed strain design (yellow) on top of the wild type model (blue), the suppressed fluxes (orange) and the protected fluxes (green). The designed strain is forced to produce 1,4-butanediol but is still able to grow at a rate higher than 0.2 1/h.

[6]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.2
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints='EX_14bdo_e + 0.3 EX_glc__D_e
↪<= 0',
                                               show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# plotting designed strain
knockouts = [[{s:1.0},'=',0.0] for s in sols.reaction_sd[0]]
_,          _,      plot4 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints=knockouts,
                                               show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
```

(continues on next page)

(continued from previous page)

```
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



An easy way to compute gene-based MCS is to set the `gene_kos` parameter to True. All genes then are regarded as knockout candidates.

[7]:
```
import logging
logging.basicConfig(level=logging.INFO)
# Compute strain designs
sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_suppress, module_protect],
                                 time_limit = 300,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 solution_approach = sd.ANY,
                                 gene_kos = True)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example knockout set: {[s for s in sols.gene_sd[0]]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 21836
```

(continues on next page)

(continued from previous page)

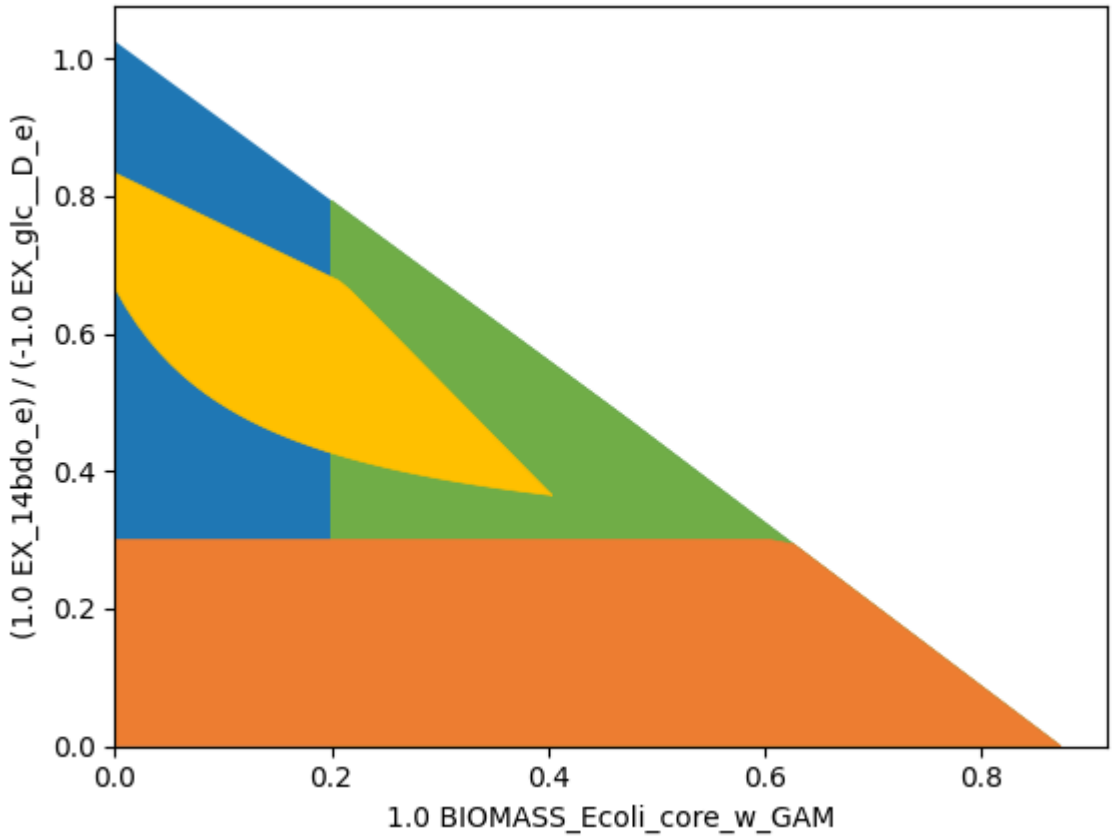
```
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 113 genes and 56 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (307 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 164 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 142 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 135 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 134 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 133 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (133 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 133 reactions, 78 metabolites
INFO:root:  47 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding (also non-optimal) strain designs ...
INFO:root:Searching in full search space.
INFO:root:Minimizing number of interventions in subspace with 10 possible targets.
INFO:root:Strain design with cost 11.0: {'PYK*pykF*R0_g_b1676_or_g_b1854*pykA*R1_g_b1676_
↪or_g_b1854': -1, 'ME1*maeA': -1, 'ME2*maeB': -1, 'pgi': -1, 'kgtP': -1, 'lpd': -1,
↪'fumC*R0_g_b1611_or_g_b1612_or_g_b4122*fumA*R1_g_b1611_or_g_b1612_or_g_b4122*fumB*R2_g_
↪b1611_or_g_b1612_or_g_b4122': -1, 'gltP': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:1 solutions found.
```

```
One compressed solution with cost 11.0 found and expanded to 1 solutions in the␣
↪uncompressed netork.
Example knockout set: ['pgi', 'kgtP', 'lpd', 'gltP', 'maeA', 'maeB', 'pykF', 'pykA',
↪'fumC', 'fumA', 'fumB']
```

We may now analyze the resulting strain designs stored in `sols.gene_sd`. When strain designs were computed with genetic interventions, such interventions need to be translated back to the reaction level in order to enable network analysis tools. This is done automatically at the end of each computation. The reaction-intervention equivalent can be

accessed through the field `sols.reaction_sd`.

[8]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.2
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints='EX_14bdo_e + 0.3 EX_glc__D_e
↪<= 0',
                                               show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# plotting designed strain
knockouts = [[{s:1.0},'=',0.0] for s in sols.reaction_sd[0]]
_,          _,      plot4 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints=knockouts,
                                               show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



As before, the designed strain complies with the specified MCS strain design goals.

In step (1), we added two pathways that can be used to produce the intermediate succinic semialdehyde branching from alpha-ketoglutarate. In fact, only one reaction, AKGDC or SSCOARx, is needed for producing 1,4-BDO. If we want to save experimental effort, we may want to consider the addition of only one of the two respective enzymes.

For the MCS computation, we will therefore mark the reactions AKGDC and SSCOARx as "addition candidates". The algorithm will now combine a knockout strategy with the addition of one of the two reactions. For this computation example we will:

- relax the strain design demands.
- allow the knockout of all genes (apart from the pseudogene for spontanous reactions)
- allow the knockout of O2, thereby simulating anaerobic conditions
- allow the limitation of O2 uptake down to a rate of below 1
- treat AKGDC and SSCOARx as addition candidates (Instead of reaction additions, one could also use gene-additions. Since the gene-reaction-association is 1:1, the computation results would be identical.). We associate different intervention costs to the addition of these reactions. AKGDC addition has the cost 1, SSCOARx addition has the cost 5.

[9]:
```
module_suppress = sd.SDModule(ecc_14bdo,sd.SUPPRESS,constraints='EX_14bdo_e + 0.05 EX_
↪glc__D_e <= 0')
module_protect  = sd.SDModule(ecc_14bdo,sd.PROTECT, constraints='BIOMASS_Ecoli_core_w_
↪GAM>=0.02')
```

(continues on next page)

(continued from previous page)

```
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# possible throttling of O2 uptake
reg_cost = {'-EX_o2_e <= 1' : 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':5}

# compute strain designs
import logging
logging.basicConfig(level=logging.INFO)
sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_suppress, module_protect],
                                 max_cost = 9,
                                 solution_approach = sd.POPULATE,
                                 ko_cost = ko_cost,
                                 gko_cost = gko_cost,
                                 reg_cost = reg_cost,
                                 ki_cost = ki_cost)
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 22847
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 111 genes and 52 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (301 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 155 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 133 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 126 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 125 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (124 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 124 reactions, 73 metabolites
```

(continues on next page)

(continued from previous page)

```
INFO:root:  48 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Enumerating strain designs ...
INFO:root:Strain designs with cost 7.0: [{'CYTBD*EX_o2_e*O2t*cbdA*cbdB*R_g_b0978_and_g_
↪b0979*R0_g_b0733_and_g_b0734_or_g_b0978_and_g_b0979*cydA*cydB*R_g_b0733_and_g_b0734*R1_
↪g_b0733_and_g_b0734_or_g_b0978_and_g_b0979': -1.0, 'SSCOARx': 1.0, 'pgi': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'SSCOARx': 1.0, 'pgi': -1.0, 'focA*R0_g_b0904_
↪or_g_b2492*focB*R1_g_b0904_or_g_b2492': -1.0, 'n-1.0_EX_o2_e_le_1p0': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'SSCOARx': 1.0, 'pgi': -1.0, 'mhpF*R0_g_b0351_
↪or_g_b1241': -1.0, 'adhE': -1.0, 'n-1.0_EX_o2_e_le_1p0': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'PYK*pykF*R0_g_b1676_or_g_b1854*pykA*R1_g_
↪b1676_or_g_b1854': -1.0, 'ICL*MALS*aceA*glcB*R0_g_b2976_or_g_b4014*aceB*R1_g_b2976_or_
↪g_b4014': -1.0, 'AKGDC': 1.0, 'pgi': -1.0, 'kgtP': -1.0, 'sucC*sucD*R_g_b0728_and_g_
↪b0729': -1.0, 'gltP': -1.0, 'mdh': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'PYK*pykF*R0_g_b1676_or_g_b1854*pykA*R1_g_
↪b1676_or_g_b1854': -1.0, 'ICL*MALS*aceA*glcB*R0_g_b2976_or_g_b4014*aceB*R1_g_b2976_or_
↪g_b4014': -1.0, 'AKGDC': 1.0, 'pgi': -1.0, 'kgtP': -1.0, 'lpd': -1.0, 'gltP': -1.0,
↪'mdh': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'AKGDH*sucA*sucB*R_g_b0116_and_g_b0726_and_g_
↪b0727': -1.0, 'PYK*pykF*R0_g_b1676_or_g_b1854*pykA*R1_g_b1676_or_g_b1854': -1.0,
↪'ICL*MALS*aceA*glcB*R0_g_b2976_or_g_b4014*aceB*R1_g_b2976_or_g_b4014': -1.0, 'AKGDC':␣
↪1.0, 'pgi': -1.0, 'kgtP': -1.0, 'gltP': -1.0, 'mdh': -1.0}]
INFO:root:Strain designs with cost 9.0: [{'PFL*EX_for_e*pflA*pflB*R_g_b0902_and_g_
↪b0903*R0_g_b0902_and_g_b0903_or_g_b0902_and_g_b3114_or_g_b3951_and_g_b3952*tdcE*R_g_
↪b0902_and_g_b3114*R1_g_b0902_and_g_b0903_or_g_b0902_and_g_b3114_or_g_b3951_and_g_
↪b3952...': -1.0, 'SSCOARx': 1.0, 'pgi': -1.0, 'n-1.0_EX_o2_e_le_1p0': -1.0}]
INFO:root:Finished solving strain design MILP.
INFO:root:7 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:14 solutions found.
```

[10]:
```
for i,s in enumerate(sols.gene_sd):
    st = [['+'+t if v>0 else '-'+t][0] for t,v in s.items() if v != 0]
    print('solution '+str(i+1)+': '+', '.join(st))
```

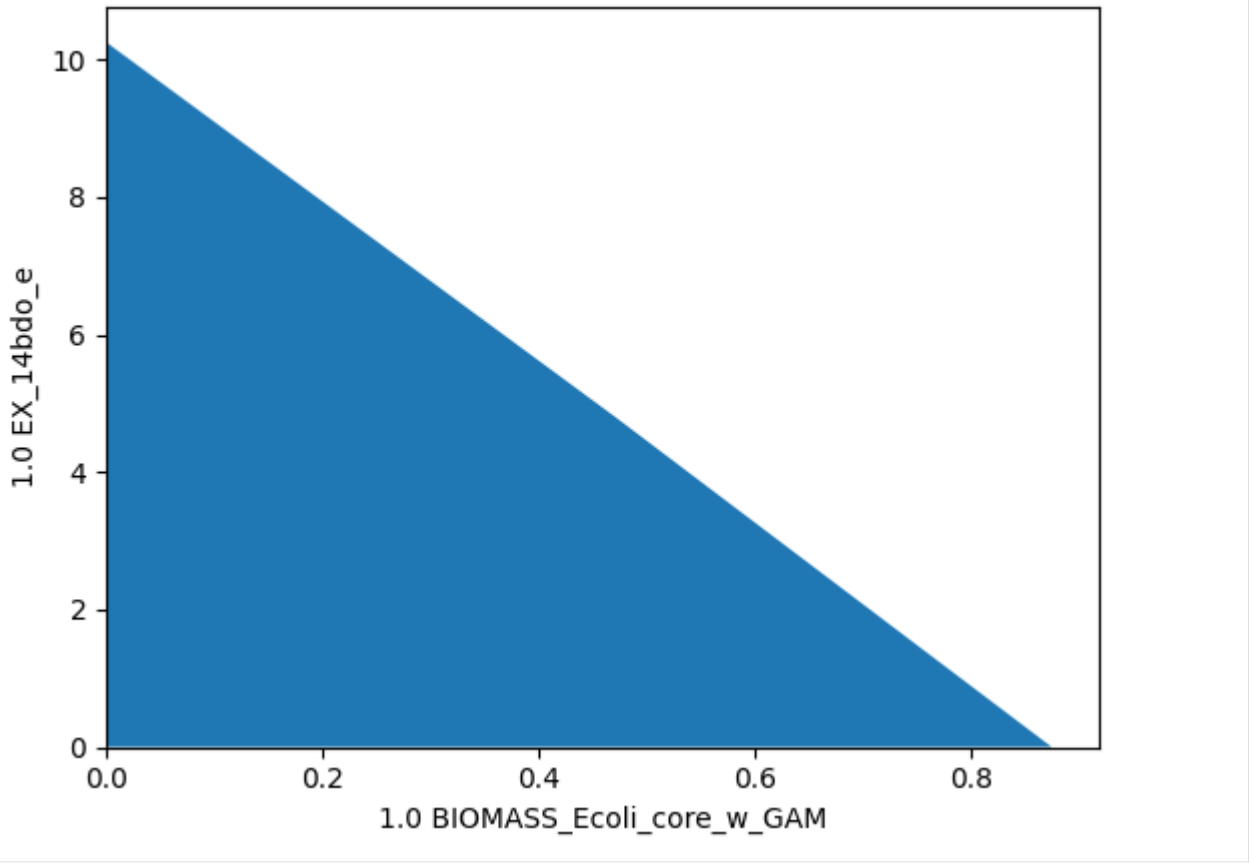
```
solution 1: +SSCOARx, -pgi, -EX_o2_e
solution 2: +SSCOARx, -pgi, -cbdB, -cydA
solution 3: +SSCOARx, -pgi, -cbdB, -cydB
solution 4: +AKGDC, -pgi, -kgtP, -gltP, -mdh, -aceA, -pykF, -pykA, -sucC
solution 5: +AKGDC, -pgi, -kgtP, -gltP, -mdh, -aceA, -pykF, -pykA, -sucD
solution 6: +SSCOARx, -pgi, -pflA, -pflD, +-EX_o2_e <= 1
solution 7: +SSCOARx, -pgi, -pflA, -pflC, +-EX_o2_e <= 1
solution 8: +AKGDC, -pgi, -kgtP, -gltP, -mdh, -sucA, -aceA, -pykF, -pykA
solution 9: +AKGDC, -pgi, -kgtP, -gltP, -mdh, -sucB, -aceA, -pykF, -pykA
solution 10: +SSCOARx, -pgi, -cbdA, -cydA
solution 11: +SSCOARx, -pgi, -cbdA, -cydB
solution 12: +SSCOARx, -pgi, -focA, -focB, +-EX_o2_e <= 1
solution 13: +SSCOARx, -pgi, -adhE, -mhpF, +-EX_o2_e <= 1
solution 14: +AKGDC, -pgi, -kgtP, -lpd, -gltP, -mdh, -aceA, -pykF, -pykA
```

4.5.3 Example 2: Enforce product (1,4-BDO) synthesis at all growth states (dGCP strain design)

For computing dGCP strain designs with MCS, we follow the identical steps from Example 1:

1) The production pathways, i.e., their metabolites and reactions are added to the model.
2) Analysis tools are used to identify adequate strain design goals.
3) Set up the according strain design problem by specifying two strain design *modules*, one that demand the suppression of flux states with low product yields, the other one protecting the functions that are essential for bacterial growth.
4) The strain design function is called, passing the model and the strain design module(s) as function arguments.
5) The results are analyzed.

[11]:
```
# Plot production envelope
sd.plot_flux_space(ecc_14bdo, ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'));
```



For dGCP, we demand that all flux states of microbial growth carry production. Ideally, we want to (a) ensure a minimum ratio of product synthesis rate and growth rate $\frac{v_{product}}{v_{growth}} > Y^{min}_{P/BM}$ (2) ensure that growth is still possible. Hence, we define one suppression and one protection module.

Suppress module (removing flux states, where $\frac{v_{product}}{v_{growth}} \leq Y^{min}_{P/BM}$):

$$v_{product} - Y^{min}_{P/BM} v_{growth} \leq 0$$

$$v_{growth} \geq 0.01\ h^{-1}$$

The latter constraint is used to explicitly exclude the $\mathbf{v} = \mathbf{0}$ vector from the suppressed flux states. Here, we use an arbitrary threshold of $Y_{P/BM}^{min} = 5$ that we inferred from the production envelope.

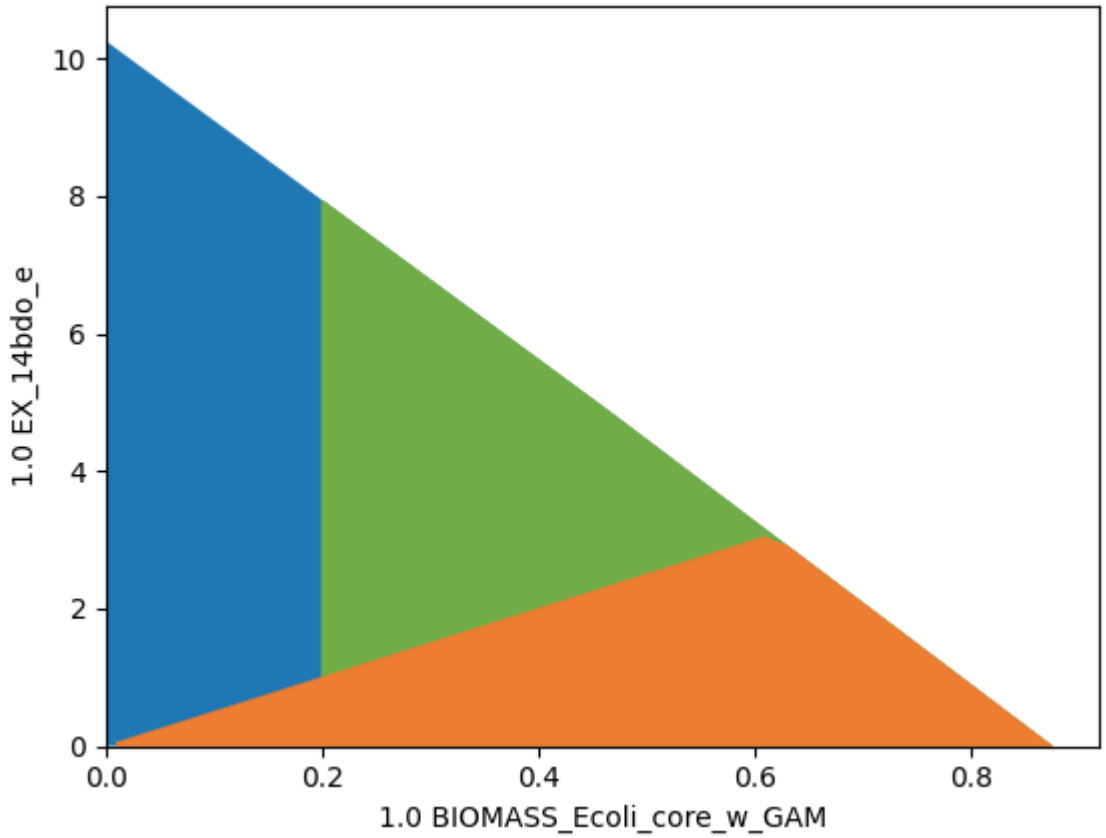
Protect module:

$$v_{growth} \geq 0.2\ h^{-1}$$

[12]:
```
module_suppress = sd.SDModule(ecc_14bdo,sd.SUPPRESS,constraints=['EX_14bdo_e - 5 BIOMASS_
↪Ecoli_core_w_GAM <= 0',
                                                                 'BIOMASS_Ecoli_
↪core_w_GAM >= 0.01'])
module_protect  = sd.SDModule(ecc_14bdo,sd.PROTECT, constraints='BIOMASS_Ecoli_core_w_
↪GAM>=0.2')
```

Plotting the flux spaces in the production envelope returns:

[13]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.2
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints=['EX_14bdo_e - 5 BIOMASS_
↪Ecoli_core_w_GAM <= 0',
                                                            'BIOMASS_Ecoli_core_w_GAM >=␣
↪0.01'],
                                               show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



[14]:
```
import logging
logging.basicConfig(level=logging.ERROR)
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

# compute strain designs
sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_suppress, module_protect],
                                 max_solutions = 1,
                                 max_cost = 30,
                                 solution_approach = sd.BEST,
                                 ko_cost = ko_cost,
                                 gko_cost = gko_cost,
                                 ki_cost = ki_cost)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.reaction_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")
print(f"Knockout set on the reaction level: {[s for s in sols.reaction_sd[0]]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 47779
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 111 genes and 52 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (299 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 153 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 131 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (122 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 122 reactions, 72 metabolites
INFO:root:  47 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Strain design with cost 6.0: {'ME2*maeB': -1, 'AKGDC': 1, 'kgtP': -1,
↪'sucC*sucD*R_g_b0728_and_g_b0729': -1, 'pntB*pntA*R_g_b1602_and_g_b1603': -1, 'zwf': -
↪1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:4 solutions found.
```

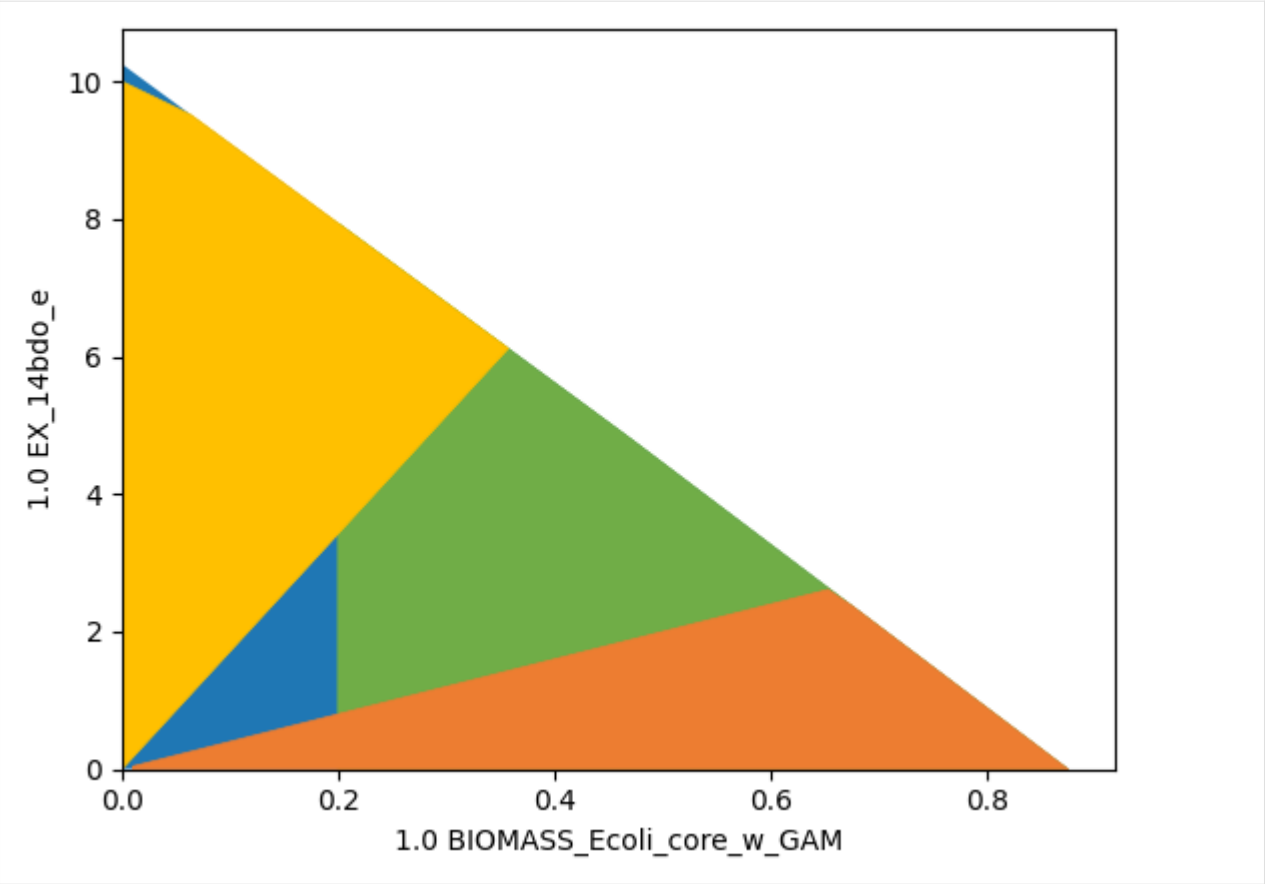
```
One compressed solution with cost 6.0 found and expanded to 4 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-kgtP', '-zwf', '-maeB', '-sucC', '-pntB']
Knockout set on the reaction level: ['THD2', 'AKGt2r', 'ME2', 'SUCOAS', 'G6PDH2r', 'AKGDC
↪', 'SSCOARx']
```

[15]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
```

(continues on next page)

(continued from previous page)

```
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                show=False);
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                constraints='BIOMASS_Ecoli_core_w_GAM>=0.2
↪',
                                                show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=['EX_14bdo_e - 4 BIOMASS_
↪Ecoli_core_w_GAM <= 0',
                                                             'BIOMASS_Ecoli_core_w_GAM >=␣
↪0.01'],
                                                show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# plotting designed strain
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot4 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
                                                show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



The resulting strain design shows a linear increase of ensured production with increasing growth.

4.5.4 Example 3: Suppress flux states that are optimal with respect to a pre-defined objective function (wGCP strain design)

The MCS approach also allows the suppression (or protection) of flux vectors that are optimal regarding a pre-defined objective function. With an inner objective function, the SUPPRESS module can be used to disrupt any growth-maximal flux states that don't carry production and thereby enforce - *at least* - weakly growth-coupled production. To avoid lethal knockouts and ensure that growth is still possible at reasonable growth rates, we additionally use a PROTECT region.

[16]:
```
# Wild-type plot
print('Preparing plot of wild type and strain design setup.')
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               show=False);
# wGCP protect plot
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.
↪15',
                                               show=False);
plot2.set_facecolor('#70AD47')
```

(continues on next page)

(continued from previous page)

```
plot2.set_edgecolor('#70AD47')
# wGCP suppress plot
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints='EX_14bdo_e + 0.25 EX_glc__D_
↪e <= 0',
                                                show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')


print('Computing MCS.')
module_wgcp = sd.SDModule(  ecc_14bdo,sd.SUPPRESS,
                            inner_objective='BIOMASS_Ecoli_core_w_GAM',
                            constraints='EX_14bdo_e + 0.25 EX_glc__D_e <= 0')
module_protect = sd.SDModule(  ecc_14bdo,sd.PROTECT,
                            constraints='BIOMASS_Ecoli_core_w_GAM >= 0.15')

# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_wgcp,module_protect],
                                 time_limit = 300,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")

# wGCP design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot4 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                                # The sign of the glucose exchange␣
```

(continues on next page)

(continued from previous page)

```
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
                                                show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```

```
Preparing plot of wild type and strain design setup.
Computing MCS.
```

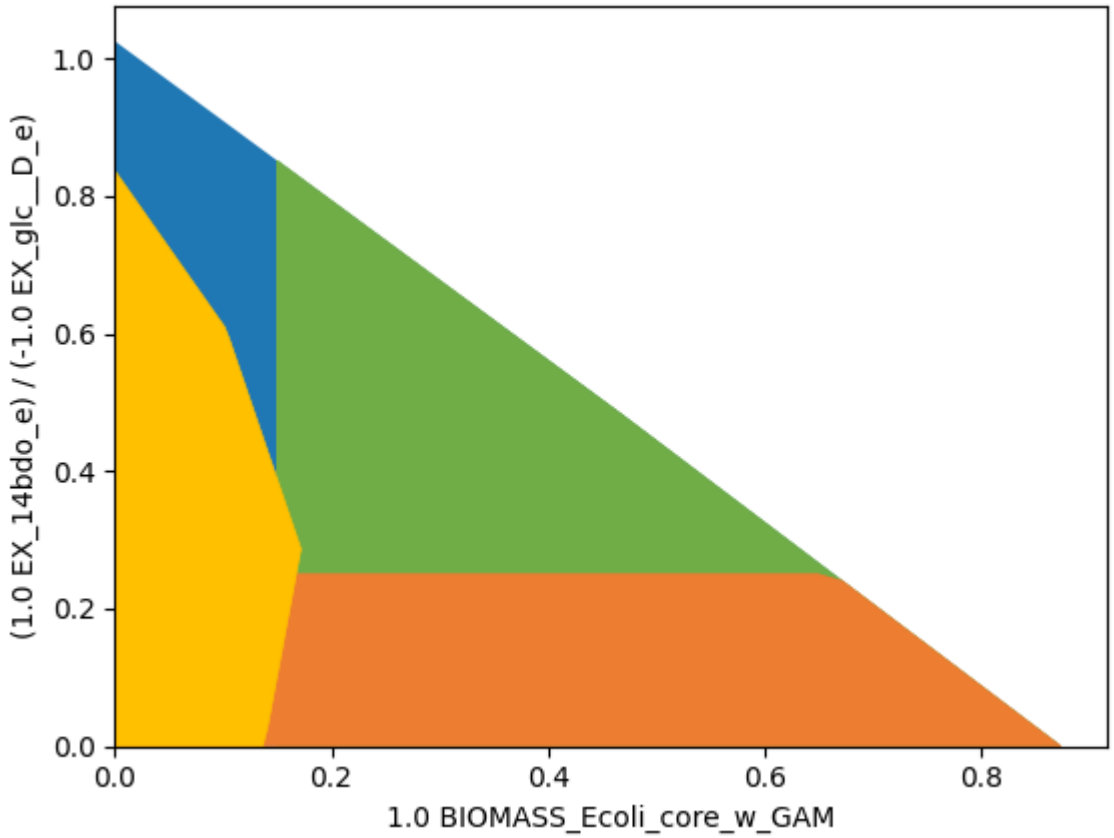
```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 63305
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 111 genes and 52 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (299 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 153 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 131 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (122 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 122 reactions, 72 metabolites
INFO:root:  47 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Strain design with cost 4.0: {'CYTBD*EX_o2_e*O2t*cbdA*cbdB*R_g_b0978_and_g_
↪b0979*R0_g_b0733_and_g_b0734_or_g_b0978_and_g_b0979*cydA*cydB*R_g_b0733_and_g_b0734*R1_
↪g_b0733_and_g_b0734_or_g_b0978_and_g_b0979': -1, 'SSCOARx': 1, 'mhpF*R0_g_b0351_or_g_
```

(continues on next page)

(continued from previous page)

```
↪b1241': -1, 'adhE': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:5 solutions found.
```

```
One compressed solution with cost 4.0 found and expanded to 5 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+SSCOARx', '-adhE', '-EX_o2_e', '-mhpF']
```



As specified by the additional *inner objective function*, product yields inferior to 0.25 (orange) are supressed at growth-maximal flux states, only. At lower growth rates, the strain design *is allowed to have lower product yields*. The protected region, marking healthy growth rates (green), has to be intersected.

4.5.5 Example 4: Protect flux states that are optimal with respect to a pre-defined objective function (pGCP strain design)

The following setup shows how this function can be used to generate strain designs with at least potentially growth-coupled production. We specify a PROTECT module to ensure that there are growth-maximal flux states with a growth rate of at least 0.4 and a product yield of at least $0.4\ \frac{mol_{1,4\text{-}BDO}}{mol_{D\text{-}Glucose}}$.

[17]:
```
# Wild-type plot
print('Preparing plot of wild type and strain design setup.')
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
```

(continues on next page)

(continued from previous page)

```
                                                ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                                show=False);
# wGCP protect plot
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                                constraints=['BIOMASS_Ecoli_core_w_GAM>=0.
↪4','EX_14bdo_e + 0.4 EX_glc__D_e >= 0'],
                                                show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')

print('Computing MCS.')
module_pgcp = sd.SDModule(  ecc_14bdo,sd.PROTECT,
                            inner_objective='BIOMASS_Ecoli_core_w_GAM',
                            constraints=[   'BIOMASS_Ecoli_core_w_GAM >= 0.4',
                                            'EX_14bdo_e + 0.4 EX_glc__D_e >= 0'])

# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = module_pgcp,
                                 time_limit = 300,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")

# wGCP design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot4 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM',('EX_14bdo_e',
↪'-EX_glc__D_e')),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
```

(continues on next page)

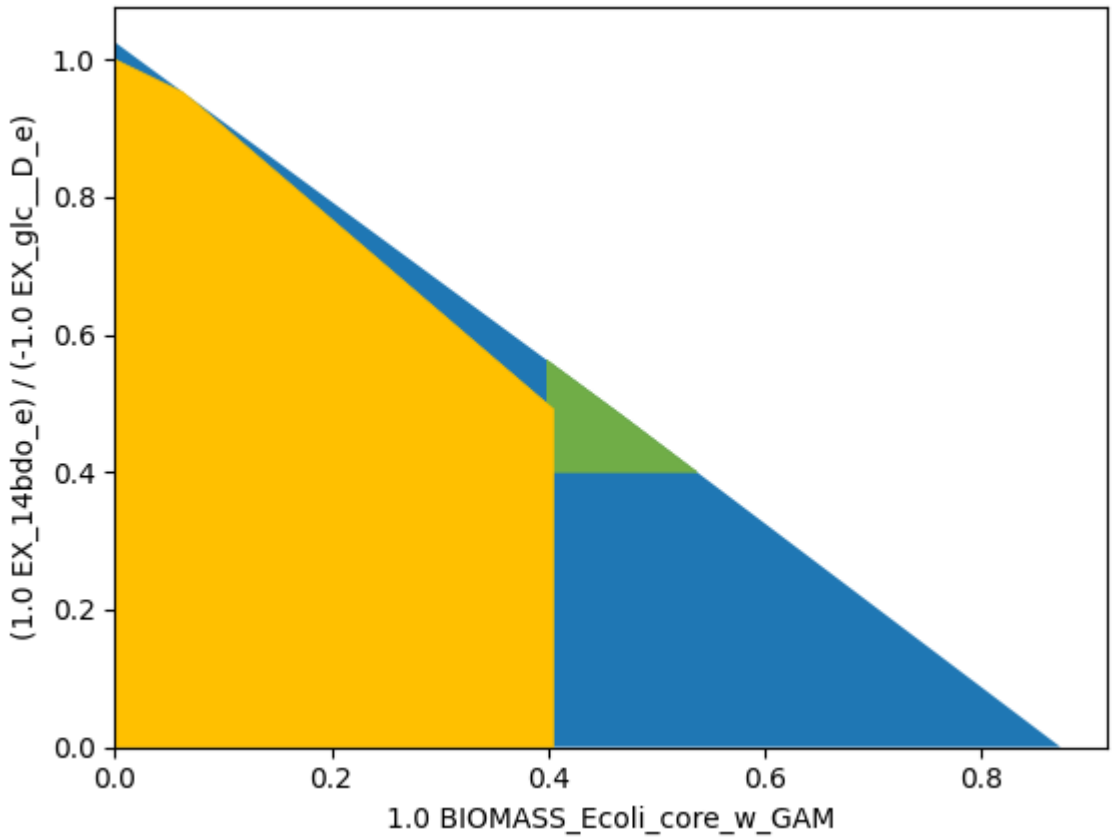
(continued from previous page)

```
                                                show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```

```
Preparing plot of wild type and strain design setup.
Computing MCS.
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 59309
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 86 genes and 49 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (267 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 149 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 128 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 121 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 120 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (120 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 120 reactions, 71 metabolites
INFO:root:  44 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Strain design with cost 6.0: {'PPS*ppsA': -1, 'ME2*maeB': -1, 'AKGDC': 1,
↪'pntB*pntA*R_g_b1602_and_g_b1603': -1, 'zwf': -1, 'mdh': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:2 solutions found.
```

```
One compressed solution with cost 6.0 found and expanded to 2 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-zwf', '-mdh', '-ppsA', '-maeB', '-pntB']
```



The designed strain has potentially growth coupled production. It must be noted that *only one PROTECT region* (green) was used in this computation. Without the *inner objective function*, the wild type strain already fulfills the modules demands, because flux states with simultanous growth and production already exist in the wildtype. In this strain design computation, we enforce that *growth-maximal* flux states exist with growth and product synthesis.

4.5.6 Example 5: All single gene knockouts that prohibit growth (synthetic lethals).

The MCS approach can be used to identify essential genes. For this type of computation we may use the MCS approach with a single suppress-module that targets flux states of microbial growth, i.e. flux states in which the inequality

$$v_{Biomass} > 0$$

approximated as

$$v_{Biomass} \geq 0.001$$

holds. We construct the corresponding module and launch an exhaustive computation of all solutions of the size 1.

[18]:
```
# Construct module
module_suppress  = sd.SDModule(ecc,sd.SUPPRESS, constraints='BIOMASS_Ecoli_core_w_GAM>=0.
↪001')
# Compute strain designs
```

(continues on next page)

(continued from previous page)

```
sols = sd.compute_strain_designs(ecc,
                                 sd_modules = module_suppress,
                                 max_cost = 1,
                                 solution_approach = sd.POPULATE,
                                 gene_kos = True)
# Print solutions
print(f"{len(sols.gene_sd)} lethal single gene knockouts were found.")
for i,sol in enumerate(sols.gene_sd):
    print(f"Solution {i+1}: {[s for s in sol][0]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 2364
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 135 genes and 69 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (348 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 198 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 169 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 160 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 158 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 156 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (156 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 156 reactions, 91 metabolites
INFO:root:  60 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Enumerating strain designs ...
INFO:root:Strain designs with cost 1.0: [{'eno': -1.0}]
INFO:root:Strain designs with cost 1.0: [{'gapA': -1.0}]
INFO:root:Strain designs with cost 1.0: [{'CS*gltA': -1.0}]
INFO:root:Strain designs with cost 1.0: [{'icd': -1.0}]
INFO:root:Strain designs with cost 1.0: [{'EX_glc__D_e*GLCpts*ptsH*ptsI*manX*manY*manZ*R_
↪g_b1817_and_g_b1818_and_g_b1819_and_g_b2415_and_g_b2416...': -1.0}]
INFO:root:Strain designs with cost 1.0: [{'pgk': -1.0}]
```

(continues on next page)

(continued from previous page)

```
INFO:root:Finished solving strain design MILP.
INFO:root:6 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:7 solutions found.
```

```
7 lethal single gene knockouts were found.
Solution 1: eno
Solution 2: gapA
Solution 3: icd
Solution 4: pgk
Solution 5: gltA
Solution 6: ptsH
Solution 7: ptsI
```

4.5.7 Example 6: Genome-scale strain designs with a minimum product (1,4-butanediol) yield (SUCP strain design)

Supplementary preparation steps are necessary for computing strain designs in a genome-scale setup. Here, we again use the example of enforcing substrat-uptake-coupled production of 1,4-butanediol.

1) Add production pathways, i.e., metabolites and reactions, to the model.
2) Ensure that only relevant exchange reactions are kept open in the model.
3) Analyze production capacities.
4) Set up the strain design problem through modules.
5) Launch the strain design computation.
6) Analyze the results.

[19]:
```
import straindesign as sd
import cobra

cobra.Configuration().solver = 'cplex'
iml = cobra.io.load_model('iML1515')

# Create copy of model to which pathway will be added
iml_14bdo = iml.copy()
# If available, set the solver to cplex or gurobi. This will increase the chances
# of sucess enormously
iml_14bdo.solver = 'cplex'

# Add metabolites to model
iml_14bdo.add_metabolites([ cobra.Metabolite('4hb_c'),    # 4-Hydroxybutanoate
                            cobra.Metabolite('4hbcoa_c'),# 4-Hydroxybutyryl-CoA
                            cobra.Metabolite('4hbal_c'), # 4-Hydroxybutanal
                            cobra.Metabolite('14bdo_c'), # Butane-1,4-diol (cytopl.)
                            cobra.Metabolite('14bdo_p'), # Butane-1,4-diol (peripl.)
                            cobra.Metabolite('14bdo_e')  # Butane-1,4-diol (extrac.)
                            ])

# Create reactions
```

(continues on next page)

(continued from previous page)

```
AKGDC      = cobra.Reaction('AKGDC')
SSCOARx    = cobra.Reaction('SSCOARx')
HBD        = cobra.Reaction('4HBD')
HBCT       = cobra.Reaction('4HBCT')
HBDH       = cobra.Reaction('4HBDH')
HBDx       = cobra.Reaction('4HBDx')
BDOtpp     = cobra.Reaction('14BDOtpp')
BDOtex     = cobra.Reaction('14BDOtex')
EX_14bdo_e = cobra.Reaction('EX_14bdo_e')

# Add reactions to model
iml_14bdo.add_reactions([SSCOARx,
                         AKGDC,
                         HBD,
                         HBCT,
                         HBDH,
                         HBDx,
                         BDOtpp,
                         BDOtex,
                         EX_14bdo_e])

# Define reaction equations
SSCOARx.reaction    = '1 h_c + 1 nadph_c + 1 succoa_c -> 1 coa_c + 1 nadp_c + 1 sucsal_c'
AKGDC.reaction      = '1 akg_c + 1 h_c -> 1 co2_c + 1 sucsal_c'
HBD.reaction        = '1 h_c + 1 nadh_c + 1 sucsal_c  -> 1 4hb_c + 1 nad_c'
HBCT.reaction       = '1 4hb_c + 1 accoa_c            -> 1 4hbcoa_c + 1 ac_c'
HBDH.reaction       = '1 4hbcoa_c + 1 h_c + 1 nadh_c  -> 1 4hbal_c + 1 coa_c + 1 nad_c'
HBDx.reaction       = '1 4hbal_c + 1 h_c + 1 nadh_c   -> 1 14bdo_c + 1 nad_c'
BDOtpp.reaction     = '1 14bdo_c                      -> 1 14bdo_p'
BDOtex.reaction     = '1 14bdo_p                      -> 1 14bdo_e'
EX_14bdo_e.reaction = '1 14bdo_e                      ->'

# define GPR rules for knock-inable genes
SSCOARx.gene_reaction_rule  = 'gsscoar'
AKGDC.gene_reaction_rule    = 'gakgdc'
iml_14bdo.genes.gsscoar.name = 'sscoar'
iml_14bdo.genes.gakgdc.name = 'akgdc'

# Verify that pathway is operational
sol = sd.fba(iml_14bdo,obj='EX_14bdo_e',obj_sense='max')
print(f"Maximum possible 1,4-BDO synthesis rate: {sol.objective_value}.")
```

```
Maximum possible 1,4-BDO synthesis rate: 10.489195402311951.
```

The iML1515 model contains metabolite exchange capacities that are usually not observed in experiments such as exchanges of CoA-associated or phosphorylated metabolites. While unrealistic, these exchanges are still represented in the model and stoichiometrically possible, hence the MCS approach would have to target many of them in order to enforce SUCP of 1,4-BDO. Closing unrealistic exchanges ahead of the computation reduces the computation effort and aviods the excessive introduction of interventions.

We block the import of all carbon-containing metabolites apart from D-glucose.

We here opt to block the export of all metabolites apart from CO2, Ethanol, Acetate, Formate, D-Lactate, Succinate, Methanol, O2, H+, H2, H2O2 and tungsten. We furthermore allow a number of exchange reactions with the prefix

DM_ that are required for the iML biomass synthesis pseudoreaction.

We replace long and complex GPR rules with shorter ones. The integrated compression routine would compresses GPR rules, which would allow a fast computation. However decompression of computed strain designs can result in countless equivalent strain designs. For instance, the simultanous knockouts of CYTBO3_4pp (cyoA & cyoB & cyoC & cyoD) and FRD2 (frdA & frdB & frdC & frdD) can be achieved by any KO combination of the subunits and hence by permutations (cyoA, frdA) or (cyoA, frdB) or (cyoB, frdA) and so on … This is an effect we try to minimize by substituting GPR rules.

[20]:
```
exchange_reacs = [r for r in iml_14bdo.reactions if all(s<0 for s in r.metabolites.
↪values())]
# shut all exchange fluxes
for r in exchange_reacs:
    r.upper_bound = 0.0

# shut CO2 uptake
iml_14bdo.reactions.EX_co2_e.lower_bound    = 0.0

# keep main fermentation products open
iml_14bdo.reactions.EX_14bdo_e.upper_bound  = 1000.0
iml_14bdo.reactions.EX_ac_e.upper_bound     = 1000.0
iml_14bdo.reactions.EX_co2_e.upper_bound    = 1000.0
iml_14bdo.reactions.EX_etoh_e.upper_bound   = 1000.0
iml_14bdo.reactions.EX_for_e.upper_bound    = 1000.0
iml_14bdo.reactions.EX_h2_e.upper_bound     = 1000.0
iml_14bdo.reactions.EX_h2o2_e.upper_bound   = 1000.0
iml_14bdo.reactions.EX_h2o_e.upper_bound    = 1000.0
iml_14bdo.reactions.EX_h_e.upper_bound      = 1000.0
iml_14bdo.reactions.EX_lac__D_e.upper_bound = 1000.0
iml_14bdo.reactions.EX_meoh_e.upper_bound   = 1000.0
iml_14bdo.reactions.EX_o2_e.upper_bound     = 1000.0
iml_14bdo.reactions.EX_succ_e.upper_bound   = 1000.0
iml_14bdo.reactions.EX_tungs_e.upper_bound  = 1000.0
iml_14bdo.reactions.DM_4crsol_c.upper_bound = 1000.0
iml_14bdo.reactions.DM_5drib_c.upper_bound  = 1000.0
iml_14bdo.reactions.DM_aacald_c.upper_bound = 1000.0
iml_14bdo.reactions.DM_amob_c.upper_bound   = 1000.0
iml_14bdo.reactions.DM_mththf_c.upper_bound = 1000.0
iml_14bdo.reactions.DM_oxam_c.upper_bound   = 1000.0

# substitute GPR rule in ATPS4rpp (all genes occur only in ATPS)
atps_genes = [g for g in iml_14bdo.reactions.ATPS4rpp.genes]
iml_14bdo.reactions.ATPS4rpp.gene_reaction_rule = str(atps_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,atps_genes[1:])
atps_genes[0].name = 'atpX'

# substitute GPR rules in NADH16pp, NADH17pp, NADH18pp (all genes occur only in NADHxxpp␣
↪and GPR rules are identical)
nuo_genes = [g for g in iml_14bdo.reactions.NADH16pp.genes]
iml_14bdo.reactions.NADH16pp.gene_reaction_rule = str(nuo_genes[0])
iml_14bdo.reactions.NADH17pp.gene_reaction_rule = str(nuo_genes[0])
iml_14bdo.reactions.NADH18pp.gene_reaction_rule = str(nuo_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,nuo_genes[1:])
nuo_genes[0].name = 'nuoX'
```

(continues on next page)

(continued from previous page)

```
# substitute GPR rules in FRD2, FRD3 (genes occur only in FRD2 and FRD3 and GPR rules␣
↪are identical)
frd_genes = [g for g in iml_14bdo.reactions.FRD2.genes]
iml_14bdo.reactions.FRD2.gene_reaction_rule = str(frd_genes[0])
iml_14bdo.reactions.FRD3.gene_reaction_rule = str(frd_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,frd_genes[1:])
frd_genes[0].name = 'frdX'

# substitute GPR rule in CYTBO3_4pp
cyo_genes = [g for g in iml_14bdo.reactions.CYTBO3_4pp.genes]
iml_14bdo.reactions.CYTBO3_4pp.gene_reaction_rule = str(cyo_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,cyo_genes[1:])
cyo_genes[0].name = 'cyoX'

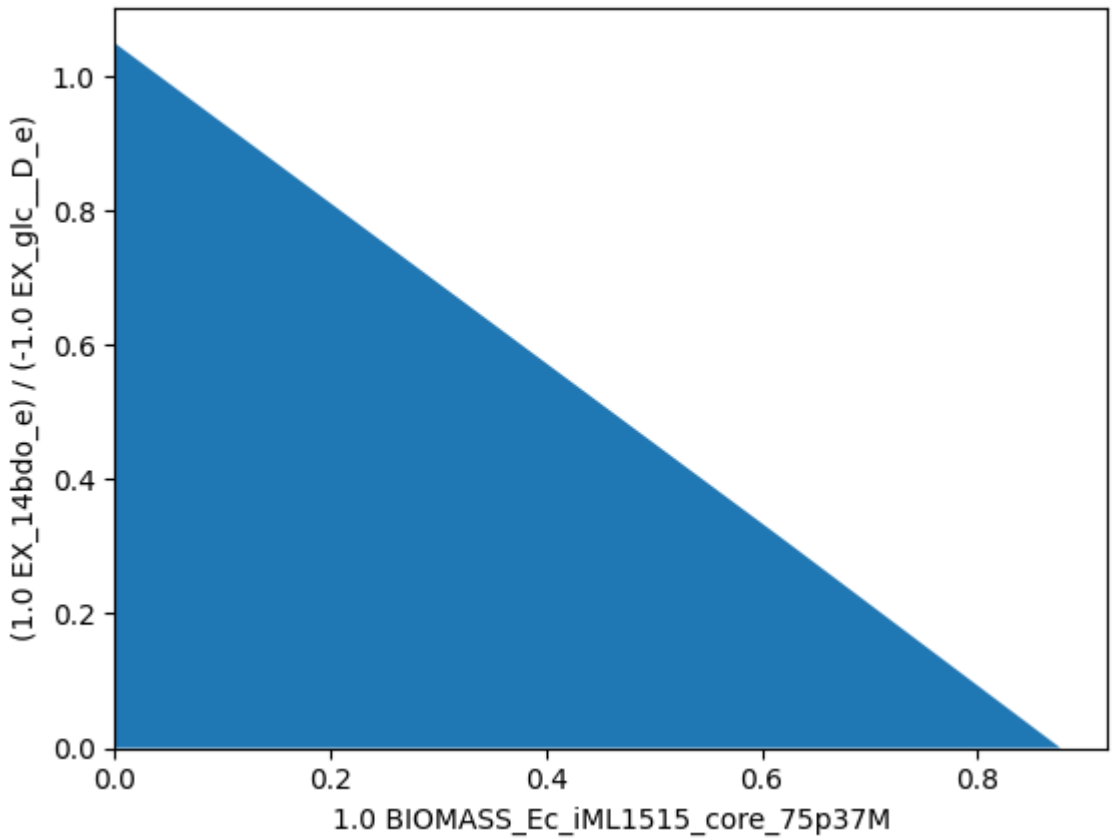
# substitute GPR rule in THD2pp
pnt_genes = [g for g in iml_14bdo.reactions.THD2pp.genes]
iml_14bdo.reactions.THD2pp.gene_reaction_rule = str(pnt_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,pnt_genes[1:])
pnt_genes[0].name = 'pntX'

# substitute GPR rule in PDH and AKGDH
ace_genes = [g for g in iml_14bdo.genes if g.name in ['aceE', 'aceF']]
sucAB_genes = [g for g in iml_14bdo.genes if g.name in ['sucA', 'sucB']]
lpd = [g for g in iml_14bdo.genes if g.name == 'lpd'][0]
iml_14bdo.reactions.PDH.gene_reaction_rule = str(ace_genes[0])+' and '+str(lpd)
iml_14bdo.reactions.AKGDH.gene_reaction_rule = str(sucAB_genes[0])+' and '+str(lpd)
cobra.manipulation.remove_genes(iml_14bdo,ace_genes[1:])
cobra.manipulation.remove_genes(iml_14bdo,sucAB_genes[1:])
ace_genes[0].name = 'aceEF'
sucAB_genes[0].name = 'sucAB'

# substitute GPR rule in SUCOAS
sucCD_genes = [g for g in iml_14bdo.reactions.SUCOAS.genes]
iml_14bdo.reactions.SUCOAS.gene_reaction_rule = str(sucCD_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,sucCD_genes[1:])
sucCD_genes[0].name = 'sucCD'

# substitute GPR rule in SUCDi
sdh_genes = [g for g in iml_14bdo.reactions.SUCDi.genes]
iml_14bdo.reactions.SUCDi.gene_reaction_rule = str(sdh_genes[0])
cobra.manipulation.remove_genes(iml_14bdo,sdh_genes[1:])
sdh_genes[0].name = 'sdhX'
```

[21]:
```
sd.plot_flux_space(iml_14bdo, ('BIOMASS_Ec_iML1515_core_75p37M',('EX_14bdo_e','-EX_glc__
↪D_e')));
```



As in the small-scale setup, we set up again the SUPRESS and PROTECT modules to enforce substrate-uptake-coupled production of 1,4-BDO. The genome-scale model contains many more secondary pathways that can be used to disrupt SUCP. We therefore start with a conservative setup of demanding a minimum product yield of $0.1\ \frac{mol_{1,4\text{-}BDO}}{mol_{D\text{-}Glucose}}$. At the same time we ensure that growth is still possible at rates of $0.05\ h^{-1}$ and above.

[22]:
```
min_14bdo_yield = 0.25
min_growth = 0.1
module_suppress = sd.SDModule(iml_14bdo,sd.SUPPRESS,constraints=f'EX_14bdo_e + {min_
↪14bdo_yield} EX_glc__D_e <= 0')
module_protect  = sd.SDModule(iml_14bdo,sd.PROTECT, constraints=f'BIOMASS_Ec_iML1515_
↪core_75p37M>={min_growth}')
```

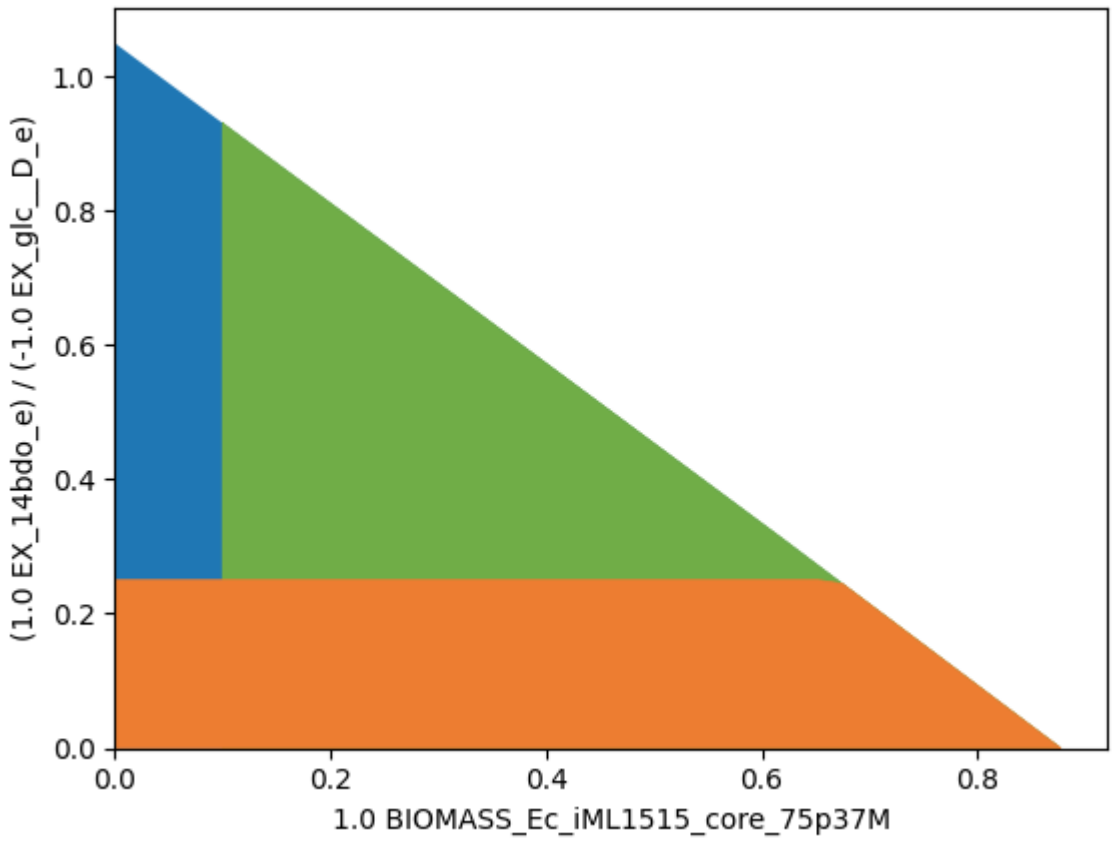
Plotted:

[23]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(iml_14bdo,
                                               ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(iml_14bdo,
                                               (f'BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                               constraints=f'BIOMASS_Ec_iML1515_core_
↪75p37M >= {min_growth}',
```

(continues on next page)

(continued from previous page)

```
                                                    show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
_,          _,      plot3 = sd.plot_flux_space(iml_14bdo,
                                                    ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                                    # The sign of the glucose exchange␣
↪reaction is flipped since
                                                    # reaction is defined in the direction of␣
↪secretion.
                                                    constraints=f'EX_14bdo_e + {min_14bdo_
↪yield} EX_glc__D_e <= 0',
                                                    show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



In the genome-case setup using iML1515, it is still unnecessary to exclude the 0-vector explicitly, since it is already excluded by default due to the minimum ATP maintenance demand.

We can now proceed with the strain design computation. Since we normally don't know if solutions to our strain design

problems exist, and even more so in genome-scale setups, we will start the computation with the most relaxed settings possible. This means, we compute only one single solution, without a time limit, while omitting the minimality demand in the solutions and allow up to 25 knockouts. We also activate logging to follow the progress of the computation. If this computation is not successful on the first run, we should retry. The success of genome-scale computations often depends on the MILP search tree, whose construction varies with different computation seeds. A repeated computation may therefore conclude in a different (but also valid) strain design solution. Generally, it is possible that a computation completes in a couple of minutes, while the repetitions runs into timeout.

[24]:
```
import logging
logging.basicConfig(level=logging.INFO)

# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in iml_14bdo.genes}
gko_cost.pop('s0001')
gko_cost.pop('akgdc')
gko_cost.pop('sscoar')
# allow knock-in of akgdc and sscoar
gki_cost = {'akgdc':1, 'sscoar':1}
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# Compute strain designs
sols = sd.compute_strain_designs(iml_14bdo,
                                 sd_modules = [module_suppress, module_protect],
                                 max_solutions = 1,
                                 max_cost = 40,
                                 ko_cost = ko_cost,
                                 gko_cost = gko_cost,
                                 gki_cost = gki_cost,
                                 solution_approach = sd.ANY)
# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.reaction_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 3183
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (1485 genes, 2268 gpr rules).
INFO:root:  Simplifyied to 1229 genes and 1853 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (5252 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 2450 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 2341 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 2298 reactions.
```

(continues on next page)

(continued from previous page)

```
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 2292 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 2288 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (2288 reactions).
INFO:root:  Network compression completed. (5 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 2288 reactions, 968 metabolites
INFO:root:  569 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding (also non-optimal) strain designs ...
INFO:root:Searching in full search space.
INFO:root:Minimizing number of interventions in subspace with 30 possible targets.
INFO:root:Strain design with cost 13.0: {'THD2pp*pntX': -1, 'EX_o2_e': -1,
↪'SSCOARx*sscoar': 1, 'adhE': -1, 'adhP*R1_g_b1241_or_g_b1478': -1, 'ybbO*R0_g_b0493_or_
↪g_b3003_or_g_b3011_or_g_b3588_or_g_b4269*yghA*R1_g_b0493_or_g_b3003_or_g_b3011_or_g_
↪b3588_or_g_b4269': -1, 'yqhD': -1, 'aldB': -1, 'ahr': -1, 'mdh': -1, 'glcA*R0_g_b2975_
↪or_g_b3603*lldP*R1_g_b2975_or_g_b3603': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:1 solutions found.
```

```
One compressed solution with cost 13.0 found and expanded to 1 solutions in the␣
↪uncompressed netork.
Example intervention set: ['-EX_o2_e', '-adhE', '-yqhD', '-aldB', '-ahr', '-mdh', '-pntX
↪', '+sscoar', '-adhP', '-ybbO', '-yghA', '-glcA', '-lldP']
```

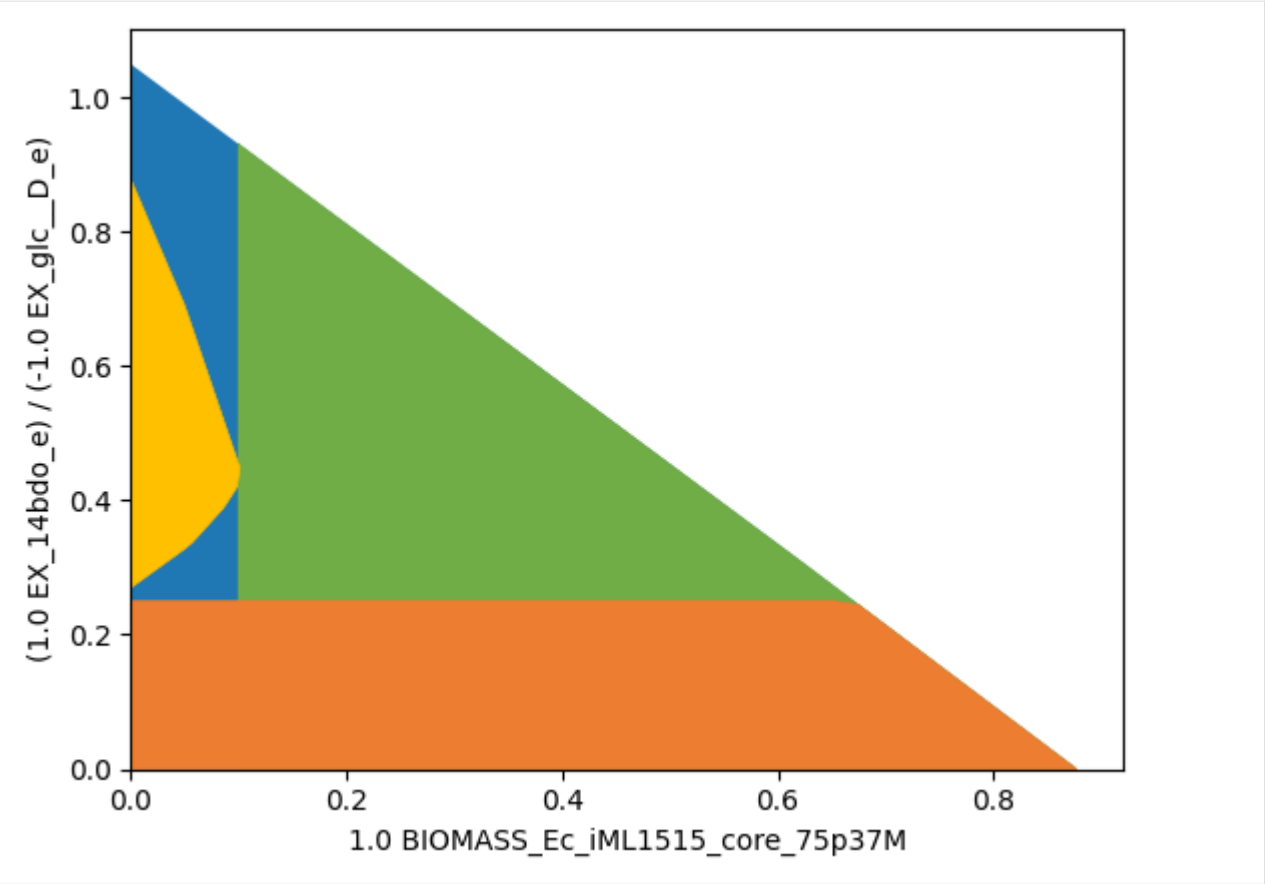
We may plot the computed strain design (yellow) on top of the wild type model (blue), the suppressed fluxes (orange) and the protected fluxes (green). The designed strain is forced to produce 1,4-butanediol but is still able to grow at a relatively high rate.

[25]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(iml_14bdo,
                                               ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                               show=False);
_,          _,      plot2 = sd.plot_flux_space(iml_14bdo,
                                               ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                               constraints=f'BIOMASS_Ec_iML1515_core_
↪75p37M>={min_growth}',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')
# pGCP design plot
```

(continues on next page)

(continued from previous page)

```
_,          _,      plot3 = sd.plot_flux_space(iml_14bdo,
                                                ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=f'EX_14bdo_e + {min_14bdo_
↪yield} EX_glc__D_e <= 0',
                                                show=False);
plot3.set_facecolor('#ED7D31')
plot3.set_edgecolor('#ED7D31')
# plotting designed strain
interventions = [{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot4 = sd.plot_flux_space(iml_14bdo,
                                                ('BIOMASS_Ec_iML1515_core_75p37M',('EX_
↪14bdo_e','-EX_glc__D_e')),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
                                                show=False);
plot4.set_facecolor('#FFC000')
plot4.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot4.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot4.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



MCS suggests the addition of only one of the two enzymes SSCOAR or AKGDC and suggests the knockout of plenty of other genes. The knockout suggestions proposed by MCS are often overly conservative. In practice, a subset of knockouts is sufficient, in particular, knockouts may be unnecessary when the affected pathway does not carry a large flux in the first place.

4.5.8 Example 7: Suppress flux states in a toy network

The MCS concept can be showcased in a small example network:

Suppose, we want block all metabolic flux through **R4** to avoid the production of metabolite **E**. One can now identify irreducible sets of reaction knockouts to achieve this. Each of these set is called a minimal cut set, short MCS. This figure shows all possible MCS for blocking reaction R4.

Set up the according strain design problem by specifying a *module* that demands the suppression of flux states with $R4 > 0$.

Since strict inequalities are not allowed in mixed integer linear programming (MILP), we need to approximate it by an inclusive inequality and a sufficiently small value $\varepsilon > 0$. Here we pick $\varepsilon = 1$, such that the flux states that we aim to delete are those that fulfill the inequality:

$$R4 >= 1$$

[26]:
```
modules  =  [sd.SDModule(model,sd.SUPPRESS,constraints='R4 >= 1')]
# modules +=  [sd.SDModule(model,sd.PROTECT, constraints='R3 >= 1')]


sols = sd.compute_strain_designs(model,sd_modules = modules)
for s in sols.reaction_sd:
    print(s)
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 4983
INFO:root:  Using cplex for solving LPs during preprocessing.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Compressing Network (10 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 8 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (8 reactions).
INFO:root:  Network compression completed. (1 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 8 reactions, 4 metabolites
INFO:root:  8 targetable reactions
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Strain design with cost 1.0: {'R4*R7*R10': -1}
INFO:root:Strain design with cost 1.0: {'R1': -1}
INFO:root:Strain design with cost 1.0: {'R3': -1}
INFO:root:Strain design with cost 2.0: {'R6': -1, 'R8': -1}
INFO:root:Strain design with cost 3.0: {'R2': -1, 'R5': -1, 'R6': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:5 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:7 solutions found.
```

```
{'R1': -1.0}
{'R3': -1.0}
{'R4': -1.0}
{'R7': -1.0}
{'R10': -1.0}
{'R6': -1.0, 'R8': -1.0}
{'R2': -1.0, 'R5': -1.0, 'R6': -1.0}
```

An adequate value for ε can inferred from the model, i.e., the coefficients of the stoichiometric matrix and the flux boundaries. In the shown example values of 1e-7 up to 50 will yield the same results. However, too small values may result in longer runtimes or numerical issues. While large values may not approximate the strict inequality well enough.

4.5.9 Example 8: Suppress and protect flux states in a toy network

It may sometimes be required to protect certain flux states, for instance, to guarantee that the model stays feasible despite the deletion, or to guarantee that microbial growth is still possible despite the introduced where **R4** must be deleted and additionally demand that **R3** must still be able to carry flux.

[27]:
```
import straindesign as sd
import cobra

model = cobra.io.read_sbml_model('../../../tests/model_small_example.xml')

modules =   [sd.SDModule(model,sd.SUPPRESS,constraints='R4 >= 1')]
modules +=  [sd.SDModule(model,sd.PROTECT, constraints='R3 >= 1')]

sols = sd.compute_strain_designs(model,sd_modules = modules)
for s in sols.reaction_sd:
    print(s)
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 47615
INFO:root:  Using cplex for solving LPs during preprocessing.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Compressing Network (10 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 8 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (8 reactions).
INFO:root:  Network compression completed. (1 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 8 reactions, 4 metabolites
INFO:root:  7 targetable reactions
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Strain design with cost 1.0: {'R4*R7*R10': -1}
INFO:root:Strain design with cost 1.0: {'R1': -1}
INFO:root:Strain design with cost 2.0: {'R6': -1, 'R8': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:3 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:5 solutions found.
```

```
{'R1': -1.0}
{'R4': -1.0}
{'R7': -1.0}
{'R10': -1.0}
{'R6': -1.0, 'R8': -1.0}
```

As can be seen, the computation returns the 5 out of 7 MCS that block R4 but not R3. The set of theses *constrained* MCS (cMCS) is a subset of the former MCS solution pool.

4.5.10 Theoretical background

In the simplest case, the Minimal Cut Set approach seeks to identify the smallest set of knockouts that render undesired flux states, such as flux states with poor productivity or yield, unattainable. In reference to the space of steady-state flux vectors: one introduces *knockouts that exclude a certain flux-subspace from the space of feasible flux states*.

(Mixed integer) linear programming operates on *feasible* problem spaces and, *per se*, does not allow sub-problems to be infeasible. To express the infeasibility of a certain sub-problem, one needs to formulate an equivalent but "mirrored" problem whose feasibility under any conditions coincides with the infeasibility of the original problem. Farkas' Lemma is a theorem that will serve for exactly that purpose. It states that, to a system of linear inequalities, a second system of linear inequalities can be defined such that *one and only one* of the two problems is feasible. We call the original system the primary and secondary system the Farkas-dual. The feasibility of one problem is a *certificate for the infeasibility* of the other one.

Exactly one of the following is true:

$$\mathbf{A}\,\mathbf{x} \leq \mathbf{b} \qquad \text{or} \qquad \mathbf{A}^\intercal\mathbf{y} = \mathbf{0}, \quad \mathbf{y} \geq \mathbf{0}, \quad \mathbf{y}^\intercal\mathbf{b} < 0.$$

Farkas-duality is closely related to Lagrange-duality in Linear Progamming and variables in one problem correspond to constraints in the other. When we introduce interventions (i.e., we force certain flux variables to take the value 0), we need to adapt the Farkas-dual that we use in the MCS-MILP accordingly. When a variable is more constrained in one problem (or fixed to zero), the other problem relaxes in the corresponding constraint (up to the complete lifting of the constraint).

In the following we show how Farkas' Lemma can be used to construct the MCS-MILP. We start with the original MCS-system simplified as $\mathbf{A_T} \cdot \mathbf{x} = \mathbf{0} \leq \mathbf{b_T}$ that is rendered infeasible through the introduction of knockouts as additional constraints.

$$\begin{bmatrix} \mathbf{A_T} \\ \mathbf{I_{KO}} \end{bmatrix} \mathbf{x} \begin{matrix} \leq \\ = \end{matrix} \begin{bmatrix} \mathbf{b_T} \\ \mathbf{0} \end{bmatrix}.$$

The introduced permanent knockouts will later be controlled through binary decision variables. It must be noted that the constraints only render the primal system infeasible when they *contradict* the primal system. As was mentioned above, it is therefore not possible to use undesired systems that have the zero vector $\mathbf{x} = \mathbf{0}$ as a feasible solution because then even the knockout of all reactions would not make the primal system infeasible.

Using Farkas-dualization, the dual system is given by:

$$\begin{bmatrix} \mathbf{A_T^\intercal} & \mathbf{I_{KO}} \\ \mathbf{b_T^\intercal} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{y} \\ \mathbf{v} \end{bmatrix} = \leq \begin{bmatrix} \mathbf{0} \\ -1 \end{bmatrix}$$
$$\mathbf{y} \geq \mathbf{0},$$

which is, per definition, feasible. The single inequality in the last row corresponds to $\mathbf{b}^\intercal\mathbf{y} < 0$ of the Farkas-dual system. The latter needs to be replaced with $\mathbf{b_T^\intercal y} \leq -1$ because the system will later be used in a MILP that cannot handle strict inequalities. The replacement is allowed because any found solution of the Farkas-dual can be scaled to also fulfill $\mathbf{b_T^\intercal y} \leq -1$ without affecting the support of the solution (which makes up the MCS).

As can be seen, the variable knockouts in the primal ($\mathbf{I_{KO}}\,\mathbf{x} = \mathbf{0}$) translate to the variables $\mathbf{v}$ in the Farkas-dual system that mime the knockout of all dual constraints by allowing for arbitrary large slack. This system can now be used to identify MCS, since a minimal subset of constraint relaxations (indicated by $v_i \neq 0$) that solves the Farkas-dual system corresponds directly to a minimal subset of primal knockout-constraints within $\mathbf{I_{KO}}\,\mathbf{x} = \mathbf{0}$ that keeps the primal system infeasible. Hence, every solution of the problem with a support-minimal vector $\mathbf{v}$ represents one MCS.

A constraint can be switched on or off by controlling its slack variable v_i by a corresponding binary variable z_i either via indicator constraints,

$$z_i = 0 \rightarrow v_i = 0,$$

or with the big-M method (with M being a sufficiently large number)

$$-M \cdot z_i \leq v_i \leq M \cdot z_i.$$

In this case, there is a 1:1 association of metabolic knockouts, indicated by z_i and slack variables v_i. With the binary variables z_i at hand, we may now finally pose a MILP problem with an objective function that minimizes the number of interventions to block the target system (we use here the version with indicator constraints):

$$\begin{aligned} \text{minimize} \quad & \sum z_i \\ \text{subject to} \quad & \begin{bmatrix} \mathbf{A_T^\intercal} & \mathbf{I_{KO}} \\ \mathbf{b_T^\intercal} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{y} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ -1 \end{bmatrix} \\ & \forall i : z_i = 0 \rightarrow v_i = 0 \\ & \mathbf{y} \geq \mathbf{0}, \quad z_i \in \{0,1\}. \end{aligned}$$

This MILP finds the smallest irreducible set of interventions (support-minimal in $\mathbf{v}$) that blocks the target system, hence an MCS with the smallest possible cardinality.

We note that an even more concise formulation can be constructed by omitting the slack-variables v_i and linking the removal of constraints directly to z_i:

$$\begin{aligned} \text{minimize} \quad & \sum z_i \\ \text{subject to} \quad & \begin{bmatrix} \mathbf{b_T^\intercal} & \mathbf{0} \\ \mathbf{0} & \mathbf{A_D} \end{bmatrix} \begin{bmatrix} \mathbf{y} \\ \mathbf{x} \end{bmatrix} \leq \begin{bmatrix} -1 \\ \mathbf{b_D} \end{bmatrix} \\ & \forall i : z_i = 0 \rightarrow \mathbf{A}_{\mathbf{T},i}^\intercal \mathbf{y} = 0 \\ & \forall i : z_i = 1 \rightarrow x_i = 0 \\ & \mathbf{y} \geq \mathbf{0}, \quad z_i \in \{0,1\}. \end{aligned}$$

The MCS S (containing the indices of the knocked-out reactions), computed by the MILP, is given by $S = \{i | z_i = 1\}$. Multiple MCS solutions (with increasing cardinality) can be found by excluding previously found solutions and their supersets through integer cut constraints and solving the MILP repeatedly.

4.6 Multi-level strain optimization approaches

Several strain design approaches use MILPs with nested optimization to enforce growth-coupled production. As the first nested optimization algorithm, OptKnock [2] aimed to resolve the conflict between the microbial objective of fast growth with the engineering goal of fast production. It therefore constructs a max-max problem for the maximization of product synthesis under the assumption that the cell itself, will maximize its growth rate. One problem is that this formulation leads to overly optimistic strain designs, since it assumes that a cell would maximize production when attaining its maximal growth rate. In the worst case, however (potentially growth coupled prodcution), a cell might turn off production completely.

Successors of OptKnock, such as RobustKnock [3] and OptCouple [4] have overcome this problem since they guarantee production at maximum growth (weakly growth-coupled production). In the following sections we will explain how one can compute strain designs with OptKnock, RobustKnock and OptCouple, and how these methods can be combined with the minimial cut set approach.

[3]:
```
import straindesign as sd
import cobra

ecc = cobra.io.load_model('e_coli_core')
```

For the following examples we again look at 1,4-BDO production. Hence, we first need to introduce the 1,4-BDO pathway again into the e_coli_core model and ensure that it is operational.

[4]:
```
# Create copy of model to which pathway will be added
ecc_14bdo = ecc.copy()

# Add metabolites to model
ecc_14bdo.add_metabolites([ cobra.Metabolite('sucsal_c'),# Succinic semialdehyde
                            cobra.Metabolite('4hb_c'),   # 4-Hydroxybutanoate
                            cobra.Metabolite('4hbcoa_c'),# 4-Hydroxybutyryl-CoA
                            cobra.Metabolite('4hbal_c'), # 4-Hydroxybutanal
                            cobra.Metabolite('14bdo_c'), # Butane-1,4-diol (cytopl.)
                            cobra.Metabolite('14bdo_p'), # Butane-1,4-diol (peripl.)
                            cobra.Metabolite('14bdo_e')  # Butane-1,4-diol (extrac.)
                            ])

# Create reactions
SSCOARx    = cobra.Reaction('SSCOARx')
AKGDC      = cobra.Reaction('AKGDC')
HBD        = cobra.Reaction('4HBD')
HBCT       = cobra.Reaction('4HBCT')
HBDH       = cobra.Reaction('4HBDH')
HBDx       = cobra.Reaction('4HBDx')
BDOtpp     = cobra.Reaction('14BDOtpp')
BDOtex     = cobra.Reaction('14BDOtex')
EX_14bdo_e = cobra.Reaction('EX_14bdo_e')

# Add reactions to model
ecc_14bdo.add_reactions([SSCOARx,
                         AKGDC,
                         HBD,
                         HBCT,
                         HBDH,
                         HBDx,
                         BDOtpp,
                         BDOtex,
                         EX_14bdo_e])

# Define reaction equations
SSCOARx.reaction    = '1 h_c + 1 nadph_c + 1 succoa_c -> 1 coa_c + 1 nadp_c + 1 sucsal_c'
AKGDC.reaction      = '1 akg_c + 1 h_c -> 1 co2_c + 1 sucsal_c'
HBD.reaction        = '1 h_c + 1 nadh_c + 1 sucsal_c  -> 1 4hb_c + 1 nad_c'
HBCT.reaction       = '1 4hb_c + 1 accoa_c            -> 1 4hbcoa_c + 1 ac_c'
HBDH.reaction       = '1 4hbcoa_c + 1 h_c + 1 nadh_c  -> 1 4hbal_c + 1 coa_c + 1 nad_c'
HBDx.reaction       = '1 4hbal_c + 1 h_c + 1 nadh_c   -> 1 14bdo_c + 1 nad_c'
BDOtpp.reaction     = '1 14bdo_c                      -> 1 14bdo_p'
BDOtex.reaction     = '1 14bdo_p                      -> 1 14bdo_e'
EX_14bdo_e.reaction = '1 14bdo_e                      ->'

# Verify that pathway is operational
sol = sd.fba(ecc_14bdo,obj='EX_14bdo_e',obj_sense='max')
print(f"Maximum possible 1,4-BDO synthesis rate: {sol.objective_value}.")
```

```
Read LP format model from file /tmp/tmpzacr20gi.lp
Reading time = 0.00 seconds
: 72 rows, 190 columns, 720 nonzeros
```

(continues on next page)

(continued from previous page)

```
Maximum possible 1,4-BDO synthesis rate: 10.252923076923079.
```

4.6.1 OptKnock

4.6.1.1 Example 9: OptKnock strain design

Optknock [2] is based on a bi-level optimization problem:

$$
\begin{array}{lll}
\text{maximize} & v_{production} & \\
 & \text{maximize} & v_{biomass} \\
 & \text{subject to} & \mathbf{S}\,\mathbf{v} = \mathbf{0} \\
\text{subject to} & & v_{BM} \geq v_{BM}^{min} \\
 & & (1 - z_i) \cdot lb_i \leq v_i \leq (1 - z_i) \cdot ub_i,\ \forall i \in \{1, \ldots, n\} \\
 & & \sum z_i \leq \text{maxKOs} \\
 & & z_i \in \{0, 1\}
\end{array}
$$

The nested optimization is translated into a single-layer problem and can then be solved as a mixed-integer linear problem (MILP).

Translating the nested optimization into a single level optimization yields:

$$
\begin{gathered}
\text{maximize } v_{production} \\
\text{subject to} \\
\begin{bmatrix} \mathbf{G} & \mathbf{0} & \mathbf{0} \\ \mathbf{D} & \mathbf{0} & \mathbf{0} \\ -\mathbf{c}^\intercal & \mathbf{g}^\intercal & \mathbf{0} \\ \mathbf{0} & \mathbf{G}^\intercal & \mathbf{I_{KO}} \end{bmatrix} \begin{bmatrix} \mathbf{v} \\ \mathbf{y} \\ \mathbf{s} \end{bmatrix} \begin{matrix} \leq \\ \leq \\ \leq \\ = \end{matrix} \begin{bmatrix} \mathbf{g} \\ \mathbf{d} \\ 0 \\ \mathbf{c} \end{bmatrix} \\
\forall i : z_i = 1 \rightarrow v_i = 0 \\
\forall i : z_i = 0 \rightarrow s_i = 0 \\
\sum z_i \leq MaxNoKO \\
\mathbf{y} \geq \mathbf{0}, \quad z \in \{0, 1\}
\end{gathered}
$$

The steps of translating a bi-level problem to a MILP are automated in StrainDesign. In the following, we will compute OptKnock strain designs using `e_coli_core`.

An advantage of OptKnock strain designs is that they often allow for a higher maximal growth rate. However, the predicted strain designs are often overly optimistic since they imply that the cells are able to actually reach their stoichiometrically highest possible growth rate and then also tune teir metabolism towards production. OptKnock strain designs might be preferred if the production pathway is known to be well expressed and active, even without introducing knockouts. OptKnock may then assure that laboratory evolution selects against production. We set up the strain design module for computing OptKnock strain desings. The OptKnock module consists of an outer and inner objective and additional constraints. The additional constraints are used to enforce the minimal desired growth rate.

[5]:
```
module_optknock = sd.SDModule(ecc_14bdo,sd.OPTKNOCK,
                              inner_objective='BIOMASS_Ecoli_core_w_GAM',
                              outer_objective='EX_14bdo_e',
                              constraints='BIOMASS_Ecoli_core_w_GAM >= 0.5')
```

We then call the strain design function with the constructed module. Since OptKnock uses an outer objective, we should use the solution approach 'BEST' to enforce optimality. By default, inner and outer objective functions are defined in the sense of *maximization* for minimization, the `inner_opt_sense` and `outer_opt_sense` can be set to `'maximize'` or one can simply use negative coefficients in the objective functions.

[6]:
```
import logging
logging.basicConfig(level=logging.INFO)
## Compute strain designs
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = module_optknock,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
# Print solution
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 10833
INFO:root:  Using gurobi for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 87 genes and 49 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (270 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 151 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 130 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Reduced to 121 reactions.
INFO:root:  Compression 7: Applying compression from EFM-tool module.
INFO:root:  Reduced to 120 reactions.
INFO:root:  Compression 8: Lumping parallel reactions.
```

(continues on next page)

(continued from previous page)

```
INFO:root:  Last step could not reduce size further (120 reactions).
INFO:root:  Network compression completed. (7 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 120 reactions, 71 metabolites
INFO:root:  44 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: gurobi.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 2.6195399376392943
INFO:root:Minimizing number of interventions in subspace with 16 possible targets.
INFO:root:Strain design with cost 11.0: {'AKGDH*sucA*sucB*R_g_b0116_and_g_b0726_and_g_
↪b0727': -1, 'PYK*pykA*R0_g_b1676_or_g_b1854*pykF*R1_g_b1676_or_g_b1854': -1,
↪'ICL*MALS*aceA*aceB*R0_g_b2976_or_g_b4014*glcB*R1_g_b2976_or_g_b4014': -1, 'AKGDC': 1,
↪'pgi': -1, 'kgtP': -1, 'tdcD*R0_g_b1849_or_g_b2296_or_g_b3115*ackA*R1_g_b1849_or_g_
↪b2296_or_g_b3115*purT*R2_g_b1849_or_g_b2296_or_g_b3115': -1, 'gdhA': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:4 solutions found.
```

```
One compressed solution with cost 11.0 found and expanded to 4 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-pgi', '-kgtP', '-gdhA', '-sucA', '-aceA', '-tdcD',
↪ '-ackA', '-purT', '-pykA', '-pykF']
```

We can now compare the computed strain design to the wild type and the minimal enforced growth rate. As OptKnock is rate instead of yield based, we plot our strain designs in the production envelope.

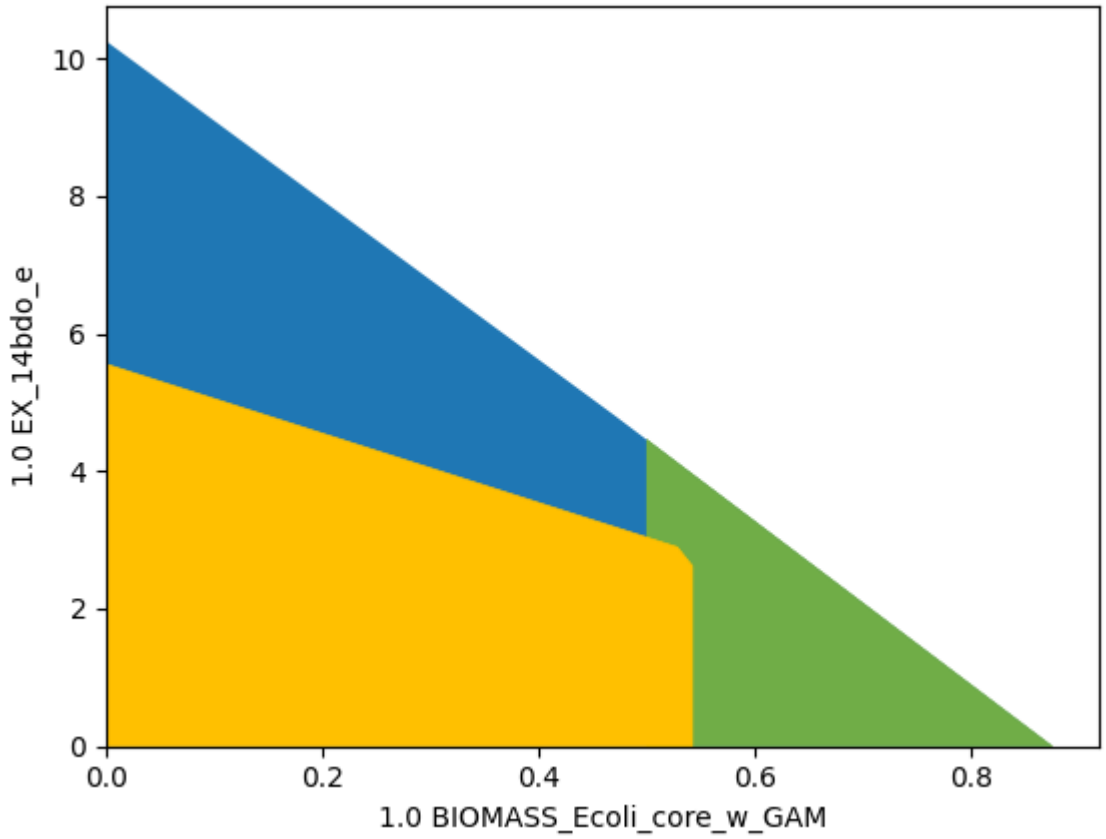
[7]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               show=False);
# Plot minimal enforced growth rate
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.5
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')

# OptKnock design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
```

(continues on next page)

(continued from previous page)

```
↪secretion.
                                                    constraints=interventions,
                                                    show=False);
plot3.set_facecolor('#FFC000')
plot3.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



The computed strain design generates a pGCP strain design with a relatively high maximal growth rate, however, at the cost of relatively many knockouts. The plot shows that OptKnock has exploited all means to improve maximal production at maximum growth. The maximum growth rate is only slightly above the demanded minimum.

4.6.1.2 Example 10: OptKnock strain design with a tilted objective function

The last example showed that predicted strain designs not always *guarantee* product synthesis at maximum growth. A tilted objective function can be used to simulate that the cell not only maximizes growth but also minimizes product synthesis and thus opposes the engineering goal. Factoring in this secondary goal by the cell with a sufficiently small objective coefficient will lead to more agressive stain designs that counteract the cells possible minimization of production.

[8]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
```

(continues on next page)

(continued from previous page)

```
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                show=False);
# Plot minimal enforced growth rate
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                constraints='BIOMASS_Ecoli_core_w_GAM>=0.5
↪',
                                                show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')

module_tilted_optknock = sd.SDModule(ecc_14bdo,sd.OPTKNOCK,
                              inner_objective='BIOMASS_Ecoli_core_w_GAM - 0.001 EX_14bdo_
↪e',
                              outer_objective='EX_14bdo_e',
                              constraints='BIOMASS_Ecoli_core_w_GAM >= 0.5')

# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = module_tilted_optknock,
                                 time_limit = 300,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)

# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")

# OptKnock design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                                ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
                                                show=False);
plot3.set_facecolor('#FFC000')
```

(continues on next page)

(continued from previous page)

```
plot3.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```

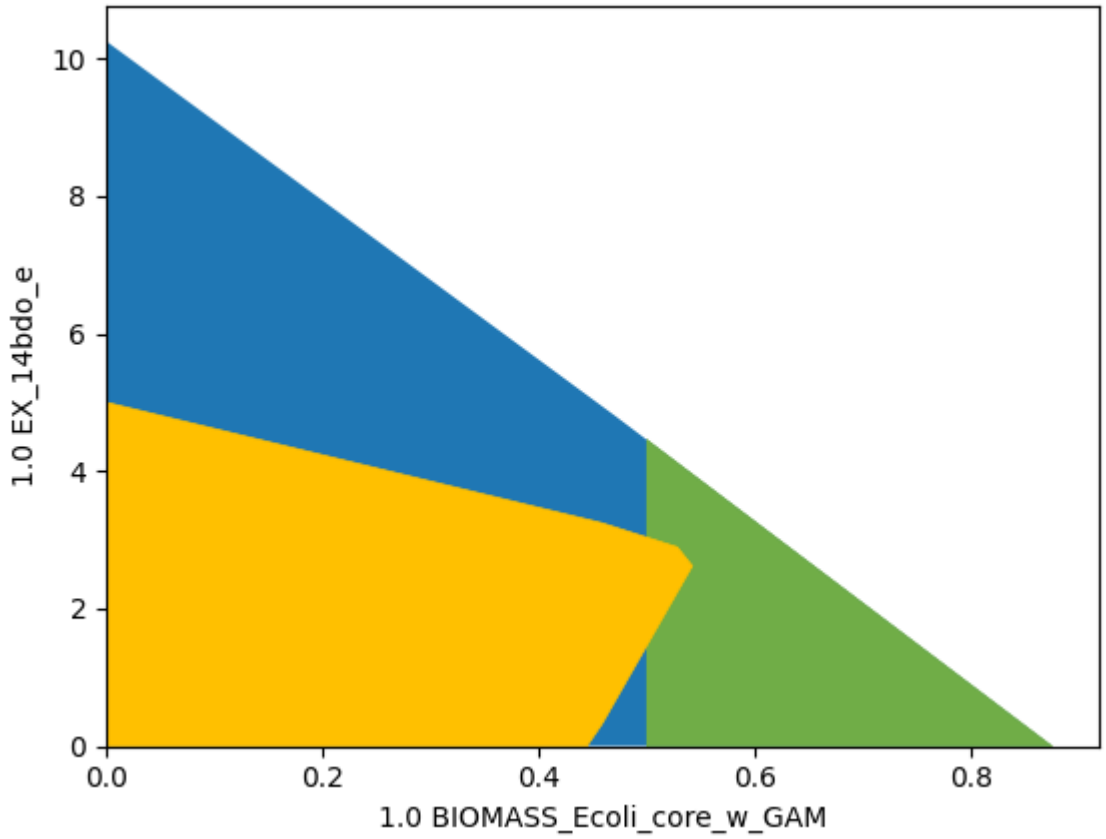
```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 54233
INFO:root:  Using gurobi for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 87 genes and 49 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (270 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 151 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 130 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Reduced to 121 reactions.
INFO:root:  Compression 7: Applying compression from EFM-tool module.
INFO:root:  Reduced to 120 reactions.
INFO:root:  Compression 8: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (120 reactions).
INFO:root:  Network compression completed. (7 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 120 reactions, 71 metabolites
INFO:root:  44 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: gurobi.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 2.619539937639321
INFO:root:Minimizing number of interventions in subspace with 16 possible targets.
INFO:root:Strain design with cost 16.0: {'PFL*EX_for_e*pflA*pflB*grcA*R_g_b0902_and_g_
↪b0903_and_g_b2579*R0_g_b0902_and_g_b0903_or_g_b0902_and_g_b0903_and_g_b2579_or_g_b0902_
↪and_g_b3114_or_g_b3951_and_g_b3952...': -1, 'PYK*pykA*R0_g_b1676_or_g_b1854*pykF*R1_g_
↪b1676_or_g_b1854': -1, 'ICL*MALS*aceA*aceB*R0_g_b2976_or_g_b4014*glcB*R1_g_b2976_or_g_
↪b4014': -1, 'ME1*maeA': -1, 'ME2*maeB': -1, 'AKGDC': 1, 'pgi': -1, 'kgtP': -1,
```

(continues on next page)

(continued from previous page)

```
↪'tdcD*R0_g_b1849_or_g_b2296_or_g_b3115*ackA*R1_g_b1849_or_g_b2296_or_g_b3115*purT*R2_g_
↪b1849_or_g_b2296_or_g_b3115': -1, 'sucC*sucD*R_g_b0728_and_g_b0729': -1, 'gdhA': -1,
↪'gltP': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:24 solutions found.
```

```
One compressed solution with cost 16.0 found and expanded to 24 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-pgi', '-kgtP', '-gdhA', '-gltP', '-pflA', '-aceA',
↪ '-maeA', '-maeB', '-pflD', '-tdcD', '-ackA', '-purT', '-pykA', '-pykF', '-sucC']
```



As a result, we obtain an at least weakly growth coupled strain design.

4.6.1.3 Example 11: Genome-scale OptKnock strain design

Other than RobustKnock and OptCouple, OptKnock's computational effort is moderate so that the approach may be used in genome scale metabolic models. The first steps of model manipulation are identical to the ones in example 6.

[9]:
```
import straindesign as sd
import cobra

cobra.Configuration().solver = 'cplex'
```

(continues on next page)

(continued from previous page)

```
ijo = cobra.io.load_model('iJO1366')

# Create copy of model to which pathway will be added
ijo_14bdo = ijo.copy()
# If available, set the solver to cplex or gurobi. This will increase the chances
# of sucess enormously
ijo_14bdo.solver = 'cplex'

# Add metabolites to model
ijo_14bdo.add_metabolites([ cobra.Metabolite('4hb_c'),    # 4-Hydroxybutanoate
                            cobra.Metabolite('4hbcoa_c'),# 4-Hydroxybutyryl-CoA
                            cobra.Metabolite('4hbal_c'), # 4-Hydroxybutanal
                            cobra.Metabolite('14bdo_c'), # Butane-1,4-diol (cytopl.)
                            cobra.Metabolite('14bdo_p'), # Butane-1,4-diol (peripl.)
                            cobra.Metabolite('14bdo_e')  # Butane-1,4-diol (extrac.)
                            ])

# Create reactions
AKGDC      = cobra.Reaction('AKGDC')
SSCOARx    = cobra.Reaction('SSCOARx')
HBD        = cobra.Reaction('4HBD')
HBCT       = cobra.Reaction('4HBCT')
HBDH       = cobra.Reaction('4HBDH')
HBDx       = cobra.Reaction('4HBDx')
BDOtpp     = cobra.Reaction('14BDOtpp')
BDOtex     = cobra.Reaction('14BDOtex')
EX_14bdo_e = cobra.Reaction('EX_14bdo_e')

# Add reactions to model
ijo_14bdo.add_reactions([SSCOARx,
                         AKGDC,
                         HBD,
                         HBCT,
                         HBDH,
                         HBDx,
                         BDOtpp,
                         BDOtex,
                         EX_14bdo_e])

# Define reaction equations
SSCOARx.reaction    = '1 h_c + 1 nadph_c + 1 succoa_c -> 1 coa_c + 1 nadp_c + 1 sucsal_c'
AKGDC.reaction      = '1 akg_c + 1 h_c -> 1 co2_c + 1 sucsal_c'
HBD.reaction        = '1 h_c + 1 nadh_c + 1 sucsal_c  -> 1 4hb_c + 1 nad_c'
HBCT.reaction       = '1 4hb_c + 1 accoa_c            -> 1 4hbcoa_c + 1 ac_c'
HBDH.reaction       = '1 4hbcoa_c + 1 h_c + 1 nadh_c  -> 1 4hbal_c + 1 coa_c + 1 nad_c'
HBDx.reaction       = '1 4hbal_c + 1 h_c + 1 nadh_c   -> 1 14bdo_c + 1 nad_c'
BDOtpp.reaction     = '1 14bdo_c                      -> 1 14bdo_p'
BDOtex.reaction     = '1 14bdo_p                      -> 1 14bdo_e'
EX_14bdo_e.reaction = '1 14bdo_e                      ->'

# define gene rules for knock-inable genes
SSCOARx.gene_reaction_rule  = 'gsscoar'
```

(continues on next page)

(continued from previous page)

```
AKGDC.gene_reaction_rule    = 'gakgdc'

exchange_reacs = [r for r in ijo_14bdo.reactions if all(s<0 for s in r.metabolites.
↪values())]
# shut all exchange fluxes
for r in exchange_reacs:
    r.upper_bound = 0.0

# shut CO2 uptake
ijo_14bdo.reactions.EX_co2_e.lower_bound    = 0.0

# keep main fermentation products open
ijo_14bdo.reactions.EX_14bdo_e.upper_bound  = 1000.0
ijo_14bdo.reactions.EX_ac_e.upper_bound     = 1000.0
ijo_14bdo.reactions.EX_co2_e.upper_bound    = 1000.0
ijo_14bdo.reactions.EX_etoh_e.upper_bound   = 1000.0
ijo_14bdo.reactions.EX_for_e.upper_bound    = 1000.0
ijo_14bdo.reactions.EX_h2_e.upper_bound     = 1000.0
ijo_14bdo.reactions.EX_h2o2_e.upper_bound   = 1000.0
ijo_14bdo.reactions.EX_h2o_e.upper_bound    = 1000.0
ijo_14bdo.reactions.EX_h_e.upper_bound      = 1000.0
ijo_14bdo.reactions.EX_lac__D_e.upper_bound = 1000.0
ijo_14bdo.reactions.EX_meoh_e.upper_bound   = 1000.0
ijo_14bdo.reactions.EX_o2_e.upper_bound     = 1000.0
ijo_14bdo.reactions.EX_succ_e.upper_bound   = 1000.0
ijo_14bdo.reactions.EX_tungs_e.upper_bound  = 1000.0
ijo_14bdo.reactions.DM_4crsol_c.upper_bound = 1000.0
ijo_14bdo.reactions.DM_5drib_c.upper_bound  = 1000.0
ijo_14bdo.reactions.DM_aacald_c.upper_bound = 1000.0
ijo_14bdo.reactions.DM_amob_c.upper_bound   = 1000.0
ijo_14bdo.reactions.DM_mththf_c.upper_bound = 1000.0
ijo_14bdo.reactions.DM_oxam_c.upper_bound   = 1000.0

# Verify that pathway is operational
sol = sd.fba(ijo_14bdo,obj='EX_14bdo_e',obj_sense='max')
print(f"Maximum possible 1,4-BDO synthesis rate: {sol.objective_value}.")
```

```
INFO:cobra.io.web.load:Attempting to fetch 'iJO1366' from the Cobrapy repository.
```

```
Maximum possible 1,4-BDO synthesis rate: 10.659770114941239.
```

Now that we introduced the 1,4-BDO pathway into the model and prepared it for strain design computation, we can continue with the problem formulation. We reduce the enforced attainable growth rate to relax the strain design problem. By introducing a minimum production threshold, we are able to still use the 'ANY' approach. This guarantees at least potentially growth-coupled production, although it is unlikely to identify the strategy with the highetst production potential this way. If the computation is successful, we can repeat the computation with the 'BEST' approach. If the computation is unsuccessful, one should consider using smaller models, such as iJO1366 or compute reaction instead of gene interventions.

[10]:
```
import matplotlib.pyplot as plt
import logging
logging.basicConfig(level=logging.INFO)
```

(continues on next page)

(continued from previous page)

```
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ijo_14bdo,
                                               ('BIOMASS_Ec_iJO1366_core_53p95M ','EX_
↪14bdo_e'),
                                               show=False);
# Plot minimal enforced growth rate
_,          _,      plot2 = sd.plot_flux_space(ijo_14bdo,
                                               ('BIOMASS_Ec_iJO1366_core_53p95M ','EX_
↪14bdo_e'),
                                               constraints=['BIOMASS_Ec_iJO1366_core_
↪53p95M >=0.2',
                                                            'EX_14bdo_e >=3'],
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')

module_tilted_optknock = sd.SDModule(ijo_14bdo,sd.OPTKNOCK,
                             inner_objective='BIOMASS_Ec_iJO1366_core_53p95M ', #  - 0.
↪001 EX_14bdo_e
                             outer_objective='EX_14bdo_e',
                             constraints=['BIOMASS_Ec_iJO1366_core_53p95M  >= 0.2','EX_
↪14bdo_e >=3'])

# add AKGDC and SSCOARx as adition candidates
ki_cost = {'AKGDC': 1.0, 'SSCOARx':1}
# possible knockout of O2
ko_cost = {r.id : 1.0 for r in ijo_14bdo.reactions if r.genes and ijo_14bdo.genes.s0001␣
↪not in r.genes}
ko_cost.update({'EX_o2_e': 1.0})
# remove AKGDC and SSCOARx from knockout candidates
ko_cost.pop('AKGDC')
ko_cost.pop('SSCOARx')
# Compute strain designs
sols = sd.compute_strain_designs(ijo_14bdo,
                                 sd_modules = module_tilted_optknock,
                                 max_solutions = 1,
                                 max_cost = 3,
                                 ki_cost = ki_cost,
                                 ko_cost = ko_cost,
                                 solution_approach = sd.BEST)

# Print solutions
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.reaction_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.reaction_
↪sd[0].items() if v!=0]}")

# OptKnock design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot3 = sd.plot_flux_space(ijo_14bdo,
                                               ('BIOMASS_Ec_iJO1366_core_53p95M ','EX_
↪14bdo_e'),
```

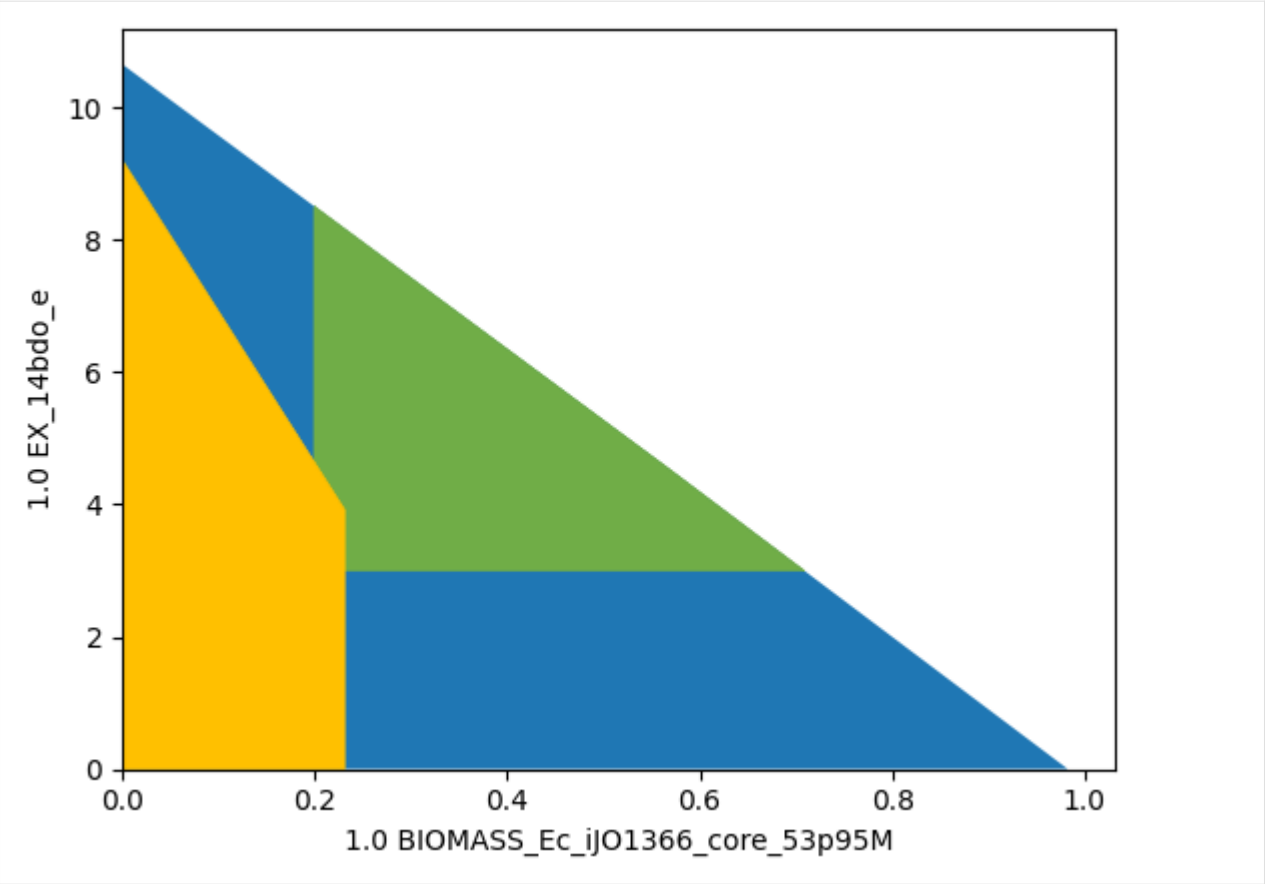
(continues on next page)

(continued from previous page)

```
                                                # The sign of the glucose exchange␣
↪reaction is flipped since
                                                # reaction is defined in the direction of␣
↪secretion.
                                                constraints=interventions,
                                                show=False);
plot3.set_facecolor('#FFC000')
plot3.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 60462
INFO:root:  Using cplex for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Compressing Network (2592 reactions).
INFO:root:  Removing blocked reactions.
/home/pschneider/miniconda3/envs/cnapy-1.2.4/lib/python3.10/site-packages/cobra/core/
↪group.py:147: UserWarning: need to pass in a list
  warn("need to pass in a list")
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 931 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (931 reactions).
INFO:root:  Network compression completed. (1 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 931 reactions, 421 metabolites
INFO:root:  811 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: cplex.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 3.9226348048271396
INFO:root:Minimizing number of interventions in subspace with 3 possible targets.
INFO:root:Strain design with cost 3.0: {'ACKr*PTAr': -1, 'AKGDH': -1, 'ATPS4rpp': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:2 solutions found.
```

```
One compressed solution with cost 3.0 found and expanded to 2 solutions in the␣
↪uncompressed netork.
Example intervention set: ['-AKGDH', '-ATPS4rpp', '-ACKr']
```



4.6.2 RobustKnock

The RobustKnock [3] approach usues multi-level optimization in a similar way to OptKnock. Whereas Optknock guarantees potentially growth-coupled production, RobusKnock guarantees at least weakly growth coupled production. This is achieved by introducing a third optimization layer. In order to avoid growth-maximal flux states with no production, RobustKnock *maximizes the production rate at maximum growth under the premise that is previously minimized*. The problem can be formalized as follows:

$$
\begin{array}{lll}
\text{maximize} & v_{production} & \\
 & \text{minimize} & v_{production} \\
 & & \text{maximize} \quad v_{biomass} \\
 & & \text{subject to} \quad \mathbf{S}\,\mathbf{v} = \mathbf{0} \\
\text{subject to} & \text{subject to} & v_{BM} \geq v_{BM}^{min} \\
 & & (1 - z_i) \cdot lb_i \leq v_i \leq (1 - z_i) \cdot ub_i,\ \forall i \in \{1, \ldots, n\} \\
 & & \sum z_i \leq \text{maxKOs} \\
 & & z_i \in \{0, 1\}
\end{array}
$$

To translate the presented problem into a single-layer probem that can be used in a Mixed-Integer Linear Problem, one applies the duality principle twice, once on the original problem, and then once again to the linearized nested problem.

This yields the MILP:

$$\text{maximize } v_{production}$$

$$\text{subject to}$$

$$\begin{bmatrix} \mathbf{G} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{D} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -\mathbf{c}_{\mathbf{prod,min}}^{\mathsf{T}} & \mathbf{g}^{\mathsf{T}} & \mathbf{0} & \mathbf{c}_{\mathbf{BM}}^{\mathsf{T}} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}^{\mathsf{T}} & -\mathbf{c}_{\mathbf{BM}}^{\mathsf{T}} & \mathbf{0} & \mathbf{I_{KO}} \\ \mathbf{0} & \mathbf{0} & \mathbf{g} & \mathbf{G} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{v} \\ \mathbf{u} \\ \mathbf{t} \\ \mathbf{r} \\ \mathbf{s} \end{bmatrix} \begin{matrix} \leq \\ \leq \\ = \\ = \\ \geq \end{matrix} \begin{bmatrix} \mathbf{g} \\ \mathbf{d} \\ 0 \\ \mathbf{c}_{\mathbf{BM}}^{\mathsf{T}} \\ \mathbf{0} \end{bmatrix}$$

$$\forall i : z_i = 1 \rightarrow v_i = 0$$

$$\forall i : z_i = 1 \rightarrow r_i = 0$$

$$\forall i : z_i = 0 \rightarrow s_i = 0$$

$$\sum z_i \leq MaxNoKO$$

$$\mathbf{u} \geq \mathbf{0}, \quad z \in \{0,1\}$$

4.6.2.1 Example 12: RobustKnock strain design

In StrainDesign, computing OptKnock strain designs is not any different from computing OptKnock strategies. One simply declares a RobustKnock instead of an Optknock module.

[11]:
```
module_robustknock = sd.SDModule(ecc_14bdo,sd.ROBUSTKNOCK,
                                 inner_objective='BIOMASS_Ecoli_core_w_GAM',
                                 outer_objective='EX_14bdo_e',
                                 constraints='BIOMASS_Ecoli_core_w_GAM >= 0.5')
```

Launching the strain design computation:

[12]:
```
import logging
logging.basicConfig(level=logging.INFO)
## Compute strain designs
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = module_robustknock,
                                 max_solutions = 1,
                                 max_cost = 30,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
# Print solution
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")
```

```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 34494
INFO:root:  Using gurobi for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 87 genes and 49 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (270 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 151 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 130 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Reduced to 121 reactions.
INFO:root:  Compression 7: Applying compression from EFM-tool module.
INFO:root:  Reduced to 120 reactions.
INFO:root:  Compression 8: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (120 reactions).
INFO:root:  Network compression completed. (7 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 120 reactions, 71 metabolites
INFO:root:  44 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: gurobi.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 2.6195399376393107
INFO:root:Minimizing number of interventions in subspace with 14 possible targets.
INFO:root:Strain design with cost 14.0: {'PYK*pykA*R0_g_b1676_or_g_b1854*pykF*R1_g_b1676_
↪or_g_b1854': -1, 'ICL*MALS*aceA*aceB*R0_g_b2976_or_g_b4014*glcB*R1_g_b2976_or_g_b4014':
↪ -1, 'ME1*maeA': -1, 'ME2*maeB': -1, 'AKGDC': 1, 'pgi': -1, 'kgtP': -1, 'tdcD*R0_g_
↪b1849_or_g_b2296_or_g_b3115*ackA*R1_g_b1849_or_g_b2296_or_g_b3115*purT*R2_g_b1849_or_g_
↪b2296_or_g_b3115': -1, 'sucC*sucD*R_g_b0728_and_g_b0729': -1, 'gdhA': -1, 'gltP': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:4 solutions found.
```

```
One compressed solution with cost 14.0 found and expanded to 4 solutions in the␣
```

(continues on next page)

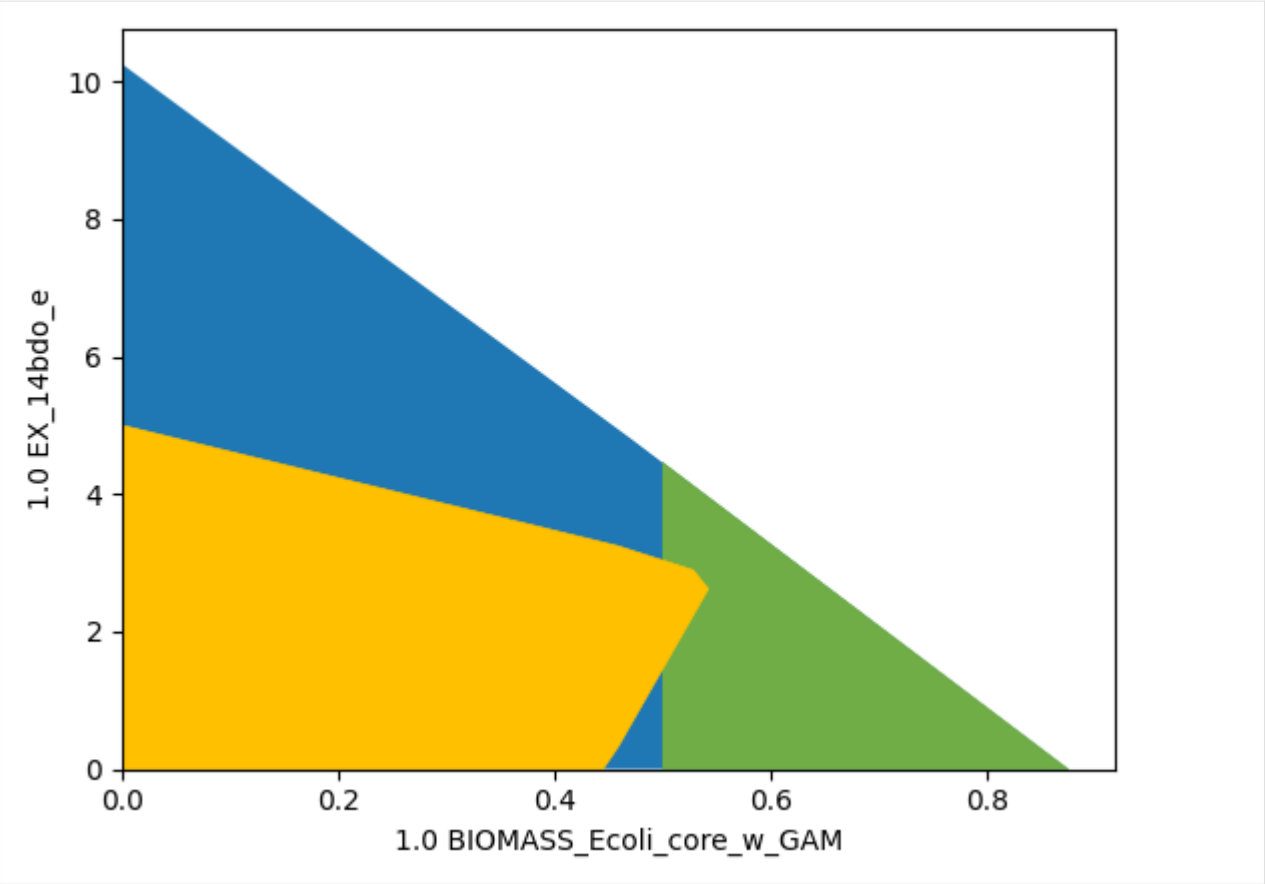
(continued from previous page)

```
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-pgi', '-kgtP', '-gdhA', '-gltP', '-aceA', '-maeA',
↪ '-maeB', '-tdcD', '-ackA', '-purT', '-pykA', '-pykF', '-sucC']
```

Plotting results:

[13]:
```
import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               show=False);
# Plot minimal enforced growth rate
_,          _,      plot2 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               constraints='BIOMASS_Ecoli_core_w_GAM>=0.5
↪',
                                               show=False);
plot2.set_facecolor('#70AD47')
plot2.set_edgecolor('#70AD47')

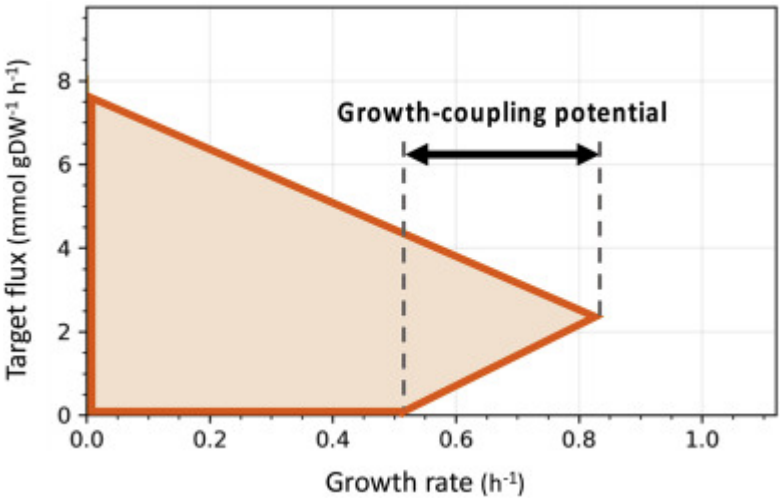
# OptKnock design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints=interventions,
                                               show=False);
plot3.set_facecolor('#FFC000')
plot3.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



The RobustKnock problem is significantly larger than the OptKnock problem. Genome-scale strain design problems are often computationally too expensive to solve. To assess whether genome-scale computation is possible, one should first define a very relaxed problem, use reaction KOs and solve with the 'ANY' approach before attempting more complex setups.

4.6.3 OptCouple

Optcouple [4] pursues a slightly different approach from OptKnock and RobustKnock as it aims to maximize the "growth-coupling potential" (GCP), the span of growth rates ranging from (left) the maximum growth rate without product synthesis to (right) the global maximum growth rate (Figure below taken from the original publication).



Mathematically, the OptCouple appraoch has many parallels to other nested multi-level approaches. In particular OptCouple demands the solution of two distinct flux states. The first subproblem determines the globally maximum flux state. Herefore, LP-duality may be employed to construct a subproblem that is identical to an MCS PROTECT module with an implicit optimization of growth (and optionally a minimum threshold for productivity). The second flux state that needs to be referenced has maximum growth under the condition that there is no product synthesis. This is again similar to the MCS PROTECT module, however the constraint $v_{production} = 0$ must be taken into account in the primal and its dual system. A weakness of OptCouple is that it does not return strain designs with substrate-uptake-coupled production, since a flux state with no production *must* exist as a lower reference point.

4.6.3.1 Example 13: OptCouple strain design

In StrainDesign, the computation of OptCouple designs is as straightforward as for the other approaches.

[14]:
```
module_optcouple = sd.SDModule(ecc_14bdo,sd.OPTCOUPLE,
                               inner_objective='BIOMASS_Ecoli_core_w_GAM',
                               prod_id='EX_14bdo_e',
                               min_gcp=0.3)

import logging
logging.basicConfig(level=logging.INFO)
## Compute strain designs
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = module_optcouple,
                                 max_solutions = 1,
                                 max_cost = 15,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
```

(continues on next page)

(continued from previous page)

```
# Print solution
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")

import matplotlib.pyplot as plt
# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               show=False);

# OptKnock design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,          _,      plot3 = sd.plot_flux_space(ecc_14bdo,
                                               ('BIOMASS_Ecoli_core_w_GAM','EX_14bdo_e'),
                                               # The sign of the glucose exchange␣
↪reaction is flipped since
                                               # reaction is defined in the direction of␣
↪secretion.
                                               constraints=interventions,
                                               show=False);
plot3.set_facecolor('#FFC000')
plot3.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot3.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot3.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```

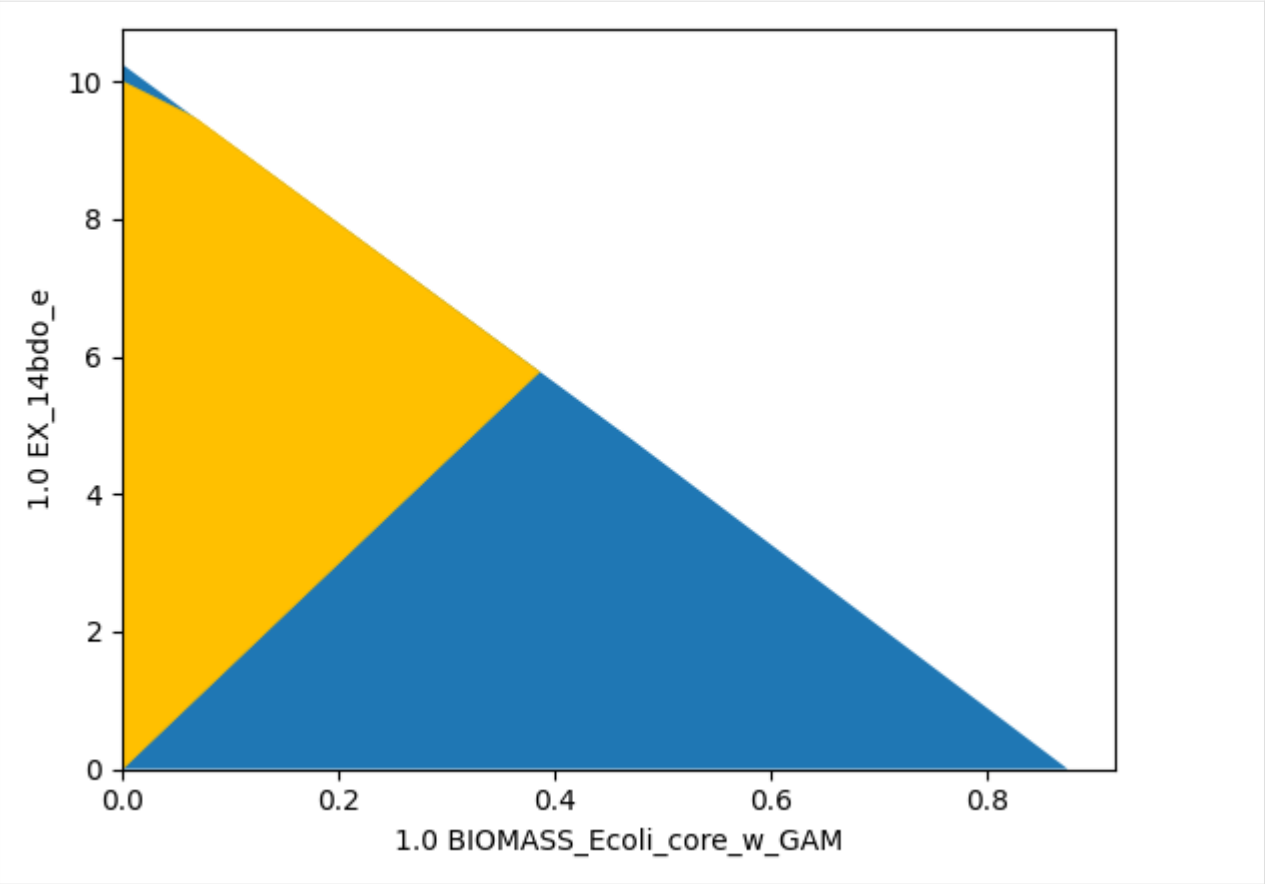
```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 29484
INFO:root:  Using gurobi for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 129 genes and 63 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (342 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 186 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 157 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 148 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 147 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 146 reactions.
```

(continues on next page)

(continued from previous page)

```
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Reduced to 145 reactions.
INFO:root:  Compression 7: Applying compression from EFM-tool module.
INFO:root:  Reduced to 144 reactions.
INFO:root:  Compression 8: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (144 reactions).
INFO:root:  Network compression completed. (7 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 144 reactions, 84 metabolites
INFO:root:  58 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: gurobi.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 0.3853916420255001
INFO:root:Minimizing number of interventions in subspace with 12 possible targets.
INFO:root:Strain design with cost 9.0: {'PFL*EX_for_e*pflA*pflB*grcA*R_g_b0902_and_g_
↪b0903_and_g_b2579*R0_g_b0902_and_g_b0903_or_g_b0902_and_g_b0903_and_g_b2579_or_g_b0902_
↪and_g_b3114_or_g_b3951_and_g_b3952...': -1, 'ME2*maeB': -1, 'AKGDC': 1, 'kgtP': -1,
↪'rpe*R0_g_b3386_or_g_b4301*sgcE*R1_g_b3386_or_g_b4301': -1, 'sucC*sucD*R_g_b0728_and_g_
↪b0729': -1, 'pntB*pntA*R_g_b1602_and_g_b1603': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:24 solutions found.
```

```
One compressed solution with cost 9.0 found and expanded to 24 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-kgtP', '-pflA', '-maeB', '-pflD', '-rpe', '-sgcE',
↪ '-sucC', '-pntB']
```



As in this case, OptCouple computations often result in directionally growth-coupled strain designs, as the maximum growth coupling potential is often obtained when the maximal growth rate without production is zero. In our exampe, we observe a growth-coupling potential of 0.4.

4.6.4 Combining nested optimization strain design with MCS

Nested strain design methods can be combined with the MCS approach to enforce additional properties. This allows flux space tailoring with unprecedented possibilities and precision.

> **Warning**
>
> In genome-scale setups the MILP of combined approaches is usually too large for efficient solving.

4.6.4.1 Example 14: Combining OptKnock with a tilted objective function and the MCS approach

Let's construct a strain design with at least weakly growth-coupled production, while we avoid ethanol nor succinate as by-products.

We construct the strain design problems with an OptKnock with a tilted objective function and an MCS module as follows:

[15]:
```
import logging
import matplotlib.pyplot as plt
logging.basicConfig(level=logging.INFO)
```

(continues on next page)

(continued from previous page)

```
# Enforce 1,4-BDO production at maximal growth
module_optknock = sd.SDModule(ecc_14bdo,sd.OPTKNOCK,
                              inner_objective='BIOMASS_Ecoli_core_w_GAM - 0.01 EX_14bdo_e
↪',
                              outer_objective='EX_14bdo_e',
                              constraints=['BIOMASS_Ecoli_core_w_GAM>=0.5'])
# Suppress Ethanol and Succinate production at maximal growth
module_mcs_suppress_etoh_succ = sd.SDModule(ecc_14bdo,sd.SUPPRESS,
                              inner_objective='BIOMASS_Ecoli_core_w_GAM',
                              constraints=['EX_etoh_e + EX_succ_e >= 0.1'])

## Compute strain designs
# allow all gene knockouts except for spontanuos
gko_cost = {g.name:1 for g in ecc_14bdo.genes}
gko_cost.pop('s0001')
# possible knockout of O2
ko_cost = {'EX_o2_e': 1}
# addition candidates
ki_cost = {'AKGDC':1, 'SSCOARx':1} # AKGDC was added in example 1.c)

sols = sd.compute_strain_designs(ecc_14bdo,
                                 sd_modules = [module_optknock,module_mcs_suppress_etoh_
↪succ],
                                 max_solutions = 1,
                                 max_cost = 15,
                                 gko_cost = gko_cost,
                                 ko_cost = ko_cost,
                                 ki_cost = ki_cost,
                                 solution_approach = sd.BEST)
# Print solution
print(f"One compressed solution with cost {sols.sd_cost[0]} found and "+\
      f"expanded to {len(sols.gene_sd)} solutions in the uncompressed netork.")
print(f"Example intervention set: {['+'+s if v>0 else '-'+s for s,v in sols.gene_sd[0].
↪items() if v!=0]}")


%matplotlib inline
# Aerobic design plot
interventions = [[{s:1.0},'=',0.0] for s,v in sols.reaction_sd[0].items() if v < 1]
_,_,plot1 = sd.plot_flux_space(ecc_14bdo,('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e + EX_
↪succ_e','EX_14bdo_e'),
                   constraints=interventions,points=50,show=False);
plot1._axes.view_init(20, 60)
plt.show()
```

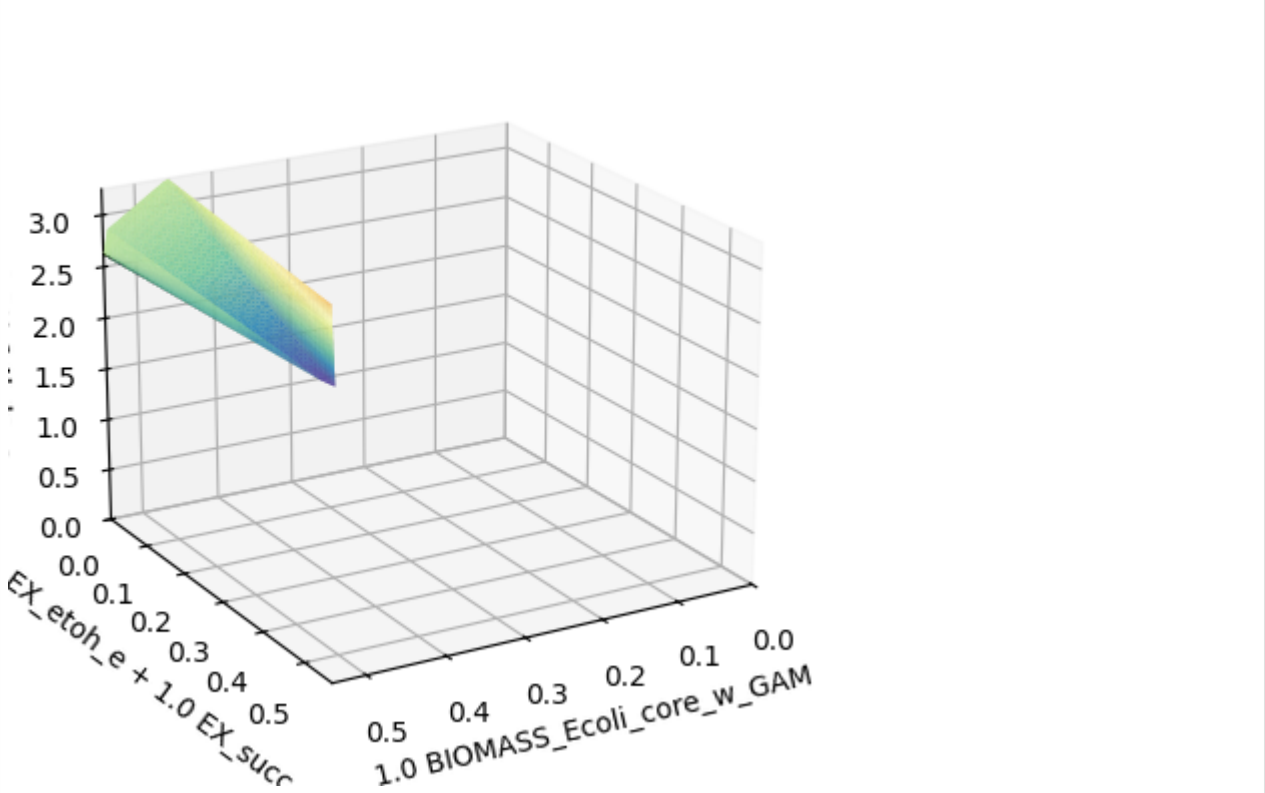
```
INFO:root:Preparing strain design computation.
INFO:root:  Using random seed 64490
INFO:root:  Using gurobi for solving LPs during preprocessing.
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:  FVA to identify blocked reactions and irreversibilities.
INFO:root:  FVA(s) to identify essential reactions.
```

(continues on next page)

(continued from previous page)

```
INFO:root:Preprocessing GPR rules (137 genes, 69 gpr rules).
INFO:root:  Simplifyied to 87 genes and 49 gpr rules.
INFO:root:  Extending metabolic network with gpr associations.
INFO:root:Compressing Network (270 reactions).
INFO:root:  Removing blocked reactions.
INFO:root:  Translating stoichiometric coefficients to rationals.
INFO:root:  Removing conservation relations.
INFO:root:  Compression 1: Applying compression from EFM-tool module.
INFO:root:  Reduced to 151 reactions.
INFO:root:  Compression 2: Lumping parallel reactions.
INFO:root:  Reduced to 130 reactions.
INFO:root:  Compression 3: Applying compression from EFM-tool module.
INFO:root:  Reduced to 124 reactions.
INFO:root:  Compression 4: Lumping parallel reactions.
INFO:root:  Reduced to 123 reactions.
INFO:root:  Compression 5: Applying compression from EFM-tool module.
INFO:root:  Reduced to 122 reactions.
INFO:root:  Compression 6: Lumping parallel reactions.
INFO:root:  Reduced to 121 reactions.
INFO:root:  Compression 7: Applying compression from EFM-tool module.
INFO:root:  Reduced to 120 reactions.
INFO:root:  Compression 8: Lumping parallel reactions.
INFO:root:  Last step could not reduce size further (120 reactions).
INFO:root:  Network compression completed. (7 compression iterations)
INFO:root:  Translating stoichiometric coefficients back to float.
INFO:root:  FVA(s) in compressed model to identify essential reactions.
INFO:root:Finished preprocessing:
INFO:root:  Model size: 120 reactions, 71 metabolites
INFO:root:  44 targetable reactions
WARNING:root:  Removing reaction bounds when larger than the cobra-threshold of 1000.
INFO:root:Constructing strain design MILP for solver: gurobi.
INFO:root:  Bounding MILP.
INFO:root:Finding optimal strain designs ...
INFO:root:Found solution with objective value 2.6195399376392556
INFO:root:Minimizing number of interventions in subspace with 12 possible targets.
INFO:root:Strain design with cost 15.0: {'AKGDH*sucA*sucB*R_g_b0116_and_g_b0726_and_g_
↪b0727': -1, 'PYK*pykA*R0_g_b1676_or_g_b1854*pykF*R1_g_b1676_or_g_b1854': -1,
↪'FRD7*frdD*frdC*frdB*frdA*R_g_b4151_and_g_b4152_and_g_b4153_and_g_b4154': -1, 'ME1*maeA
↪': -1, 'ME2*maeB': -1, 'AKGDC': 1, 'pgi': -1, 'kgtP': -1, 'tdcD*R0_g_b1849_or_g_b2296_
↪or_g_b3115*ackA*R1_g_b1849_or_g_b2296_or_g_b3115*purT*R2_g_b1849_or_g_b2296_or_g_b3115
↪': -1, 'gdhA': -1, 'gltP': -1, 'mdh': -1}
INFO:root:Finished solving strain design MILP.
INFO:root:1 solutions to MILP found.
INFO:root:  Decompressing.
INFO:root:  Preparing (reaction-)phenotype prediction of gene intervention strategies.
INFO:root:8 solutions found.
```

```
One compressed solution with cost 15.0 found and expanded to 8 solutions in the␣
↪uncompressed netork.
Example intervention set: ['+AKGDC', '-pgi', '-kgtP', '-gdhA', '-gltP', '-mdh', '-sucA',
↪'-frdD', '-maeA', '-maeB', '-tdcD', '-ackA', '-purT', '-pykA', '-pykF']
```



The plot shows the flux space of the designed strain. While at lower growth rates ethanol and succinate production is still possible, there is a growth-based selection against ethanol and succinate producers and in favor of 1,4-BDO producing phenotypes. The "waves" on top of the shape are artifacts of the plotting grid and would disappear with infinte resolution.

4.7 Standalone network compression

An effective network compression is essential to any strain design computation. Since it may also be of interest outside the context of strain design, this example may help you using the network compression routine independently. Likewise, StrainDesign also offers the integration of GPR rules into the metabolic networks as a separate function.

The network compression routine removes blocked reactions, removes conservation relations and then alternately lumps **coupled** reactions (compress_model_coupled, using a sparse integer RREF nullspace algorithm by default) and **parallel** reactions (compress_model_parallel). The compression returns a compressed network and a list of so-called "compression maps". Each map consists of a dictionary that contains complete information for reversing the compression steps successively and expand information obtained from the compressed model to the full model. Each entry of each map contains the id of a compressed reaction, associated with the original reaction names and their factor (provided as a rational number) with which they were lumped.

The default backend is backend='sparse_rref' (pure Python, no extra dependencies). A legacy Java-based backend backend='efmtool_rref' is also available via pip install straindesign[java].

Furthermore, the user can select reactions that should be exempt from the parallel compression. In the following, we provide the code snippet that can be used to call the compression.

[1]:
```
from straindesign import compress_model, remove_dummy_bounds
import cobra
import logging
```

(continues on next page)

(continued from previous page)

```
logging.basicConfig(level=logging.INFO)

# load model
iml1515 = cobra.io.load_model('iML1515')
# replace dummy bounds with infinite
logging.info(f"Replacing dummy bounds of +/- 1000 with infinite.")
remove_dummy_bounds(iml1515)
logging.info(f"The original model contains {len(iml1515.reactions)} reactions.")
cmp_map = compress_model(iml1515)
```

```
Set parameter Username
```

```
INFO:gurobipy:Set parameter Username
```

```
Academic license - for non-commercial use only - expires 2026-12-09
```

```
INFO:gurobipy:Academic license - for non-commercial use only - expires 2026-12-09
INFO:root:Replacing dummy bounds of +/- 1000 with infinite.
WARNING:straindesign.compression:Removing reaction bounds >= 1000.
INFO:root:The original model contains 2712 reactions.
INFO:straindesign.compression:  Removing blocked reactions.
INFO:straindesign.compression:  Converting coefficients to rationals.
INFO:straindesign.compression:  Removing conservation relations.
INFO:straindesign.compression:  Compression 1: Lumping coupled reactions.
INFO:straindesign.compression:Compression complete: 6 iterations, 234 zero-flux, 602␣
↪contradicting, 970 coupled, 1250 unused metabolites
INFO:straindesign.compression:  Reduced to 1237 reactions.
INFO:straindesign.compression:  Compression 2: Lumping parallel reactions.
INFO:straindesign.compression:  Reduced to 1217 reactions.
INFO:straindesign.compression:  Compression 3: Lumping coupled reactions.
INFO:straindesign.compression:Compression complete: 3 iterations, 0 zero-flux, 0␣
↪contradicting, 4 coupled, 2 unused metabolites
INFO:straindesign.compression:  Reduced to 1215 reactions.
INFO:straindesign.compression:  Compression 4: Lumping parallel reactions.
INFO:straindesign.compression:  Reduced to 1213 reactions.
INFO:straindesign.compression:  Compression 5: Lumping coupled reactions.
INFO:straindesign.compression:Compression complete: 3 iterations, 0 zero-flux, 0␣
↪contradicting, 0 coupled, 0 unused metabolites
INFO:straindesign.compression:  No further reduction (1213 reactions).
INFO:straindesign.compression:  Compression complete (4 iterations).
```

Here, the original model of iML1515 was compressed from 2712 to 1213 reactions in 4 steps. The greatest reduction in size is achieved in the first step. Still, the benefit of the parallel compression should not be underestimated since it removes superfluous degrees of freedom in the model that don't add significant information. The resulting compression maps can be used to translate data between the compressed and the original network.

[2]:
```
# Let us track the compression of the ADK4 reaction
orig_reac = 'ADK4'

for i,c in enumerate(cmp_map):
    logging.info(f"Compression step {i+1} was {['parallel' if c['parallel'] else 'linear
↪'][0]}.")
    lumped_reaction = [k for k,v in c['reac_map_exp'].items() if orig_reac in v][0]
    if len(c['reac_map_exp'][lumped_reaction]) == 1:
```

(continues on next page)

(continued from previous page)

```
        logging.info(f"Reaction '{orig_reac}' was not affected by this reaction")
    else:
        logging.info(f"Reaction '{orig_reac}' was lumped to {lumped_reaction} with the␣
↪coefficients {c['reac_map_exp'][lumped_reaction]}.")
    orig_reac = lumped_reaction
```

```
INFO:root:Compression step 1 was linear.
INFO:root:Reaction 'ADK4' was lumped to ADK4*NTP10 with the coefficients {'ADK4':␣
↪Fraction(1, 1), 'NTP10': Fraction(-1, 1)}.
INFO:root:Compression step 2 was parallel.
INFO:root:Reaction 'ADK4*NTP10' was not affected by this reaction
INFO:root:Compression step 3 was linear.
INFO:root:Reaction 'ADK4*NTP10' was not affected by this reaction
INFO:root:Compression step 4 was parallel.
INFO:root:Reaction 'ADK4*NTP10' was lumped to␣
↪ADK4*NTP10*NADPHXD*NADPHXE*NADPHHR*NADPHHS*NADHHR*NADHXE*NADHHS*NADHXD with the␣
↪coefficients {'ADK4*NTP10': 1, 'NADPHXD*NADPHXE*NADPHHR*NADPHHS': 1,
↪'NADHHR*NADHXE*NADHHS*NADHXD': 1}.
```

4.7.1 Standalone GPR-integraton

GPR rules can be introduced to the metabolic model in a way that the logical GPR-terms are reflected and the gene presence or absence can be simulated by setting flux bounds. This can be useful to investigate the space of feasible steady-state flux vectors after gene knockouts or be used to investigate the role of genes in different elementary flux modes or vectors.

[3]:
```
from straindesign import extend_model_gpr
import cobra
import logging
logging.basicConfig(level=logging.INFO)
# load model
e_coli_core = cobra.io.load_model('e_coli_core')
# extend model with GPR rules using gene-IDs
extend_model_gpr(e_coli_core)
# printing the last 10 reactions (corresponding to genes and GPR rules) of the GPR-
↪extend network
logging.info('The first 95 reactions are original reactions from e_coli_core. All other␣
↪reactions result from the integration of GPR rules.')
logging.info('Here, we only print the last 10 reactions of the model for showcasing.')
[print(r) for r in e_coli_core.reactions[-10:]];
```

```
INFO:root:The first 95 reactions are original reactions from e_coli_core. All other␣
↪reactions result from the integration of GPR rules.
INFO:root:Here, we only print the last 10 reactions of the model for showcasing.
```

```
R_g_b2276_and_g_b2277_and_g_b2278_and_g_b2279_and_g_b2280_and_g_b2281_and_g_b2282_and_g_
↪b2283_and_g_b2284_and_g_b2285_and_g_b2286_and_g_b2287_and_g_b2288: g_b2276 + g_b2277 +␣
↪g_b2278 + g_b2279 + g_b2280 + g_b2281 + g_b2282 + g_b2283 + g_b2284 + g_b2285 + g_
↪b2286 + g_b2287 + g_b2288 --> g_b2276_and_g_b2277_and_g_b2278_and_g_b2279_and_g_b2280_
↪and_g_b2281_and_g_b2282_and_g_b2283_and_g_b2284_and_g_b2285_and_g_b2286_and_g_b2287_
↪and_g_b2288
b3962:  --> g_b3962
R0_g_b1602_and_g_b1603_or_g_b3962: g_b3962 --> g_b1602_and_g_b1603_or_g_b3962
```

(continues on next page)

(continued from previous page)

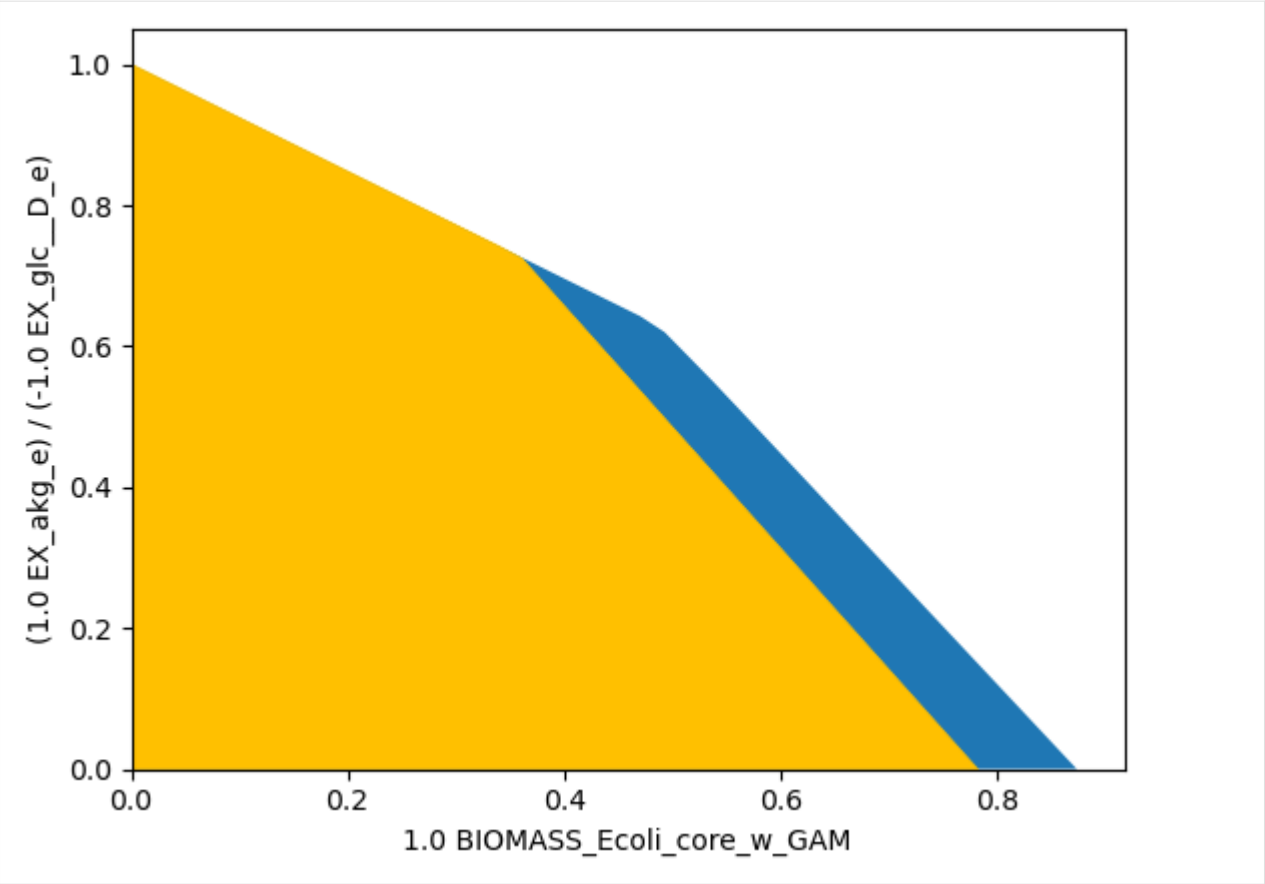
```
R1_g_b1602_and_g_b1603_or_g_b3962: g_b1602_and_g_b1603 --> g_b1602_and_g_b1603_or_g_b3962
b0451:  --> g_b0451
R0_g_b0451_or_g_s0001: g_s0001 --> g_b0451_or_g_s0001
R1_g_b0451_or_g_s0001: g_b0451 --> g_b0451_or_g_s0001
b0114:  --> g_b0114
b0115:  --> g_b0115
R_g_b0114_and_g_b0115_and_g_b0116: g_b0114 + g_b0115 + g_b0116 --> g_b0114_and_g_b0115_
↪and_g_b0116
```

4.7.1.1 Gene perturbation studies

GPR-extended models can be used to study how single or multiple gene-KOs affect the steady-state flux space. We can therefore integrate the GPR-rules in the model and then plot flux spaces that take into account knockouts. In the plots below, we show how knocking out the gene *lpd* affects bacterial growth and AKG yields.

[4]:
```
import straindesign as sd
import cobra
import logging
import matplotlib.pyplot as plt
logging.basicConfig(level=logging.INFO)
# load model
e_coli_core = cobra.io.load_model('e_coli_core')
# extend model with GPR rules using gene-names instead of IDs.
sd.extend_model_gpr(e_coli_core, use_names=True)

# Wild-type plot
datapoints, triang, plot1 = sd.plot_flux_space(e_coli_core,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_akg_e','-
↪EX_glc__D_e')),
                                               show=False);
# plotting designed strain
_,          _,      plot2 = sd.plot_flux_space(e_coli_core,
                                               ('BIOMASS_Ecoli_core_w_GAM',('EX_akg_e','-
↪EX_glc__D_e')),
                                               constraints=['lpd = 0'],
                                               show=False);
plot2.set_facecolor('#FFC000')
plot2.set_edgecolor('#FFC000')
# adjust axes limits and show plot
plot2.axes.set_xlim(0, 1.05*max([a[0] for a in datapoints]))
plot2.axes.set_ylim(0, 1.05*max([a[1] for a in datapoints]))
plt.show()
```



The blue shape represents the wild type and the yellow shape the *lpd* knockout strain flux space. The model predicts that the knockout will not affect the globally attainable -ketoglutarate yield, but reduces the maximal possible growth rate. At higher growth rates, -ketoglutarate yield is reduced.

4.8 CNApy interface

CNApy, a GUI-featured toolbox for metabolic modeling offers a graphical user interface for the modeling and design of metabolic networks. The software provides a dialog box for specifying strain design problems. A full documentation can be found here.

4.9 StrainDesign API

4.9.1 straindesign

StrainDesign package for computational metabolic engineering

4.9.1.1 Submodules

straindesign.compression

Metabolic network compression using nullspace-based coupling detection.

This module provides network compression for COBRA models. The main function is compress_cobra_model() which compresses a model and returns transformation matrices for converting between original and compressed flux spaces.

API:

```
>>> from straindesign.compression import compress_cobra_model
>>> result = compress_cobra_model(model)
>>> compressed_model = result.compressed_model
```

Note: The model should be preprocessed first (rational coefficients, conservation relations removed). Use network-tools.compress_model() for automatic preprocessing.

Module Contents

class straindesign.compression.**CompressionConverter**(*reaction_map: Dict[str, Dict[str, float | fractions.Fraction]]*, *metabolite_map: Dict[str, Dict[str, float | fractions.Fraction]]*, *flipped_reactions: List[str]*)

Bidirectional transformer for expressions between original and compressed spaces.

expand_expression(*expression: Dict[str, float]*, *remove_missing: bool = False*) → Dict[str, float]

Transform expression from compressed back to original space.

class straindesign.compression.**CompressionMethod**

Bases: enum.Enum

Compression methods for metabolic network compression.

classmethod standard() → List[*CompressionMethod*]

Standard compression methods (recommended).

class straindesign.compression.**CompressionRecord**(*pre:* RationalMatrix, *cmp:* RationalMatrix, *post:* RationalMatrix, *meta_names: List[str]*, *stats:* CompressionStatistics | *None = None*)

Compression result with transformation matrices.

Contains: pre @ stoich @ post == cmp For EFM expansion: efm_original = post @ efm_compressed

class straindesign.compression.**CompressionResult**(*compressed_model*, *compression_converter*, *pre_matrix*, *post_matrix*, *reaction_map*, *metabolite_map*, *statistics*, *methods_used*, *original_reaction_names*, *original_metabolite_names*, *flipped_reactions*)

Result of COBRA model compression.

class straindesign.compression.**CompressionStatistics**

Tracks compression statistics for logging.

class straindesign.compression.**RationalMatrix**(*rows: int*, *cols: int*)

Sparse rational matrix using dual int sparse storage (numerators + denominators).

add_scaled_column(*dst_col: int*, *src_col: int*, *scalar_num: int*, *scalar_den: int*) → None

Add scalar * column[src] to column[dst]. dst[i] += (num/den) * src[i]

begin_batch_edit()

Enter batch edit mode - delays cache invalidation.

clone() → *RationalMatrix*

Create a deep copy.

end_batch_edit()

Exit batch edit mode - converts back to CSR and invalidates cache.

classmethod from_cobra_model(*model*, *max_precision: int = 6*, *max_denom: int = 100*) → *RationalMatrix*

Create RationalMatrix from COBRA model stoichiometry.

classmethod from_numpy(*arr: numpy.ndarray*, *max_precision: int = 6*, *max_denom: int = 100*) → *RationalMatrix*

Create RationalMatrix from numpy array.

get_signum(*row: int*, *col: int*) → int

Return sign of element: -1, 0, or 1.

classmethod identity(*size: int*) → *RationalMatrix*

Create identity matrix.

iter_column_fractions(*col: int*) → Iterator[Tuple[int, fractions.Fraction]]

Iterate over non-zero entries in column as (row, Fraction) pairs.

remove_columns(*keep_indices: List[int]*) → None

Keep only the specified columns.

remove_rows(*keep_indices: List[int]*) → None

Keep only the specified rows.

submatrix(*rows: int*, *cols: int*) → *RationalMatrix*

Extract top-left submatrix of given dimensions.

to_numpy() → numpy.ndarray

Convert to numpy float array.

to_sparse_csr() → Tuple[scipy.sparse.csr_matrix, int]

Return sparse representation and common denominator.

Returns numerator matrix scaled by LCM of denominators, plus the LCM.

to_sparse_pattern() → Tuple[scipy.sparse.csr_matrix, Dict[int, Dict[int, fractions.Fraction]]]

Return sparsity pattern as CSR and row-wise Fraction data.

For pattern analysis (coupled reaction detection) without integer overflow. :returns: CSR matrix with 1s at non-zero positions (for indptr/indices)

row_data: {row: {col: Fraction}} for actual values

Return type
pattern

class straindesign.compression.**StoichMatrixCompressor**(**methods:* CompressionMethod)

Nullspace-based metabolic network compression.

compress(*stoich:* RationalMatrix, *meta_names: List[str]*, *reac_names: List[str]*, *suppressed: Set[str] | None = None*, *bounds: List[Tuple[float, float]] | None = None*) → *CompressionRecord*

Compress network, return transformation matrices.

Single-pass approach: each iteration computes the nullspace once, then removes zero-flux reactions AND merges coupled groups from the same kernel. Re-iterates only when contradicting groups were removed (which may expose new couplings).

straindesign.compression.**basic_columns**(*matrix:* RationalMatrix) → List[int]

Find indices of basic (pivot) columns using integer RREF.

straindesign.compression.**basic_columns_from_numpy**(*mx: numpy.ndarray*) → List[int]

Find basic columns from a numpy array.

straindesign.compression.**compress_cobra_model**(*model*, *methods: List[str | CompressionMethod] | None = None*, *in_place: bool = True*, *suppressed_reactions: Set[str] | None = None*) → *CompressionResult*

Compress a COBRA model using nullspace-based coupling detection.

Note: Model should be preprocessed first (rational coefficients, conservation relations removed). Use network-tools.compress_model() for automatic preprocessing.

Parameters

- **model** – COBRA model to compress (should be preprocessed)
- **methods** – Compression methods. Default: CompressionMethod.standard()
- **in_place** – Modify original model (True) or work on copy (False)
- **suppressed_reactions** – Reaction names to exclude from compression

Returns

CompressionResult with compressed model and transformation data

straindesign.compression.**compress_model**(*model*, *no_par_compress_reacs=set()*, *compression_backend='sparse_rref'*, *propagate_gpr=False*)

Compress a metabolic model using multiple techniques.

Performs blocked reaction removal, conservation relation removal, and alternating dependent/parallel reaction lumping until no further compression is possible.

Parameters

- **model** – COBRA model to compress in-place
- **no_par_compress_reacs** – Reactions exempt from parallel compression
- **compression_backend** – Compression backend to use: - 'sparse_rref' (default): Pure Python sparse integer RREF.

  No external dependencies beyond NumPy/SciPy.

  – 'efmtool_rref' (legacy): Java-based EFMTool via JPype. Requires a JVM and the jpype1 package.

- **propagate_gpr** – If True, propagate and simplify GPR rules through compression (AND for coupled, OR for parallel merges). Empty GPR rules are correctly handled: skipped in

AND (always active), and absorb in OR (result is always active). Uses sympy for boolean simplification. Default False.

Returns
Compression maps for reversing each compression step

Return type
list of dict

straindesign.compression.**compress_model_coupled**(*model*, *compression_backend='sparse_rref'*, *propagate_gpr=False*, *suppressed_reactions=None*)

Compress by lumping stoichiometrically coupled (dependent) reactions.

Identifies groups of reactions whose flux vectors are proportional in every steady state (i.e. they share a common nullspace direction) and merges each group into a single lumped reaction. Both the pure-Python and legacy Java backends perform this operation; the compression_backend controls the nullspace algorithm.

Parameters

- **model** – COBRA model to compress in-place
- **compression_backend** – 'sparse_rref' (default, Python) or 'efmtool_rref' (Java legacy)
- **propagate_gpr** – If True, AND-combine GPR rules of merged reactions (with sympy simplification). Empty GPRs are skipped. Default False.
- **suppressed_reactions** – Set of reaction IDs to exclude from compression (Java backend only). Used to protect reactions referenced in strain design constraints from being deleted by the Java compressor's CoupledContradicting logic. Ignored for the Python backend (which handles contradicting groups correctly via bounds intersection).

Returns
Mapping {compressed_id: {orig_id: factor, ...}}

Return type
dict

straindesign.compression.**compress_model_parallel**(*model*, *protected_rxns=set()*, *propagate_gpr=False*)

Compress by lumping parallel reactions.

Parameters

- **model** – COBRA model to compress in-place
- **protected_rxns** – Reactions exempt from parallel compression
- **propagate_gpr** – If True, OR-combine GPR rules of lumped reactions (with sympy simplification). Default False.

Returns
Mapping {compressed_id: {orig_id: factor, ...}}

Return type
dict

straindesign.compression.**detect_max_precision**(*model*) → int

Detect maximum decimal precision needed for model coefficients.

straindesign.compression.**float_to_rational**(*val*, *max_precision: int = 6*, *max_denom: int = 100*) → fractions.Fraction

Convert float to Fraction with bounded denominators.

straindesign.compression.**nullspace**(*matrix:* RationalMatrix) → *RationalMatrix*

Compute right nullspace (kernel). Returns K where matrix @ K = 0.

Uses integer RREF with column/row pre-sorting and GCD reduction. All arithmetic is Python arbitrary-precision integers — no overflow possible.

Parameters
matrix – Input RationalMatrix

Returns
Kernel matrix K where matrix @ K = 0

straindesign.compression.**remove_blocked_reactions**(*model*) → List

Remove blocked reactions (bounds == (0, 0)) from a network.

straindesign.compression.**remove_conservation_relations**(*model*) → None

Remove conservation relations (dependent metabolites) from a model.

This reduces the number of metabolites while maintaining the original flux space. Uses exact rational arithmetic to find linearly independent rows.

Parameters
model – COBRA model to modify in-place

straindesign.compression.**remove_dummy_bounds**(*model*) → None

Replace COBRA standard bounds with +/-inf.

straindesign.compression.**remove_ext_mets**(*model*) → None

Remove external metabolites from 'External_Species' compartment.

straindesign.compression.**stoichmat_coeff2float**(*model*) → None

Convert stoichiometric coefficients to floats.

straindesign.compression.**stoichmat_coeff2rational**(*model*) → None

Convert stoichiometric coefficients to rational numbers.

straindesign.compute_strain_designs

Function: computing metabolic strain designs (compute_strain_designs)

Module Contents

straindesign.compute_strain_designs.**compute_strain_designs**(*model: cobra.Model*, ***kwargs: dict*) → straindesign.SDSolutions

Computes strain designs for a user-defined strain design problem

A number of arguments can be specified to detail the problem and influence the solution process. This function supports the computation of Minimal Cut Sets (MCS), OptKock, RobustKnock and OptCouple strain designs. It is possible to combine any of the latter ones with the MCS approach, e.g., to engineer growth coupled production, but also suppress the production of an undesired by-product. The computation can be started in two different ways. Either by specifying the computation parameters indivdually or reuse a parameters dictionary from a previous computation. CNApy stores strain design setup dics as JSON ".sd"-files that can be loaded in python and used as an input for this function.

Example

sols = compute_strain_designs(model, sd_modules=[sd_module1, sd_module2], solution_approach = 'any')

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class. The model may or may not contain genes/GPR-rules.
- **sd_setup** (*dict*) – sd_setup should be a dictionary containing a set of parameters for strain design computation. The allowed keywords are the same listed hereafter. Therefore, *sd_setup and other arguments (except for model) must not be used together*.
- **sd_modules** (*[straindesign.SDModule]*) – List of strain design modules that describe the sub-problems, such as the MCS-like protection or suppression of flux subspaces or the OptKnock, RobustKnock or OptCouple objective and constraints. The list of modules determines the global objective function of the strain design computation. If only SUPPRESS and PROTECT modules are used, the strain design computation is MCS-like, such that the number of interventions is minimized. If a module for one of the nested optimization approaches is used, the global objective function is retrieved from this module. The number of SUPPRESS and PROTECT modules is unrestricted and can be combined with the other modules, however only one of the modules OPTKNOCK, ROBUSKNOCK and OPTCOUPLE may be used at a time. For details, see SDModule.
- **solver** (*optional (str)*) – (Default: same as defined in model / COBRApy) The solver that should be used for preparing and carrying out the strain design computation. Allowed values are 'cplex', 'gurobi', 'scip' and 'glpk'.
- **max_cost** (*optional (int)*) – (Default: inf): The maximum cost threshold for interventions. Every possible intervention is associated with a cost value (1, by default). Strain designs cannot exceed the max_cost threshold. Individual intervention cost factors may be defined through ki_cost, ko_cost, gki_cost, gko_cost and reg_cost.
- **max_solutions** (*optional (int)*) – (Default: inf) The maximum number of MILP solutions that are generated for a strain design problem. The number of returned strain designs is usually larger than the number of max_solutions, since a MILP solution is decompressed to multiple strain designs. When the compress-flag is set to 'False' the number of returned solutions is equal to max_solutions.
- **M** (*optional (int)*) – (Default: None) If this value is specified (and non-zero, not None), the computation uses the big-M method instead of indicator constraints. Since GLPK does not support indicator constraints it uses the big-M method by default (with M=1000). M should be chosen 'sufficiently large' to avoid computational artifacts and 'sufficiently small' to avoid numerical issues.
- **compress** (*optional (bool)*) – (Default: True) If 'True', the interative network compressor is used.
- **gene_kos** (*optional (bool)*) – (Default: False) If 'True', strain designs are computed based on gene-knockouts instead of reaction knockouts. This parameter needs not be defined if any of ki_cost, ko_cost, gki_cost, gko_cost and reg_cost is used. By default, reactions are considered as knockout targets.
- **ko_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated knockout costs. If not specified, all reactions are treated as knockout candidates, equivalent to ko_cost = {'r1':1, 'r2':1, ...}. If a subset of reactions is listed in the dict, all other are not considered as knockout candidates.
- **ki_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated costs for addition. If not specified, all reactions are treated as knockout can-

didates. Reaction addition candidates must be present in the original model with the intended flux boundaries **after** insertion. Additions are treated adversely to knockouts, meaning that their exclusion from the network is not associated with any cost while their presence entails intervention costs.

- **gko_cost** (*optional (dict)*) – (Default: None) A dictionary of gene identifiers and their associated knockout costs. To reference genes, gene IDs can be used,as well as gene names. If not specified, genes are not treated as knockout candidates. An exception is the 'gene_kos' argument. If 'gene_kos' is used, all genes are treated as knockout candidates with intervention costs of 1. This is equivalent to gko_cost = {'g1':1, 'g2':1, ...}.
- **gki_cost** (*optional (dict)*) – (Default: None) A dictionary of gene identifiers and their associated addition costs. To reference genes, gene IDs can be used, as well as gene names. If not specified, none of the genes are treated as addition candidates.
- **reg_cost** (*optional [dict]*) – (Default: None) Regulatory interventions candidates can be optionally specified as a list. Thereby, the constraint marking the regulatory intervention is put as key and the associated intervention cost is used as the corresponding value. E.g., reg_cost = {'1 EX_o2_e = -1': 1, ... <other regulatory interventions>}. Instead of strings, constraints can also be passed as lists. reg_cost = {['EX_o2_e':1}, '=', -1]: 1, ...}
- **solution_approach** (*optional (str)*) – (Default: 'best') The approach used to find strain designs. Possible values are 'any', 'best' or 'populate'. 'any' is usually the fastest option, since optimality is not enforced. Hereby computed MCS are still irreducible intervention sets, however, not MCS with the fewest possible number of interventions. 'best' computes globally optimal strain designs, that is, MCS with the fewest number of interventions, OptKnock strain designs with the highest possible production rate, OptCouple strain designs with the hightest growth coupling potential etc.. 'populate' does the same as 'best', but makes use of CPLEX' and Gurobi's populate function to generate multiple strain designs. It is identical to 'best' when used with SCIP or GLPK. Attention: If 'any' used with OptKnock, for instance, the MILP may return the wild type as a possible immediately. Technically, the wiltype fulfills the criterion of maximal growth (inner objective) and maximality of the global objective is omitted by using 'any', so that carrying no product synthesis is permitted. Additional constraints can be used in the OptKnock problem to circumvent this. However, Optknock should generally be used with the 'best' option.
- **time_limit** (*optional (int)*) – (Default: inf) The time limit in seconds for the MILP-solver.
- **advanced** (*optional (bool)*) – Dummy parameters used for the CNApy interface.
- **use_scenario** (*optional (bool)*) – Dummy parameters used for the CNApy interface.

Returns

An object that contains all computed strain designs. If strain designs were computed as gene-interventions, the solution object contains a set of corresponding reaction-interventions that facilitate the analysis of the computed strain designs with COBRA methods.

Return type

(*SDSolutions*)

straindesign.compute_strain_designs.**compute_strain_designs_from_preprocessed**(*dump*, *seed=None*, *solver=None*, *solution_approach=None*, *max_solutions=None*, *time_limit=None*)

Load preprocessed model and run MILP solve with optional overrides.

Parameters

- **dump** (*str or dict*) – Path to pickle file created by dump_preprocessed, or the dict itself (e.g. after unpickling and modifying).
- **seed** (*int, optional*) – Override MILP solver seed.
- **solver** (*str, optional*) – Override solver.
- **solution_approach** (*str, optional*) – Override solution approach ('any', 'best', 'populate').
- **max_solutions** (*int or float, optional*) – Override max_solutions.
- **time_limit** (*int or float, optional*) – Override time limit.

Returns
Strain design solutions.

Return type
(*SDSolutions*)

straindesign.compute_strain_designs.**postprocess_reg_sd**(*reg_cost*, *sd*)

Postprocess regulatory interventions

Mark regulatory interventions with true or false

straindesign.cplex_interface

CPLEX solver interface for LP and MILP

Module Contents

class straindesign.cplex_interface.**Cplex_MILP_LP**(*c=None*, *A_ineq=None*, *b_ineq=None*, *A_eq=None*, *b_eq=None*, *lb=None*, *ub=None*, *vtype=None*, *indic_constr=None*, *seed=None*, *milp_threads=None*)

Bases: cplex.Cplex

CPLEX interface for MILP and LP

This class is a wrapper for the CPLEX-Python API to offer bindings and namings for functions for the construction and manipulation of MILPs and LPs in an vector-matrix-based manner that are consistent with those of the other solver interfaces in the StrainDesign package. The purpose is to unify the instructions for operating with MILPs and LPs throughout StrainDesign.

The CPLEX interface provides support for indicator constraints as well as for the populate function.

Accepts a (mixed integer) linear problem in the form:
minimize(c), subject to: A_ineq * x <= b_ineq, A_eq * x = b_eq, lb <= x <= ub, forall(i) type(x_i) = vtype(i) (continous, binary, integer), indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

cplex = Cplex_MILP_LP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub, vtype, indic_constr)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).

- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.
- **ub** (*list of float*) – (Default: None) The upper variable bounds.
- **vtype** (*str*) – (Default: None) A character string that specifies the type of each variable: 'c'ontinous, 'b'inary or 'i'nteger
- **indic_constr** (IndicatorConstraints) – (Default: None) A set of indicator constraints stored in an object of IndicatorConstraints (see reference manual or docstring).
- **seed** (*int16*) – (Default: None) An integer value serving as a seed to make MILP solving reproducible.
- **Returns** – (Cplex_MILP_LP):

  A CPLEX MILP/LP interface class.

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_ineq** (*list of float*) – The right hand side vector

get_basis()

Return the current LP basis (variable and constraint statuses).

Returns

dict with 'vbasis' (list of int) and 'cbasis' (list of int). CPLEX codes: 0=free/superbasic, 1=basic, 2=at ub, 3=at lb.

get_lp_method()

Return the current LP method as a solver-neutral string.

populate(*n*) → Tuple[List, float, float]

Generate a solution pool for MILPs

Example

sols_x, optim, status = cplex.populate()

Returns

(Tuple[List of lists, float, float])

solution_vectors, optimal_value, optimization_status

set_basis(*basis*)

Load a previously saved basis for warm-starting.

Parameters

basis – dict from get_basis() with 'vbasis' and 'cbasis'.

set_ineq_constraint(*idx*, *a_ineq*, *b_ineq*)

Replace a specific inequality constraint

Replace the constraint with the index idx with the constraint a_ineq*x ~ b_ineq

Parameters

- **idx** (*int*) – Index of the constraint
- **a_ineq** (*list of float*) – The coefficient vector
- **b_ineq** (*float*) – The right hand side value

set_lp_method(*method*)

Set the LP solving method.

Parameters

method – LP_METHOD_AUTO, LP_METHOD_PRIMAL, LP_METHOD_DUAL, or LP_METHOD_BARRIER

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

Set the computation time limit (in seconds)

set_ub(*ub*)

Set the upper bounds to a given vector

slim_solve() → float

Solve the MILP or LP, but return only the optimal value

Example

optim = cplex.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the MILP or LP

Example

sol_x, optim, status = cplex.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

straindesign.efmtool_cmp_interface

EFMtool compression interface for straindesign.

This module provides compression utilities for metabolic networks. By default, the pure Python 'sparse_rref' compression backend is used. The Java EFMTool backend is available via compression_backend='efmtool_rref' (requires jpype1).

For the documentation of the compression API provided by StrainDesign, refer to straindesign.compression.compress_model.

Module Contents

straindesign.efmtool_cmp_interface.**basic_columns_rat**(*mx*, *tolerance=0*)

Find basic columns using exact rational arithmetic (FLINT or sympy).

straindesign.efmtool_cmp_interface.**basic_columns_rat_java**(*mx*, *tolerance=0*)

Find basic columns using Java Gaussian elimination.

Legacy implementation using jpype and Java efmtool. Requires jpype1 and sympy to be installed.

Parameters

- **mx** – Matrix (numpy array or Java matrix)
- **tolerance** – Tolerance (unused in exact arithmetic)

Returns

Array of indices of basic columns

straindesign.efmtool_cmp_interface.**compress_model_java**(*model*, *suppressed_reactions=None*)

Legacy Java compression using jpype (requires jpype and sympy).

Parameters

- **model** – COBRA model (will be modified in place)
- **suppressed_reactions** – Set of reaction IDs to exclude from compression. These reactions are kept as standalone entries with identity mapping. Used to protect reactions referenced in strain design constraints from being deleted by the Java compressor's CoupledContradicting logic.

Returns

Reaction map from compressed to original reactions with scaling factors

Return type

dict

straindesign.efmtool_cmp_interface.**jBigFraction2sympyRat**(*val*)

Convert Java BigFraction to sympy Rational (requires Java init).

straindesign.efmtool_cmp_interface.**jBigIntegerPair2sympyRat**(*numer*, *denom*)

Convert Java BigInteger pair to sympy Rational (requires sympy).

straindesign.efmtool_cmp_interface.**jpypeArrayOfArrays2numpy_mat**(*jmat*)

Convert jpype array of arrays to numpy matrix.

straindesign.efmtool_cmp_interface.**numpy_mat2jpypeArrayOfArrays**(*npmat*)

Convert numpy matrix to jpype array of arrays (requires Java init).

straindesign.efmtool_cmp_interface.**sympyRat2jBigIntegerPair**(*val*)

Convert Fraction or sympy Rational to Java BigInteger pair (requires Java init).

straindesign.glpk_interface

GLPK solver interface for LP and MILP

Module Contents

class straindesign.glpk_interface.**GLPK_MILP_LP**(*c=None*, *A_ineq=None*, *b_ineq=None*, *A_eq=None*, *b_eq=None*, *lb=None*, *ub=None*, *vtype=None*, *indic_constr=None*, *M=None*)

GLPK interface for MILP and LP

This class is a wrapper for the GLPK-Python API to offer bindings and namings for functions for the construction and manipulation of MILPs and LPs in an vector-matrix-based manner that are consistent with those of the other solver interfaces in the StrainDesign package. The purpose is to unify the instructions for operating with MILPs and LPs throughout StrainDesign.

The GLPK interface does not natively support indicator constraints. They are hence translated to bigM-constraints when passed to the GLPK constructor (see docstring of IndicatorConstraints). The GLPK interface does not natively support the populate function. A high level implementation emulates the behavior of populate.

Accepts a (mixed integer) linear problem in the form:

minimize(c), subject to: A_ineq * x <= b_ineq, A_eq * x = b_eq, lb <= x <= ub, forall(i) type(x_i) = vtype(i) (continous, binary, integer), indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

glpk = GLPK_MILP_LP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub, vtype, indic_constr, M)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).
- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.
- **ub** (*list of float*) – (Default: None) The upper variable bounds.

- **vtype** (*str*) – (Default: None) A character string that specifies the type of each variable: 'c'ontinous, 'b'inary or 'i'nteger
- **indic_constr** (IndicatorConstraints) – (Default: None) A set of indicator constraints stored in an object of IndicatorConstraints. To make GLPK compatible with indicator constraints, they are translated into bigM-constraints (see reference manual or docstring of IndicatorConstraints).
- **M** (*int*) – (Default: None) A large value that is used in the translation of indicator constraints to bigM-constraints. If no value is provided, 1000 is used.
- **Returns** – (GLPK_MILP_LP):

  A GLPK MILP/LP interface class.

addExclusionConstraintsIneq(*x*)

Function to add exclusion constraint (GLPK compatibility function)

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_ineq** (*list of float*) – The right hand side vector

getSolution(*status*) → list

Retrieve solution from GLPK backend

get_basis()

Return the current LP basis (column and row statuses).

Returns

dict with 'vbasis' (list of int) and 'cbasis' (list of int). GLPK codes: 1=basic, 2=at lb, 3=at ub, 4=free, 5=fixed.

get_lp_method()

Return the current LP method as a solver-neutral string.

populate(*pool_limit*) → Tuple[List, float, float]

Generate a solution pool for MILPs

This is only a high-level implementation of the populate function. There is no native support in GLPK.

Example

sols_x, optim, status = glpk.populate()

Returns

(Tuple[List of lists, float, float])

solution_vectors, optimal_value, optimization_status

set_basis(*basis*)

Load a previously saved basis for warm-starting.

Parameters

basis – dict from get_basis() with 'vbasis' and 'cbasis'.

set_ineq_constraint(*idx*, *a_ineq*, *b_ineq*)

Replace a specific inequality constraint

Replace the constraint with the index idx with the constraint a_ineq*x ~ b_ineq

Parameters

- **idx** (*int*) – Index of the constraint
- **a_ineq** (*list of float*) – The coefficient vector
- **b_ineq** (*float*) – The right hand side value

set_lp_method(*method*)

Set the LP solving method.

Parameters

method – LP_METHOD_AUTO, LP_METHOD_PRIMAL, LP_METHOD_DUAL, or LP_METHOD_BARRIER

Note: GLPK does not support barrier. LP_METHOD_BARRIER falls back to dual.

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

Set the computation time limit (in seconds)

set_ub(*ub*)

Set the upper bounds to a given vector

slim_solve() → float

Solve the MILP or LP, but return only the optimal value

Example

optim = glpk.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the MILP or LP

Example

sol_x, optim, status = glpk.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

solve_MILP_LP() → Tuple[float, int, bool]

Trigger GLPK solution through backend

straindesign.gurobi_interface

Gurobi solver interface for LP and MILP

Module Contents

class straindesign.gurobi_interface.**Gurobi_MILP_LP**(*c=None*, *A_ineq=None*, *b_ineq=None*, *A_eq=None*, *b_eq=None*, *lb=None*, *ub=None*, *vtype=None*, *indic_constr=None*, *seed=None*, *milp_threads=None*)

Bases: gurobipy.Model

Gurobi interface for MILP and LP

This class is a wrapper for the Gurobi-Python API to offer bindings and namings for functions for the construction and manipulation of MILPs and LPs in an vector-matrix-based manner that are consistent with those of the other solver interfaces in the StrainDesign package. The purpose is to unify the instructions for operating with MILPs and LPs throughout StrainDesign.

The Gurobi interface provides support for indicator constraints as well as for the populate function.

Accepts a (mixed integer) linear problem in the form:

minimize(c), subject to: A_ineq * x <= b_ineq, A_eq * x = b_eq, lb <= x <= ub, forall(i) type(x_i) = vtype(i) (continous, binary, integer), indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

gurobi = Gurobi_MILP_LP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub, vtype, indic_constr)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).
- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.

- **ub** (*list of float*) – (Default: None) The upper variable bounds.
- **vtype** (*str*) – (Default: None) A character string that specifies the type of each variable: 'c'ontinous, 'b'inary or 'i'nteger
- **indic_constr** (IndicatorConstraints) – (Default: None) A set of indicator constraints stored in an object of IndicatorConstraints (see reference manual or docstring).
- **seed** (*int16*) – (Default: None) An integer value serving as a seed to make MILP solving reproducible.
- **Returns** – (Gurobi_MILP_LP):

  A Gurobi MILP/LP interface class.

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_ineq** (*list of float*) – The right hand side vector

getSolution() → list

Retrieve solution from Gurobi backend

getSolutions() → list

Retrieve solution pool from Gurobi backend

get_basis()

Return the current LP basis (variable and constraint statuses).

Returns

dict with 'vbasis' (list of int) and 'cbasis' (list of int). Gurobi codes: -1=superbasic, 0=basic, -2=at lb, -3=at ub.

get_lp_method()

Return the current LP method as a solver-neutral string.

populate(*n*) → Tuple[List, float, float]

Generate a solution pool for MILPs

Example

sols_x, optim, status = cplex.populate()

Returns

(Tuple[List of lists, float, float])

solution_vectors, optimal_value, optimization_status

set_basis(*basis*)

Load a previously saved basis for warm-starting.

Parameters

basis – dict from get_basis() with 'vbasis' and 'cbasis'.

set_ineq_constraint(*idx*, *a_ineq*, *b_ineq*)

Replace a specific inequality constraint

Replace the constraint with the index idx with the constraint a_ineq*x ~ b_ineq

Parameters

- **idx** (*int*) – Index of the constraint
- **a_ineq** (*list of float*) – The coefficient vector
- **b_ineq** (*float*) – The right hand side value

set_lp_method(*method*)

Set the LP solving method.

Parameters

method – LP_METHOD_AUTO, LP_METHOD_PRIMAL, LP_METHOD_DUAL, or LP_METHOD_BARRIER

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

Set the computation time limit (in seconds)

set_ub(*ub*)

Set the upper bounds to a given vector

slim_solve() → float

Solve the MILP or LP, but return only the optimal value

Example

optim = gurobi.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the MILP or LP

Example

sol_x, optim, status = gurobi.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

straindesign.indicatorConstraints

Class for indicator contraints (IndicatorConstraints)

Module Contents

class straindesign.indicatorConstraints.**IndicatorConstraints**(*binv*, *A*, *b*, *sense*, *indicval*)

A class for storing indicator contraints

This class is a container for indicator constraints. Indicator constraints are used to link the fulfillment of a constraint to an indicating variable. For instance the indicator constraint: z = 1 -> 2*d - 1*e <= 3 can be described as: If z=1, then 2*d - 1*e <= 3 An alternative formulation of this association is possible with a bigM constraint: 2*d - 1*e - z*M <= 3, with M = very large The constraint z = 0 -> 2*d - 1*e <= 3 would translate to 2*d - 1*e + z*M <= 3 + M Generally, indicator constraints are preferred over bigM, because they provide better numerical stability. However, not all solvers support indicator constraints

Indicator constraints have the form: x_binv = indicval -> a * x <sense> b e.g.,: x_35 = 1 -> 2 * x_2 + 3 *x_3 'L' 6

(<=)

This class contains a set of indicator constraints: x_binv_1 = indicval_1 -> A_1 * x <sense_1> b_1 x_binv_2 = indicval_2 -> A_2 * x <sense_2> b_2 ...

Example

ic = IndicatorConstraints(binv, A, b, sense, indicval)

Parameters

- **binv** (*list of int*) – (e.g.: [25, 27, 30]) The index of the binary, indicating variables (indicators) for all indicator constraints. Integers are allowed ot occur more than once.
- **A** (*sparse.csr_matrix*) – Coefficient vectors for all indicator constraints, stored in one matrix, whereas each row is used in one indicator constraint. (num_columns = number of variables, num_rows = number of indicator constraints)
- **b** (*list of float*) – Right hand sides of all indicator constraints. (e.g.: [0.1, 2.0, 3])
- **sense** (*str*) – (In)equality signs for all indicator constraints. 'L'ess or equal, 'E'qual or, 'G'reater or equal (e.g.: 'EEGLGLGE')
- **indicval** (*list of int*) – Indicator values for all indicator constraints. Which value of the indicator enforces the constraint, 0 or 1? (e.g., 0001010110)

Returns
An object of the IndicatorConstraints class to pass indicator constraints.

Return type
(*IndicatorConstraints*)

straindesign.lptools

A collection of functions for the LP-based analysis of metabolic networks

Module Contents

straindesign.lptools.**ceil_dec**(*v*, *n*)

Round up v to n decimals

straindesign.lptools.**fba**(*model*, ***kwargs*) → cobra.core.Solution

Flux Balance Analysis (FBA), parsimonius Flux Balance Analysis (pFBA),

Flux Balance Analysis optimizes a *linear objective function* in a space of steady-state flux vectors given by a constraint-based metabolic model. FVA is often used to determine the (stoichiometrically) maximal possible growth rate, or flux rate towards a particular product. This FBA function allows to us a custom objective function and sense and allows the user to narrow down the flux states with additional constraints. In addition, one may use different types of parsimonious FBAs to either reduce the total sum of fluxes or the total number of active reactions after the primary objective is optimized.

Example

optim = fba(model, constraints='EX_o2_e=0', solver='gurobi', pfba=1)

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class. If no custom objective function is provided, the model's objective function is retrieved from the fields model.reactions[i].objective_coefficient.
- **solver** (*optional (str)*) – The solver that should be used for FBA.
- **constraints** (*optional (str) or (list of str) or (list of [dict,str, float])*) – (Default: '') List of *linear* constraints to be applied on top of the model: signs + or -, scalar factors for reaction rates, inclusive (in)equalities and a float value on the right hand side. The parsing of the constraints input allows for some flexibility. Correct (and identical) inputs are, for instance: constraints='-EX_o2_e <= 5, ATPM = 20' or constraints=['-EX_o2_e <= 5', 'ATPM = 20'] or constraints=[[{'EX_o2_e':-1},'<=',5], [{'ATPM':1},'=',20]]
- **obj** (*optional (str) or (dict)*) – As a custom objective function, any linear expression can be used, either provided as a single string or as a dict. Correct (and identical) inputs are, for instance: inner_objective='BIOMASS_Ecoli_core_w_GAM' inner_objective={'BIOMASS_Ecoli_core_w_GAM': 1}
- **obj_sense** (*optional (str)*) – (Default: 'maximize') The optimization direction can be set either to 'maximize' (or 'max') or 'minimize' (or 'min').
- **pfba** (*optional (int)*) – (Default: 0) The level of parsimonious FBA that should be applied. 0: no pFBA, only optmize the primary objective, 1: minimize sum of fluxes after the primary objective is optimized, 2: minimize the number of active reactions after the primary objective is optimized.

Returns

A solution object that contains the objective value, an optimal flux vector and the optmization status.

Return type

(cobra.core.Solution)

straindesign.lptools.**floor_dec**(*v*, *n*)

Round down v to n decimals

straindesign.lptools.**fva**(*model*, ***kwargs*) → pandas.DataFrame

Flux Variability Analysis (FVA)

Flux Variability Analysis determines the global flux ranges of reactions by minimizing and maximizing the flux through all reactions of a given metabolic network. This FVA function additionally allows the user to narrow down the flux states with additional constraints.

Uses an accelerated two-phase approach: global scan LPs with dual simplex warm-start resolve ~50% of bounds cheaply, then individual LPs for the rest. Large models (>= 200 reactions) are automatically compressed via coupled reaction lumping. Multiprocessing is used when >= 1000 reactions and cobra.Configuration().processes > 1.

Example

flux_ranges = fva(model, constraints='EX_o2_e=0', solver='gurobi')

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **solver** (*optional (str)*) – The solver that should be used for FVA.
- **constraints** (*optional (str) or (list of str) or (list of [dict,str, float])*) – (Default: '') List of *linear* constraints to be applied on top of the model: signs + or -, scalar factors for reaction rates, inclusive (in)equalities and a float value on the right hand side. The parsing of the constraints input allows for some flexibility. Correct (and identical) inputs are, for instance: constraints='-EX_o2_e <= 5, ATPM = 20' or constraints=['-EX_o2_e <= 5', 'ATPM = 20'] or constraints=[[{'EX_o2_e':-1},'<=',5], [{'ATPM':1},'=',20]]

Returns
A data frame containing the minimum and maximum attainable flux rates for all reactions.

Return type
(pandas.DataFrame)

straindesign.lptools.**fva_legacy**(*model*, ***kwargs*) → pandas.DataFrame

Legacy FVA implementation (brute-force 2*n LPs, no scan/compression).

Kept as fallback for debugging. Use fva() for production.

straindesign.lptools.**fva_worker_compute**(*i*) → Tuple[int, float]

Helper function for parallel FVA

Run a single LP as a step of FVA. Is executed on workers, not on main thread.

Parameters
i (*int*) – Index of the computation step.

straindesign.lptools.**fva_worker_compute_glpk**(*i*) → Tuple[int, float]

Helper function for parallel FVA

Run a single LP for GLPK as a step of FVA. Is executed on workers, not on main thread.

Parameters
i (*int*) – Index of the computation step.

straindesign.lptools.**fva_worker_init**(*A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*, *solver*)

Helper function for parallel FVA

Initialize the LP that will be solved iteratively. Is executed on workers, not on main thread.

Parameters

- **A_ineq** – The LP.
- **b_ineq** – The LP.
- **A_eq** – The LP.
- **b_eq** – The LP.
- **lb** – The LP.
- **ub** – The LP.
- **solver** (*str*) – Solver to be used.

straindesign.lptools.**fva_worker_init_glpk**(*A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*)

Helper function for parallel FVA

Initialize the LP for GLPK that will be solved iteratively. Is executed on workers, not on main thread.

Parameters

- **A_ineq** – The LP.
- **b_ineq** – The LP.
- **A_eq** – The LP.
- **b_eq** – The LP.
- **lb** – The LP.
- **ub** – The LP.

straindesign.lptools.**idx2c**(*i*, *prev*) → list

Helper function for parallel FVA

Builds the objective function for minimizing or maximizing the flux through the reaction with the index floor(i / 2). If i is even, there is a maximization.

Parameters

- **i** (*float*) – An index between 0 and 2*num_reacs.
- **prev** (*optional (str)*) – Index of the previously optimized reaction.

Returns
An optimization vector.

Return type
(list)

straindesign.lptools.**plot_flux_space**(*model*, *axes*, ***kwargs*) → Tuple[list, list, list]

Plot projections of the space of steady-state flux vectors onto two or three dimensions.

This function uses LP and matplotlib to generate lower dimensional representations of the flux space. Custom *linear* or *fractional-linear* expressions can be used for the plot axis. The most commonly used flux space reprentations are the *production envelope* that plots the growth rate (x) vs the product synthesis rate (y) and the *yield space plot* that plots the biomass yield (x) vs the product yiel (y). One may specify additional constraints to investigate subspaces of the metabolic steady-state flux space.

Example

plot_flux_space(model,('BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'))

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **axes** (*(list of lists) or (list of str)*) – A set of linear expressions that specify which reactions/expressions/dimensions should be used on the axis. Examples: axes=['BIOMASS_Ecoli_core_w_GAM','EX_etoh_e'] or axes=[['BIOMASS_Ecoli_core_w_GAM','-EX_glc_e'],['EX_etoh_e','-EX_glc_e']] or axes=[['BIOMASS_Ecoli_core_w_GAM'],['EX_etoh_e','-EX_glc_e']]
- **solver** (*optional (str)*) – The solver that should be used for scanning the flux space.
- **constraints** (*optional (str) or (list of str) or (list of [dict,str, float])*) – (Default: '') List of *linear* constraints to be applied on top of the model: signs + or -, scalar factors for reaction rates, inclusive (in)equalities and a float value on the right hand side. The parsing of the constraints input allows for some flexibility. Correct (and identical) inputs are, for instance: constraints='-EX_o2_e <= 5, ATPM = 20' or constraints=['-EX_o2_e <= 5', 'ATPM = 20'] or constraints=[[{'EX_o2_e':-1},'<=',5], [{'ATPM':1},'=',20]]
- **plt_backend** (*optional (str)*) – The matplotlib backend that should be used for plotting: interactive backends: GTK3Agg, GTK3Cairo, GTK4Agg, GTK4Cairo, MacOSX, nbAgg, QtAgg, QtCairo,

  TkAgg, TkCairo, WebAgg, WX, WXAgg, WXCairo, Qt5Agg, Qt5Cairo

  non-interactive backends: agg, cairo, pdf, pgf, ps, svg, template
- **show** (*optional (bool)*) – (Default: True) Should matplotlib show the plot or should it stop after plot generation. show=False can be useful if flux spaces should be plotted and saved in a non-interactive environment, multiple flux spaces should be plotted at once or the plot should be modified before been shown.
- **cmap** (*optional (str)*) – (Default: 'managua') The matplotlib colormap used for 3D face coloring.
- **points** (*optional (int)*) – (Default: 25 (3D) or 40 (2D)) Controls resolution of non-exact (approximate) plot regions. For rate-only axes, boundary tracing finds exact vertices and this parameter is ignored. For yield axes, this controls: - 2D yield plots: max refinement depth = max(5, log2(points)) - 3D with 1 yield axis: number of slices along the yield axis - 3D with 2+ yield axes: number of grid intervals per axis

Returns

(datapoints, triang, plot1). The array of datapoints from which the plot was generated. These datapoints are optimal values for different optimizations within the flux space. The triang variable contains information about which datapoints need to be connected in triangles to render a closed surface. The last variable contains the matplotlib object.

Return type

(Tuple)

straindesign.lptools.**remove_redundant_bounds**(*model*, ***kwargs*) → pandas.DataFrame

Remove non-binding bounds from a model using FVA.

Runs FVA and relaxes bounds that never bind at steady state: - If fva_min > lb + tol: set lb = -inf (lower bound is not binding) - If fva_max < ub - tol: set ub = +inf (upper bound is not binding)

Modifies the model IN-PLACE. Returns the FVA DataFrame.

Parameters

- **model** (*cobra.Model*) – A metabolic model. Modified in-place.
- **solver** (*optional (str)*) – Solver for FVA.
- **constraints** (*optional*) – Constraints passed through to fva().
- **compress** (*optional (bool)*) – Compress before FVA (passed through).
- **threads** (*optional (int)*) – Parallel threads for FVA (passed through).
- **tol** (*optional (float)*) – (Default: 1e-6) Tolerance for considering a bound as binding.

Returns
FVA results with 'minimum' and 'maximum' columns.

Return type
(pandas.DataFrame)

straindesign.lptools.**select_solver**(*solver=None*, *model=None*) → str

Select a solver for subsequent MILP/LP computations

This function will determine the solver to be used for subsequend MILP/LP computations. If no argument is provided, this function will try to determine the currently selected solver from the COBRA configuration. If unavailable, the solver will be inferred from the packages available at package initialization and one of the solvers will be picked and retured in the prioritized order: 'glpk', 'cplex', 'gurobi', 'scip' One may provide a solver or a model manually. This function then checks if the selected solver is available, or else, if the solver indicated in the model is available. If yes, this function returns the name of the solver as a str. If both arguments are specified, the function prefers 'solver' over 'model'.

Example

solver = select_solver('cplex')

Parameters

- **solver** (*optional (str)*) – A user preferred solver, that should be checked for availability: 'glpk', 'cplex', 'gurobi' or 'scip'.
- **model** (*optional (cobra.Model)*) – A metabolic model that is an instance of the cobra.Model class. The function will try to dertermine the selected solver by accessing the field model.solver.

Returns
The selected solver name as a str (one of the following: 'glpk', 'cplex', 'gurobi', 'scip').

Return type
(str)

straindesign.lptools.**yopt**(*model*, ***kwargs*) → cobra.core.Solution

Yield optmization (YOpt)

Yield optimization optimizes a *fractional objective function* in a space of steady-state flux vectors given by a constraint-based metabolic model. Yield optimization employs linear fractional programming, and is often utilized to determine the (stoichiometrically) maximal possible product yield, that is, the fraction between the product exchange and the substrate uptake flux. This function requires a custom fractional objective function specified by a *linear* numerator and denominator terms. Coefficients in the linear numerator or denominator expression can be used to optimize for carbon recovery, for instance: objective: (3*pyruvate_ex)/(2*ac_up+6*glc_up) The user may also specify the optimization sense. In addition, additional constraints can be specified to narrow down the flux space.

Yield optimization can fail because of several reasons. Here is how the function reacts:

1. **The model is infeasible:**
   The function returns infeasible with no flux vector
2. **The denominator is fixed to zero:**
   The function returns infeasible with no flux vector
3. **The numerator is unbounded:**
   The function returns unbounded with no flux vector
4. **The denominator can become zero:**
   The function returns unbounded, and a flux vector is computed by fixing the the denominator

Example

optim = yopt(model, obj_num='2 EX_etoh_e', obj_den='-6 EX_glc__D_e', constraints='EX_o2_e=0')

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **obj_num** (*(str) or (dict)*) – The numerator of the fractional objective function, provided as a linear expression, either as a character string or as a dict. E.g.: obj_num='EX_prod_e' or obj_num={'EX_prod_e': 1}
- **obj_den** (*(str) or (dict)*) – The denominator of the fractional objective function, provided as a linear expression, either as a character string or as a dict. E.g.: obj_num='1.0 EX_subst_e' or obj_num={'EX_subst_e': 1}
- **obj_sense** (*optional (str)*) – (Default: 'maximize') The optimization direction can be set either to 'maximize' (or 'max') or 'minimize' (or 'min').
- **solver** (*optional (str)*) – The solver that should be used for YOpt.
- **constraints** (*optional (str) or (list of str) or (list of [dict,str, float])*) – (Default: '') List of *linear* constraints to be applied on top of the model: signs + or -, scalar factors for reaction rates, inclusive (in)equalities and a float value on the right hand side. The parsing of the constraints input allows for some flexibility. Correct (and identical) inputs are, for instance: constraints='-EX_o2_e <= 5, ATPM = 20' or constraints=['-EX_o2_e <= 5', 'ATPM = 20'] or constraints=[[{'EX_o2_e':-1},'<=',5], [{'ATPM':1},'=',20]]

Returns
A solution object that contains the objective value, an optimal flux vector and the optmization status.

Return type
(cobra.core.Solution)

straindesign.names

Static strings used in the StrainDesign package

Model and module

MODEL_ID = 'model_id'

PROTECT = 'mcs_lin'

SUPPRESS = 'mcs_bilvl'

SUPPRESS = 'suppress'

PROTECT = 'protect'

OPTKNOCK = ‘optknock’

ROBUSTKNOCK = ‘robustknock’

OPTCOUPLE = ‘optcouple’

MODULE_TYPE = ‘module_type’

CONSTRAINTS = ‘constraints’

INNER_OBJECTIVE = ‘inner_objective’

INNER_OPT_SENSE = ‘inner_opt_sense’

OUTER_OBJECTIVE = ‘outer_objective’

OUTER_OPT_SENSE = ‘outer_opt_sense’

PROD_ID = ‘prod_id’

MIN_GCP = ‘min_gcp’

Solvers and status codes

SOLVER = ‘solver’

CPLEX = ‘cplex’

GUROBI = ‘gurobi’

SCIP = ‘scip’

GLPK = ‘glpk’

OPTIMAL = ‘optimal’ # from optlang interface

INFEASIBLE =’infeasible’ # from optlang interface

TIME_LIMIT = ‘time_limit’ # from optlang interface

UNBOUNDED = ‘unbounded’ # from optlang interface

TIME_LIMIT_W_SOL = ‘time_limit_w_sols’

ERROR = ‘error’

Strain design setup

KOCOST = ‘ko_cost’

KICOST = ‘ki_cost’

GKOCOST = ‘gko_cost’

GKICOST = ‘gki_cost’

REGCOST = ‘reg_cost’

MODULES = ‘sd_modules’

SETUP = ‘sd_setup’

MAX_SOLUTIONS = ‘max_solutions’

MAX_COST = ‘max_cost’

T_LIMIT = ‘time_limit’

SOLUTION_APPROACH = ‘solution_approach’

ANY = ‘any’

BEST = ‘best’

POPULATE = ‘populate’

SEED = ‘seed’

Analysis

MAXIMIZE = ‘maximize’

MINIMIZE = ‘minimize’

straindesign.networktools

Functions for metabolic network extension with GPR rules and strain design module handling.

Compression functions have been moved to straindesign.compression. This module re-exports them for backwards compatibility.

Module Contents

straindesign.networktools.**bound_blocked_or_irrevers_fva**(*model*, ***kwargs*)

Use FVA to determine the flux ranges. Use this information to update the model bounds

If flux ranges for a reaction are narrower than its bounds in the mode, these bounds can be omitted, since other reactions must constrain the reaction flux. If (upper or lower) flux bounds are found to be zero, the model bounds are updated to reduce the model complexity.

straindesign.networktools.**compress_ki_ko_cost**(*kocost*, *kicost*, *cmp_mapReac*)

Compress knockout/addition cost vectors to match with a compressed model

When knockout/addition cost vectors have been specified (as dicts) and the original metabolic model was compressed, one needs to update the knockout/addition cost vectors. This function takes care of this. In particular it makes sure that the resulting costs are calculated correctly.

E.g.: r_ko_a (cost 1) and r_ko_b (cost 2) are lumped parallel: The resulting cost of r_ko_ab is 3 If they are lumped as dependent reactions the resulting cost is 1. If one of the two reactions is an addition candidate, the resulting reaction will be an addition candidate when lumped as dependent reactions and a knockout candidate when lumped in parallel. There are various possible cases that are treated by this function.

Example

kocost, kicost, cmp_mapReac = compress_ki_ko_cost(kocost, kicost, cmp_mapReac)

Parameters

- **kocost** (*dict*) – Knockout and addition cost vectors
- **kicost** (*dict*) – Knockout and addition cost vectors
- **cmp_mapReac** (*list of dicts*) – Compression map obtained from cmp_mapReac = compress_model(model)

Returns

Updated vectors of KO costs and KI costs and an updated compression map that contains information on how to expand strain designs and correctly distinguish between knockouts and additions.

Return type

(Tuple)

straindesign.networktools.**compress_modules**(*sd_modules*, *cmp_mapReac*)

Compress strain design modules to match with a compressed model

When a strain design task has been specified with modules and the original metabolic model was compressed, one needs to refit the strain design modules (objects of the SDModule class) to the new compressed model. This function takes a list of modules and a compression map and returns the strain design modules for a compressed network.

Example

comression_map = compress_modules(sd_modules, cmp_mapReac)

Parameters

- **model** (*list of* SDModule) – A list of strain design modules
- **cmp_mapReac** (*list of dicts*) – Compression map obtained from cmp_mapReac = compress_model(model)

Returns
A list of strain design modules for the compressed network

Return type
(list of *SDModule*)

straindesign.networktools.**estimate_expansion_size**(*compressed_sds*, *cmp_mapReac*)

Estimate total expanded solutions without doing the expansion.

Walks cmp_mapReac in reverse for each compressed solution. For each compression step, for each reaction that maps to multiple originals: - KO + parallel -> factor 1 (all ko'd together) - KO + coupled -> factor = count of knockable originals - KI + parallel -> factor = count of KI-able originals - KI + coupled -> factor 1

Returns int (upper bound, exact for single-step compression).

straindesign.networktools.**evaluate_gpr_ast**(*node*, *gene_states*)

Evaluate a GPR AST node with given gene states.

Supports arbitrary nesting of AND/OR operators (not limited to DNF/CNF). Used by both gene_kos_to_constraints and reduce_gpr.

Parameters

- **node** – An ast.Name or ast.BoolOp node from a parsed GPR rule (e.g. reaction.gpr.body).
- **gene_states** (*dict*) – Maps gene IDs to True, False, or None. Genes not in the dict are treated as None (undetermined).

Returns
True if the GPR is satisfied, False if knocked out, None if undetermined.

straindesign.networktools.**expand_sd**(*sd*, *cmp_mapReac*)

Expand computed strain designs from a compressed to a full model

Needed after computing strain designs in a compressed model

Example

expanded_sd = expand_sd(compressed_sds, cmp_mapReac)

Parameters

- **sd** (SDSolutions) – Solutions of a strain design computation that refer to a compressed model

- **cmp_mapReac** (*list of dicts*) – Compression map obtained from cmp_mapReac = compress_model(model) and updated with kocost, kicost, cmp_mapReac = compress_ki_ko_cost(kocost, kicost, cmp_mapReac)

Returns
Strain design solutions that refer to the uncompressed model

Return type
(*SDSolutions*)

straindesign.networktools.**extend_model_gpr**(*model*, *use_names=False*)

Integrate GPR-rules into a metabolic model as pseudo metabolites and reactions using AST parsing

COBRA modules often have gene-protein-reaction (GPR) rules associated with each reaction. These can be integrated into the metabolic network structure through pseudo reactions and variables. As GPR rules are integrated into the metabolic network, the metabolic flux space does not change. After integration, the gene-pseudoreactions can be fixed to a flux of zero to simulate gene knockouts. Gene pseudoreactions are referenced either by the gene name or the gene identifier (user selected).

GPR-rule integration enables the computation of strain designs based on genetic interventions.

This function now uses AST parsing to handle both DNF (disjunctive normal form) and non-DNF GPR rules. It processes the reaction.gpr.body AST structure directly, enabling proper handling of complex nested boolean expressions.

Example

reac_map = extend_model_gpr(model):

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class containing GPR rules
- **use_names** (*bool*) – (Default: False) If set to True, the gene pseudoreactions will carry the gene name as reaction identifier. If False, the gene identifier will be used. By default this option is turned off because many models do not provide gene names.

Returns
A dictionary to reference old and new reaction identifiers, for reversible reactions that were split (when they are associated with GPR rules). Entries have the form: {'Reaction1' : {'Reaction1' : 1, 'Reaction1_reverse_a59c' : -1}}

Return type
(dict)

straindesign.networktools.**extend_model_regulatory**(*model*, *reg_itv*)

Extend a metabolic model to account for regulatory constraints

This function emulates regulatory interventions in a network. These can either be added permanently or linked to a pseudoreation whose boundaries can be fixed to zero used to activate the regulatory constraint.

Accounting for regulatory interventions, such as applying an upper or lower bound to a reaction or gene pseudoreaction, can be achieved by combining different pseudometabolites and reactions. For instance, to introduce the regulatory constraint:

2*r_1 + 3*r_2 <= 4

and make it 'toggleable', one adds 1 metabolite 'm' and 2 reactions, 'r_bnd' to account for the bound/rhs and r_ctl to control whether the regulatory intervention is active or not:

dm/dt = 2*r_1 + 3*r_2 - r_bnd + r_ctl = 0, -inf <= r_bnd <= 4, -inf <= r_ctl <= inf

When r_ctl is fixed to zero, the constraint 2*r_1 + 3*r_2 <= 4 is enforced, otherwise, the constraint is non binding, thus virtually non-existant. To use this mechanism for strain design, we add the metabolite and reactions as described above and tag r_ctl as knockout candidate. If the algorithm decides to knockout r_ctl, this means, it choses to add the regulatory intervention 2*r_1 + 3*r_2 <= 4.

If the constraint is be added permanently, this function completely omits the r_ctl reaction.

Example

reg_itv_costs = extend_model_regulatory(model, {'1 PDH + 1 PFL <= 5' : 1, '-EX_o2_e <= 2' : 1.5})

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class
- **reg_itv** (*dict or list of str or str*) – A set of regulatory constraints that should be added to the model. If reg_itv is a string or a list of strings, regulatory constraints are added permanently. If reg_itv is a dict, regulatory interventions are added in a controllable manner. The id of the reaction that controls the constraint is contained in the return variable. The constraint to be added will be parsed from strings, so ensure that you use the correct reaction identifiers. Valid inputs are: reg_itv = '-EX_o2_e <= 2' # A single permanent regulatory constraint reg_itv = ['1 PDH + 1 PFL <= 5', '-EX_o2_e <= 2'] # Two permanent constraints reg_itv = {'1 PDH + 1 PFL <= 5' : 1, '-EX_o2_e <= 2' : 1.5} # Two controllable constraints # one costs '1' and the other one '1.5' to be added. The function returns a dict with # {'p1_PDH_p1_PFK_le_5' : 1 'nEX_o2_e_le_2' : 1.5}. Fixing the reaction # p1_PDH_p1_PFK_le_5 to zero will activate the constraint in the model.

Returns
A dictionary that contains the cost of adding each constraint; e.g., {'p1_PDH_p1_PFK_le_5' : 1 'n1EX_o2_e_le_2' : 1.5}

Return type
(dict)

straindesign.networktools.**filter_sd_maxcost**(*sd*, *max_cost*, *kocost*, *kicost*)

Filter out strain designs that exceed the maximum allowed intervention costs

Returns
Strain design solutions complying with the intervention costs limit

Return type
(*SDSolutions*)

straindesign.networktools.**gene_kos_to_constraints**(*model*, *gene_kos*)

Translate gene knockouts to reaction-level constraints.

Given a cobra model and a set of knocked-out genes, evaluates GPR rules (via AST parsing) to determine which reactions become non-functional. Returns a list of constraints (in straindesign list format) that fix those reactions to zero flux.

Supports arbitrary GPR nesting (not limited to DNF/CNF).

Gene identifiers can be gene IDs or gene names (case-sensitive). Unknown identifiers are silently ignored.

Note on the SD solution grammar:
SDSolutions uses the following convention for intervention values: -1 = knockout (KO), +1 = knock-in (KI), 0 = non-added KI.

When passing gene constraints to `fba()` or `fva()`, both `gene = 0` and `gene = -1` are treated as knockouts, and `gene = 1` is ignored (the gene is active). This means SD solution values can be used directly as constraints.

The returned *reaction* constraints use the linear constraint format `[{reaction_id: 1}, '=', 0]`, meaning `1 * v = 0`.

Example

```
>>> constraints = gene_kos_to_constraints(model, ['b0727', 'b1241'])
>>> fba(model, constraints=constraints)
```

Parameters

- **model** (*cobra.Model*) – A metabolic model.
- **gene_kos** (*list or set*) – Gene IDs or names to knock out.

Returns

Constraints in list format, e.g.
[[{'AKGDH': 1}, '=', 0], [{'PDH': 1}, '=', 0], ...]

Return type
list

straindesign.networktools.**modules_coeff2float**(*sd_modules*)

Convert coefficients occurring in SDModule objects to floats

straindesign.networktools.**modules_coeff2rational**(*sd_modules*)

Convert coefficients to rational numbers using sympy.Rational

straindesign.networktools.**reduce_gpr**(*model*, *essential_reacs*, *gkis*, *gkos*)

Simplify GPR rules by removing non-targetable genes and reducing boolean expressions

This function is used in preprocessing of computational strain design computations. Often, certain reactions, for instance, reactions essential for microbial growth can/must not be targeted by interventions. That can be exploited to reduce the set of genes in which interventions need to be considered.

Given a set of essential reactions that is to be maintained operational, some genes can be removed from a metabolic model, either because they only affect only blocked reactions or essential reactions, or because they are essential reactions and must not be removed. As a consequence, the GPR rules of a model can be simplified using AST parsing for both DNF and non-DNF rules.

Example

reduce_gpr(model, essential_reacs, gkis, gkos):

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class containing GPR rules
- **essential_reacs** (*list of str*) – A list of identifiers of essential reactions.
- **gkis** (*dict*) – Dictionaries that contain the costs for gene knockouts and additions. E.g., gkos={'adhE': 1.0, 'ldhA' : 1.0 ...}
- **gkos** (*dict*) – Dictionaries that contain the costs for gene knockouts and additions. E.g., gkos={'adhE': 1.0, 'ldhA' : 1.0 ...}

Returns
An updated dictionary of the knockout costs in which irrelevant genes are removed.

Return type
(dict)

straindesign.networktools.**resolve_gene_constraints**(*model*, *constraints*)

Scan constraints for gene IDs/names and replace with reaction constraints.

Constraints that reference gene identifiers (rather than reaction IDs) are interpreted using the strain design solution grammar:

- `gene = -1` — knockout (KO)
- `gene = 0` — knockout (non-added, absence of knock-in)
- `gene = 1` — ignored (knock-in, gene is active)

This is consistent with the SDSolutions convention, so values from solution dicts can be passed directly as constraints.

Non-gene constraints are passed through unchanged.

Accepted gene constraint formats:

```
'b0727 = 0'                          # string - KO
'b0727 = -1'                         # string - KO (SD grammar)
'b1241 = 1'                          # string - ignored (KI)
['b0727 = 0', 'b1241 = 0']           # list of strings
[[{'b0727': 1}, '=', 0]]             # list format
```

Gene identifiers are matched case-sensitively against both gene IDs and gene names in the model.

This function is called automatically by `fba()`, `fva()`, and `fva_legacy()` before constraint parsing, so users can pass gene knockouts directly:

```
fba(model, constraints='b0727 = 0')
fva(model, constraints=['b0727 = 0', 'EX_o2_e <= 5'])
```

Parameters

- **model** (*cobra.Model*) – A metabolic model.
- **constraints** – Constraints in any format accepted by straindesign (str, list of str, or list of [dict, str, float]).

Returns
Constraints with gene entries replaced by reaction entries.

Return type
list

straindesign.networktools.**suppress_lp_context**(*model*)

Context manager that suppresses all solver-touching operations.

Patches both optlang (coefficient/bounds updates) and cobra (property setters, Model.remove_reactions, Model.remove_metabolites) at the class level. On exit, restores originals and rebuilds the solver so it reflects the current model state.

Nests safely: if already suppressed by an outer context, this is a no-op.

straindesign.networktools.**with_suppressed_lp**(*func*)

Decorator that wraps a function in *suppress_lp_context()*.

The first positional argument must be a cobra Model.

straindesign.parse_constr

Functions for parsing and converting constraints and linear expressions

Module Contents

straindesign.parse_constr.**get_rids**(*expr*, *reaction_ids*)

Get reaction identifiers that are present in string

E.g.: input: D={"R1":-1.0, "R3": 2.0}, translates to the string: "- 1.0 R1 + 2.0 R3"

Parameters

- **expr** (*str*) – A character string
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names

Returns
A list of strings containing the reaction/variable strings present in the input string

Return type
(list of str)

straindesign.parse_constr.**lineq2list**(*equations*, *reaction_ids*) → List

Translates *linear* (in)equalities to list format: [lhs,sign,rhs]

Input inequalities in the form of strings are translated into a specific list format that facilitates the readout of left-hand-side, equality sign and right-hand-side of the inequality.

equations = ["2*c - b +3*a <= 2","c - b = 0","2*b -5…], reaction_ids = ["a","b","c"]

This will be translated to the [[{"a":3.0,"b":-1.0,"c":2.0},"<=",2.0],[{"b":-1.0,"c":1.0},"=",0.0], …]

Parameters

- **equations** (*list of str*) – (List of) (in)equalities in string form equations=["r1 + 3*r2 = 0.3", "-5*r3 -r4 <= -0.5"]
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names that are used to recognize variables in the provided (in)equalities

Returns

(In)equalities presented in the form: [[{"a":3.0,"b":-1.0,"c":2.0},"<=",2.0], # e1

[{"b":-1.0,"c":1.0},"=",0.0], # e2 …] # …

Return type
(list of lists)

straindesign.parse_constr.**lineq2mat**(*equations*, *reaction_ids*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple]

Translates *linear* (in)equalities to matrices

Input inequalities in the form of strings is translated into matrices and vectors. The reaction list defines the order of variables and thus the columns of the resulting matrices, the order of (in)equalities will be preserved in the output matrices. As an example, take the input:

equations = ["2*c - b +3*a <= 2","c - b = 0","2*b -a >=-2"], reaction_ids = ["a","b","c"]

This will be translated to the form A_ineq * x <= b_ineq, A_eq * x = b_eq and hence to

A_ineq = sparse.csr_matrix([[3,-1,2],[1,-2,0]]), b_ineq = [2,2], A_eq = sparse.csr_matrix([[1,-2,0]]), b_eq = [0]

Parameters

- **equations** (*list of str*) – (List of) (in)equalities in string form equations=[“r1 + 3*r2 = 0.3”, “-5*r3 -r4 <= -0.5”]
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names that are used to recognize variables in the provided (in)equalities

Returns
A_ineq, b_ineq, A_eq, b_eq. Coefficient matrices and right hand sides that represent the input (in)equalities as matrix-vector multiplications

Return type
(Tuple)

straindesign.parse_constr.**lineqlist2mat**(*D*, *reaction_ids*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple]

Translates *linear* (in)equalities presented in the list of lists format to matrices

Input inequalities in the list of lists form is translated into matrices and vectors. The reaction list defines the order of variables and thus the columns of the resulting matrices, the order of (in)equalities will be preserved in the output matrices. As an example, take the input:

D = [[{“a”:3.0,”b”:-1.0,”c”:2.0},”<=”,2.0],[{“b”:-1.0,”c”:1.0},”=”,0.0], [{“a”:-1,”b”:2.0},”>=”,-2.0]]

This will be translated to the form A_ineq * x <= b_ineq, A_eq * x = b_eq and hence to

A_ineq = sparse.csr_matrix([[3,-1,2],[1,-2,0]]), b_ineq = [2,2], A_eq = sparse.csr_matrix([[1,-2,0]]), b_eq = [0]

Parameters

- **D** (*list of dict*) – (List of) (in)equalities in the list of list form: [[{“a”:3.0,”b”:-1.0,”c”:2.0},”<=”,2.0],[{“b”:-1.0,”c”:1.0},”=”,0.0], . . .]
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names that are used to recognize variables in the provided (in)equalities

Returns
A_ineq, b_ineq, A_eq, b_eq. Coefficient matrices and right hand sides that represent the input (in)equalities as matrix-vector multiplications

Return type
(Tuple)

straindesign.parse_constr.**lineqlist2str**(*D*)

Translates a *linear* (in)equality from the list format [lhs,sign,rhs] to a string

E.g. input: D=[{“a”:3.0,”b”:-1.0,”c”:2.0},”<=”,2.0]] is translated to: out=”3.0 a - 1.0 b + 2.0 c <= 2”

Parameters
D (*list*) – (In)equality in list form, e.g.: D=[{“a”:3.0,”b”:-1.0,”c”:2.0},”<=”,2.0]]

Returns
A list of (in)equalities in string form

Return type
(str)

straindesign.parse_constr.**linexpr2dict**(*expr*, *reaction_ids*) → dict

Translates a linear expression into a dictionary

E.g.: input: expr=”2 R3 - R1”, reaction_ids=[“R1”, “R2”, “R3”, “R4”] translates to a dict D={“R1”:-1.0, “R3”: 2.0}

Parameters

- **expr** (*str*) – (In)equalities as a character string, e.g.: expr="2 R3 - R1"
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names that are used to recognize variables in the input

Returns
A dictionary that contains the variable names and the variable coefficients in the linear expression

Return type
(dict)

straindesign.parse_constr.**linexpr2mat**(*expr*, *reaction_ids*) → scipy.sparse.csr_matrix

Translates a linear expression into a vector

E.g.: input: expr="2 R3 - R1", reaction_ids=["R1", "R2", "R3", "R4"] translates into sparse matrix: A = [-1 0 2 0]

Parameters

- **expr** (*str*) – (In)equality as a character string: e.g., expr="2 R3 - R1"
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names that are used to recognize variables in the input

Returns
A single-row coefficient matrix that represents the input expression when multiplied with the variable vector

Return type
(sparse.csr_matrix)

straindesign.parse_constr.**linexprdict2mat**(*D*, *reaction_ids*) → scipy.sparse.csr_matrix

Translates a linear expression from dict into a matrix

E.g.: input: D={"R1":-1.0, "R3": 2.0}, reaction_ids=["R1", "R2", "R3", "R4"] translates into sparse matrix: A = [-1 0 2 0]

Parameters

- **D** (*dict*) – Linear expression as a dictionary
- **reaction_ids** (*list of str*) – List of reaction identifiers or variable names

Returns
A single-row coefficient matrix that represents the input expression when multiplied with the variable vector

Return type
(sparse.csr_matrix)

straindesign.parse_constr.**linexprdict2str**(*D*)

Translates a linear expression from dict into a caracter string

E.g.: input: D={"R1":-1.0, "R3": 2.0}, translates to the string: "- 1.0 R1 + 2.0 R3"

Parameters
D (*dict*) – Linear expression as a dictionary

Returns
The input linear expression as a character string

Return type
(str)

straindesign.parse_constr.**parse_constraints**(*constr*, *reaction_ids*) → list

Parses linear constraints written as strings

Parses one or more *linear* constraints written as strings.

Parameters

- **constr** (*str or list of str*) – (List of) constraints in string form. E.g.: ["r1 + 3*r2 = 0.3", "-5*r3 -r4 <= -0.5"] or "1.0 r1 + 3.0*r2 =0.3,-r4-5*r3<=-0.5" or ...
- **reaction_ids** (*list of str*) – List of reaction identifiers.

Returns

List of constraints. Each constraint is a list of three elements. E.g.: [[{"r1":1.0,"r2":3.0},"=",0.3],[{"r3":-5.0,"r4":-1.0},"<=",-0.5],...]

Return type

(List of dicts)

straindesign.parse_constr.**parse_linexpr**(*expr*, *reaction_ids*) → List

Parses linear expressions written as strings

Parses one or more *linear* expressions written as strings.

Parameters

- **expr** (*str or list of str*) – (List of) expressions in string form. E.g.: ["r1 + 3*r2", "-5*r3 -r4"] or "1.0 r1 + 3.0*r2,-r4-5*r3" or ...
- **reaction_ids** (*list of str*) – List of reaction identifiers.

Returns

List of expressions. Each expression is a dictionary. E.g.: [{"r1":1.0,"r2":3.0},{"r3":-5.0,"r4":-1.0},...]

Return type

(List of dicts)

straindesign.pool

Provide a process pool with enhanced performance on Windows, copied and slightly adapted from cobra.

Module Contents

class straindesign.pool.**SDPool**(*processes: int | None = None*, *initializer: Callable | None = None*, *initargs: Tuple = ()*, *maxtasksperchild: int | None = None*, *context=None*)

Bases: multiprocessing.pool.Pool

Multiprocessing process pool with enhanced Windows compatibility

Initialize a process pool.

Add a thin layer on top of the *multiprocessing.Pool* that, on Windows, passes initialization code to workers via a pickle file rather than directly. This is done to avoid a performance issue that exists on Windows. Please, also see the discussion [*1*].

References

__exit__(**args*, ***kwargs*)

Clean up resources when leaving a context

close()

Call cleanup function and close

straindesign.scip_interface

SCIP and SoPlex solver interface for LP and MILP

Module Contents

class straindesign.scip_interface.**SCIP_LP**(*c*, *A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*)

Bases: pyscipopt.LP

SoPlex interface for LP

This class is a wrapper for the SoPlex-Python API to offer bindings and namings for functions for the construction and manipulation of LPs in an vector-matrix-based manner that are consistent with those of the other solver interfaces in the StrainDesign package. The purpose is to unify the instructions for operating with MILPs and LPs throughout StrainDesign.

Constructor of the SCIP (SoPlex) LP interface class

Accepts a (mixed integer) linear problem in the form:

minimize(c) subject to: A_ineq * x <= b_ineq

A_eq * x = b_eq lb <= x <= ub forall(i) type(x_i) = vtype(i) (continous, binary, integer) indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

scip = SCIP_LP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).
- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.
- **ub** (*list of float*) – (Default: None) The upper variable bounds.
- **Returns** – (SCIP_LP):

  A SCIP LP interface class.

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix

- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_ineq** (*list of float*) – The right hand side vector

get_basis()

Reconstruct vbasis/cbasis from SoPlex getBasisInds + solution.

Uses getBasisInds() to identify basic variables, then checks solution values against bounds to determine non-basic status.

Returns

dict with 'vbasis' (list of int) and 'cbasis' (list of int). Status codes: 0=basic, -2=at lower bound, -3=at upper bound.

get_lp_method()

Return LP_METHOD_AUTO — SCIP_LP does not expose method selection.

set_basis(*basis*)

No-op — pyscipopt LP does not expose basis loading.

set_lp_method(*method*)

Set the LP solving method (no-op for SCIP_LP — SoPlex manages internally).

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

No-op — SCIP_LP does not support time limits.

slim_solve() → float

Solve the LP, but return only the optimal value

Example

optim = scip.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the LP

Example

sol_x, optim, status = scip.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

class straindesign.scip_interface.**SCIP_MILP**(*c=None*, *A_ineq=None*, *b_ineq=None*, *A_eq=None*, *b_eq=None*, *lb=None*, *ub=None*, *vtype=None*, *indic_constr=None*, *seed=None*, *milp_threads=None*)

Bases: `pyscipopt.Model`

SCIP interface for MILP

This class is a wrapper for the SCIP-Python API to offer bindings and namings for functions for the construction and manipulation of MILPs in an vector-matrix-based manner that are consistent with those of the other solver interfaces in the StrainDesign package. The purpose is to unify the instructions for operating with MILPs and LPs throughout StrainDesign.

The SCIP interface provides support for indicator constraints as well as for the populate function. The SCIP interface does not natively support the populate function. A high level implementation emulates the behavior of populate.

Accepts a (mixed integer) linear problem in the form:
minimize(c), subject to: A_ineq * x <= b_ineq, A_eq * x = b_eq, lb <= x <= ub, forall(i) type(x_i) = vtype(i) (continous, binary, integer), indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

scip = SCIP_MILP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub, vtype, indic_constr)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).
- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.
- **ub** (*list of float*) – (Default: None) The upper variable bounds.
- **vtype** (*str*) – (Default: None) A character string that specifies the type of each variable: ‘c’ontinous, ‘b’inary or ‘i’nteger
- **indic_constr** (IndicatorConstraints) – (Default: None) A set of indicator constraints stored in an object of IndicatorConstraints (see reference manual or docstring).
- **seed** (*int16*) – (Default: None) An integer value serving as a seed to make MILP solving reproducible.

- **Returns** – (SCIP_MILP):

  A SCIP MILP interface class.

addExclusionConstraintIneq(*x*)

Function to add exclusion constraint (SCIP compatibility function)

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_ineq** (*list of float*) – The right hand side vector

getSolution() → list

Retrieve solution from SCIP backend

abstract get_basis()

Not available for SCIP MILP — LP basis is discarded after MIP solve.

get_lp_method()

Return the current LP method as a solver-neutral string.

abstract set_basis(*basis*)

Not available for SCIP MILP — pyscipopt does not expose basis loading.

set_ineq_constraint(*idx*, *a_ineq*, *b_ineq*)

Replace a specific inequality constraint

Replace the constraint with the index idx with the constraint a_ineq*x ~ b_ineq

Parameters

- **idx** (*int*) – Index of the constraint
- **a_ineq** (*list of float*) – The coefficient vector
- **b_ineq** (*float*) – The right hand side value

set_lp_method(*method*)

Set the LP solving method.

Parameters

method – LP_METHOD_AUTO, LP_METHOD_PRIMAL, LP_METHOD_DUAL, or LP_METHOD_BARRIER

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

Set the computation time limit (in seconds)

set_ub(*ub*)

Set the upper bounds to a given vector

slim_solve() → float

Solve the MILP, but return only the optimal value

Example

optim = scip.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the MILP

Example

sol_x, optim, status = scip.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

straindesign.solver_interface

Unified solver interface for LPs and MILPs (MILP_LP)

Module Contents

class straindesign.solver_interface.**MILP_LP**(***kwargs*)

Bases: object

Unified MILP and LP interface

This class is a wrapper for several solver interfaces to offer unique and consistent bindings for the construction and manipulation of MILPs and LPs in an vector-matrix-based manner and their solution.

Accepts a (mixed integer) linear problem in the form:

minimize(c), subject to: A_ineq * x <= b_ineq, A_eq * x = b_eq, lb <= x <= ub, forall(i) type(x_i) = vtype(i) (continous, binary, integer), indicator constraints: x(j) = [0|1] -> a_indic * x [<=|=|>=] b_indic

Please ensure that the number of variables and (in)equalities is consistent

Example

milp = MILP_LP(c, A_ineq, b_ineq, A_eq, b_eq, lb, ub, vtype, indic_constr)

Parameters

- **c** (*list of float*) – (Default: None) The objective vector (Objective sense: minimization).
- **A_ineq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static inequalities.
- **b_ineq** (*list of float*) – (Default: None) The right hand side of the static inequalities.
- **A_eq** (*sparse.csr_matrix*) – (Default: None) A coefficient matrix of the static equalities.
- **b_eq** (*list of float*) – (Default: None) The right hand side of the static equalities.
- **lb** (*list of float*) – (Default: None) The lower variable bounds.
- **ub** (*list of float*) – (Default: None) The upper variable bounds.
- **vtype** (*str*) – (Default: None) A character string that specifies the type of each variable: 'c'ontinous, 'b'inary or 'i'nteger
- **indic_constr** (IndicatorConstraints) – (Default: None) A set of indicator constraints stored in an object of IndicatorConstraints (see reference manual or docstring).
- **M** (*int*) – (Default: None) A large value that is used in the translation of indicator constraints to bigM-constraints for solvers that do not natively support them. If no value is provided, 1000 is used.
- **solver** (*str*) – (Default: taken from avail_solvers) Solver backend that should be used: 'cplex', 'gurobi', 'glpk' or 'scip'
- **skip_checks** (*bool*) – (Default: False) Upon MILP construction, the dimensions of all provided vectors and matrices are checked to verify their consistency. If skip_checks=True is set, these checks are skipped.
- **tlim** (*float*) – Solution time limit in seconds.
- **Returns** – (MILP_LP):

  A MILP/LP solver interface class.

add_eq_constraints(*A_eq*, *b_eq*)

Add equality constraints to the model

Additional equality constraints have the form A_eq * x = b_eq. The number of columns in A_eq must match with the number of variables x in the problem.

Parameters

- **A_eq** (*sparse.csr_matrix*) – The coefficient matrix
- **b_eq** (*list of float*) – The right hand side vector

add_ineq_constraints(*A_ineq*, *b_ineq*)

Add inequality constraints to the model

Additional inequality constraints have the form A_ineq * x <= b_ineq. The number of columns in A_ineq must match with the number of variables x in the problem.

Parameters

- **A_ineq** (*sparse.csr_matrix*) – The coefficient matrix

- **b_ineq** (*list of float*) – The right hand side vector

clear_objective()

Clear objective

Set all coefficients in the objective vector to 0.

get_basis()

Return the current LP basis for warm-starting.

Returns
dict with solver-specific basis data, or None if not supported. Pass the returned dict to set_basis() on the same solver type.

get_lp_method()

Return the current LP method as a solver-neutral string.

populate(*n*) → Tuple[List, float, float]

Generate a solution pool for MILPs

Example

sols_x, optim, status = cplex.populate()

Returns

(Tuple[List of lists, float, float])

solution_vectors, optimal_value, optimization_status

set_basis(*basis*)

Load a previously saved basis for warm-starting.

Parameters
basis – dict from get_basis() (same solver type required).

set_ineq_constraint(*idx*, *a_ineq*, *b_ineq*)

Replace a specific inequality constraint

Replace the constraint with the index idx with the constraint a_ineq*x ~ b_ineq

Parameters

- **idx** (*int*) – Index of the constraint
- **a_ineq** (*list of float*) – The coefficient vector
- **b_ineq** (*float*) – The right hand side value

set_lp_method(*method*)

Set the LP solving method.

Uses solver-neutral constants from straindesign.names:
LP_METHOD_AUTO — solver default LP_METHOD_PRIMAL — primal simplex LP_METHOD_DUAL — dual simplex LP_METHOD_BARRIER — barrier / interior point

Note: GLPK and SCIP_LP do not support barrier; it falls back to dual simplex (GLPK) or is ignored (SCIP_LP).

set_objective(*c*)

Set the objective function with a vector

set_objective_idx(*C*)

Set the objective function with index-value pairs

e.g.: C=[[1, 1.0], [4,-0.2]]

set_time_limit(*t*)

Set the computation time limit (in seconds)

set_ub(*ub*)

Set the upper bounds to a given vector

slim_solve() → float

Solve the MILP or LP, but return only the optimal value

Example

optim = cplex.slim_solve()

Returns

(float)

Optimum value of the objective function.

solve() → Tuple[List, float, float]

Solve the MILP or LP

Example

sol_x, optim, status = milp.solve()

Returns

(Tuple[List, float, float])

solution_vector, optimal_value, optimization_status

straindesign.speedy_fva

Accelerated FVA using scan LPs and bound scanning.

Standard FVA solves 2*n independent LPs (max and min for each reaction). This implementation reduces LP count via a two-phase approach:

Phase 1 — Scan LPs (cheap, resolve ~50-70% of bounds):

a. v=0 feasibility: free resolutions when zero flux is feasible

b. min(sum(|x|)): pushes reactions toward zero, resolves lb=0/ub=0 bounds

c. Push-to-bounds: directed objectives push unresolved reactions toward their variable bounds, with dual simplex warm-start for fast re-solves

Each scan LP solution is processed by bound scanning (vectorized at-bound check that marks reactions whose flux equals their variable bound).

Phase 2 — Individual LPs for remaining unresolved objectives:
Sequential (warm-started) or parallel (SDPool) dispatch depending on problem size and thread count. Sequential mode includes co-optimization scanning: each LP vertex is checked for other reactions at their bounds.

Module Contents

straindesign.speedy_fva.**speedy_fva**(*model*, ***kwargs*)

Accelerated FVA using global scan LPs and KKT-based optimality propagation.

Returns the same DataFrame as straindesign.lptools.fva.

Parameters

- **model** (*cobra.Model*)
- **solver** (*str, optional*)
- **constraints** (*str or list, optional*)
- **compress** (*bool or None, optional (default None)*) – Compress the model (coupled compression + conservation removal) before running FVA, then expand results back. None = auto (True if n >= 200).
- **precheck** (*bool or None, optional (default None)*) – Run global scan LPs (min/max sum(|x|)) in Phase 1 to pre-resolve many objectives without individual LPs. None = auto (always True).
- **threads** (*int or None, optional (default None)*) – Number of parallel workers for Phase 2 dispatch. None = auto (Configuration().processes if n >= 1000, else 1).
- **verbose** (*bool, optional (default False)*)

Return type

pandas.DataFrame with columns 'minimum' and 'maximum', indexed by reaction ID.

straindesign.strainDesignMILP

Classes and function for the solution of strain design MILPs

Module Contents

class straindesign.strainDesignMILP.**SDMILP**(*model: straindesign.Model*, *sd_modules: List[straindesign.SDModule]*, ***kwargs*)

Bases: straindesign.SDProblem, straindesign.MILP_LP

Class that contains functions for the solution of the strain design MILP

This class is a wrapper and inherited from the casses SDProblem, MILP_LP. The constructor of SDProblem (see strainDesignProblem.py) translates a given problem into a MILP. The constructor of MILP_LP (see solver_interface.py) then sets up the solver interface for the selected solver. In addition to the functions from SDProblem and MILP_LP, SDMILP provides functions for the solution of the strain design MILP, such as verification of strain design solutions or introduction of exclusion constraints for computing multiple solutions.

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **sd_modules** (*(list of) straindesign.SDModule*) – Modules that specify the strain design problem, e.g., protected or suppressed flux states for MCS strain design or inner and outer objective functions for OptKnock. See description of SDModule for more information on how to set up modules.
- **ko_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated knockout costs. If not specified, all reactions are treated as knockout candidates, equivalent to ko_cost = {'r1':1, 'r2':1, ...}. If a subset of reactions is listed in the dict, all other are not considered as knockout candidates.

- **ki_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated costs for addition. If not specified, all reactions are treated as knockout candidates. Reaction addition candidates must be present in the original model with the intended flux boundaries **after** insertion. Additions are treated adversely to knockouts, meaning that their exclusion from the network is not associated with any cost while their presence entails intervention costs.
- **max_cost** (*optional (int)*) – (Default: inf): The maximum cost threshold for interventions. Every possible intervention is associated with a cost value (1, by default). Strain designs cannot exceed the max_cost threshold. Individual intervention cost factors may be defined through ki_cost and ko_cost.
- **solver** (*optional (str)*) – (Default: same as defined in model / COBRApy) The solver that should be used for preparing and carrying out the strain design computation. Allowed values are 'cplex', 'gurobi', 'scip' and 'glpk'.
- **M** (*optional (int)*) – (Default: None) If this value is specified (and non-zero, not None), the computation uses the big-M method instead of indicator constraints. Since GLPK does not support indicator constraints it uses the big-M method by default (with COBRA standard M=1000). M should be chosen 'sufficiently large' to avoid computational artifacts and 'sufficiently small' to avoid numerical issues.
- **essential_kis** (*optional (set)*) – A set of reactions that are marked as addable and that are essential for at least one of the strain design modules. Providing such "essential knock-ins" may speed up the strain design computation.

Returns

An instance of SDProblem containing the strain design MILP and providing several functions for its solution

Return type

(*SDMILP*)

add_exclusion_constraints(*z*)

Exclude binary solution in z and all supersets from MILP

add_exclusion_constraints_ineq(*z*)

Exclude exact binary solution in z (but not its supersets) from MILP.

For each solution row in z, adds: sum(z_active) - sum(z_inactive) <= **|active|** - 1 This excludes the exact pattern without blocking supersets or subsets.

build_sd_solution(*sd_dict*, *status*, *solution_approach*)

Build the strain design solution object

compute(***kwargs*)

Compute arbitrary solutions of the strain design MILP and iteratively find further solutions

Parameters

- **max_solutions** (*optional (int)*) – (Default: inf) The maximum number of MILP solutions that are generated for a strain design problem.
- **time_limit** (*optional (int)*) – (Default: inf) The time limit in seconds for the MILP-solver.
- **show_no_ki** (*optional (bool)*) – (Default: True) Indicate non-added addition candidates in a solution specifically with a value of 0

Returns

Strain design solutions provided as an SDSolutions object

Return type
(*SDSolutions*)

compute_optimal(***kwargs*)

Compute the global optimum of the strain design MILP and iteratively find the next best solution

Parameters

- **max_solutions** (*optional (int)*) – (Default: inf) The maximum number of MILP solutions that are generated for a strain design problem.
- **time_limit** (*optional (int)*) – (Default: inf) The time limit in seconds for the MILP-solver.
- **show_no_ki** (*optional (bool)*) – (Default: True) Indicate non-added addition candidates in a solution specifically with a value of 0

Returns
Strain design solutions provided as an SDSolutions object

Return type
(*SDSolutions*)

enumerate(***kwargs*)

Find all globally optimal solutions to the strain design MILP and iteratively construct pools for the suboptimal values

Parameters

- **max_solutions** (*optional (int)*) – (Default: inf) The maximum number of MILP solutions that are generated for a strain design problem.
- **time_limit** (*optional (int)*) – (Default: inf) The time limit in seconds for the MILP-solver.
- **show_no_ki** (*optional (bool)*) – (Default: True) Indicate non-added addition candidates in a solution specifically with a value of 0

Returns
Strain design solutions provided as an SDSolutions object

Return type
(*SDSolutions*)

fixObjective(*c*, *cx*)

Enforce a certain objective function and value (or any other constraint of the form c*x <= cx)

populateZ(*n*) → Tuple[List, int]

Populate MILP, and return only binary variables rounded to 5 decimals (should return ints)

resetObjective()

Reset objective to the one set upon MILP construction

resetTargetableZ()

Reset targetable/switchable intervention indicators / allow all intervention candidates

sd2dict(*sol*, **args*) → Dict

Translate binary solution vector to dictionary for human-readable output

setMinIntvCostObjective()

Reset minimization of intervention costs as global objective

setTargetableZ(*sol*)

Only allow a subset of intervention candidates

solveZ() → Tuple[List, int]

Solve MILP, and return only binary variables rounded to 5 decimals (should return ints)

verify_sd(*sols*) → List

Verify computed strain design

straindesign.strainDesignModule

Class: strain design module (SDModule)

Module Contents

class straindesign.strainDesignModule.**SDModule**(*model*, *module_type*, **args*, ***kwargs*)

Bases: straindesign.parse_constr.Dict

Strain design modules are used to specify the goal of a strain design computation

(Lists of) SDModule objects are passed to compute_strain_design to specify the goal strain design computation. Strain design modules indicate the appraoch that should be used (OptKnock, RobustKnock, OptCouple or MCS) and the parameters for each approach. In each strain design computation, one of the following modules can be used at most once: OptKnock, RobustKnock and OptCouple. Additionally an arbitrary number of MCS modules (PROTECT or SUPPRESS) may me used. The global objective of a strain design computation depends on the specified modules. If an OptKnock or RobustKnock module is used, the global objective function will be defined by 'outer_objective' and 'outer_opt_sense'. If OptCouple is used, the global objective is derived from by 'inner_objective' and 'inner_opt_sense'. If only MCS-like modules (suppress, protect) are used in a computation, the number of interventions is globally minimized.

In the following, the modules and their mandatory/optional arguments are presented in detail.

OptKnock:

Globally maximize an *outer objective* subjected to the maximization/optimization of an *inner objective*. For instance, maximize product synthesis, assuming that the strain maximizes its growth rate. When used with the 'best' or 'populate' approach (see compute_strain_designs), this module will guarantee that the found intervention set allows for the highest possible outer objective (e.g., production) under the premise that the inner objective (growth) is forced to be maximal. This means that the *production potential* is maximal. However it does not necessarily mean that production is enforced at high growth rates. For enforced growth coupling, refer to the other module types. Additional constraints can be used to impose certain properties on the designed strains: constraints='growth >= 0.5' will guarantee that the designed strain can reach growth rates above 0.5. constraints=['growth >= 0.5', 'EX_byprod_e <= 2'] additionally guarantees that the synthesis rate of a by-product stays below 2 at growth maximal flux states. Alternative inner or outer objective functions (e.g., ATP maintenance) can be used for diffent types of strain design.

mandatory arguments: model, module_type='optknock', inner_objective, outer_objective optional arguments: constraints, inner_opt_sense, outer_opt_sense, skip_checks (Detailed description of the arguments follow below)

RobustKnock:

Globally minimize the maximum of an *outer objective* subjected to the maximization/optimization of an *inner objective*. For instance, maximize the minimal product synthesis rate, assuming that the strain maximizes its growth rate. When used with the 'best' or 'populate' approach (see compute_strain_designs), this module will guarantee that the found intervention set enforces that the minimum of the outer objective (e.g., maximal production) is maximal (max-min), given that the inner objective (growth) is forced to be maximal. This means that the *minimal guaranteed production* is maximal and production is guaranteed

at growth-maximal flux states. Additional constraints can be used to impose certain properties on the designed strains: constraints='growth >= 0.5' will guarantee that the designed strain can reach growth rates above 0.5/h. constraints=['growth >= 0.5', 'EX_byprod_e<=2'] additionally guarantees that the synthesis rate of a by-product stays below 2 mmol/gCDW/h at growth maximal flux states.

mandatory arguments: model, module_type='robustknock', inner_objective, outer_objective optional arguments: constraints, inner_opt_sense, outer_opt_sense, skip_checks, reac_ids (Detailed description of the arguments follow below)

OptCouple:

Globally maximize the *growth-coupling potential*, that is, the difference between the maximal growth rate without product synthesis and the maximum overall growth rate (with product synthesis). This strain design approach often leads to directionally growth-coupled strain designs. Again, alternative definitions of this objective are possible to, for instance, couple production to the synthesis of ATP. To specify the product for which coupling should be engineered, the reaction identifier of the product exchange (pseudo)reaction is passed through the *prod_id* parameter (see below). One may addidionally define a minimum growth-coupling potential through the parameter *min_gcp*.

mandatory arguments: model, module_type='optcouple', inner_objective, prod_id optional arguments: constraints, inner_opt_sense, min_gcp, skip_checks, reac_ids (Detailed description of the arguments follow below)

Suppress:

MCS-like suppression of a subspace of all steady-state flux vectors. The 'constraints' parameter is used to descirbe the flux states that should not be eliminated from the flux space. Depending on the goal of the strain design computation, undesired flux states may be those with production of an undesired by-product, low product synthesis rates, low product yields or even flux states with microbial growth. In addition to constraints, an inner objective function can be defined that is enforced. In that case flux states are suppressed that are optimal with respect to the specified inner objective function and additionally fulfil the specified constraints. If the goal is to find minimal sets of knockouts that are lethal for an organism, one can use a suppress module with the constraint parameter: constraints='growth >= 0.01'. The algorithm will then find thes smallest sets of knockouts that render flux states with growth >= 0.01 infeasible. If we take the example of production strain design, one could use the suppress module to enforce a certain product yield (prod/subst > min_yield) growth-maximal flux states. This can be done by defining an inner objective function for optmizing growth and selecting a minimum production threshold to be attained at maximum growth: inner_objective='1.0 growth', constraints='EX_prod_e - min_yield*UP_subst_e <= 0'. If used without any constraints, the suppress module ensures that the model is infeasible.

mandatory arguments: model, module_type='suppress' optional arguments: constraints, inner_objective, inner_opt_sense, skip_checks, reac_ids (Detailed description of the arguments follow below)

Protect:

MCS-like protection of a subspace of all steady-state flux vectors. The 'constraints' parameter is used to descirbe flux states that should become/stay feasible when/by introducing interventions. This can be used to maintain or protect certain metabolic functions, like microbial growth, despite engineering a strain for bioroduction. When provided with an inner objective, the protect module will ensure that flux vectors optimal with respect to the inner objective function will be able to fulfil the constraints set in the 'constraints' parameter. This can be used to design potentially growth-coupled strains with a minimum set of metabolic interventions. As an example, if one uses the protect module to ensure that growth with rates above 0.1/h remains feasible, one sets constraints='growth >= -0.1'. If the goal is to ensure that product synthesis is possible at rates of more than 5 mmol/gCDW/h at maximal growth, one sets: inner_objective='1.0 growth' and constraints='EX_prod_e >= 5'. If used without any constraints, the protect module just ensures that the model is feasible.

mandatory arguments: model, module_type='protect' optional arguments: constraints,inner_objective, inner_opt_sense, skip_checks, reac_ids (Detailed description of the arguments follow below)

Example

m = SDModule(model,'optknock',outer_objective='growth', inner_objective='EX_etoh_e', constraints='growth >= 0.2')

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class. Instead of a model, a dummy object can be used as long as it has the field 'id'. If a dummy is used, skip_checks=True must be used and a list of reaction ids must be provided through reac_ids=*list_of_strings*.
- **module_type** (*str*) – A string that specifies the module type. Allowed values are 'optknock', 'robustknock', 'optcouple', 'protect', 'suppress'. Depending on the specified module type, other parameters must be set accordingly (see description above).
- **constraints** (*optional (str) or (list of str) or (list of [dict,str, float])*) – (Default: '')

  List of *linear* constraints to be used in the module, e.g., to be enforced, suppressed or taken into account: signs + or -, scalar factors for reaction rates, inclusive (in)equalities and a float value on the right hand side. The parsing of the constraints input allows for some flexibility. Correct (and identical) inputs are, for instance: constraints='-EX_o2_e <= 5, ATPM = 20' or constraints=['-EX_o2_e <= 5', 'ATPM = 20'] or constraints=[[{'EX_o2_e':-1},'<=',5], [{'ATPM':1},'=',20]]
- **inner_objective** (*optional (str) or (dict)*) – The *linear* inner objective function for any module type. This parameter is mandatory for OptKnock, RobustKnock and OptCouple modules and optional for suppress and protect modules. If used, an optimization is nested into the global problem. It can be used to account for the biological objective of growth (inner_objective='BIOMASS_Ecoli_core_w_GAM'). Any linear expression can be used as input, either as a single string or as a dict. Correct (and identical) inputs are, for instance: inner_objective='BIOMASS_Ecoli_core_w_GAM - 0.05 EX_etoh_e' inner_objective={'BIOMASS_Ecoli_core_w_GAM': 1, 'EX_etoh_e': -0.05}
- **inner_opt_sense** (*optional (str)*) – (Default: 'maximize')

  Sense of the inner optimization (maximization or minimization). Allowed values are 'minimize' and 'maximize'.
- **outer_objective** (*optional (str) or (dict)*) – The *linear* outer objective function for any module type. This parameter is mandatory for OptKnock and RobustKnock and cannot be used with OptCouple, suppress and protect modules. If applied, this objective function is used as the global objective function. In case of OptCouple, the global objective is the maximization of the growth-coupling potential and the outer objective is not specified manually. Typical outer objectives are the optimization of product synthesis (outer_objective='BIOMASS_Ecoli_core_w_GAM'), but also combinations of product synthesis and growth are possible. Any linear expression can be used as input, either as a single string or as a dict. Correct (and identical) inputs are, for instance: outer_objective='BIOMASS_Ecoli_core_w_GAM + 0.1 EX_prod_e' outer_objective={'BIOMASS_Ecoli_core_w_GAM': 1, 'EX_prod_e': 0.1}
- **outer_opt_sense** (*optional (str)*) – (Default: 'maximize')

  Sense of the outer optimization (maximization or minimization). Allowed values are 'minimize' and 'maximize'.
- **prod_id** (*optional (str) or (dict)*) – The reaction id of the product of interest. This parameter is *only* used in OptCouple strain design and will have no effect as part of any

other module. Permitted is any linear expression either in the form of a string or as a dict: prod_id='EX_etoh' prod_id={'EX_etoh': 1}

- **min_gcp** (*optional (float)*) – (Default: 0.0)

  Minimial growth-coupling potential (GCP). I.e., the minimum difference between maximum growth with and without production. In practice there are two nested optimizations, one that optimizes inner_objective and one that optimizes inner_objective and additionally demands that the constraint: prod_id=0 holds. Therefore, min_gcp presents the minimum enforced growth-coupling potential, so, a minimum objective value. This parameter is supposed to be used with the 'any' approach and has (virtually) no effect when used with 'best' or 'populate', since GCP is maximized anyway.

- **skip_checks** (*optional (bool)*) – (Default: False)

  Skip the module verification. If checks are not skipped, the constructor will verify, if the module is feasible with the original model, that is, if all entered parameters parse correctly and if the given sets of constraints can be applied on the model without rendering it infeasible. Finally, it throws an error if the trivial 0-vector is feasible in the subspaces defined by a suppress or protect module, since the user should ensure that the 0-vector is excluded from these subspaces (see online documentation for detail).

Returns
A strain design module object that can be used with the function compute_strain_design. Multiple modules can be used to specify a strain design problem.

Return type
(*SDModule*)

Initialize self. See help(type(self)) for accurate signature.

copy()
Create a deep copy of a strain design module.

straindesign.strainDesignProblem

Classes and functions for the construction of strain design MILPs

This module contains functions that help construct mixed-integer linear problems, mainly functions that facilitate the construction of LP and Farkas dual problems from linear problems of the type A_ineq*x<=b_ineq, A_eq*x<=b_eq, lb<=x<=ub. The functions also help keeping track of the relationship of constraints and variables and their individual counterparts in dual problems, which is essential when simulating knockouts in dual problems. Most of the time, the sparse datatype is used to store and edit matrices for improved speed and memory.

Module Contents

class straindesign.strainDesignProblem.**ContMILP**(*A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*, *c*, *z_map_constr_ineq*, *z_map_constr_eq*, *z_map_vars*)

Continuous representation of the strain design MILP.

This MILP can be used to verify computation results. Since this class also stores the relationship between intervention variables z and corresponding (in)equality constraints and variables in the problem, it can be used to verify computed designs quickly and in a numerically stable manner.

class straindesign.strainDesignProblem.**SDProblem**(*model: cobra.Model*, *sd_modules: List[straindesign.SDModule]*, **args*, ***kwargs*)

Strain design MILP

The construcor of this class translates a model and strain design modules into a mixed integer linear problem. This class, however, is the backbone of the strain design computation. Preprocessing steps that enable gene, reaction and regulatory interventions, or network compression usually preceed the construction of an SDProblem-object and are integrated in the function *compute_strain_designs*.

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **sd_modules** (*(list of) straindesign.SDModule*) – Modules that specify the strain design problem, e.g., protected or suppressed flux states for MCS strain design or inner and outer objective functions for OptKnock. See description of SDModule for more information on how to set up modules.
- **ko_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated knockout costs. If not specified, all reactions are treated as knockout candidates, equivalent to ko_cost = {'r1':1, 'r2':1, ...}. If a subset of reactions is listed in the dict, all other are not considered as knockout candidates.
- **ki_cost** (*optional (dict)*) – (Default: None) A dictionary of reaction identifiers and their associated costs for addition. If not specified, all reactions are treated as knockout candidates. Reaction addition candidates must be present in the original model with the intended flux boundaries **after** insertion. Additions are treated adversely to knockouts, meaning that their exclusion from the network is not associated with any cost while their presence entails intervention costs.
- **max_cost** (*optional (int)*) – (Default: inf): The maximum cost threshold for interventions. Every possible intervention is associated with a cost value (1, by default). Strain designs cannot exceed the max_cost threshold. Individual intervention cost factors may be defined through ki_cost and ko_cost.
- **solver** (*optional (str)*) – (Default: same as defined in model / COBRApy) The solver that should be used for preparing and carrying out the strain design computation. Allowed values are 'cplex', 'gurobi', 'scip' and 'glpk'.
- **M** (*optional (int)*) – (Default: None) If this value is specified (and non-zero, not None), the computation uses the big-M method instead of indicator constraints. Since GLPK does not support indicator constraints it uses the big-M method by default (with COBRA standard M=1000). M should be chosen 'sufficiently large' to avoid computational artifacts and 'sufficiently small' to avoid numerical issues.
- **essential_kis** (*optional (set)*) – A set of reactions that are marked as addable and that are essential for at least one of the strain design modules. Providing such "essential knock-ins" may speed up the strain design computation.

Returns

An instance of SDProblem containing the strain design MILP

Return type

(*SDProblem*)

addModule(*sd_module*)

Generate module LP and z-linking-matrix for each module and add them to the strain design MILP

Parameters

sd_module (*straindesign.SDModule*) – Modules to describe strain design problems like protected or suppressed flux states for MCS strain design or inner and outer objective functions for OptKnock. See description of SDModule for more information on how to set up modules.

link_z()

Connect binary intervention variables to variables and constraints of the strain design problem

Function that uses the maps between intervention indicators z and variables and constraints of the linear strain design (in)equality system (self.z_map_constr_ineq, self.z_map_constr_eq and self.z_map_vars) to set up the strain design MILP.

MILP construction uses the following steps:

(1) Translate equality-KOs/KIs to two inequality-KOs/KIs

(2) Translate variable-KOs/KIs to inequality-KIs/KOs

(3) Try to bound the problem with LPs

(4) Use LP-determined bounds to link z-variables, where such bounds were found

(5) Translate remaining inequalities back to equalities when possible and link z via indicator constraints. If necessary, the solver interface will translate them to big-M constraints. (6) Remove redundant equalities from static problem

straindesign.strainDesignProblem.**LP_dualize**(*A_ineq_p*, *b_ineq_p*, *A_eq_p*, *b_eq_p*, *lb_p*, *ub_p*, *c_p*, *z_map_constr_ineq_p=None*, *z_map_constr_eq_p=None*, *z_map_vars_p=None*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple, Tuple, Tuple, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix]

Translate a primal system to its LP dual system

The primal system must be given in the standard form: A_ineq x <= b_ineq, A_eq x = b_eq, lb <= x < ub, min{c'x}. The LP duality theorem defines a set of two problems. If one of the LPs is a maximization and and optimum exists, the optimal value of this LP is identical to the minimal optimum of its LP dual problem. LP duality can be used for nested optimization, since solving the primal and the LP dual problem, while enfocing equality of the objective value, guarantees optimality.

Construction of the LP dual:

Variables translate to constraints:
x={R} -> = x>=0 -> >= (new constraint is multiplied with -1 to translate to <= e.g. -A_i' y <= -c_i)
x<=0 -> <=

Constraints translate to variables:
= -> y={R} <= -> y>=0

Parameters

- **A_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **b_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **A_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **b_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **lb_p** (*list of float*) – Upper and lower variable bounds in vector form.
- **ub_p** (*list of float*) – Upper and lower variable bounds in vector form.

- **c_p** (*list of float*) – The objective coefficient vector of the primal minimization-LP. z_map_constr_ineq_p, z_map_constr_eq_p, z_map_vars_p
- **z_map_constr_ineq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.
- **z_map_constr_eq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.
- **z_map_vars** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.

Returns

(Tuple): The LP dual of the problem in the format: A_ineq, b_ineq, A_eq, b_eq, c, lb, ub and optionally also z_map_constr_ineq, z_map_constr_eq, z_map_vars

straindesign.strainDesignProblem.**build_primal_from_cbm**(*model*, *V_ineq=None*, *v_ineq=None*, *V_eq=None*, *v_eq=None*, *c=None*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple, Tuple, Tuple, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix]

Builds primal LP from constraint-based model and (optionally) additional constraints.

standard form: A_ineq x <= b_ineq, A_eq x = b_eq, lb <= x <= ub, min{c'x}. Additionally, this function also returns a set of matrices that associate each variable (and constraint) with reactions. In the primal problems all variables correspond to reactions (z), therefore, the z_map_vars matrix is an identity matrix. The constraints correspond to metabolites, thus z_map_constr_ineq, z_map_constr_eq are all-zero.

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class
- **V_ineq** (*sparse.csr_matrix, list of float*) – Linear inequality constraints of the form V_ineq*x <= v_ineq. Ensure that the number of columns in V_ineq is identical to the

number of reactions in the model.

- **v_ineq** (*sparse.csr_matrix, list of float*) – Linear inequality constraints of the form V_ineq*x <= v_ineq. Ensure that the number of columns in V_ineq is identical to the number of reactions in the model.
- **V_eq** (*sparse.csr_matrix, list of float*) – Linear equality constraints of the form V_eq*x = v_eq. Ensure that the number of columns in V_eq is identical to the number of reactions in the model.
- **v_eq** (*sparse.csr_matrix, list of float*) – Linear equality constraints of the form V_eq*x = v_eq. Ensure that the number of columns in V_eq is identical to the number of reactions in the model.
- **c** (*list of float*) – Object coefficient vector (same lenght as variable vector).

Returns

A_ineq, b_ineq, A_eq, b_eq, lb, ub, c, z_map_constr_ineq, z_map_constr_eq, z_map_vars. A constraint-based steady-state model in the form of a linear (in)equality system. The matrices z_map_constr_ineq, z_map_constr_eq, z_map_vars contain the association between reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities.

Return type

(Tuple)

straindesign.strainDesignProblem.**farkas_dualize**(*A_ineq_p*, *b_ineq_p*, *A_eq_p*, *b_eq_p*, *lb_p*, *ub_p*, *z_map_constr_ineq_p=None*, *z_map_constr_eq_p=None*, *z_map_vars_p=None*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple, Tuple, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix]

Translate a primal system of linear (in)equality to its Farkas dual

The primal system must be given in the standard form: A_ineq x <= b_ineq, A_eq x = b_eq, lb <= x < ub. Farkas' lemma defines a set of two systems of linear (in)equalities of which exactly one is feasible. Since the feasibility of one is a certificate for the infeasibility of the other one, this theorem can be used to set up problems that imply the infeasibility and thus exclusion of a certain subspace. This priciple is used for MCS calculation (the SUPPRESS module).

Consider that the following is not implemented: In the case of (1) A x = b, (2) x={R}, (3) b~=0, Farkas' lemma is special, because b'y ~= 0 is required to make the primal infeasible instead of b'y < 0. 1. This does not occur very often. 2. Splitting the equality into two inequalities that translate to y>=0 would be posible, and yield b'y < 0 in the farkas' lemma. Maybe splitting is required, but I actually don't think so. Using the special case of b'y < 0 for b'y ~= 0 should be enough.

Parameters

- **A_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **b_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **A_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **b_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **lb_p** (*list of float*) – Upper and lower variable bounds in vector form.

- **ub_p** (*list of float*) – Upper and lower variable bounds in vector form.
- **z_map_constr_ineq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.
- **z_map_constr_eq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.
- **z_map_vars** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated for the dualized LP, if not, the dual problem is constructed without returning information about these relationships.

Returns

(Tuple): The Farkas dual of the linear (in)equality system in the format: A_ineq, b_ineq, A_eq, b_eq, lb, ub and optionally also z_map_constr_ineq, z_map_constr_eq, z_map_vars

straindesign.strainDesignProblem.**prevent_boundary_knockouts**(*A_ineq*, *b_ineq*, *lb*, *ub*, *z_map_constr_ineq*, *z_map_vars*) → Tuple[scipy.sparse.csr_matrix, Tuple, Tuple, Tuple, scipy.sparse.csr_matrix]

Put negative lower bounds and positive upper bounds into (notknockable) inequalities

This is a helper function that puts negative lower bounds and positive upper bounds into (not-knockable) inequalities. Later on, one may simulate the knockouts by multiplying the upper and lower bounds with a binary variable z. This functions prevents that

Parameters

- **A_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **b_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **lb_p** (*list of float*) – Upper and lower variable bounds in vector form.
- **ub_p** (*list of float*) – Upper and lower variable bounds in vector form.

- **z_map_constr_ineq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated. Otherwise, all reactions are assumed to be knockable and thus all negative upper and positive lower bounds are translated into constraints.
- **z_map_vars** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated. Otherwise, all reactions are assumed to be knockable and thus all negative upper and positive lower bounds are translated into constraints.

Returns

(Tuple): A linear (in)equality system in the format: A_ineq, b_ineq, A_eq, b_eq, lb, ub and optionally also updated z_map_constr_ineq, z_map_constr_eq

straindesign.strainDesignProblem.**reassign_lb_ub_from_ineq**(*A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*, *z_map_constr_ineq=None*, *z_map_constr_eq=None*, *z_map_vars=None*) → Tuple[scipy.sparse.csr_matrix, Tuple, scipy.sparse.csr_matrix, Tuple, Tuple, Tuple, scipy.sparse.csr_matrix, scipy.sparse.csr_matrix]

Remove single constraints in A_ineq or A_eq in favor of lower and upper bounds on variables

Constraints on single variables instead translated into lower and upper bounds (lb, ub). This is useful to filter out redundant bounds on variables and keep the (in)equality system concise. To avoid interference with the knockout logic, negative upper bounds and positive lower bounds are not put into lb and ub, when reactions are flagged knockable with z_map_vars.

Parameters

- **A_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **b_ineq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear inequalities of the primal LP A_ineq_p*x <= b_ineq_p
- **A_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **b_eq_p** (*sparse.csr_matrix and list of float*) – A coefficient matrix and a vector that describe the linear equalities of the primal LP A_eq_p*x <= b_eq_p
- **lb_p** (*list of float*) – Upper and lower variable bounds in vector form.
- **ub_p** (*list of float*) – Upper and lower variable bounds in vector form.
- **z_map_constr_ineq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions,

metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated. Otherwise, all reactions are assumed to be notknockable and thus all constraints on single variables put into lb and ub.

- **z_map_constr_eq** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated. Otherwise, all reactions are assumed to be notknockable and thus all constraints on single variables put into lb and ub.
- **z_map_vars** (*optional (sparse.csr_matrix)*) – Matrices that contain the relationship between metabolic reactions and different parts of the LP, such as reactions, metabolites or other (in)equalities. These matrices help keeping track of the parts of the LP that are affected by reaction knockouts and additions. When a reaction (i) knockout removes the variable or constraint (j), the respective matrix contains a coefficient 1 at this position. -1 marks additions. E.g.: If the knockout of reaction i corresponds to the removal of inequality constraint j, there is a matrix entry z_map_constr_ineq_(i,j) = 1. If these matrices are provided, they are updated. Otherwise, all reactions are assumed to be notknockable and thus all constraints on single variables put into lb and ub.

Returns

(Tuple): A linear (in)equality system in the format: A_ineq, b_ineq, A_eq, b_eq, lb, ub and optionally also updated z_map_constr_ineq, z_map_constr_eq

straindesign.strainDesignProblem.**worker_compute**(*i*) → Tuple[int, float]

Helper function for determining bounds on linear expressions

straindesign.strainDesignProblem.**worker_init**(*A*, *A_ineq*, *b_ineq*, *A_eq*, *b_eq*, *lb*, *ub*, *solver*, *seed*)

Helper function for determining bounds on linear expressions

straindesign.strainDesignSolutions

Container for strain design solutions (SDSolutions)

Module Contents

class straindesign.strainDesignSolutions.**SDSolutions**(*model*, *sd*, *status*, *sd_setup*, ***, *_lazy_init=None*)

Bases: object

Container for strain design solutions

Objects of this class are returned by strain design computations. This class contains the metabolic interventions on the gene, reaction or regulation level alongside with information about the strain design setup, including the model used and the strain design modules. Strain design solutions can be accessed either through the fields or through specific functions that preprocess or reformat strain designs for different purposes.

Instances of this class are not meant to be created by StrainDesign users.

Parameters

- **model** (*cobra.Model*) – A metabolic model that is an instance of the cobra.Model class.
- **sd** (*list of dict*) – A list of dicts every dict represents an intervention set. Keys in each dict are reaction/gene identifiers and the associated value determines if it is added (1), not added (0) or knocked out (-1). For regulatory interventions, (1) means active regulation and (0) means regulatory intervention not added. These will be translated to True and False.
- **status** (*str*) – Status string of the computation (e.g.: 'optimal')
- **sd_setup** (*dict*) – A dictionary containing information about the problem setup. This dict can/should contain the keys MODEL_ID, MODULES, MAX_SOLUTIONS, MAX_COST, TIME_LIMIT, SOLVER, KOCOST, KICOST, REGCOST, GKICOST, GKOCOST

  These entries can be set like this: sd_setup[straindesign.MODEL_ID] = model.id

Returns

(SDSolutions): Strain design solutions

__add__(*other*)

Merge two SDSolutions objects, returning a new object.

__iadd__(*other*)

In-place merge of two SDSolutions objects (deduplicates at compressed level).

expand_all(*n_per_group=None*)

Expand all compressed groups.

Parameters

n_per_group – None means all, int means keep up to n per group.

expand_group(*grp_idx*)

Expand one compressed group on demand.

Returns list of expanded solution dicts. Also updates reaction_sd, sd_cost, itv_bounds, and group_map in place.

get_gene_reac_sd_assoc(*i=None*)

Get reaction and gene-based strain design solutions, and show which reaction-based solution corresponds to which gene-based.

Often the association is not 1:1 but n:1.

get_gene_reac_sd_assoc_mark_no_ki(*i=None*)

Get reaction and gene-based strain design solutions, but also tag knock-ins that were not made with a 0

Often the association is not 1:1 but n:1.

get_gene_sd(*i=None*)

Get gene-based strain design solutions

get_gene_sd_mark_no_ki(*i=None*)

Get gene-based strain design solutions, but also tag knock-ins that were not made with a 0

get_group(*i*)

Get all expanded solution indices that belong to the same compressed group as solution i.

Returns a list of indices into reaction_sd that share the same compressed solution origin. Requires that compute_strain_designs was called with compression enabled.

get_num_groups()

Get the number of distinct compressed solution groups.

get_num_materialized()

Number of currently materialized solutions in reaction_sd.

get_num_sols()

Get number of solutions

get_reaction_sd(*i=None*)

Get reaction-based strain design solutions

Gene-based intervention sets are translated to the reaction level. This can be helpful to understand the impact of gene interventions. GPR-rules are accounted for automatically.

get_reaction_sd_bnds(*i=None*)

Get reaction-based strain design solutions represented by upper and lower bounds

Knocked-out reactions will show as upper and lower bounds of zero.

get_reaction_sd_mark_no_ki(*i=None*)

Get reaction-based strain design solutions, but also tag knock-ins that were not made with a 0

This can be helpful to analyze gene intervention sets in original metabolic models. GPR-rules are accounted for automatically.

get_representative_sd()

Get one representative expanded solution per compressed group.

Returns a list of dicts, one per unique compressed solution.

get_strain_design_costs(*i=None*)

Get costs of i-th strain design or of all in a list

get_strain_designs(*i=None*)

Get i-th strain design (intervention set) or all in original format

classmethod load(*filename*)

Load strain design solutions from a file.

save(*filename*)

Save strain design solutions to a file.

property is_lazy

True if lazy expansion is active (some groups unexpanded).

straindesign.strainDesignSolutions.**get_subset**(*sd*, *i*)

SDSolutions internal function: getting a subset of solutions

straindesign.strainDesignSolutions.**gpr_eval**(*cj_terms*, *interv*)

SDSolutions internal function: evaluate a GPR term

straindesign.strainDesignSolutions.**strip_non_ki**(*sd*)

SDSolutions internal function: removing non-added reactions or genes

4.9.1.2 Package Contents

class straindesign.**DisableLogger**

Environment in which logging is disabled

4.10 Legacy Methods

This page documents optional legacy functionality that requires additional dependencies beyond the core StrainDesign installation.

4.10.1 Java-based EFMTool compression (`backend='efmtool_rref'`)

The default compression backend is `backend='sparse_rref'`, a pure Python implementation with no extra dependencies. A legacy Java-based backend is available for comparison or reproducibility purposes.

To use it, install the optional Java dependency:

```
pip install straindesign[java]
```

or:

```
pip install jpype1
```

Then pass `backend='efmtool_rref'` to `compress_model()` or to `compute_strain_designs` via the `backend` keyword argument.

4.10.2 JAVA_HOME path

In some cases, using the `efmtool_rref` backend may fail with:

```
JVMNotFoundException: No JVM shared library file (libjli.dylib) found. Try setting up the
JAVA_HOME environment variable.
```

In this case, make sure Java is installed correctly and the JAVA_HOME variable is set. See JAVA_HOME environment variable for platform-specific instructions.

If you're on OS X and get the error

```
OSError: [Errno 0] JVM DLL not found
```

check that your Java and the JPype library is set up correctly. The easiest way to avoid this error is to use conda to install StrainDesign and Java together.

4.11 StrainDesign Developer's Guide

> **Reference paper:** Schneider P., Bekiaris P. S., von Kamp A., Klamt S. — *StrainDesign: a comprehensive Python package for computational design of metabolic networks.* Bioinformatics, btac632 (2022). DOI: 10.1093/bioinformatics/btac632

4.11.1 Table of Contents

1. Architecture Overview
2. Repository Structure
3. Public API & Entry Points
4. Preprocessing Pipeline
    - 4.1 Model Cleaning
    - 4.2 Compression Pass 1

- 4.3 FVA
- 4.4 GPR Integration
- 4.5 Compression Pass 2
- 4.6 Network Compression — Algorithm Details
- 4.7 Regulatory Interventions

5. MILP Construction
   - 5.1 Overview & z-map Matrices
   - 5.2 Primal LP from COBRApy Model
   - 5.3 SUPPRESS Module — Farkas' Lemma Dual
   - 5.4 PROTECT Module
   - 5.5 OPTKNOCK — LP Duality for Bilevel Optimization
   - 5.6 ROBUSTKNOCK — Three-Level Duality
   - 5.7 OPTCOUPLE — Growth Coupling Potential
   - 5.8 Linking Binary Variables: link_z
   - 5.9 Big-M Bounding
   - 5.10 Indicator Constraints vs. Big-M
6. Solver Backends
   - 6.1 MILP_LP Unified Interface
   - 6.2 CPLEX
   - 6.3 Gurobi
   - 6.4 SCIP
   - 6.5 GLPK
7. Computation Approaches — SDMILP
   - 7.1 ANY
   - 7.2 BEST
   - 7.3 POPULATE
   - 7.4 Exclusion Constraints
   - 7.5 Solution Verification
8. Post-processing & Result Container
   - 8.1 Solution Decompression
   - 8.2 GPR Translation
   - 8.3 SDSolutions
9. Constraint & Expression Parsing
10. Supporting Utilities
    - 10.1 LP Suppression (networktools.py)
11. Known Issues, Urgent Actions & Future Work

12. MILP Performance Notes
13. Testing

4.11.2 1. Architecture Overview

StrainDesign solves **metabolic strain design** problems: given a genome-scale metabolic model (COBRApy `Model`) and a set of biological objectives, find the minimal sets of gene/reaction knockouts (and additions) that force the network into a desired flux state.

All methods reduce to **Mixed-Integer Linear Programs (MILP)**. The key intellectual contribution is the translation of nested/bilevel biological objectives into a flat MILP via LP duality and Farkas' lemma, so a single MILP call suffices to find strain designs. The pipeline is:

```
User (model + SDModules)
        │

   Preprocessing        networktools.py, compression.py, lptools.py
   – FVA cleanup
   – GPR extension
   – Network compression
        │

   MILP Construction    strainDesignProblem.py  (SDProblem)
   – build_primal_from_cbm
   – farkas_dualize / LP_dualize
   – link_z (bounding + indicator constraints)
        │

   Solver Execution     strainDesignMILP.py  (SDMILP)
   – ANY / BEST / POPULATE loops
   – verify_sd
   – exclusion constraints
        │

   Post-processing      compute_strain_designs.py, strainDesignSolutions.py
   – expand_sd (decompress)
   – GPR translation
   – SDSolutions
```

All methods (**MCS/suppress/protect**, **OptKnock**, **RobustKnock**, **OptCouple**) use the same pipeline — only how `addModule` builds the LP blocks differs.

4.11.3 2. Repository Structure

```
straindesign/
├── __init__.py                    # Package exports & avail_solvers detection
├── names.py                       # All string constants (solver names, module types,␣
↪etc.)
│
├── compute_strain_designs.py      # Main user-facing orchestration function
```

(continues on next page)

(continued from previous page)

```
├── strainDesignModule.py           # SDModule: problem specification (a dict subclass)
├── strainDesignProblem.py          # SDProblem: translates model+modules → MILP␣
↪matrices
├── strainDesignMILP.py             # SDMILP: solves the MILP, manages solution loop
├── strainDesignSolutions.py        # SDSolutions: result container, GPR translation
│
├── compression.py                  # Network compression (RREF, RationalMatrix,␣
↪nullspace)
├── networktools.py                 # GPR extension, regulatory extension, compress␣
↪wrappers, LP suppression
├── lptools.py                      # FVA, flux space plotting, solver selection
├── parse_constr.py                 # Constraint/expression string → matrix conversion
├── indicatorConstraints.py         # IndicatorConstraints data class
├── solver_interface.py             # MILP_LP factory: instantiates the right solver
├── cplex_interface.py              # CPLEX backend (Cplex_MILP_LP)
├── gurobi_interface.py             # Gurobi backend (Gurobi_MILP_LP)
├── scip_interface.py               # SCIP backend (SCIP_MILP_LP)
├── glpk_interface.py               # GLPK backend (GLPK_MILP_LP)
├── efmtool_cmp_interface.py        # EFMtool JAR interface (legacy compression backend)
├── pool.py                         # SDPool: Windows-compatible multiprocessing pool
└── efmtool.jar                     # Bundled EFMtool binary
```

Tests are in:

```
tests/
└── *.py   (pytest test files, one per feature area)
```

Configuration / packaging:

```
setup.py           # python_requires=">=3.7"; classifiers include 3.9-3.13
conda-recipe/meta.yaml
.github/workflows/CI-test.yml
```

4.11.4 3. Public API & Entry Points

4.11.4.1 Primary function

```python
from straindesign import compute_strain_designs, SDModule

sols = compute_strain_designs(
    model,
    sd_modules   = [sd_module1, sd_module2],
    solver       = 'gurobi',      # 'cplex' | 'gurobi' | 'scip' | 'glpk'
    max_cost     = 3,
    max_solutions= 10,
    solution_approach = 'best',   # 'any' | 'best' | 'populate'
    ko_cost      = {'r1': 1, 'r2': 2},   # reaction knockouts
    ki_cost      = {'r3': 1},             # reaction additions
    gko_cost     = {'geneA': 1},          # gene knockouts
    gki_cost     = {'geneB': 1},          # gene additions
```

(continues on next page)

(continued from previous page)

```
    time_limit    = 3600,
)
```

4.11.4.2 SDModule specification

```
# MCS-style: suppress undesired subspace, protect desired one
SDModule(model, 'suppress', constraints='EX_product_e >= 0.1')
SDModule(model, 'protect',  constraints='BIOMASS >= 0.05')

# With inner optimization (force to inner optimum first)
SDModule(model, 'suppress',
         constraints='EX_product_e >= 0.1',
         inner_objective='BIOMASS', inner_opt_sense='maximize')

# OptKnock
SDModule(model, 'optknock',
         inner_objective='BIOMASS',
         outer_objective='EX_product_e')

# RobustKnock
SDModule(model, 'robustknock',
         inner_objective='BIOMASS',
         outer_objective='EX_product_e')

# OptCouple
SDModule(model, 'optcouple',
         inner_objective='BIOMASS',
         prod_id='EX_product_e',
         min_gcp=0.1)
```

4.11.4.3 Result access

```
sols.status           # 'optimal', 'infeasible', 'time_limit', ...
sols.reaction_sd      # list of dicts: {reac_id: -1 (KO) | +1 (KI) | 0 (not added KI)}
sols.gene_sd          # list of dicts: gene-level interventions (if gene_kos=True)
sols.get_strain_designs()          # same as reaction_sd
sols.get_gene_reac_sd_assoc()      # (reac_sd, assoc, gene_sd) with GPR linkage
sols.get_strain_design_costs()     # list of floats
sols.get_reaction_sd_bnds()        # intervention expressed as flux bounds
```

4.11.4.4 Lower-level access

```
from straindesign import SDProblem, SDMILP, MILP_LP
from straindesign import fva, select_solver
from straindesign.compression import compress_cobra_model
```

4.11.5 4. Preprocessing Pipeline

`compute_strain_designs` (`compute_strain_designs.py`) orchestrates all preprocessing before constructing the MILP. The pipeline uses **two-pass compression**: the model is compressed once *before* GPR extension (so that FVA and GPR processing operate on a smaller model), then again *after* GPR extension (to compress the newly added gene pseudoreactions). This roughly halves model size at each downstream step and yields a ~4x end-to-end speedup on genome-scale models like iML1515.

4.11.5.1 4.1 Model Cleaning

Functions: `remove_ext_mets`, `remove_dummy_bounds` (in `networktools.py`, re-exported from `compression.py`).

Steps:

1. **Remove dummy bounds** — COBRApy uses ± 1000 as a proxy for $\pm\infty$. Anything above a `bound_thres` (from `cobra.Configuration`) is replaced with $\pm$`inf` so big-M bounding works correctly.
2. **Deep-copy** the model so the user's model is never mutated.
3. **Remove external metabolites** — metabolites with no stoichiometric coupling to the rest of the network (boundary metabolites already handled by COBRApy exchange reactions, but orphaned ones are pruned).

4.11.5.2 4.2 Compression Pass 1 (before GPR extension)

File: `compression.py` **Orchestrated by:** `networktools.compress_model`

Before FVA or GPR processing, the model is compressed with `propagate_gpr=True`. This merges coupled and parallel reactions while propagating GPR rules through the compression map: when reactions are lumped, their GPR rules are AND-combined (using sympy `And`/`Or` constructors for flattening and deduplication). The result is a much smaller model that retains correct GPR annotations.

After compress #1:

- `compress_modules(sd_modules, cmp_mapReac_1)` — rewrites constraint/objective reaction IDs.
- `compress_ki_ko_cost(ko_cost, ki_cost, cmp_mapReac_1)` — maps reaction and regulatory costs (gene costs are not yet present and are handled separately after GPR extension).

4.11.5.3 4.3 FVA

Function: `bound_blocked_or_irrevers_fva`, `fva` (from `lptools.py`), `speedy_fva` (from `speedy_fva.py`)

Now running on the **compressed model** (roughly half the reactions of the original), FVA identifies:

- **Blocked reactions** (FVA min = FVA max = 0): forced to zero, removed from intervention candidates.
- **Irreversible reactions** (FVA min >= 0 or FVA max <= 0): bounds tightened.
- **Essential reactions** (cannot be zero under module constraints): excluded from knockout candidates.

`fva()` in `lptools.py` delegates to `speedy_fva()`, which replaces the legacy implementation with a two-phase approach that reduces LP count by 70-80% on genome-scale models.

speedy_fva algorithm

Phase 1 — Scan LPs resolve the majority of bounds cheaply:

1. **v=0 feasibility (Phase 1a):** If `lb[j] <= 0 <= ub[j]`, then zero flux is feasible, so `min x_j = 0` (or `max x_j = 0` when the bound is one-sided). No LP needed.

2. **min(sum|x|) scan LP (Phase 1b):** Builds an extended LP with splitting variables for reversible reactions to minimize total absolute flux. The optimal vertex pushes many reactions to zero or their bounds. A vectorized **bound scan** marks any reaction whose flux equals its variable bound (within tolerance).
3. **Push-to-bounds iteration (Phase 1c):** Using the same LP with dual simplex warm-start, iteratively sets directed objectives that push unresolved reactions toward their upper/lower bounds. Each round: set `c[j] = -1` for unresolved-max reactions, solve, scan; then `c[j] = +1` for unresolved-min reactions, solve, scan. Stops when a round resolves fewer than 5 new bounds. Warm-started dual simplex makes each re-solve near-instant (the basis stays primal feasible after objective changes).

Phase 2 — Individual LPs for remaining unresolved objectives:

- **Sequential mode** (default for small/medium models): warm-started dual simplex with periodic LP rebuild (every 200 solves) to limit degeneration. Each LP vertex is checked for **co-optimization**: a bound scan on the full solution vector that may resolve other unresolved directions for free.
- **Parallel mode** (for large models, configurable via `threads`): process-based `SDPool` with per-worker LP instances.

Optional compression: `speedy_fva` can compress the model internally (auto-enabled for n >= 200). This uses single-pass nullspace compression + conservation removal, with a fast model copy that avoids expensive `deepcopy(solver)`.

Benchmarks (Gurobi, iML1515, compressed, single-threaded):

Configuration	Standard FVA	speedy_fva	LP count	Speedup
Unconstrained	90s	15s	71% fewer	6.0x
Biomass >= 0.1	17s	13s	41% fewer	1.3x

Phase 1 resolves 70-80% of objectives in the unconstrained case, but only ~40% when constraints limit the feasible space (fewer reactions can achieve their variable bounds).

Design decisions and rejected alternatives

KKT dual certification (investigated, not pursued): We explored using KKT optimality conditions to certify co-optimal reactions without solving additional LPs. After solving `min x_j`, we obtain a vertex solution and its basis. For another reaction k, optimality at the same vertex requires dual variables satisfying `S^T lambda + mu_ub - mu_lb = e_k` with correct complementarity. This can be checked by:

1. Extracting the basis B from the solver
2. Computing reduced costs for objective `e_k`: `pi = B^{-T} e_{pos(k)}`, then `r_i = -A_i^T pi` for nonbasic variables
3. Verifying sign conditions: `r_i >= 0` for nonbasic-at-lb, `r_i <= 0` for nonbasic-at-ub

The problem is that **bound scanning already handles nonbasic variables** (they are at their bounds by definition in a simplex vertex). KKT certification only adds value for *basic* variables (at interior points), which requires a sparse LU factorization of B per LP solve (1-5ms for m=900) — comparable to or exceeding the cost of the warm-started dual simplex LP itself (~0.5ms). The overhead exceeds the savings.

An earlier implementation (`DualChecker`) used null-space projection to test dual feasibility via sparse LU. On **compressed models**, this produced zero certifications because compression creates linearly dependent column subsets (coupled reactions with diagonal values of 2/3 instead of 1), causing the factorization check to reject all candidates. On uncompressed models certifications were possible but the null-space dimension was too large (d=39-304 on iMLcore) for efficient LP-based certification. The approach was removed in favor of the simpler bound scan, which captures the same information for nonbasic variables at zero cost.

Intelligent objective ordering (considered, marginal benefit): After each LP solve, the basis determines which variables are basic (interior) vs nonbasic (at bounds). Solving next for a basic variable should require fewer dual

simplex pivots than jumping to a distant nonbasic variable. However, after bound scanning, the remaining unresolved reactions are precisely those that were basic (interior) in *every* solution seen — these "hard" reactions resist being pushed to bounds regardless of ordering. The objective change is always a rank-1 update, and pivot count depends more on problem structure than ordering. Warm-started dual simplex is already so fast per-LP (~0.5ms) that ordering heuristics cannot recover their own overhead.

4.11.5.4 4.4 GPR Integration

File: `networktools.py` **Key functions:** `reduce_gpr`, `extend_model_gpr`

When gene-level interventions are requested (`gko_cost` or `gki_cost` provided), the model is **extended** with artificial reactions that represent gene-level knockouts. This allows the MILP's binary variables `z` to operate on genes rather than reactions. Because the model has already been compressed, there are fewer reactions to extend and fewer GPR rules to process.

Step 1 — Reduce GPR rules (`reduce_gpr`)

Uses **AST-based GPR evaluation** (`evaluate_gpr_ast`):

- Each reaction's GPR rule is parsed to an AST (`ast.parse`).
- Genes are removed if their knockout/addition cannot affect any non-blocked, non-essential reaction.
- This is determined by evaluating the DNF/CNF GPR expression with gene states `{True, False, None (undetermined)}`.
- Reduces the gene intervention space before extension.

Step 2 — Extend model with artificial gene reactions (`extend_model_gpr`)

For each gene/isoenzyme group that participates in the GPR rules, artificial reactions and metabolites are added to encode the boolean logic:

- **AND gate**: If a reaction requires `g1 AND g2`, an artificial metabolite and two artificial reactions are added. Knocking out either gene removes the pathway.
- **OR gate**: If `g1 OR g2`, parallel paths are created. Both must be knocked out to abolish the reaction.

The function returns a `reac_map` dict mapping original reactions to the new artificial ones in the extended model. This map is used later to:

1. Translate `gko_cost`/`gki_cost` into reaction costs on the extended model.
2. Expand computed solutions back to gene-level interventions.

Important: Gene names starting with digits are prefixed with `'g'` (`cobra.manipulation.rename_genes`) because Python AST cannot parse identifiers starting with digits.

Step 3 — GPR solution translation (`SDSolutions.__init__`)

After computation, the artificial reactions in solutions are translated back to gene-level interventions using `gpr_eval`. This evaluates each solution's intervention set against the DNF GPR rules to determine which reactions are truly knocked out (or added).

`gpr_eval(cj_terms, interv)`:

- `cj_terms`: the GPR rule as a list of "conjunctive clauses" (AND-groups within OR expression).
- `interv`: current gene intervention state.
- Returns `True/False/nan` (active/inactive/undetermined).

4.11.5.5 4.5 Compression Pass 2 (after GPR extension)

After GPR extension adds gene pseudoreactions, the model is compressed a second time. This pass merges newly coupled gene pseudoreactions and parallel reactions introduced by the GPR encoding. The `no_par_compress_reacs` set is rebuilt from the (now rewritten) `sd_modules` to protect constraint/objective reactions.

The final compression map is the concatenation of both passes: `cmp_mapReac = cmp_mapReac_1 + cmp_mapReac_2`. When decompressing solutions, `expand_sd` processes this list in reverse order (pass 2 first, then pass 1).

A final FVA pass identifies essential reactions in the fully compressed model and removes them from the knockout candidates.

4.11.5.6 4.6 Network Compression — Algorithm Details

File: `compression.py` **Orchestrated by:** `networktools.compress_model`

Network compression reduces model size while preserving the strain design problem. It is **lossless** — all solutions in the compressed space correspond to solutions in the original, and vice versa (via the `cmp_mapReac` mapping).

RationalMatrix

The core data structure for numerically exact RREF computation. It uses **sparse dual-integer storage** (numerator matrix + denominator matrix, both `scipy.sparse.csr_matrix`). Operations are performed in rational arithmetic using `fractions.Fraction` and `sympy.Rational`. The model stays in exact rational arithmetic throughout both compression passes; no float conversion is performed between passes.

Key operations: row operations, scaling, finding pivots, extracting nullspace.

Floating-point coefficients are converted via `float_to_rational` with configurable precision (`detect_max_precision` scans the model's stoichiometry).

Compression Algorithm (StoichMatrixCompressor)

The main class `StoichMatrixCompressor` performs the following in each cycle, repeating until no further reduction is possible:

1. **Parallel reactions** (`compress_model_parallel`): Reactions with identical stoichiometry (same column in `S`) are merged via hash-based O(nnz) comparison. This is cheap and runs first to shrink the model before the expensive RREF step. The resulting "super-reaction" carries the combined cost, and the z-variable controls all of them simultaneously. Hash matches are verified with exact index/data comparison to prevent collisions.
2. **Conservation relation removal** (`remove_conservation_relations`): Compute the left nullspace of the stoichiometric matrix `S` over the rationals (homogeneous dependencies among metabolite rows). Linearly dependent rows are removed. This reduces the row count of `S` for the subsequent RREF computation.
3. **Exit check**: If the loop has completed at least one full cycle (including a coupled step) and neither parallel nor coupled compression reduced the reaction count, exit. This ensures the model is always fully cleaned (conservation relations removed) on exit.
4. **Coupled reactions** (`compress_model_coupled`): Two reactions `r_i` and `r_j` are **coupled** if there exists a scalar `alpha` such that in every feasible steady state, `v_i = alpha * v_j`. These can be lumped into one reaction. Coupling is detected by computing the right nullspace (RREF) and identifying proportional columns. Lumpable reactions are merged and their costs combined. When `propagate_gpr=True`, GPR rules are AND-combined during merging. **Contradicting groups** (where bounds intersection yields lb > ub or lb = ub = 0) are detected and removed.

Blocked reactions are removed once before the loop via `remove_blocked_reactions`.

Compression Result

`cmp_mapReac`: a list of dicts, one per compression pass. Each dict maps compressed reaction IDs to their original reactions with scaling factors.

This is used by:

- `compress_modules(sd_modules, cmp_mapReac)` — rewrites constraints/objectives in terms of compressed reaction IDs.
- `compress_ki_ko_cost(cmp_ko_cost, cmp_ki_cost, cmp_mapReac)` — maps intervention costs.
- `expand_sd(sd, cmp_mapReac)` — after solving, maps binary solutions back to original reactions (processes passes in reverse order).

Backend Selection

The compression backend is chosen by the `compression_backend` parameter:

- `'sparse_rref'` (default): Pure Python, uses `RationalMatrix` for exact arithmetic. No Java dependency.
- `'efmtool_rref'` (legacy): Uses the bundled `efmtool.jar` via JPype for RREF. Requires `pip install straindesign[java]`. Available via `compress_model_efmtool`.

4.11.5.7 4.7 Regulatory Interventions

Function: `extend_model_regulatory` (`networktools.py`)

Regulatory interventions represent changes that activate or silence a pathway without directly knocking out a gene. They are encoded similarly to gene interventions: artificial reactions are added to the model, and their "presence" or "absence" is controlled by binary z-variables.

Regulatory constraints are **split by parsability**: constraints that reference only existing reaction IDs (e.g., `r6 >= 4.5`) are applied before compression pass 1; constraints that reference gene IDs (e.g., `g4 <= 0.4`) are deferred until after `extend_model_gpr` creates the corresponding pseudoreactions.

After solving, `postprocess_reg_sd` in `compute_strain_designs.py` converts regulatory intervention values from numeric (`1`/`0`) to boolean (`True`/`False`) in the solution dict.

4.11.6 5. MILP Construction

File: `strainDesignProblem.py` **Classes:** `SDProblem`, `ContMILP` **Key functions:** `build_primal_from_cbm`, `LP_dualize`, `farkas_dualize`, `reassign_lb_ub_from_ineq`, `prevent_boundary_knockouts`, `link_z`

4.11.6.1 5.1 Overview & z-map Matrices

The MILP has the form:

```
minimize   c' * x
subject to:
  A_ineq * x  <=  b_ineq
  A_eq   * x   =  b_eq
  lb  <=  x  <=  ub
  x[0..num_z-1]   {0,1}         (binary z-variables: interventions)
  x[num_z..]                   (continuous: flux variables + dual variables)
  indicator constraints or big-M
```

The first `num_z` variables are binary: `z_i = 1` means reaction `i` is knocked out (or added, if `z_inverted[i]`).

Three **z-map matrices** track which z-variables affect which parts of the LP:

- `z_map_vars[i, j]`: z-variable `i` controls continuous variable `j` (knockout/addition of that reaction flux).
- `z_map_constr_ineq[i, j]`: z-variable `i` controls inequality constraint `j`.
- `z_map_constr_eq[i, j]`: z-variable `i` controls equality constraint `j`.

A value of +1 means "z=1 removes this constraint/variable" (knockout); -1 means "z=1 adds this" (knock-in).

These maps are propagated through all dualization steps (see below) and ultimately used in `link_z` to wire up big-M coefficients or indicator constraints.

MILP header rows (fixed positions):

- Row 0 (`idx_row_maxcost`): `cost' z  0` (ensures non-negative cost)
- Row 1 (`idx_row_mincost`): `cost' z  max_cost`
- Row 2 (`idx_row_obj`): objective constraint (used by `fixObjective`)

4.11.6.2 5.2 Primal LP from COBRApy Model

Function: `build_primal_from_cbm(model, V_ineq, v_ineq, V_eq, v_eq, c=None)`

Constructs the standard LP from a COBRApy model:

```
A_eq  * v  =  b_eq    (stoichiometric matrix S stacked with V_eq)
A_ineq* v  <= b_ineq  (V_ineq constraints, e.g. flux ratio constraints)
lb <= v <= ub         (reaction bounds from model)
```

- `A_eq = [S; V_eq]`, `b_eq = [0; v_eq]`
- `z_map_vars = I` (identity — each z directly maps to its reaction variable)
- `z_map_constr_ineq = 0`, `z_map_constr_eq = 0` (in the primal, knockouts affect variables, not constraints)

`prevent_boundary_knockouts`: If a variable has `lb < 0` or `ub > 0`, knocking it out to zero requires converting these non-zero bounds into inequality constraints. This prevents situations where a "zero flux" constraint on a reaction with `lb = 1000` would be applied incorrectly.

4.11.6.3 5.3 SUPPRESS Module — Farkas' Lemma Dual

Function: `farkas_dualize(A_ineq_p, b_ineq_p, A_eq_p, b_eq_p, lb_p, ub_p, ...)`

The **SUPPRESS** module wants to make a certain flux subspace infeasible. Farkas' lemma provides a certificate of infeasibility:

Farkas' Lemma: The system `{A_ineq x  b_ineq, A_eq x = b_eq, lb  x  ub}` is **infeasible** if and only if there exist multipliers `y` such that:

```
A_ineq' y_ineq + A_eq' y_eq = 0   (dual feasibility)
b_ineq' y_ineq + b_eq' y_eq < 0   (certificate: b'y < 0)
y_ineq  0
```

The Farkas dual is constructed by calling `LP_dualize` with zero objective (`c = 0`), yielding dual variables `y`. Then the constraint `c_dual' y  1` is appended (equivalently: `b_ineq' y_ineq + b_eq' y_eq  1`), which enforces the Farkas infeasibility certificate.

When an intervention (z) removes a primal constraint or variable, the corresponding dual variable is also removed — this is the mechanism by which knockouts "activate" the SUPPRESS condition. When the dual system becomes feasible with the 1 constraint, the corresponding primal flux subspace is infeasible, i.e., the targeted flux state is suppressed.

Known limitation: For the special case `A x = b, x , b 0` (homogeneous equality with unrestricted variable), the Farkas certificate requires `b'y 0` rather than `< 0`. This edge case is **not currently implemented** (see section 11).

4.11.6.4 5.4 PROTECT Module

The **PROTECT** module keeps a flux subspace feasible. It uses the **primal** LP directly (`build_primal_from_cbm`) without dualization, then calls `reassign_lb_ub_from_ineq` to tighten bounds. As long as the primal system is feasible, the protected state is reachable. The objective is set to zero (feasibility only).

4.11.6.5 5.5 OPTKNOCK — LP Duality for Bilevel Optimization

OptKnock solves:

```
maximize  outer_objective(v)
s.t.      v  argmax { inner_objective(v) : v feasible }
```

This bilevel program is converted to a single-level MILP using **LP duality**:

LP Duality Theorem: For a minimization LP `{min c'x : Ax b, x 0}`, the dual is `{max b'y : A'y c, y 0}`. At optimality, primal and dual objectives are equal: `c'x* = b'y*`.

`LP_dualize(A_ineq_p, b_ineq_p, A_eq_p, b_eq_p, lb_p, ub_p, c_p, ...)`

Constructs the dual LP. The primal constraints become dual variables and vice versa:

Primal	→	Dual
`x 0`		`y c_i` (inequality constraint)
`x`		`y = c_i` (equality constraint)
`x 0`		`y c_i` (flipped inequality)
`A_ineq row`		`y_ineq 0`
`A_eq row`		`y_eq`
Objective `b'y`		`max` → `min`, `b` becomes new objective

The OptKnock MILP block is assembled as:

```
# Build: [primal (with constraints) | dual (without constraints)]
# Connect: c_inner_primal == c_inner_dual (strong duality)
A_eq_combined = block_diag(A_eq_v, A_eq_dual) stacked with
                hstack(c_primal, c_dual_objective) == 0
```

The outer objective is set as the MILP objective. The strong duality equality guarantees that `v` is at the inner optimum.

4.11.6.6 5.6 ROBUSTKNOCK — Three-Level Duality

RobustKnock solves a **min-max** problem:

```
maximize  min_v { outer(v) : v  argmax inner(v) }
```

This requires **two applications of LP duality**:

1. First, the inner bilevel problem `{inner(v) at max}` is dualized into primal+dual pair (like OptKnock).
2. The combined primal+dual system is treated as a new primal, and **dualized again** to convert the outer minimization to a maximization.

The result is a three-layer LP construction assembled into a single MILP block. The outer objective (minimized production, maximized over worst case) becomes the MILP objective.

This is the most complex module and the most sensitive to numerical issues.

4.11.6.7 5.7 OPTCOUPLE — Growth Coupling Potential

OptCouple maximizes the **Growth Coupling Potential (GCP)**:

```
GCP = max_v { inner(v) }  max_v { inner(v) | prod = 0 }
```

The two terms are the max growth with and without production. The objective is their difference.

Construction:

1. Build primal for the "with production" LP → `(A_ineq_p, ..., c_p)`.
2. Build primal for the "baseline" LP (no production constraint) → `(A_ineq_b, ..., c_b)`.
3. Dualize the baseline LP → `c_dual_b` (dual objective coefficients).
4. Assemble combined block with strong duality for both LPs.
5. Final objective: `c_p - c_b` (growth with production minus baseline).

`min_gcp` parameter sets a lower bound on GCP in the MILP.

4.11.6.8 5.8 Linking Binary Variables: `link_z`

Function: `link_z` in `SDProblem` (called at end of `__init__`).

This is the step that physically wires the binary variables `z` into the MILP. It proceeds in 5 steps:

Step 1 — Split knockable equalities into pairs of inequalities: An equality `A_eq[j] x = b` that is controlled by z is split into `A_ineq x  b` and `A_ineq x  b`. Both inequalities are individually linked to z, so when z is activated, both are activated together (enforcing zero).

Step 2 — Convert variable knockouts to inequality knockouts: A variable knockout (controlled by `z_map_vars`) is converted to: `x  0` and `x  0`. These bound constraints are added to `A_ineq` and linked to z via `z_map_constr_ineq`.

Step 3 — Compute big-M values via LP bounding (parallel): For each knockable constraint `A[i] x  b[i]`, compute:

```
M_i = max { A[i] x : x in most-relaxed feasible LP }
```

The "most-relaxed LP" removes all knockable constraints (allowing all knockouts) to get an upper bound on `A[i] x`. This gives the tightest valid big-M for each constraint.

Parallelized via `SDPool` when there are > 1000 constraints. Each worker (`worker_init`/`worker_compute`) holds a private LP object.

Step 4 — Link z via big-M for bounded constraints (finite M): For constraints where a finite `M_i` was found:

- z=1 knocks out `A[i] x  b`: add coefficient `M_i` in column z_i of row i: `A[i] x  M_i * z  b` → when z=1: `A[i] x  b + M_i` (always satisfied if `M_i` tight enough).
- z=0 knocks out: `A[i] x + (M_i  b) * z  M_i`.

Step 5 — Create indicator constraints for remaining knockables: Constraints where `M_i` = ∞ (unbounded LP subspace) cannot use big-M. These become **indicator constraints** passed to the solver backend:

```
z_i = indicval  →  A[i] x  b[i]   (sense 'L')
z_i = indicval  →  A[i] x  = b[i]  (sense 'E')
```

4.11.6.9 5.9 Big-M Bounding

The quality of big-M values is critical:

- Too small → valid solutions may be cut off (incorrect results).
- Too large → numerical instability, especially for GLPK.
- M = ∞ → the constraint must use an indicator constraint (solver-dependent, slower).

StrainDesign computes per-constraint tight bounds via LP. The LP maximizes `A[i] x` subject to the static (non-knockable) constraints, giving the tightest valid `M_i` for each knockable constraint.

For GLPK (which has no indicator constraint support), `M` defaults to the COBRApy bound threshold (default: 1000). This is conservative and may cause numerical issues on models with large flux ranges. Setting `M` manually via the `M` parameter allows tuning.

Round-up: computed Ms are ceiled to 5 significant decimal digits.

4.11.6.10 5.10 Indicator Constraints vs. Big-M

Feature	Indicator Constraints	Big-M
Solver support	CPLEX, Gurobi, SCIP	All (required for GLPK)
Numerical stability	Excellent (no large coefficients)	Can be poor with large M
Solver speed	Typically faster (native branching)	Depends on M tightness
When used	Unbounded constraints (M = ∞)	Bounded constraints (finite M)
Forced by user	`M=some_value` forces big-M everywhere	Default for GLPK

The `IndicatorConstraints` class (`indicatorConstraints.py`) stores:

- `binv`: indices of binary variables that trigger each constraint
- `A`: coefficient matrix for the triggered constraints
- `b`: RHS
- `sense`: `'L'` (), `'E'` (=), `'G'` ()
- `indicval`: which value of z (0 or 1) activates the constraint

Each solver backend translates `IndicatorConstraints` into its native format. For GLPK, the solver interface translates them to big-M constraints using the provided `M`.

Sign convention in z-maps (subtle!):

- `z_map_constr_ineq[i, j]` = +1: z_i = 1 → constraint j removed (knockout)
- `z_map_constr_ineq[i, j]` = 1: z_i = 1 → constraint j active (knock-in)

This sign is inverted when KIs vs. KOs are assigned (in `z_kos_kis` diagonal matrix at end of `SDProblem.__init__`), so the final indicator constraint `indicval` correctly maps to 0 (KO: constraint active when z=0) or 1 (KI: constraint active when z=1).

4.11.7 6. Solver Backends

4.11.7.1 6.1 MILP_LP Unified Interface

File: `solver_interface.py` **Class:** `MILP_LP`

`MILP_LP` is a **factory/dispatcher**: its `__init__` detects the requested solver and instantiates the appropriate backend class (`Cplex_MILP_LP`, `Gurobi_MILP_LP`, etc.). All backends expose the same interface:

```
class MILP_LP:
    solve()                         # → (x, opt, status)
    slim_solve()                    # → float (objective value only, fast)
    populate(n)                     # → (x_list, opt, status)  [CPLEX/Gurobi only]
    set_objective(c)
    set_ub(ub_list)                 # [(idx, val), ...]
    set_time_limit(t)
    add_ineq_constraints(A, b)
    add_eq_constraints(A, b)
    set_ineq_constraint(row, c, b) # Overwrite a specific row
    clear_objective()
    set_objective_idx(idx_val_list)
```

`slim_solve()` is used heavily for verification — it runs the LP and returns just the objective value (or `nan` for infeasible). This avoids extracting the full solution vector.

4.11.7.2 6.2 CPLEX

File: `cplex_interface.py` **Class:** `Cplex_MILP_LP`

- Native indicator constraint support (`cplex.indicator_constraints.add`)
- Solution pool (`populate_solution_pool`) for enumerating multiple solutions
- Thread control via `cplex.parameters.threads`
- Seed via `cplex.parameters.randomseed`
- Status mapped from CPLEX codes to StrainDesign names (`OPTIMAL`, `INFEASIBLE`, etc.)
- **Advantages:** Best solution pool support, fastest for large MILPs in practice, population-based enumeration.
- **Limitation:** Licensed (academic/commercial). No indicator constraints → automatic big-M fallback.

4.11.7.3 6.3 Gurobi

File: `gurobi_interface.py` **Class:** `Gurobi_MILP_LP`

- Native indicator constraint support (`gp.Model.addGenConstrIndicator`)
- Solution pool via Gurobi `SolCount` / `PoolSolutions`
- Seed via `Seed` parameter
- Status mapped from Gurobi `Status` codes
- **Advantages:** Often fastest solver, excellent solution pool. MIPFocus parameter available for tuning.
- **Note:** Requires a Gurobi license (free for academic use).

4.11.7.4 6.4 SCIP

File: `scip_interface.py` **Class:** `SCIP_MILP_LP`

- Indicator constraints via `pyscipopt addConsIndicator`
- No native solution pool → falls back to iterative `compute_optimal`
- Seed via `setSeed`
- **Advantages:** Open-source, no license needed, handles large models well.
- **Limitation:** No populate support; POPULATE mode falls back to BEST.

4.11.7.5 6.5 GLPK

File: `glpk_interface.py` **Class:** `GLPK_MILP_LP`

- **No indicator constraint support** — all indicator constraints are translated to big-M at construction time.
- Uses `swiglpk` (bundled with COBRApy).
- No solution pool → iterative enumeration only.
- **Advantages:** Always available without additional installation.
- **Limitation:** Poorest performance on large MILPs. Sensitive to M value choice. Default `M = 1000` (COBRApy bound threshold).

4.11.8 7. Computation Approaches — SDMILP

File: `strainDesignMILP.py` **Class:** `SDMILP` (inherits from `SDProblem` and `MILP_LP`)

`SDMILP.__init__` calls `SDProblem.__init__` to build the MILP matrices, then `MILP_LP.__init__` to set up the solver backend.

4.11.8.1 7.1 ANY

Method: `SDMILP.compute(**kwargs)`

Finds feasible solutions without enforcing global optimality:

1. Solve MILP as-is (no cost-minimization objective, just feasibility).
2. Verify each solution.
3. Add exclusion constraint.
4. Repeat until `max_solutions` or infeasibility.

For MCS problems, `z_sum  max_cost` and `z_sum  0` are the only cost constraints. Results may have suboptimal (higher) cardinality compared to BEST.

Warning: For OptKnock/RobustKnock, ANY may return the wild-type (z=0) immediately if that satisfies the outer objective. Use BEST for these methods.

4.11.8.2 7.2 BEST

Method: `SDMILP.compute_optimal(**kwargs)`

Finds globally optimal solutions:

1. Solve MILP with full objective (minimize cost for MCS; maximize outer for OptKnock, etc.).

2. Verify solution.
3. Add exclusion constraint (excludes solution and supersets for MCS; only the exact solution for OptKnock via `add_exclusion_constraints_ineq`).
4. Fix objective: `fixObjective(c_bu, opt)` — constrain next solution to have at least the same objective value.
5. Among solutions with equal objective, minimize intervention cost: `setMinIntvCostObjective()`.
6. Repeat until `max_solutions`, infeasibility, or time limit.

For MCS computations, the MILP first checks if the wild-type already satisfies all modules (via `verify_sd(sparse.csr_matrix((1, num_z)))`).

4.11.8.3 7.3 POPULATE

Method: `SDMILP.enumerate(**kwargs)`

Uses the solver's native solution pool (CPLEX/Gurobi):

1. Call `populateZ(max_solutions)` to generate a pool.
2. Verify all solutions in the pool.
3. Add exclusion constraints for invalid solutions.
4. Repeat.

Falls back to `compute_optimal` for SCIP and GLPK (which lack native pool support).

Gurobi pool notes: Pool quality/diversity can be tuned via Gurobi's `PoolSearchMode` and `PoolGap` parameters, but this is not currently exposed in the interface.

4.11.8.4 7.4 Exclusion Constraints

Methods: `add_exclusion_constraints(z)`, `add_exclusion_constraints_ineq(z)`

For a solution with binary vector `z`:

Case 1 — z is all zeros (wild-type): Add `sum(z_i)  1` → forces at least one intervention next time. Implemented as `1  1` (infeasible if all z = 0).

Case 2 — z has exactly one non-zero entry (single intervention): Set `ub[idx] = 0` → permanently disable that intervention.

Case 3 — z has multiple non-zero entries: Integer cut: `sum(z_i : i active)  sum(z_i_current)  1` This excludes the exact solution and all its **supersets** (for MCS: if removing r1 and r2 works, we don't want r1+r2+r3 as a "new" solution since it's a non-minimal superset).

`add_exclusion_constraints_ineq` uses a different cut that excludes **only the exact solution**, not its supersets (used for OptKnock where we want all optima).

4.11.8.5 7.5 Solution Verification

Method: `SDMILP.verify_sd(sols)`

After each MILP solution, the binary vector `z` is verified by building and solving a **continuous LP** with the corresponding constraint/variable removal applied:

1. From `cont_MILP` (saved before `link_z`): the original LP matrices with z-map information.
2. For each z_i in the solution: determine which variables (`z_map_vars`) and constraints (`z_map_constr_ineq`, `z_map_constr_eq`) are deactivated.
3. Build the restricted LP: remove deactivated constraints/variables.

4. Solve via `slim_solve()`.
5. If `slim_solve()` returns `nan` → infeasible → solution is valid (for SUPPRESS) or invalid (for PROTECT).

This catches numerical artifacts where the MILP returns a slightly non-integer solution that passes rounding but does not actually satisfy the biological constraint.

`cont_MILP` is the continuous pre-`link_z` version of the problem. It preserves the structural relationship between z-variables and constraints/variables before big-M coefficients obscure it. This is why the continuous validation is numerically robust even if M values were imprecise.

4.11.9 8. Post-processing & Result Container

4.11.9.1 8.1 Solution Decompression

Function: `expand_sd(sd, cmp_mapReac)` in `networktools.py`

After solving on the compressed model, solutions refer to compressed reaction IDs. `expand_sd` maps each compressed intervention back to the set of original reactions it represents (using `cmp_mapReac`).

- A single compressed KO may expand to multiple original reaction KOs (coupled/parallel reactions).
- The expanded solution preserves the cost structure.

Function: `filter_sd_maxcost(sd, max_cost, uncmp_ko_cost, uncmp_ki_cost)` After expansion, the cost of a solution in original space may differ from compressed space (due to parallel reactions with non-unit costs). Solutions exceeding `max_cost` in original space are filtered out.

4.11.9.2 8.2 GPR Translation

SDSolutions translates gene interventions to reaction phenotypes using `gpr_eval`.

For each gene-level solution `gene_sd`:

1. Collect all affected reactions from the model's GPR rules.
2. Evaluate each reaction's GPR with the gene states using `gpr_eval`.
3. Reactions whose GPR evaluates to `False` (given knockouts) are collected as reaction knockouts.
4. Reactions requiring knocked-in genes that are `True` (given additions) are collected as reaction additions.

`get_gene_reac_sd_assoc()` returns three parallel lists:

- `reac_sd`: reaction-level intervention dicts
- `assoc`: association dict showing which genes caused which reaction change
- `gene_sd`: the original gene interventions

4.11.9.3 8.3 SDSolutions

File: `strainDesignSolutions.py`

`SDSolutions` stores:

- `reaction_sd`: list of dicts `{reac_id: -1 | 0 | +1}` for each solution
- `gene_sd`: list of gene-level dicts (if gene interventions were used)
- `status`: solver status string
- `sd_setup`: full setup dict (model_id, modules, costs, solver, etc.) — can be serialized to JSON and reloaded

Serialization: `sd_setup` is JSON-compatible, allowing CNApy to save `.sd` files that can be reloaded into `compute_strain_designs` via the `sd_setup` dict parameter.

4.11.10 9. Constraint & Expression Parsing

File: `parse_constr.py`

Handles user-supplied constraints and objectives in multiple formats:

Input formats (all equivalent):

```
# String
"r1 + 3*r2 = 0.3, -5*r3 - r4 <= -0.5"

# List of strings
["r1 + 3*r2 = 0.3", "-5*r3 - r4 <= -0.5"]

# List of structured constraints
[[{'r1': 1.0, 'r2': 3.0}, '=', 0.3], [{'r3': -5.0, 'r4': -1.0}, '<=', -0.5]]
```

Key functions:

Function	Input → Output
`parse_constraints(constr, reaction_ids)`	Any format → `[[dict, sense, rhs], ...]`
`parse_linexpr(expr, reaction_ids)`	Any format → `[dict, ...]`
`lineq2mat(equations, reaction_ids)`	String list → `(A_ineq, b_ineq, A_eq, b_eq)` sparse matrices
`lineqlist2mat(eqlist, reaction_ids)`	Structured list → sparse matrices
`linexpr2dict(expr, reaction_ids)`	String → `{reac_id: coeff}`
`linexprdict2mat(expr_dict, reaction_ids)`	Dict → sparse row vector

The `parse_constraints` function uses regex splitting on `','` and `'\n'` to separate multiple constraints in a single string, then dispatches to `lineq2list` which uses further regex for coefficient/identifier/operator extraction.

4.11.11 10. Supporting Utilities

4.11.11.1 `lptools.py`

- **`fva(model, variables, constraints, solver)`**: Flux variability analysis. Parallelized via `SDPool` using `fva_worker_init`/`fva_worker_compute`. Returns a `pandas.DataFrame` with min/max flux values.
- **`select_solver(solver, model)`**: Selects solver based on availability, preference, and model settings. Priority: CPLEX > Gurobi > SCIP > GLPK.
- **`plot_flux_space(model, ...)`**: 2D/3D convex hull visualization using `scipy.spatial.ConvexHull` and Matplotlib.
- **`DisableLogger`**: Context manager to suppress logging during FVA.

4.11.11.2 networktools.py — LP Suppression

Problem

COBRApy's `Model` is tightly coupled to its solver backend (optlang). Every mutation — setting a reaction's bounds, renaming a reaction, adding/removing metabolites — immediately updates the solver's LP representation via `set_bounds`, `set_linear_coefficients`, `_populate_solver`, etc. During compression and preprocessing, hundreds of such mutations occur on a model copy. Synchronizing each one to the solver is pure overhead: the LP is rebuilt from scratch at the end anyway.

Early approaches patched optlang classes (`Constraint.set_linear_coefficients`, `Variable.set_bounds`, `Objective.set_linear_coefficients`) at the class level during suppression. This worked for optlang-level calls but missed cobra-level code paths: `Reaction.add_metabolites` accesses `self.forward_variable`, `self.reverse_variable`, and `model.constraints[met.id]` to build solver constraint updates. These paths reached the solver through cobra internals, not optlang's public API.

Architecture: class-level cobra patching

`suppress_lp_context(model)` is a context manager that patches **both optlang and cobra classes** at the class level for the duration of the block. All solver-bound operations become no-ops, and cobra methods that would access the solver are redirected to safe stubs.

Patches applied by `_suppress_lp_updates`:

Target	Original	Suppressed replacement
`Reaction._set_id_with_model`	Renames solver variable	`_suppressed_set_id`: writes `_id` + updates `DictList._dict` index only
`Reaction.lower_bound` (setter)	Calls `update_variable_bounds` → solver	`_suppressed_set_lb`: writes `_lower_bound` only
`Reaction.upper_bound` (setter)	Calls `update_variable_bounds` → solver	`_suppressed_set_ub`: writes `_upper_bound` only
`Reaction.update_variable_bounds`	Accesses `forward_variable.set_bounds(...)`	No-op
`Model._populate_solver`	Full solver rebuild	No-op
`Model.remove_reactions`	Removes solver variables/constraints	`_suppressed_remove_reactions`: removes from model lists only
`Model.remove_metabolites`	Removes solver constraints	`_suppressed_remove_metabolites`: removes from model lists only
`Container.__getitem__`	Raises `KeyError` for missing keys	`_permissive_container_getitem`: returns `_SolverStub` for missing keys
optlang `Constraint.set_linear_coefficients`	Updates solver constraint	No-op
optlang `Variable.set_bounds`	Updates solver variable bounds	No-op
optlang `Objective.set_linear_coefficients`	Updates solver objective	No-op

`_SolverStub` is a lightweight null object returned by the permissive `Container.__getitem__` when cobra code tries to access a solver variable/constraint that doesn't exist (because `_populate_solver` is suppressed). It is hashable (used as a dict key in `set_linear_coefficients` calls) and has no-op `set_bounds`/`set_linear_coefficients` methods. This single class eliminates the need to patch every individual cobra method that might touch the solver.

`_suppressed_set_id` and DictList invariant: cobra's `DictList._dict` maps `id → int index` (not `id → object`). The suppressed setter must maintain this invariant: it pops the old key and inserts the new key with the same integer index. Violating this causes `ValueError` in `DictList.__setitem__` due to its duplicate-check logic.

Objective preservation through compression

When `suppress_lp_context` enters, it captures the model's objective coefficients into `model._suppressed_obj` (a `dict` mapping `reaction_id → float coefficient`). This dict is updated by compression code as reactions are merged:

- **Coupled compression** (`compression.py`, `_apply_compression_to_model`): when reactions are merged, their objective contributions are weighted by compression factors and summed into the surviving reaction's entry.
- **Parallel compression** (`compression.py`, `compress_model_parallel`): when identical-stoichiometry reactions are merged, their objective contributions are summed directly.
- **Legacy Java compression** (`efmtool_cmp_interface.py`, `compress_model_java`): same logic using `jBigFraction2sympyRat` conversion factors.

On exit, if the model was modified (reaction IDs changed), the context manager rebuilds the solver from the model's current state and restores the objective from `_suppressed_obj`. If the model was not modified (e.g., `compute_strain_designs` only modifies a copy), no rebuild occurs — this is critical because an unconditional rebuild would produce a solver with the wrong objective direction for methods like RobustKnock that have inner/outer objectives.

```
# Simplified exit logic
current_ids = {r.id for r in model.reactions}
if current_ids != pre_ids:
    model._solver = solver_interface.Model()
    model._populate_solver(model.reactions, model.metabolites)
    for rxn in model.reactions:
        c = final_obj.get(rxn.id, 0.0)
        if c != 0:
            rxn.objective_coefficient = c
```

Design principles

1. **All plumbing in the context manager.** Compression code, GPR extension, and other preprocessing functions are backend-agnostic — they manipulate the cobra model without knowing whether LP updates are suppressed. The context manager handles all solver synchronization.
2. **Class-level patching, not instance-level.** Patching at the class level (e.g., `Reaction.lower_bound = property(fget, suppressed_fset)`) ensures that ALL reactions are covered, including ones created during compression. Instance-level patches would miss newly created objects.
3. **Conditional rebuild.** The solver is only rebuilt when the model was actually modified. This prevents regressions in methods like RobustKnock/OptKnock/OptCouple that have complex objective structures.

Performance impact

On iML1515, LP suppression reduces preprocessing time from ~131s to ~70s (46% reduction). The bulk of the savings comes from avoiding ~2000 `set_linear_coefficients` calls during compression, each of which would otherwise rebuild solver constraint data structures.

4.11.11.3 pool.py — SDPool

Windows-compatible `multiprocessing.Pool` subclass. On Windows, the initializer function is pickled to a temporary file and loaded by worker processes, avoiding a performance regression from passing the initializer directly (COBRApy issue #997). Uses `'spawn'` context universally to avoid `fork`-related instability.

4.11.11.4 names.py

All string constants. Importing with `from straindesign.names import *` is used throughout the codebase. Constants include module type strings, solver names, status codes, and parameter keys. This single-file approach ensures consistent naming.

Notable: `PROTECT = 'mcs_lin'` and `SUPPRESS = 'mcs_bilvl'` are defined first (legacy names), then immediately overwritten with `'protect'` and `'suppress'`. This is vestigial code — the legacy names are not used and could be removed.

4.11.11.5 __init__.py

Detects available solvers at import time:

```
avail_solvers = set()
try: import cplex; avail_solvers.add('cplex')
try: import gurobipy; avail_solvers.add('gurobi')
try: import pyscipopt; avail_solvers.add('scip')
try: import swiglpk; avail_solvers.add('glpk')
```

Exports: all public classes, functions, and the `avail_solvers` set.

Also calls `_start_jvm()` from `efmtool_cmp_interface.py` — see next section.

4.11.11.6 efmtool_cmp_interface.py — JPype/JVM Initialization

JPype and Java are optional dependencies. StrainDesign works without them; the default compression backend is `sparse_rref` (pure Python). The efmtool Java backend (`compression_backend='efmtool_rref'`) requires `jpype1` and a JVM.

Conditional eager JVM startup

`__init__.py` calls `_start_jvm()` at package import time. This function:

1. Checks if `jpype1` is installed (via `find_spec`). If not → immediate return, no-op.
2. Adds `efmtool.jar` to the classpath.
3. Starts the JVM with `jpype.startJVM("--enable-native-access=ALL-UNNAMED")`.
4. Loads all Java classes via `import jpype.imports` + Python import statements.
5. Registers `atexit` handler for clean JVM shutdown.

If any step fails (no Java installed, jar missing, etc.), the function returns silently. Users who never use `efmtool_rref` are unaffected. When `_init_java()` is later called (on first efmtool use), it checks `_JAVA_INITIALIZED` and falls back to `JClass()` loading.

Why eager, not lazy?

This design is critical for stability. In v1.14, `from .efmtool import *` in `__init__.py` started the JVM at import time. When we switched to lazy initialization (JVM started on first efmtool use), JPype began crashing with SIGBUS/SIGSEGV on Linux and macOS CI runners when processing larger matrices (iMLcore, 586 reactions).

Root causes identified:

- **JVM + OpenBLAS pthread conflict** (jpype#808): The JVM modifies pthread stack allocation. If started after NumPy/OpenBLAS has already spawned worker threads, subsequent JNI calls can trigger SIGSEGV.
- **GC finalization race** (jpype#934): Python's garbage collector can attempt to finalize JPype proxy objects during a JNI call, causing Bus error. Mitigated with `gc.disable()`/`gc.enable()` around heavy Java calls in `basic_columns_rat_java` and `compress_model_java`.
- **`import jpype.imports` vs `JClass()`**: The Python import-style class loading (`import ch.javasoft...`) sets up JNI references differently from `JClass()`. The import style (used in v1.14) is more stable on CI runners.

CI considerations

Java 21 (Temurin) is used on all CI platforms. The `--enable-native-access=ALL-UNNAMED` flag suppresses Java 17+ warnings about JPype's `System.load()` calls. Despite all mitigations, JPype JNI crashes remain non-deterministic at ~1-in-20 frequency on some Linux runners (jpype#934). The iMLcore efmtool parity tests are marked `--large` (skipped on CI) since the e_coli_core tests exercise the same code path on a smaller, more reliable matrix size.

4.11.12 11. Known Issues, Urgent Actions & Future Work

4.11.12.1 11.1 Urgent Action Items

Farkas special case (homogeneous equality): In `farkas_dualize`, there is a documented comment noting the unimplemented edge case:

> "In the case of (1) A x = b, (2) x , (3) b 0, Farkas' lemma is special, because `b'y 0` is required…"

This could cause silent incorrect results for SUPPRESS modules with equality constraints where `b 0` (e.g., free reactions with zero RHS). Impact: depends on whether the compressed, preprocessed models trigger this case. A defensive check + explicit splitting into two inequalities would fix it.

Big-M for GLPK with large flux ranges: GLPK uses `M = cobra_bound_threshold = 1000` by default. For models with non-standard bounds (e.g., ATPM = 8.39, but oxygen intake up to 10,000), this can produce incorrect strain designs. Models should always have `remove_dummy_bounds` applied, but verifying this is enforced is important.

`add_exclusion_constraints_ineq` has a bug:

```
A_ineq = [1.0 if z[j, i] else -1.0 for i in self.idx_z]
A_ineq.resize((1, self.A_ineq.shape[1]))  # ← list.resize doesn't exist
```

`list.resize` is a NumPy array method, not a list method. This would raise an `AttributeError` at runtime. This method is not used in the current main computation paths (OptKnock uses `add_exclusion_constraints`), but should be fixed. The fix: convert to `np.array` before calling `.resize`.

Legacy name double-assignment in `names.py`:

```
PROTECT = 'mcs_lin'
SUPPRESS = 'mcs_bilvl'
PROTECT = 'protect'      # overwrites silently
SUPPRESS = 'suppress'    # overwrites silently
```

The first assignments are dead code and should be removed to avoid confusion.

4.11.12.2 11.2 Low-Hanging Fruits

Solution space plotting issues (reported): `lptools.plot_flux_space` uses `scipy.spatial.ConvexHull` and `Matplotlib`. Known issues include:

- 3D plots with degenerate hulls (lower-dimensional polytopes) cause `QhullError`.
- Color scaling in `plot_flux_space_colored` may fail for single-point solutions.
- Fix: add degeneracy checks and fallback to scatter plots for 2 unique points.

Populate mode for SCIP/GLPK: Currently silently falls back to BEST. Should log a clear warning and document that SCIP/GLPK do not support POPULATE.

Gurobi pool tuning: The `populate` method in `gurobi_interface.py` could expose `PoolSearchMode=2` (focused enumeration) for better solution diversity. Currently uses default settings.

RobustKnock + POPULATE: RobustKnock with POPULATE is not well-tested. The three-level dual structure is particularly sensitive to solver pool heuristics. Add explicit test cases.

Parallel FVA compression for large models: For genome-scale models (> 2000 reactions), compression can be slow. The RREF step in `RationalMatrix` has $O(n^2)$ row operations. Potential improvement: use block-sparse RREF or exploit model structure (e.g., subsystems).

FVA Phase 2 parallelization for non-Gurobi solvers: Gurobi releases the GIL during `model.optimize()`, enabling `ThreadPoolExecutor`-based parallelism with batched warm-start (4.3x speedup measured on iML1515). CPLEX, GLPK, and SCIP hold the GIL, so they must use process-based `SDPool` with its ~4-6s spawn overhead. Investigating ctypes/Cython wrappers that release the GIL for these solvers could enable thread-based parallelism across all backends.

Essential KI detection: `essential_kis` is computed but may not always be passed correctly through the compression/GPR pipeline. Verify that the set persists through `compress_ki_ko_cost`.

Opposite-direction parallel merging: Two irreversible reactions with opposite stoichiometry and complementary bounds (e.g., `A→B (0,inf)` and `B→A (0,inf)`) could be merged into one reversible reaction `(-inf,inf)`. This would help eliminate flux cycles. However, a computed strain design might need to knock out one direction specifically (the other being essential), which limits the merger to non-knockable reactions. Few reactions are essential in both directions, so the practical benefit is small.

4.11.12.3 11.3 Future Development Directions

Nullspace-based primal/dual formalization: The referenced paper (DOI: 10.1093/bioinformatics/btz393, von Kamp & Klamt 2019) presents a nullspace-based approach to computing minimal cut sets. Integrating this formalization could:

- Avoid explicit dualization for certain model types.
- Give a more compact MILP representation.
- Improve numerical conditioning.

A key idea: the primal flux space has a right nullspace (internal cycles). The Farkas dual operates in the left nullspace. Exploiting this duality directly, rather than constructing full dual matrices, could reduce MILP size significantly.

Variable splitting: Reversible reactions split into forward/reverse can sometimes improve MILP performance (tighter LP relaxation). This is a pre-processing step that should be optional and tested for impact on different model types.

On the mathematical basis of big-M values (investigated, not pursued):

This section documents a thorough investigation into computing theoretically optimal big-M values for the Farkas dual MILP. The investigation confirmed the mathematical foundations but revealed that the problem is inherently as hard as MCS enumeration itself. No implementation was made; this section exists to prevent re-derivation.

Where M comes from — the primal view:

In the Farkas dual formulation (section 5.3), each knockable constraint `i` is linked to a binary z_i via either big-M (`A[i]x - M_i*z_i  b[i]`) or an indicator constraint (`z_i = 0 → A[i]x  b[i]`). The big-M value M_i must be large enough that when z_i = 1, the constraint `A[i]x  b[i] + M_i` is never binding — i.e., M_i max{A[i]x - b[i]} over all feasible x in every MCS that knocks out constraint i.

For SUPPRESS modules, the Farkas dual constraints encode the *primal* flux space through duality. Each knockable dual constraint corresponds to a primal reaction. The M value for reaction j's dual constraint is determined by the *primal* flux behavior:

M_j = 1 / min|v_j| taken over all Elementary Flux Vectors (EFVs) in which reaction j participates.

Here, an EFV is a support-minimal feasible flux distribution satisfying the target constraint. MCS are the minimal hitting sets of EFV supports. For reaction j to appear in any MCS of cardinality > 1, there must exist an EFV where j carries flux — and the *smallest* such flux across all EFVs determines the tightest valid M.

Why single-KO M is trivial:

For reactions that are individually essential (size-1 MCS candidates), M_j = 1/FVA_min(|v_j|). This is a single LP: minimize |v_j| subject to `Sv = 0`, bounds, and the target constraint. This is already computed during preprocessing FVA and is the basis for the current M-bounding in step 3 of `link_z`.

Why multi-KO M is hard:

For reactions that only appear in MCS of size 2, M_j depends on the *combinatorial structure* of which other reactions are simultaneously knocked out. The minimum |v_j| is not taken over the full flux space (where j can always carry large flux via alternative pathways) but over the restricted space where a specific set of other reactions are forced to zero — and j must be essential in that restricted space.

Formally, computing the optimal M_j requires solving:

```
M_j = max over all MCS K containing j of:
        1 / min{ |v_j| : Sv = 0, lb  v  ub, v_target  t,
                         v_i = 0 for i in K\{j} }
```

This is a max-min over an exponential number of MCS. Even computing the inner minimization for a *single* known MCS K is just an LP, but enumerating which K to consider requires MCS enumeration — the very problem M values are meant to help solve.

Combined primal/Farkas MILP — formulation and results:

We attempted to bypass MCS enumeration by embedding the combinatorial search in a single MILP. The formulation combines primal feasibility with a Farkas certificate:

- **Primal:** `S*v = 0`, `lb_i*(1-k_i)  v_i  ub_i*(1-k_i)` (big-M knockout), `v_target  target_lb`. Binary k_i = 1 knocks out reaction i.
- **Essential participation:** Sign binary forces `v_j` or `v_j -`.
- **Farkas certificate:** Same k_i binaries control dual constraints via indicators. When k_i = 0 (active): `S[:, i]'u - r_i + s_i = _{i=target}`. When k_i = 1 (knocked): `r_i = s_i = 0`. Farkas sum: `(ub_i*s_i - lb_i*r_i)  target_lb - .`
- **Objective:** minimize |v_j| = t, giving M_j = 1/t.

The Farkas certificate proves that removing j from the active set (while keeping the knocked reactions knocked) makes the system infeasible — i.e., j is essential in the knockout context defined by the k_i values. The optimizer simultaneously searches for the knockout set and the flux distribution where j's participation is minimized.

Practical obstacle — primal leakage:

The big-M knockout constraints `v_i  ub_i*(1-k_i)` allow leakage when k_i is within solver tolerance of 1 but not exactly 1. With Gurobi's default IntFeasTol = 10^5, a variable with k_i = 0.9999963 (accepted as integer 1) gives

leakage v_i ub_i × 3.7×10^{6}. With default bounds ±1000, this is 0.0037 per reaction. Across 15+ knocked reactions, the cumulative leakage invalidates the primal solution entirely — the model appears feasible when it should be infeasible.

This was diagnosed by inspecting k/v values at full precision (`tests/check_k_precision.py`): PIt2r showed k = 0.9999963, v = 0.003679, meaning the "knocked" reaction carried measurable flux that kept the system alive.

Mitigation — FVA-tight bounds + solver tolerances:

Two measures reduce leakage to negligible levels:

1. FVA-derived bounds (replace ±1000 with actual flux ranges, typically O(1)–O(10))
2. Tight solver tolerances (IntFeasTol = 10^{9}, FeasibilityTol = 10^{9}, NumericFocus = 3)

With both, per-reaction leakage drops to O(bound × 10^{9}) 10^{8}.

Validation on E. coli core (95 reactions, 353 MCS up to size 3):

Ground truth M values were computed independently: enumerate all MCS via indicator-based MILP (no big-M), then for each MCS, solve one LP per knocked reaction to find the minimum |v_j| in the Farkas certificate. M_gt[j] = max over all MCS containing j. (See `tests/ground_truth_M.py`.)

With tight tolerances and FVA bounds, the combined MILP matched ground truth exactly for 6 of 15 tested reactions, spanning 4 orders of magnitude:

Reaction	M_gt	M_milp	Ratio	#KO	Time
TALA	413,793	413,793	1.0000	15	~120s
TKT1	413,793	413,818	1.0001	16	~120s
NADH16	68,966	68,966	1.0000	6	~35s
EX_etoh_e	68,966	68,966	1.0000	23	~120s
ETOHt2r	68,966	68,966	1.0000	9	~120s
ATPM	0.12	0.12	1.0000	36	0.8s

Reactions with intermediate M values (PFL: 329, NADTRHD: 0.91) did not converge within 120s, returning inflated values. The combinatorial search space is too large for the solver to explore efficiently.

Why we did not pursue this further:

The investigation confirmed that the formulation is mathematically correct — it produces exact M values when the solver converges. However, it also confirmed the fundamental insight: **computing the optimal M_j for multi-reaction MCS is a combinatorial problem of the same complexity as MCS enumeration itself.** There is no shortcut:

1. For size-1 MCS: M = 1/FVA_min — already computed cheaply via LP.
2. For size-k MCS (k 2): the optimal M depends on which *other* reactions are simultaneously knocked out, requiring exploration of the space of knockout combinations. This is exactly what the MCS MILP does.
3. The "FVA over the full Farkas polyhedron" approach (computing bounds with all constraints active) gives a *lower bound* on M — it tells you the range of dual variables when no constraints are relaxed. But in an actual MCS, some constraints ARE relaxed (z = 1), and the remaining dual variables may need larger values to construct a valid Farkas certificate. The FVA approach is a natural progression from the single-KO case but does not capture the combinatorial interaction that determines the true M for multi-reaction MCS.

In summary: the relationship M_j = 1/min|v_j| over EFVs containing j is exact but not efficiently computable. Any finite M chosen without enumerating MCS risks being too small (cutting off valid MCS) or too large (degrading solver performance). Indicator constraints avoid the M problem entirely at the cost of solver performance (lazy enforcement, weaker LP relaxation). This is the fundamental trade-off; there is no free lunch.

Investigation scripts were removed after documenting findings. The formulations above can be reconstructed from the mathematical descriptions if needed.

Reaction scaling for better-conditioned M values: Trace metabolites (e.g., MoYB at 7×10^7 mmol/gDW/h) create small inhomogeneous bounds in the primal, which after dualization produce small coefficients in the Farkas objective, forcing dual variables to be very large to satisfy `b^T u  -1`. This leads to M values spanning many orders of magnitude (irreducible condition number (S)), harming LP relaxation quality.

Column scaling (per reaction) addresses this: multiply flux variable `v_j` by a power-of-10 factor `_j` so that inhomogeneous bounds move into a reasonable range (e.g., [0.1, 10]):

- Stoichiometry: `S[:, j] /= _j` (coefficients shrink)
- Bounds: `lb_j *= _j`, `ub_j *= _j` (bounds grow)
- Mass balance: `(S[:,j]/_j) × (_j × v_j) = S[:,j] × v_j` (unchanged)

Row scaling (per metabolite) rebalances after column scaling: multiply metabolite row `i` by `_i` (power of 10) to bring the max coefficient back to O(1). This does not change the solution space (just scales the mass balance equation).

Implementation would go between `build_primal_from_cbm` and dualization in `addModule`. The scaling factors must be tracked for unscaling continuous solution values in non-MCS problem types (OptKnock, etc.). For MCS, only binary z-variables are returned, so unscaling is not needed.

The FVA bounds from `bound_blocked_or_irrevers_fva` (already computed in preprocessing) provide the tight flux ranges needed to determine `_j`. A per-module FVA considering module constraints could provide even tighter ranges.

Size-1 MCS via FVA on SUPPRESS modules: The current preprocessing (line 346–356 in `compute_strain_designs.py`) runs FVA on PROTECT (and OPTKNOCK etc.) modules to find essential reactions — reactions that must carry flux in the protected region — and marks them non-knockable. SUPPRESS modules are explicitly skipped (`if m[MODULE_TYPE] != SUPPRESS`).

The idea: also run FVA on SUPPRESS modules (without inner objective) to identify reactions essential for the target (undesired) region. Combining the two FVAs yields size-1 MCS cheaply and further reduces the knockable reaction set:

Step 1 — FVA on PROTECT (already implemented): Reactions where `min(|flux|) > 0` under PROTECT constraints are essential for the protected region. These are already marked non-knockable.

Step 2 — FVA on SUPPRESS (new): Reactions where `min(|flux|) > 0` under SUPPRESS constraints are essential for the target region. Knocking out any single one of these makes the target infeasible — each is a size-1 MCS candidate.

Step 3 — Cross-check: Size-1 candidates from step 2 that are NOT in the essential set from step 1 are true size-1 MCS: they disrupt the target without being required for the protected region. Add them to the output.

Step 4 — Mark non-knockable: All reactions identified in steps 1 and 2 can be removed from the knockable set for the main MILP:

- Step 1 essentials: knocking them out violates protection (already handled).
- Step 2 essentials not in step 1: they are size-1 MCS, already found. Since any larger MCS containing a size-1 MCS is not minimal, these reactions cannot participate in any new MCS. Safe to exclude.
- Step 2 essentials also in step 1: essential for both regions, obviously non-knockable.

Benefits:

- Size-1 MCS are found for free (just FVA, no MILP). This is the cheapest possible way to discover them.
- Fewer knockable reactions in the main MILP → fewer z-variables → smaller B&B tree.
- Reactions with tiny flux bounds (e.g., trace metabolite carriers at 1e-7) that are essential in the SUPPRESS region get excluded early, improving the conditioning of the main MILP's M-value computation.
- The model may simplify further (fewer knockables may enable additional compression passes or remove degenerate constraints), though the extent of this benefit is unclear.

Caveats:

- Only finds size-1 MCS. Size 2 MCS still require the full MILP.
- The inner objective (if present in the SUPPRESS module) is ignored for this FVA. This is correct for standard MCS but may miss reactions that are only essential under the bilevel structure.
- Adds O(2n) LP solves per SUPPRESS module (one FVA). For compressed models this is fast.

MILP warm-starting: When BEST generates multiple solutions iteratively, each call starts cold. Warm-starting from the previous solution's LP relaxation (available in CPLEX/Gurobi) could speed up subsequent solves significantly.

Improving RobustKnock numerics: The three-level dual construction in ROBUSTKNOCK creates very large matrices with potential for numerical blow-up. Applying scaling (row/column normalization) to the LP before and after dualization could help. Alternatively, use iterative refinement at the inner LP level.

SOS1/SOS2 constraints for bilevel formulations: Gurobi, CPLEX, and SCIP support SOS1 (Special Ordered Sets type 1) and SOS2 constraints natively. These could be useful for connecting variables of primal and KKT parts of multi-level optimization problems, encoding complementarity conditions (primal-dual coupling) more efficiently, or as an alternative to big-M for disjunctive constraints in bilevel formulations. SOS1 constraints enforce that at most one variable in a set is nonzero, which is exactly the complementarity condition in KKT-based reformulations.

Sparse matrix format conversions: The computation pipeline contains many sparse matrix format conversions (`tocsr`, `tocsc`, `todok`, `tocoo`) that may not all be necessary. A profiling pass could identify redundant conversions. Key areas: `SDProblem.__init__` performs multiple `tocsc`/`tocsr` conversions during `link_z`; `addModule` builds matrices in one format then converts for block assembly; `compression.py` converts between formats during RREF and coupled-reaction detection. Eliminating unnecessary conversions could reduce preprocessing time, especially for large models.

Integration of regulatory networks beyond simple T/F interventions: The current regulatory intervention model (active/inactive) is binary. Future work could integrate thermodynamic constraints or kinetic feasibility checks.

SCIP native solution enumeration: SCIP supports solution enumeration beyond the incidental pool (`limits/maxsol`, default 100). Since version 2.0, the `cons_countsols` constraint handler can count and enumerate all feasible solutions of a constraint integer program via the `count` command. Key details:

- `constraints/countsols/collect = TRUE` stores detected solutions (default FALSE = count only).
- To enumerate all *optimal* solutions: solve to optimality to get `c*`, add the objective as a constraint with both bounds equal to `c*`, then run `count`.
- Restarts must be turned off during counting (use `SCIPsetParamsCountsols()`).
- SCIP uses unrestricted subtree detection, which can detect several solutions at once, so a soft solution limit may be exceeded before SCIP stops.
- Collected solutions are stored with respect to active (non-presolved) variables only; they are lifted back into the original variable space when written to file.
- This is *not* equivalent to Gurobi's ranked near-optimal pool (`PoolSearchMode`). SCIP enumerates feasibility/optimality, not a ranked set of near-optimal solutions.

Currently, StrainDesign's SCIP `populate` method uses a custom workaround (iterative solve with solution exclusion constraints). Replacing this with native `cons_countsols` enumeration could improve performance and correctness. However, pyscipopt does not yet expose the `count` command or `cons_countsols` parameters — this would require upstream pyscipopt changes or direct C API calls.

pyscipopt gaps for LP method and basis control: Several SoPlex and SCIP features that would benefit StrainDesign are not exposed through the pyscipopt Python bindings:

- *LP basis get/set:* SCIP internally maintains LP bases via SoPlex, but pyscipopt does not expose `SCIPlpGetBasisInd`, `SCIPgetLPBasisInd`, or the SoPlex `getBasis`/`setBasis` methods. This prevents explicit basis warm-starting for LPs solved through the SCIP/SoPlex interface.

- *LP algorithm selection for SCIP_LP (pure LP via `pyscipopt.LP`):* The `pyscipopt.LP` class wraps SoPlex directly but does not expose SoPlex's `setIntParam(ALGORITHM, ...)` for selecting primal vs dual simplex. The SCIP_MILP path supports this via `lp/initalgorithm` and `lp/resolvealgorithm` parameters, but the direct SoPlex LP wrapper does not.
- *Solution enumeration:* The `count` command and `cons_countsols` constraint handler (see above) are not accessible from pyscipopt.

These gaps are candidates for upstream pyscipopt issues or contributions.

Improved CNApy integration: The `sd_setup` dict format is designed for CNApy interoperability. Ensuring full round-trip serialization (JSON → `compute_strain_designs` → JSON) with all parameter types (modules, costs, constraints) is an ongoing compatibility concern.

4.11.13 12. MILP Performance Notes

The following design choices have a measurable impact on MILP enumeration performance. They are listed roughly by impact, based on benchmarking with e_coli_core (95 rxns, 353 MCS) and iML1515 (1920 compressed rxns, 393 MCS).

Unbounded reactions save constraints and variables. Reactions with infinite bounds (`-inf` / `inf`) do not generate big-M or indicator constraints because no finite bound needs enforcement. Calling `remove_dummy_bounds` early (replacing ± 1000 with $\pm$inf) is therefore not just cosmetic — it directly reduces MILP size. Tightening bounds to FVA ranges has the opposite effect: more finite bounds means more constraints.

Omit non-knockable z-variables. The SD formulation creates one binary z per reaction, but only knockable reactions (non-zero cost, non-essential) actually need one. Non-knockable z-variables have ub=0 and are fixed to zero; with solver presolve disabled they become dead weight. `_trim_z_variables` removes them before the solver sees the problem. On iML1515: 462 → 205 binaries after trimming.

Aggressive compression. Two-pass network compression (before and after GPR extension) reduces the stoichiometric matrix substantially. On iML1515: 2712 → 1920 reactions, 1728 → 974 metabolites. Every removed reaction is one fewer z-variable, one fewer column in all constraint matrices, and fewer indicator constraints.

GPR propagation through compression. Merging GPR rules during compression (via `propagate_gpr=True`) and only running `extend_model_gpr` on the compressed model avoids creating gene pseudoreactions for reactions that were already merged away. This reduces both preprocessing time (FVA on ~190 vs ~2700 reactions) and final model size.

Condensed formulation (no reaction splitting). The SD "condensed" formulation handles reversible reactions without splitting them into forward/reverse pairs. This produces fewer z-variables and fewer indicator constraints than the "split" variant. Benchmarks show condensed is consistently faster (4.0s vs 4.1s on iMLcore, but the gap widens on larger models).

Auxiliary-variable indicators do not help. Replacing SD's complex indicators (`z=0` → `A·x = b`) with FLB-style auxiliary variables (`A·x + s = b` always, `z=0` → `s = 0`) was tested. The extra continuous variables and equality constraints outweigh the benefit of simpler indicator propagation (~15% slower on iMLcore).

Solver settings matter. Gurobi's presolve is critical for indicator-constraint MILPs. With `Presolve=0` (which was temporarily needed for a Gurobi 13 bug), solve times were ~1.6× slower across all formulations. On iML1515 the full pipeline went from ~12 minutes (presolve off) to ~7 minutes (presolve on).

4.11.14 13. Testing

Tests are in `tests/`. They are run with:

```
pytest tests -v --log-cli-level=INFO --junit-xml=test-results.xml
```

CI matrix (`.github/workflows/CI-test.yml`):

- OS: `ubuntu-latest`, `windows-latest`
- Python: `3.10`, `3.11`, `3.12`, `3.13`
- Package managers: `pip`, `conda`
- CPLEX excluded for Python 3.13 (max supported: 3.12)
- Segfault on Ubuntu (JPype JVM shutdown) handled via JUnit XML exit-code check

Key test areas to verify after changes:

1. Constraint parsing — any change to `parse_constr.py` must be verified against all input formats.
2. MILP construction — after changes to `link_z`, `build_primal_from_cbm`, or dualization functions: run with a small toy model and verify solutions against known correct answers.
3. Compression — changes to `compression.py` or `networktools.py` must verify that `expand_sd` correctly reconstructs original-space solutions.
4. Solver backends — changes to any `*_interface.py` file require testing with the specific solver installed.
5. GPR translation — changes to `networktools.extend_model_gpr` or `SDSolutions.gpr_eval` must be tested with models that have AND/OR GPR logic (e.g., iJO1366 for *E. coli*).

Adding new tests:

- Use a small toy model (3-5 reactions) for unit tests of MILP construction.
- Use `e_coli_core` for integration tests of the full pipeline.
- Use `iJO1366` for performance regression tests (track solve time).
- Always test with at least GLPK (always available) and one commercial solver if possible.

Performance profiling:

```
import cProfile
cProfile.run("compute_strain_designs(model, sd_modules=[...], solver='glpk')", 'profile_
↪out')
import pstats
pstats.Stats('profile_out').sort_stats('cumulative').print_stats(30)
```

The hot spots are typically: `link_z` (LP bounding), `fva` (preprocessing), and the solver's `solve()` loop.

4.11.15 Quick Reference: Data Flow Through Key Functions

```
compute_strain_designs(model, **kwargs)
    │
    ├─ remove_dummy_bounds(model)
    ├─ cmp_model = model.copy()
    ├─ remove_ext_mets(cmp_model)
    ├─ [if reg] split regulatory constraints:
```

(continues on next page)

(continued from previous page)

```
    │       ├── reaction-based → extend_model_regulatory(cmp_model) now
    │       └── gene-based → deferred until after GPR extension
    │
    ├── ─── COMPRESS #1 (before GPR extension) ───
    ├── compress_model(cmp_model, propagate_gpr=True) → cmp_mapReac_1
    ├── compress_modules(sd_modules, cmp_mapReac_1)
    ├── compress_ki_ko_cost(rxn_ko, rxn_ki, cmp_mapReac_1)
    │
    ├── ─── FVA (on compressed model) ───
    ├── bound_blocked_or_irrevers_fva(cmp_model, ...)
    ├── fva(cmp_model, ...) → essential_reacs
    │
    ├── ─── GPR extension (on compressed model) ───
    ├── [if gene_kos] rename_genes(cmp_model, ...)
    ├── [if gene_kos] reduce_gpr(cmp_model, essential_reacs, gkis, gkos)
    ├── [if gene_kos] extend_model_gpr(cmp_model, ...) → reac_map
    ├── [if gene_kos] apply deferred gene-based regulatory constraints
    ├── [if gene_kos] merge gene costs into cmp_ko_cost / cmp_ki_cost
    │
    ├── ─── COMPRESS #2 (after GPR extension) ───
    ├── compress_model(cmp_model) → cmp_mapReac_2
    ├── compress_modules(sd_modules, cmp_mapReac_2)
    ├── compress_ki_ko_cost(all_ko, all_ki, cmp_mapReac_2)
    ├── cmp_mapReac = cmp_mapReac_1 + cmp_mapReac_2
    │
    ├── fva(cmp_model, ...) → final essential_reacs
    │
    ├── ─── MILP construction & solving ───
    ├── SDMILP(cmp_model, sd_modules, ko_cost, ki_cost, solver, M, ...)
    │       ├── SDProblem.__init__(...)
    │       │       ├── [for each module] addModule(sd_module)
    │       │       │       ├── build_primal_from_cbm(model, V_ineq, v_ineq, ...)
    │       │       │       ├── [SUPPRESS] farkas_dualize(...)
    │       │       │       ├── [PROTECT]  reassign_lb_ub_from_ineq(...)
    │       │       │       ├── [OPTKNOCK] LP_dualize(...) + block_diag assembly
    │       │       │       ├── [ROBUSTKNOCK] LP_dualize x2
    │       │       │       └── [OPTCOUPLE] LP_dualize + GCP objective
    │       │       └── link_z()
    │       │               ├── split equalities → inequalities
    │       │               ├── variable KOs → inequality KOs
    │       │               ├── parallel LP bounding → big-M values
    │       │               ├── insert big-M into A_ineq
    │       │               ├── lump duplicate pairs → equalities
    │       │               └── remaining → IndicatorConstraints
    │       └── MILP_LP.__init__(c, A_ineq, ..., solver=solver)
    │               └── instantiate Cplex/Gurobi/SCIP/GLPK backend
    ├── [BEST]     sdmilp.compute_optimal(max_solutions, time_limit)
    ├── [ANY]      sdmilp.compute(max_solutions, time_limit)
    ├── [POPULATE] sdmilp.enumerate(max_solutions, time_limit)
    │       each iteration:
    │       ├── solveZ() / populateZ(n)
    │       ├── verify_sd(z) → valid[]
```

(continues on next page)

(continued from previous page)

```
    │       └── add_exclusion_constraints(z)
    ├── expand_sd(sd, cmp_mapReac)          ← processes pass 2, then pass 1
    ├── filter_sd_maxcost(sd, max_cost, ...)
    ├── postprocess_reg_sd(sd, ...)
    └── SDSolutions(model, sd, status, sd_setup)
            └── [if gene_kos] translate gene_sd → reaction_sd via gpr_eval
```

CHAPTER

FIVE

REFERENCES:

Ebrahim, A., Lerman, J.A., Palsson, B.O. et al. - COBRApy: COnstraints-Based Reconstruction and Analysis for Python. BMC Syst Biol 7, 74 (2013)

Burgard, A. P., Pharkya, P., & Maranas, C. D. - Optknock: a bilevel programming framework for identifying gene knockout strategies for microbial strain optimization. Biotechnology and bioengineering, 84(6), 647–657 (2003)

Tepper N., Shlomi T. - Predicting metabolic engineering knockout strategies for chemical production: accounting for competing pathways, Bioinformatics. Volume 26, Issue 4, Pages 536–543 (2010)

Jensen K., Broeken V., Lærke Hansen A.S., et al. - OptCouple: Joint simulation of gene knockouts, insertions and medium modifications for prediction of growth-coupled strain designs. Metabolic Engineering Communications, Volume 8 (2019)

Bestuzheva K., Besançon M., Chen W.K. et al. - The SCIP Optimization Suite 8.0. Available at Optimization Online and as ZIB-Report 21-41, (2021)

Marco Terzer, Jörg Stelling, Large-scale computation of elementary flux modes with bit pattern trees, Bioinformatics, Volume 24, Issue 19, (2008), Pages 2229–2235,

CHAPTER

SIX

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

S

straindesign, 99
straindesign.compression, 100
straindesign.compute_strain_designs, 104
straindesign.cplex_interface, 107
straindesign.efmtool_cmp_interface, 110
straindesign.glpk_interface, 111
straindesign.gurobi_interface, 114
straindesign.indicatorConstraints, 117
straindesign.lptools, 118
straindesign.names, 123
straindesign.networktools, 125
straindesign.parse_constr, 131
straindesign.pool, 134
straindesign.scip_interface, 135
straindesign.solver_interface, 139
straindesign.speedy_fva, 142
straindesign.strainDesignMILP, 143
straindesign.strainDesignModule, 146
straindesign.strainDesignProblem, 149
straindesign.strainDesignSolutions, 156

INDEX

Symbols

__add__() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
__exit__() (*straindesign.pool.SDPool method*), 134
__iadd__() (*straindesign.strainDesignSolutions.SDSolutions method*), 157

A

add_eq_constraints() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 108
add_eq_constraints() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
add_eq_constraints() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
add_eq_constraints() (*straindesign.scip_interface.SCIP_LP method*), 135
add_eq_constraints() (*straindesign.scip_interface.SCIP_MILP method*), 138
add_eq_constraints() (*straindesign.solver_interface.MILP_LP method*), 140
add_exclusion_constraints() (*straindesign.strainDesignMILP.SDMILP method*), 144
add_exclusion_constraints_ineq() (*straindesign.strainDesignMILP.SDMILP method*), 144
add_ineq_constraints() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 108
add_ineq_constraints() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
add_ineq_constraints() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
add_ineq_constraints() (*straindesign.scip_interface.SCIP_LP method*), 136
add_ineq_constraints() (*straindesign.scip_interface.SCIP_MILP method*), 138
add_ineq_constraints() (*straindesign.solver_interface.MILP_LP method*), 140
add_scaled_column() (*straindesign.compression.RationalMatrix method*), 100
addExclusionConstraintIneq() (*straindesign.scip_interface.SCIP_MILP method*), 138
addExclusionConstraintsIneq() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
addModule() (*straindesign.strainDesignProblem.SDProblem method*), 150

B

basic_columns() (*in module straindesign.compression*), 102
basic_columns_from_numpy() (*in module straindesign.compression*), 102
basic_columns_rat() (*in module straindesign.efmtool_cmp_interface*), 110
basic_columns_rat_java() (*in module straindesign.efmtool_cmp_interface*), 110
begin_batch_edit() (*straindesign.compression.RationalMatrix method*), 101
bound_blocked_or_irrevers_fva() (*in module straindesign.networktools*), 125
build_primal_from_cbm() (*in module straindesign.strainDesignProblem*), 152
build_sd_solution() (*straindesign.strainDesignMILP.SDMILP method*), 144

C

ceil_dec() (*in module straindesign.lptools*), 118
clear_objective() (*straindesign.solver_interface.MILP_LP method*), 141
clone() (*straindesign.compression.RationalMatrix method*), 101
close() (*straindesign.pool.SDPool method*), 134
compress() (*straindesign.compression.StoichMatrixCompressor method*), 102
compress_cobra_model() (*in module straindesign.compression*), 102
compress_ki_ko_cost() (*in module straindesign.networktools*), 125
compress_model() (*in module straindesign.compression*), 102
compress_model_coupled() (*in module straindesign.compression*), 103
compress_model_java() (*in module straindesign.efmtool_cmp_interface*), 110
compress_model_parallel() (*in module straindesign.compression*), 103
compress_modules() (*in module straindesign.networktools*), 125
CompressionConverter (*class in straindesign.compression*), 100
CompressionMethod (*class in straindesign.compression*), 100
CompressionRecord (*class in straindesign.compression*), 100
CompressionResult (*class in straindesign.compression*), 100
CompressionStatistics (*class in straindesign.compression*), 100
compute() (*straindesign.strainDesignMILP.SDMILP method*), 144
compute_optimal() (*straindesign.strainDesignMILP.SDMILP method*), 145
compute_strain_designs() (*in module straindesign.compute_strain_designs*), 104
compute_strain_designs_from_preprocessed() (*in module straindesign.compute_strain_designs*), 106
ContMILP (*class in straindesign.strainDesignProblem*), 149
copy() (*straindesign.strainDesignModule.SDModule method*), 149
Cplex_MILP_LP (*class in straindesign.cplex_interface*), 107

D

detect_max_precision() (*in module straindesign.compression*), 103
DisableLogger (*class in straindesign*), 158

E

end_batch_edit() (*straindesign.compression.RationalMatrix method*), 101
enumerate() (*straindesign.strainDesignMILP.SDMILP method*), 145
estimate_expansion_size() (*in module straindesign.networktools*), 126
evaluate_gpr_ast() (*in module straindesign.networktools*), 126
expand_all() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
expand_expression() (*straindesign.compression.CompressionConverter method*), 100
expand_group() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
expand_sd() (*in module straindesign.networktools*), 126
extend_model_gpr() (*in module straindesign.networktools*), 127
extend_model_regulatory() (*in module straindesign.networktools*), 127

F

farkas_dualize() (*in module straindesign.strainDesignProblem*), 153
fba() (*in module straindesign.lptools*), 118
filter_sd_maxcost() (*in module straindesign.networktools*), 128
fixObjective() (*straindesign.strainDesignMILP.SDMILP method*), 145
float_to_rational() (*in module straindesign.compression*), 103
floor_dec() (*in module straindesign.lptools*), 118
from_cobra_model() (*straindesign.compression.RationalMatrix class method*), 101
from_numpy() (*straindesign.compression.RationalMatrix class method*), 101
fva() (*in module straindesign.lptools*), 119
fva_legacy() (*in module straindesign.lptools*), 119
fva_worker_compute() (*in module straindesign.lptools*), 119
fva_worker_compute_glpk() (*in module straindesign.lptools*), 119
fva_worker_init() (*in module straindesign.lptools*), 119

fva_worker_init_glpk() (*in module straindesign.lptools*), 120

G

gene_kos_to_constraints() (*in module straindesign.networktools*), 128
get_basis() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 108
get_basis() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
get_basis() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
get_basis() (*straindesign.scip_interface.SCIP_LP method*), 136
get_basis() (*straindesign.scip_interface.SCIP_MILP method*), 138
get_basis() (*straindesign.solver_interface.MILP_LP method*), 141
get_gene_reac_sd_assoc() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_gene_reac_sd_assoc_mark_no_ki() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_gene_sd() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_gene_sd_mark_no_ki() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_group() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_lp_method() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 108
get_lp_method() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
get_lp_method() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
get_lp_method() (*straindesign.scip_interface.SCIP_LP method*), 136
get_lp_method() (*straindesign.scip_interface.SCIP_MILP method*), 138
get_lp_method() (*straindesign.solver_interface.MILP_LP method*), 141
get_num_groups() (*straindesign.strainDesignSolutions.SDSolutions method*), 157
get_num_materialized() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_num_sols() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_reaction_sd() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_reaction_sd_bnds() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_reaction_sd_mark_no_ki() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_representative_sd() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_rids() (*in module straindesign.parse_constr*), 131
get_signum() (*straindesign.compression.RationalMatrix method*), 101
get_strain_design_costs() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_strain_designs() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
get_subset() (*in module straindesign.strainDesignSolutions*), 158
getSolution() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
getSolution() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
getSolution() (*straindesign.scip_interface.SCIP_MILP method*), 138
getSolutions() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
GLPK_MILP_LP (*class in straindesign.glpk_interface*), 111
gpr_eval() (*in module straindesign.strainDesignSolutions*), 158
Gurobi_MILP_LP (*class in straindesign.gurobi_interface*), 114

I

identity() (*straindesign.compression.RationalMatrix*

class method), 101
idx2c() (*in module straindesign.lptools*), 120
IndicatorConstraints (*class in straindesign.indicatorConstraints*), 117
is_lazy (*straindesign.strainDesignSolutions.SDSolutions property*), 158
iter_column_fractions() (*straindesign.compression.RationalMatrix method*), 101

J

jBigFraction2sympyRat() (*in module straindesign.efmtool_cmp_interface*), 110
jBigIntegerPair2sympyRat() (*in module straindesign.efmtool_cmp_interface*), 110
jpypeArrayOfArrays2numpy_mat() (*in module straindesign.efmtool_cmp_interface*), 111

L

lineq2list() (*in module straindesign.parse_constr*), 131
lineq2mat() (*in module straindesign.parse_constr*), 131
lineqlist2mat() (*in module straindesign.parse_constr*), 132
lineqlist2str() (*in module straindesign.parse_constr*), 132
linexpr2dict() (*in module straindesign.parse_constr*), 132
linexpr2mat() (*in module straindesign.parse_constr*), 133
linexprdict2mat() (*in module straindesign.parse_constr*), 133
linexprdict2str() (*in module straindesign.parse_constr*), 133
link_z() (*straindesign.strainDesignProblem.SDProblem method*), 150
load() (*straindesign.strainDesignSolutions.SDSolutions class method*), 158
LP_dualize() (*in module straindesign.strainDesignProblem*), 151

M

MILP_LP (*class in straindesign.solver_interface*), 139
module
    straindesign, 99
    straindesign.compression, 100
    straindesign.compute_strain_designs, 104
    straindesign.cplex_interface, 107
    straindesign.efmtool_cmp_interface, 110
    straindesign.glpk_interface, 111
    straindesign.gurobi_interface, 114
    straindesign.indicatorConstraints, 117
    straindesign.lptools, 118
    straindesign.names, 123
    straindesign.networktools, 125
    straindesign.parse_constr, 131
    straindesign.pool, 134
    straindesign.scip_interface, 135
    straindesign.solver_interface, 139
    straindesign.speedy_fva, 142
    straindesign.strainDesignMILP, 143
    straindesign.strainDesignModule, 146
    straindesign.strainDesignProblem, 149
    straindesign.strainDesignSolutions, 156
modules_coeff2float() (*in module straindesign.networktools*), 129
modules_coeff2rational() (*in module straindesign.networktools*), 129

N

nullspace() (*in module straindesign.compression*), 103
numpy_mat2jpypeArrayOfArrays() (*in module straindesign.efmtool_cmp_interface*), 111

P

parse_constraints() (*in module straindesign.parse_constr*), 133
parse_linexpr() (*in module straindesign.parse_constr*), 134
plot_flux_space() (*in module straindesign.lptools*), 120
populate() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 108
populate() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 112
populate() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 115
populate() (*straindesign.solver_interface.MILP_LP method*), 141
populateZ() (*straindesign.strainDesignMILP.SDMILP method*), 145
postprocess_reg_sd() (*in module straindesign.compute_strain_designs*), 107
prevent_boundary_knockouts() (*in module straindesign.strainDesignProblem*), 154

R

RationalMatrix (*class in straindesign.compression*), 100
reassign_lb_ub_from_ineq() (*in module straindesign.strainDesignProblem*), 155
reduce_gpr() (*in module straindesign.networktools*), 129

remove_blocked_reactions() (*in module straindesign.compression*), 104
remove_columns() (*straindesign.compression.RationalMatrix method*), 101
remove_conservation_relations() (*in module straindesign.compression*), 104
remove_dummy_bounds() (*in module straindesign.compression*), 104
remove_ext_mets() (*in module straindesign.compression*), 104
remove_redundant_bounds() (*in module straindesign.lptools*), 121
remove_rows() (*straindesign.compression.RationalMatrix method*), 101
resetObjective() (*straindesign.strainDesignMILP.SDMILP method*), 145
resetTargetableZ() (*straindesign.strainDesignMILP.SDMILP method*), 145
resolve_gene_constraints() (*in module straindesign.networktools*), 130

S

save() (*straindesign.strainDesignSolutions.SDSolutions method*), 158
SCIP_LP (*class in straindesign.scip_interface*), 135
SCIP_MILP (*class in straindesign.scip_interface*), 137
sd2dict() (*straindesign.strainDesignMILP.SDMILP method*), 145
SDMILP (*class in straindesign.strainDesignMILP*), 143
SDModule (*class in straindesign.strainDesignModule*), 146
SDPool (*class in straindesign.pool*), 134
SDProblem (*class in straindesign.strainDesignProblem*), 149
SDSolutions (*class in straindesign.strainDesignSolutions*), 156
select_solver() (*in module straindesign.lptools*), 122
set_basis() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_basis() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_basis() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_basis() (*straindesign.scip_interface.SCIP_LP method*), 136
set_basis() (*straindesign.scip_interface.SCIP_MILP method*), 138
set_basis() (*straindesign.solver_interface.MILP_LP method*), 141
set_ineq_constraint() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_ineq_constraint() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_ineq_constraint() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_ineq_constraint() (*straindesign.scip_interface.SCIP_MILP method*), 138
set_ineq_constraint() (*straindesign.solver_interface.MILP_LP method*), 141
set_lp_method() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_lp_method() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_lp_method() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_lp_method() (*straindesign.scip_interface.SCIP_LP method*), 136
set_lp_method() (*straindesign.scip_interface.SCIP_MILP method*), 138
set_lp_method() (*straindesign.solver_interface.MILP_LP method*), 141
set_objective() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_objective() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_objective() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_objective() (*straindesign.scip_interface.SCIP_LP method*), 136
set_objective() (*straindesign.scip_interface.SCIP_MILP method*), 138
set_objective() (*straindesign.solver_interface.MILP_LP method*), 141
set_objective_idx() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109

set_objective_idx() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_objective_idx() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_objective_idx() (*straindesign.scip_interface.SCIP_LP method*), 136
set_objective_idx() (*straindesign.scip_interface.SCIP_MILP method*), 139
set_objective_idx() (*straindesign.solver_interface.MILP_LP method*), 141
set_time_limit() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_time_limit() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_time_limit() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_time_limit() (*straindesign.scip_interface.SCIP_LP method*), 136
set_time_limit() (*straindesign.scip_interface.SCIP_MILP method*), 139
set_time_limit() (*straindesign.solver_interface.MILP_LP method*), 142
set_ub() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
set_ub() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
set_ub() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
set_ub() (*straindesign.scip_interface.SCIP_MILP method*), 139
set_ub() (*straindesign.solver_interface.MILP_LP method*), 142
setMinIntvCostObjective() (*straindesign.strainDesignMILP.SDMILP method*), 145
setTargetableZ() (*straindesign.strainDesignMILP.SDMILP method*), 145
slim_solve() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
slim_solve() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
slim_solve() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
slim_solve() (*straindesign.scip_interface.SCIP_LP method*), 136
slim_solve() (*straindesign.scip_interface.SCIP_MILP method*), 139
slim_solve() (*straindesign.solver_interface.MILP_LP method*), 142
solve() (*straindesign.cplex_interface.Cplex_MILP_LP method*), 109
solve() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 113
solve() (*straindesign.gurobi_interface.Gurobi_MILP_LP method*), 116
solve() (*straindesign.scip_interface.SCIP_LP method*), 136
solve() (*straindesign.scip_interface.SCIP_MILP method*), 139
solve() (*straindesign.solver_interface.MILP_LP method*), 142
solve_MILP_LP() (*straindesign.glpk_interface.GLPK_MILP_LP method*), 114
solveZ() (*straindesign.strainDesignMILP.SDMILP method*), 146
speedy_fva() (*in module straindesign.speedy_fva*), 143
standard() (*straindesign.compression.CompressionMethod class method*), 100
stoichmat_coeff2float() (*in module straindesign.compression*), 104
stoichmat_coeff2rational() (*in module straindesign.compression*), 104
StoichMatrixCompressor (*class in straindesign.compression*), 101
straindesign
    module, 99
straindesign.compression
    module, 100
straindesign.compute_strain_designs
    module, 104
straindesign.cplex_interface
    module, 107
straindesign.efmtool_cmp_interface
    module, 110
straindesign.glpk_interface
    module, 111
straindesign.gurobi_interface
    module, 114
straindesign.indicatorConstraints
    module, 117
straindesign.lptools
    module, 118
straindesign.names

module, 123
straindesign.networktools
module, 125
straindesign.parse_constr
module, 131
straindesign.pool
module, 134
straindesign.scip_interface
module, 135
straindesign.solver_interface
module, 139
straindesign.speedy_fva
module, 142
straindesign.strainDesignMILP
module, 143
straindesign.strainDesignModule
module, 146
straindesign.strainDesignProblem
module, 149
straindesign.strainDesignSolutions
module, 156
strip_non_ki() (*in module straindesign.strainDesignSolutions*), 158
submatrix() (*straindesign.compression.RationalMatrix method*), 101
suppress_lp_context() (*in module straindesign.networktools*), 130
sympyRat2jBigIntegerPair() (*in module straindesign.efmtool_cmp_interface*), 111

T

to_numpy() (*straindesign.compression.RationalMatrix method*), 101
to_sparse_csr() (*straindesign.compression.RationalMatrix method*), 101
to_sparse_pattern() (*straindesign.compression.RationalMatrix method*), 101

V

verify_sd() (*straindesign.strainDesignMILP.SDMILP method*), 146

W

with_suppressed_lp() (*in module straindesign.networktools*), 130
worker_compute() (*in module straindesign.strainDesignProblem*), 156
worker_init() (*in module straindesign.strainDesignProblem*), 156

Y

yopt() (*in module straindesign.lptools*), 122